Seconde

Sciences Numériques et Technologie

Cours • Exercices • Corrigés



Mathieu Thibaud

Édition 2026

Version du 20 octobre 2026 — v2026.10.20 (sources : commit 4126de1)

Sommaire

Présentation de l'année . . . 3
Méthode : débattre . . . 7
Frise chronologique . . . 10

Cours et exercices

1 Les bases de la programmation Python . . . 13
Complément : dessiner avec la tortue . . . 47
2 Les réseaux sociaux . . . 57
3 La photographie numérique . . . 81
4 Le Web . . . 107
5 Les données structurées et leur traitement . . . 129
6 Internet . . . 153
7 Localisation, cartographie et mobilité . . . 173
8 Informatique embarquée et objets connectés . . . 193

Corrigés

1 Les bases de la programmation Python . . . 214
Complément : dessiner avec la tortue . . . 225
2 Les réseaux sociaux . . . 230
3 La photographie numérique . . . 239
4 Le Web . . . 246
5 Les données structurées et leur traitement . . . 255
6 Internet . . . 262
7 Localisation, cartographie et mobilité . . . 270
8 Informatique embarquée et objets connectés . . . 277
Lexique . . . 287
Crédits iconographiques . . . 290

Présentation de l'année

Vous passez des heures chaque jour sur un téléphone, un ordinateur, une console… mais savez-vous vraiment **ce qui se passe derrière l'écran** ? Comment votre photo devient une suite de nombres, comment une page Web arrive jusqu'à vous, comment une appli sait où vous vous trouvez, ou pourquoi un réseau social vous montre *exactement* ce contenu-là ? La SNT, c'est l'année où vous passez du statut de simple **utilisateur** à celui de personne qui **comprend** — et qui garde son **esprit critique** dans un monde numérique.

L'esprit SNT

- **Comprendre** plutôt que subir : « mais comment ça marche, en vrai ? »
- **Fabriquer** un peu soi-même : quelques lignes de code, une image retouchée, une petite page Web…
- **Prendre du recul** : derrière chaque service « gratuit », des données, des choix, des enjeux de société.

I La SNT, c'est quoi ?

La SNT est un enseignement de **tronc commun** : *tout le monde* en fait en Seconde, à raison de **2 heures par semaine**. Ce n'est pas une spécialité et il n'y a **pas d'épreuve au baccalauréat** — mais le travail compte dans votre bulletin, et surtout cette culture vous servira toute votre vie. C'est aussi une belle occasion de découvrir si l'informatique vous plaît, avant de choisir (ou non) la spécialité **NSI** en Première.

Le programme s'organise autour de **sept grands thèmes** officiels, plus un outil qui nous accompagnera toute l'année : un peu de **programmation en Python**. Cela fait **huit chapitres** : l'outil Python d'abord, puis les sept thèmes. On ne cherche pas à devenir informaticien : on cherche à **comprendre**, à **manipuler** et à **réfléchir** en citoyen éclairé.

II Ce que vous allez découvrir cette année

On commence toujours par apprendre à donner quelques ordres à la machine (Python), puis on alterne les thèmes : certains demandent de « mettre les mains dans le code », d'autres invitent surtout à observer, débattre et prendre du recul.

A. De la rentrée aux vacances de Toussaint

Chapitre 1 — Les bases de la programmation (Python), l'outil de l'année, suivi d'un complément pour dessiner avec la tortue. Vous allez apprendre à écrire vos premiers petits programmes : ranger des informations dans des variables, faire faire des choix à la machine, lui faire répéter des actions. C'est l'outil qu'on retrouvera dans plusieurs thèmes de l'année.

Chapitre 2 — Les réseaux sociaux. Vous utilisez Instagram, TikTok, Snapchat, WhatsApp… mais comment fonctionnent-ils vraiment ? On parlera « amis d'amis » et petit monde, modèle économique de la « gratuité », algorithmes de recommandation, cyberharcèlement et identité numérique. Beaucoup de discussion et d'esprit critique.

B. De la Toussaint à Noël

Chapitre 3 — La photographie numérique. Surprise : une photo, ce n'est que des **nombres** ! Vous allez découvrir les pixels, le codage des couleurs (RVB), les métadonnées (EXIF, y compris la position GPS…), puis **retoucher vous-même** des images avec quelques lignes de Python. Le thème où l'on réinvestit le plus la programmation.

C. De janvier aux vacances d'hiver

Chapitre 4 — Le Web (et votre propre site !)**.** Vous allez comprendre ce qui se passe quand vous cliquez sur un lien : HTML, CSS, adresses, hypertexte… Et surtout, le temps fort de l'année : deux **projets**, dont les sujets vous seront **distribués à part** (ils ne figurent pas dans ce manuel). D'abord, seul et en temps limité, vous reproduirez un petit site modèle ; puis, par groupes de trois, vous **concevrez et réaliserez votre propre mini-site** sur un lieu, un événement ou un thème **monégasque** de votre choix. À vous de créer quelque chose que l'on pourra vraiment afficher dans un navigateur.

D. Des vacances d'hiver à celles de printemps

Chapitre 5 — Les données structurées et leur traitement. Météo, résultats sportifs, catalogues, open data… le numérique carbure aux données. Vous allez apprendre à les organiser en tables, à les trier, les filtrer et les croiser — et à vous demander qui les collecte, où elles sont stockées et ce qu'on peut en déduire.

Chapitre 6 — Internet et les réseaux. Comment un message part de votre téléphone et arrive à l'autre bout de la planète en une fraction de seconde ? Vous allez suivre le voyage des données : adresses IP, routage, « paquets », protocoles… la face cachée d'Internet.

E. Du printemps à la fin de l'année

Chapitre 7 — La localisation, la cartographie et la mobilité. Comment votre téléphone sait-il où vous êtes ? Vous allez découvrir le GPS, les cartes numériques et le calcul d'itinéraire (le plus court chemin), en gardant un œil sur ce que cela implique pour votre vie privée.

Chapitre 8 — L'informatique embarquée et les objets connectés. Montres, voitures, maisons « intelligentes »… des ordinateurs se cachent partout. Vous allez comprendre comment un objet capte son environnement, décide et agit — et pourquoi la sécurité de ces objets est un vrai sujet.

> **Remarque** Ce calendrier est indicatif : l'ordre et le rythme des thèmes peuvent évoluer au fil de l'année ; nous commençons toujours par Python.

III Comment on travaille

Nos **2 heures par semaine** mêlent plusieurs façons d'apprendre :

- **du cours**, court et illustré, pour poser les idées et le vocabulaire ;
- **des activités et des manipulations sur machine** : on programme un peu, on teste, on explore de vrais outils et de vraies données ;
- **des débats et des exposés** : sur les réseaux sociaux, les données, la vie privée — ici, votre avis argumenté compte (les sujets d'exposé sont distribués à part) ;
- **deux projets Web** : reproduire un petit site modèle, puis créer en groupe votre propre mini-site (sujets distribués à part, hors de ce manuel).

Comment vous êtes évalué

- de courtes **interrogations** et **QCM** pour vérifier l'essentiel, et un **contrôle** sur les gros thèmes ;
- vos **activités, exposés et projets Web**, qui comptent aussi.

Pas de panique : il n'y a **pas d'examen final** en SNT. Le but n'est pas de « tout retenir », mais de **comprendre comment ça marche** et de devenir un utilisateur plus lucide.

Remarque Deux sites tenus par des professeurs sont d'excellentes ressources pour réviser ou aller plus loin à la maison :

- le site de **David Roche** (`pixees.fr/informatiquelycee`), qui propose de nombreux cours et activités de **SNT** ;
- le site de **Gilles Lassus** (`glassus.github.io`), plutôt orienté **NSI**, utile si vous poursuivez cette spécialité en Première.

L'intelligence artificielle : une charte, trois pastilles

Les assistants d'IA savent écrire des textes, des réponses et même des programmes ; le but de ce cours est que **vous** compreniez comment fonctionne le numérique et sachiez vérifier ce qu'on vous affirme. Une **charte d'usage de l'IA**, distribuée en début d'année, fixe ce qui est permis. L'usage est décidé **exercice par exercice** et signalé par une pastille à côté du titre :

-  **sans IA** : tests, contrôles, exercices d'automatisme (lecture de code, questions de cours) ;
-  **IA en appui** : pour reformuler, vérifier ou comprendre une erreur, à condition d'écrire soi-même la réponse finale ;
-  **IA intégrée** : l'exercice consiste à critiquer ou corriger une réponse d'assistant.

Dans tous les cas : vous restez responsable de ce que vous rendez, vous devez pouvoir l'expliquer à l'oral, et vous dites quand l'IA a été utilisée.

IV Travailler avec ce livre

Comment est organisé un chapitre

- **Cours** : les notions, appuyées sur des **documents** (chiffres, textes, schémas) et des **activités** à faire en classe, avec un bilan ;
- **Exercices** : du plus simple au plus exigeant (★, ★★, ★★★), avec des coups de pouce en fin de rubrique ; environ un tiers sont marqués • essentiel (le minimum pour suivre). Les corrigés, et les éléments de réponse des activités, sont en fin de livre ;
- selon les thèmes, un **débat** ou un **TP** sur machine ;
- **Fiche de révision** : l'essentiel du thème sur une page, avec « Se tester » (questions sur le thème) et « Et avant ? » (questions sur les thèmes précédents, pour ne pas oublier).

En début de livre, une **frise chronologique** replace les grandes inventions dans le temps ; en fin de livre, un **lexique** rappelle le sens des mots importants.

A. Avec quel outil ?

Au lycée, on programme avec **Spyder**, un logiciel libre et gratuit (sa prise en main est expliquée au chapitre 1). Vous pouvez l'installer chez vous : l'installateur officiel se télécharge sur `spyder-ide.org` et contient Python. Sans installation, **Basthon** fonctionne dans un navigateur (`basthon.fr`) : pratique pour la plupart des exercices de ce livre.

B. Une méthode pour chaque thème

1. **Après la séance** (15 à 20 minutes) : relire le cours, puis faire les exercices • essentiel indiqués par les renvois « Exercices d'application ».
2. **Bloqué ?** Chercher un peu, puis lire le **coup de pouce** (▷) avant de regarder le corrigé.
3. **Le corrigé** se lit *après* avoir essayé : comparer et repérer son erreur (les corrigés signalent les *erreurs fréquentes*).
4. **Quelques jours après** : répondre de mémoire aux questions « Se tester » et « Et avant ? » de la fiche, puis vérifier avec le lien « ▶ réponses ».

C. Lire ce livre sur un ordinateur

- **Tout est cliquable** : le sommaire, le petit sommaire de chaque chapitre, « ▶ corrigé p. 209 » et « ↑ énoncé », les coups de pouce, les numéros d'exercices et les renvois de la frise. L'**onglet de couleur** au bord de la page ramène à la page de titre du chapitre ; le **numéro de page** ramène au sommaire.
- **Les signets** (panneau latéral « Signets » ou « Table des matières » du lecteur PDF) donnent le plan détaillé.
- **Revenir où l'on était** après un lien : le lien « ↑ énoncé », ou la commande « page précédente » du lecteur (Alt + ← dans Acrobat Reader sous Windows, Cmd + [dans Aperçu sur Mac).
- **Copier un programme** : cliquer sur l'icône ⧉ en haut à droite d'un bloc de code ; une page s'ouvre avec le programme prêt à être copié.
- **Chercher un mot** : Ctrl + F (Cmd + F sur Mac). Les numéros de page du lecteur sont ceux du livre imprimé.

Prêt(e)s à voir ce qui se cache derrière l'écran ? Bienvenue en SNT !

Méthode : débattre

Chacun des sept thèmes se termine par une **fiche débat** : une question de société, deux camps (POUR et CONTRE) et quatre documents. Cette page donne la méthode, commune à tous les débats. Les rôles sont **tirés au sort** : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement. La préparation se fait avec les documents de la fiche et le cours,  sans IA ni recherche en ligne.

1. Les rôles

Rôle	Votre tâche (préparation de 15 min, puis débat)
Groupe POUR (≈ 1/3)	Défendre la réponse **oui** : préparer au moins trois arguments, chacun appuyé sur un document ou le cours ; prévoir la réponse à deux arguments adverses ; choisir qui ouvre et qui conclut.
Groupe CONTRE (≈ 1/3)	Défendre la réponse **non** : même travail que le groupe POUR.
Modérateur (1 ou 2)	Préparer deux questions de relance ; lire la question et les règles ; donner la parole en alternant les camps, tenir le temps, ramener à la question et aux documents. Ne **pas** donner son avis.
Secrétaire (1 ou 2)	Noter au tableau, en deux colonnes POUR / CONTRE, chaque argument et le numéro du document cité ; un argument sans appui est noté avec un point d'interrogation.
Observateurs (les autres)	Suivre deux ou trois élèves (citent-ils un document ? répondent-ils à l'argument précédent ?) ; noter le meilleur argument de chaque camp et le présenter au bilan.

2. Le déroulé minuté (50 min)

Min.	Étape	Min.	Étape
0–3	**Mise en place** : tirage des rôles, groupes face à face.	26–38	**Échanges** : 12 min, parole alternée entre les camps.
3–18	**Préparation** en groupes, tableau d'arguments de la fiche.	38–42	**Observateurs** : meilleur argument de chaque camp.
18–20	**Ouverture** : le modérateur lit la question et les règles.	42–44	**Conclusions** : 1 min par groupe.
20–26	**Prises de position** : 3 min POUR, puis 3 min CONTRE.	44–50	**Bilan** : trace écrite individuelle, puis vote.

3. Les règles de prise de parole

- Lever la main et attendre que le modérateur vous donne la parole ; une seule personne à la fois, une minute au plus.
- **Argumenter** (« je pense que… **parce que**… ») et **citer** un document ou le cours (« d'après le document 2… »).
- **Répondre à l'argument précédent** avant d'en avancer un nouveau : « vous dites que…, mais… ».
- **Respecter** les autres : critiquer une idée, jamais une personne ; pas de moquerie ; ne pas raconter la vie privée de quelqu'un.

4. Construire un argument

Ingrédient	Ce que vous dites	Pour commencer la phrase
Affirmation	Ce que vous défendez, en une phrase claire.	« Je pense que… »
Appui	Un document de la fiche (et son numéro) ou une notion du cours.	« … d'après le document 3, … »
Exemple	Un cas concret qui rend l'idée visible.	« Par exemple, … »

Un avis sans appui n'est qu'une opinion, pas un argument. *Exemple (question imaginée : faut-il commencer les cours à 9 h ?)* : « Je pense qu'il faut commencer plus tard (affirmation), parce que, d'après le document 2, les adolescents s'endorment plus tard que les adultes (appui). Par exemple, un élève qui se couche à 23 h et se lève à 6 h ne dort que 7 h (exemple). »

5. Répondre à l'argument adverse

1. **Reformuler** l'argument, pour montrer que vous avez écouté : « Vous dites que… »
2. **Répondre** : vérifier l'appui (le document dit-il vraiment cela ? concerne-t-il le même pays, la même époque ?), nuancer (« c'est vrai, mais… »), ou opposer un autre document ou un contre-exemple.
3. **Relancer** avec l'un de vos arguments.

6. La trace écrite individuelle

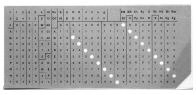
- **Avant le débat** : noter sur le cahier votre position de départ (oui, non ou je ne sais pas).
- **Pendant le débat** : compléter le tableau d'arguments de la fiche (argument et numéro du document).
- **Après le débat** : noter votre position (oui, non, je ne sais toujours pas) et si vous avez changé d'avis ; la justifier en trois à cinq lignes avec vos **deux meilleurs arguments**, dont **un de l'autre camp** que vous trouvez solide (avec son document).

Frise chronologique — de la photographie à Internet



1826 Nicéphore Niépce prend la première photographie (« Point de vue du Gras ») (ch. 3).

1888 George Eastman lance l'appareil Kodak (ch. 3).



1890 Herman Hollerith dépouille le recensement américain avec des cartes perforées (ch. 5).



1952 Grace Hopper écrit l'un des premiers compilateurs (ch. 1).

1965 Ted Nelson imagine l'hypertexte (ch. 4).



1967 Expérience du « petit monde » de Stanley Milgram (ch. 2) ; Seymour Papert crée le langage Logo (compl.).



1969 Premier message sur ARPANET (« LO ») (ch. 6) ; l'Apollo Guidance Computer guide les astronautes vers la Lune (ch. 8).

1971 La France lance le réseau Cyclades (ch. 6).



1974 Vinton Cerf et Robert Kahn inventent le protocole TCP/IP (ch. 6).



1975 Steven Sasson construit le premier appareil photo numérique (ch. 3).

Frise chronologique — du Web aux objets connectés

1983 Tous les réseaux passent à TCP/IP : naissance d'Internet ; apparition du DNS (ch. 6).



1989 Tim Berners-Lee invente le Web au CERN (ch. 4) ; Guido van Rossum commence Python (ch. 1).

1990 Un grille-pain connecté à Internet : l'un des premiers objets connectés (ch. 8).



1991 Première page web en ligne (ch. 4) ; première version publique de Python (ch. 1).

1994 Création du W3C, qui normalise le Web (ch. 4).



1995 Le GPS devient pleinement opérationnel (ch. 7).

1997 SixDegrees.com, premier réseau social en ligne (ch. 2).



1999 Kevin Ashton lance l'expression « Internet des objets » (ch. 8).



2004 Mark Zuckerberg crée Facebook (ch. 2).

2011 L'État français ouvre ses données sur data.gouv.fr (ch. 5).

2016 Premiers services du système européen Galileo (ch. 7) ; lancement de TikTok (ch. 2).

2018 Le RGPD, règlement européen sur la protection des données personnelles, entre en application (ch. 2).

2024 Monaco adopte la loi n° 1.565 sur la protection des données personnelles, contrôlée par l'APDP (ch. 2).

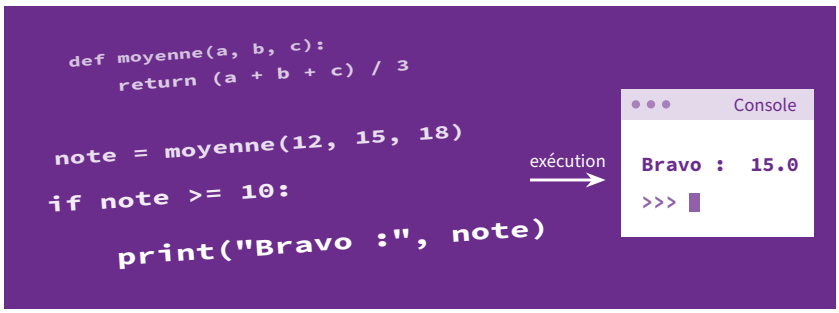
PREMIÈRE PARTIE

Cours et exercices

1

CHAPITRE 1

Les bases de la programmation Python



DANS CE CHAPITRE

- Cours 14
- Exercices 35
- Fiche de révision 45
- Corrigés 214

1

COURS

Bienvenue ! Rassurez-vous tout de suite : il n'y a rien de magique et rien de réservé aux « génies de l'informatique » dans ce chapitre. Programmer, c'est simplement donner des ordres très précis à une machine qui, soyons honnêtes, est extrêmement rapide mais aussi extraordinairement bête : elle fait exactement ce qu'on lui dit, ni plus, ni moins, même quand c'est absurde. Tout l'art consiste donc à lui parler correctement. Et bonne nouvelle : vous avez déjà fait de la programmation au collège avec Scratch. Ici, on va faire pareil… mais en tapant du texte au lieu d'emboîter des blocs.

Situation de départ

1. Vous tapez votre prénom sur un site, qui vous répond « Bonjour Léa ! ». Où le programme a-t-il gardé ce prénom pour pouvoir le réutiliser ?
2. Un petit programme doit additionner les deux nombres que vous tapez. Vous tapez 3 puis 4… et il affiche 34. Qu'est-ce qui a cloché ?
3. Dans un jeu « devinez le nombre », l'ordinateur répond « trop grand » ou « trop petit » jusqu'à ce que vous trouviez. Comment peut-il recommencer sans savoir combien d'essais il vous faudra ?

I Programmer, c'est quoi au juste ?

Programmer, c'est écrire un **programme**, c'est-à-dire une suite d'instructions données à un ordinateur pour qu'il effectue une tâche. Sans programme, un ordinateur ne sait strictement rien faire : c'est une coquille vide, un peu comme une console de jeu sans jeu.

Le seul langage que le processeur comprend vraiment est le **langage machine** : une suite interminable de 0 et de 1. Autant dire que personne (ou presque) ne programme directement comme ça aujourd'hui — ce serait aussi agréable que de rédiger une dissertation en morse.

À la place, on écrit des instructions avec des mots (souvent en anglais), plus proches du langage humain. Un programme spécial se charge ensuite de *traduire* ces instructions en langage machine. Selon la méthode de traduction, on parle d'**interpréteur** ou de **compilateur**. Python, notre langage, est un langage *interprété*.

Définition 1

Un **langage de programmation** est un langage qui permet d'écrire des programmes compréhensibles par la machine (après traduction). On en distingue deux grandes familles :

- les langages de **bas niveau** (proches de la machine, ex. l'assembleur) : très rapides, mais très pénibles à écrire ;
- les langages de **haut niveau** (proches du langage naturel, ex. Python, C, Java) : bien plus agréables pour un humain.

Nous utiliserons **Python** (version 3), un langage de haut niveau réputé pour sa clarté. C'est aussi le langage officiel du programme de SNT et de la spécialité NSI : autant prendre de bonnes habitudes tout de suite.

Pour les curieux — *Un peu d'histoire : des mots plutôt que des nombres* (facultatif)

L'idée d'écrire un programme avec des **mots** n'allait pas de soi. Dans les années 1950, on programme encore les ordinateurs avec des suites de nombres. L'Américaine **Grace Hopper** (1906–1992), mathématicienne et officière de la marine des États-Unis, pense au contraire que l'ordinateur doit s'adapter à l'humain. En 1952, elle écrit l'un des tout premiers **compilateurs** : un programme qui *traduit* des instructions lisibles en langage machine. On lui répond que « les ordinateurs ne comprennent pas l'anglais » ; elle s'obstine, et ses travaux mènent au langage COBOL (1959), utilisé dans les banques jusqu'à aujourd'hui.

Près de quarante ans plus tard, pendant les vacances de Noël 1989, le Néerlandais **Guido van Rossum** commence à écrire, à Amsterdam, un nouveau langage qu'il veut **simple à lire**. Il le baptise **Python**, non pas pour le serpent, mais en hommage à la troupe comique britannique des *Monty Python*. La première version est publiée en 1991 ; il y reprend, d'un langage plus ancien nommé ABC, l'**indentation obligatoire** que vous allez découvrir. Python est aujourd'hui l'un des langages les plus utilisés au monde, de l'enseignement à l'intelligence artificielle.



Grace Hopper en 1984



Guido van Rossum en 2006

→ frise chronologique p. 10

A. De Scratch à Python : ce qui change (et ce qui ne change pas)

Au collège, vous avez construit des programmes en **empilant des blocs** colorés dans Scratch. En Python, vous allez faire exactement la même chose, mais en **écrivant du texte**. Les grandes idées sont identiques : des variables, des boucles, des tests, des messages à afficher…

En Scratch (blocs)	En Python (texte)
dire (Bonjour) pendant (2) s	`print("Bonjour")`
mettre (score) à (0)	`score = 0`
répéter (10) fois	`for i in range(10):`
si ⟨ ⟩ alors	`if ... :`

Ce qui change vraiment, c'est la **rigueur**. Dans Scratch, un bloc mal placé, au pire, ne s'emboîte pas. En Python, une virgule oubliée, une majuscule de trop ou un guillemet manquant, et l'ordinateur boude en affichant une erreur. Ce n'est pas de la méchanceté : la machine a juste besoin qu'on lui parle *exactement* dans les règles. On s'y habitue très vite.

L'idée à garder : passer de Scratch à Python, c'est passer des *blocs* au *texte*. Les concepts sont les mêmes ; seule la façon de les écrire change.

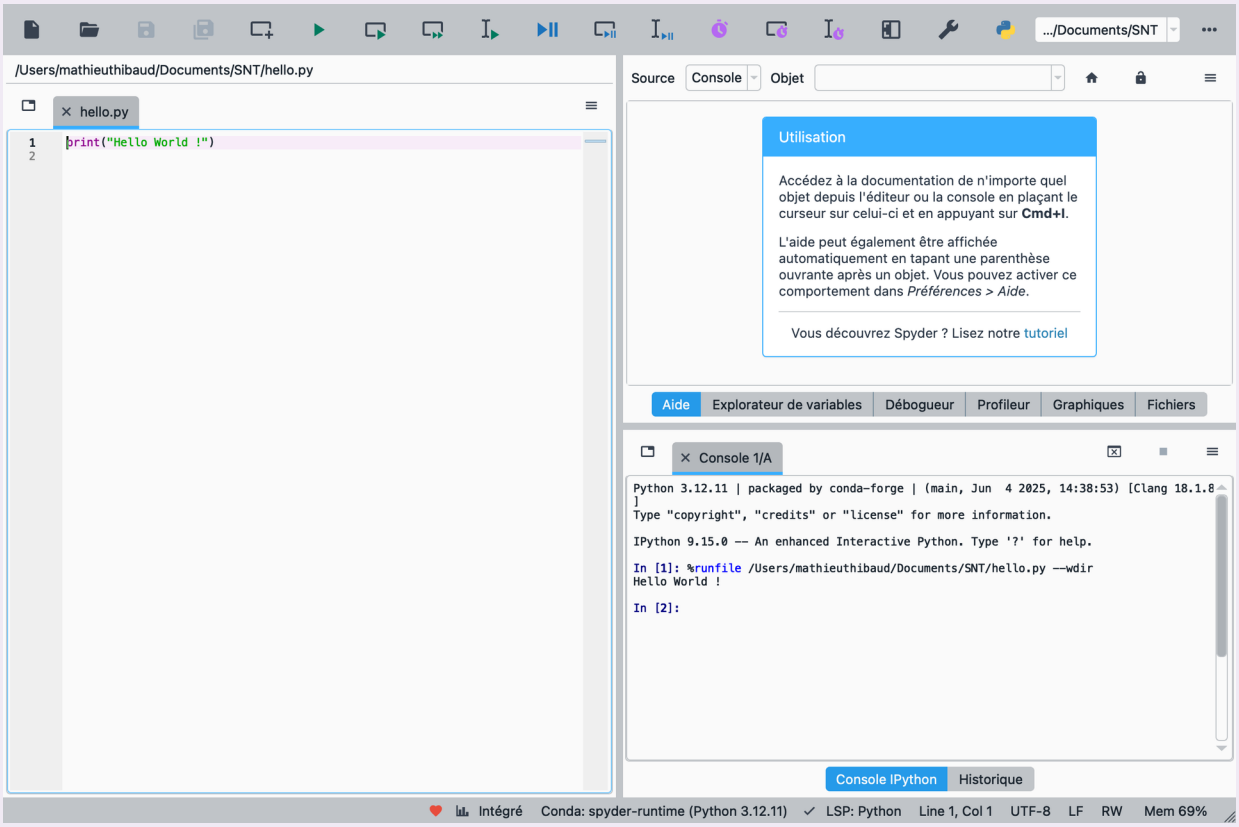
1

II L'environnement de travail

Pour écrire et exécuter nos programmes, il nous faut un outil. Nous utiliserons principalement le logiciel **Spyder**. En cas de souci (ordinateur récalcitrant, logiciel non installé...), nous aurons une solution de secours en ligne : **Basthon**.

A. Prise en main de Spyder

Une fois Spyder lancé, vous obtenez une fenêtre qui ressemble à ceci :



Spyder est divisé en plusieurs sous-fenêtres. Deux seulement vont nous intéresser pour l'instant :

- **L'éditeur** (à gauche) : c'est là qu'on *écrit* le programme. C'est notre feuille de brouillon... non, pardon, notre feuille au propre : ce qu'on y écrit est enregistré dans un fichier et peut être relancé autant de fois qu'on veut.
- **La console** (en bas à droite, avec le `In [1]:`) : c'est là que s'affichent les résultats. On peut aussi y taper des commandes « à la volée » pour tester rapidement une idée. C'est le *vrai* brouillon jetable.

Règle 1

Éditeur ou console ? La règle d'or.
- On écrit le **programme** (celui qu'on veut garder) dans l'**éditeur**.
- On utilise la **console** pour **tester** vite fait et **regarder** ce que contient une variable.

1. Le tout premier programme : « Hello World »

Il existe une tradition en informatique : le premier programme qu'on écrit dans un nouveau langage affiche « Hello World » (« Bonjour le monde »). Respectons la tradition.

Exemple

Dans la fenêtre *éditeur*, saisissez exactement :

```
1 print("Hello World !")
```

Puis cliquez sur le **triangle vert** ▶ (en haut) pour **exécuter** le programme. Spyder va vous demander d'enregistrer votre fichier : rangez-le dans un dossier de travail que vous retrouverez (par exemple `Documents/SNT/`, sous le nom `hello.py`). Dans la console, vous devez voir apparaître :

```
In [1]: %runfile .../Documents/SNT/hello.py --wdir
Hello World !

In [2]:
```

La ligne `In [1]` est écrite par Spyder quand il lance le programme ; la ligne suivante est l'affichage produit par `print`.

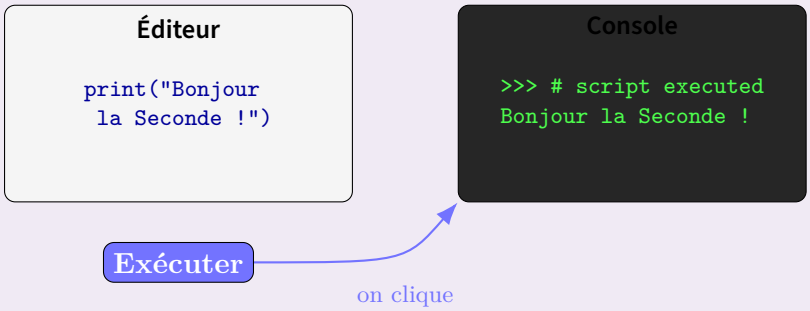
L'instruction `print(...)` (« imprimer » en anglais, au sens d'« afficher ») sert à **afficher un message dans la console**. C'est l'équivalent du bloc dire (...) de Scratch. Le message à afficher se met entre **guillemets** et entre **parenthèses**.

Remarque Attention aux petits détails, la machine ne pardonne rien :

- `print` s'écrit en minuscules (pas `Print` ni `PRINT`) ;
- il faut des *parenthèses* `( )` et des *guillemets droits* `" "` ;
- tout doit être écrit en caractères « normaux » : pas de guillemets « à la française » ni de guillemets « courbes » du traitement de texte.

B. Basthon : la solution de secours

Il arrivera qu'un poste ne veuille rien savoir : Spyder plante, n'est pas installé, ou vous travaillez sur votre propre ordinateur à la maison. Dans ce cas, direction **Basthon**, un « bac à sable » Python qui fonctionne **directement dans le navigateur**, sans rien installer.



On retrouve exactement la même organisation que dans Spyder : **on écrit à gauche**, on clique sur **Exécuter**, et **le résultat s'affiche à droite**. Rien de nouveau à apprendre, c'est rassurant.

Règle 2

Utiliser Basthon en 3 étapes :

1. Ouvrez un navigateur et allez sur `https://console.basthon.fr`.
2. Écrivez votre programme dans la zone de gauche (l'éditeur).
3. Cliquez sur **Exécuter** : le résultat apparaît dans la console de droite.

1

Remarque Basthon dépanne très bien, mais il ne conserve pas vos fichiers d'une séance à l'autre aussi facilement que Spyder. Pensez à **télécharger** votre travail (bouton en forme de flèche vers le bas) si vous voulez le garder.

Exemple — *Le détective*
Le programme ci-dessous contient **trois** erreurs qui empêchent Python de l'exécuter. Les retrouver et les corriger.

```
Print("Bonjour)
print(Au revoir)
```

▶ **Correction.** Les trois erreurs : `Print` prend une majuscule (Python écrit `print`) ; le guillemet fermant manque après `Bonjour` ; `Au revoir` est du texte, il doit être entre guillemets. Le programme corrigé :

```
1 print("Bonjour")
2 print("Au revoir")
```

▶ **Exercices d'application :** exercices 1 **et** 2 (premiers pas : afficher)

III Les variables

A. L'idée : une boîte étiquetée dans la mémoire

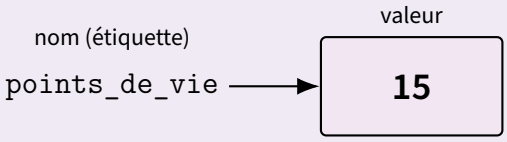
Un ordinateur passe son temps à *traiter des données* : des nombres, des mots, etc. Pour manipuler une donnée, il doit d'abord la **ranger** quelque part dans sa mémoire (la RAM). Cette mémoire est faite d'une multitude de petites cases.

Définition 2

Une **variable** est une donnée temporaire rangée dans la mémoire. Elle possède :

- un **nom** : l'étiquette qui permet de la retrouver ;
- une **valeur** : la donnée qu'elle contient (par exemple le nombre 15).

On dit qu'elle est « variable » car sa valeur peut **changer** au cours du programme.



Pour créer une variable, on utilise le signe `=`, appelé **affectation** :

```
1 points_de_vie = 15
```

Cette ligne se lit : « la variable `points_de_vie` prend la valeur 15 ». On dit aussi que `points_de_vie` *référence* le nombre 15.

Passerelle avec Scratch

Vous connaissez déjà ça ! Dans Scratch, le bloc
mettre (points_de_vie) à (15) fait exactement la même chose. En Python, on écrit simplement `points_de_vie = 15`.

Remarque Le signe `=` en Python ne veut **pas** dire « est égal à » comme en maths ! Il veut dire « **reçoit** la valeur ». La ligne `points_de_vie = 15` est un ordre : « range 15 dans la boîte `points_de_vie` ». C'est une nuance importante, on y revient au § V (« Le grand classique : `a = a + 1` »).

B. Afficher et inspecter une variable

Une fois la variable créée, comment voir ce qu'elle contient ? Deux façons.

1. Dans un programme, avec `print` :

```
1 points_de_vie = 15
2 print(points_de_vie)
```

(On écrit le nom *sans* guillemets : on veut afficher la *valeur* rangée dans la boîte, pas le mot « points_de_vie ».)

2. Dans la console, en tapant simplement le nom de la variable après avoir exécuté le programme :



C. Bien nommer ses variables

On a le droit de choisir presque n'importe quel nom… mais pas tout à fait n'importe comment.

Règle 3

Règles pour nommer une variable :

- uniquement des lettres, des chiffres et le tiret bas `_` ;
- **pas d'espace** et **pas de caractères spéciaux** (`-`, `!`, `€`, `?`, etc.) ;
- le nom ne commence **jamais** par un chiffre ;
- Python fait la différence entre majuscules et minuscules : `Age`, `age` et `AGE` sont trois variables *différentes* !

Remarque **Et les accents ?** Python accepte les lettres accentuées : `prénom` ou `année` sont des noms valides. Mais c'est une **bonne pratique** de s'en passer (`prenom`, `annee`) : les accents sont pénibles à taper sur certains claviers, et la quasi-totalité des programmes du monde est écrite sans. On évitera donc les accents dans nos noms de variables.

Conseil de pro : donnez à vos variables des noms qui *veulent dire quelque chose*. `prix_total` est mille fois plus clair que `x` ou `truc`. Votre vous du futur (celui qui relira ce code dans deux semaines) vous remerciera.

Nom	Valide ?	Pourquoi
`age`	**oui**	parfait
`prix_total`	**oui**	clair et lisible
`note2`	**oui**	un chiffre à la fin, autorisé
`2note`	**non**	commence par un chiffre
`prix total`	**non**	contient une espace
`prénom`	**oui, mais**	accepté, mais à éviter (préférez `prenom`)

► **Exercices d'application :** exercices 3 **à** 6 (les variables ; l'échange)

1

IV Les types de données

Toutes les données ne se ressemblent pas : un nombre entier, un nombre à virgule, un mot, une réponse « vrai / faux »… Python range chaque donnée dans une **catégorie**, qu'on appelle son **type**. Connaître le type, c'est savoir ce qu'on a le droit de faire avec (on n'additionne pas un mot et un nombre comme on additionne deux nombres).

Définition 3

Les quatre types de base que nous rencontrerons :

- `int` : un nombre **entier** (*integer*), ex. `12`, `-4`, `0` ;
- `float` : un nombre **à virgule** (*flottant*), ex. `5.2`, `-0.5` ;
- `str` : une **chaîne de caractères** (*string*), du texte, ex. `"Bonjour"` ;
- `bool` : un **booléen**, qui vaut `True` (vrai) ou `False` (faux). (On l'étudie au § VIII.)

Remarque **Le point à la place de la virgule !** En Python (comme dans le monde anglo-saxon), le nombre « cinq virgule deux » s'écrit `5.2` et **pas** `5,2`. Écrire une virgule donnerait un résultat inattendu (Python croit qu'on lui donne deux nombres). C'est l'erreur classique de début d'année.

Pour connaître le type d'une donnée, on utilise l'instruction `type(...)` :

Exemple

Testez ce programme, puis tapez `type(a)` puis `type(b)` dans la console :

```
1 a = 5.2
2 b = 12
```

```
In [2]: type(a)
Out[2]: float

In [3]: type(b)
Out[3]: int
```

La console répond `float` pour `a` et `int` pour `b`.

A. Changer de type (conversion)

On peut demander à Python de **transformer** une donnée d'un type vers un autre, quand c'est possible :

- `int("42")` transforme le texte `"42"` en entier `42` ;
- `float("3.14")` transforme du texte en flottant ;
- `str(42)` transforme le nombre `42` en texte `"42"`.

Ces conversions seront très utiles au § VI, quand on voudra mélanger nombres et texte.

Exemple — *Conversions*

Sans machine, donner le type *et* la valeur de chaque résultat, puis vérifier :

```
1 int("12")
2 str(12)
3 float("3.5")
4 int(3.9)
```

(La dernière ligne est un piège amusant : `int` ne fait pas d'arrondi, il coupe.)

► Correction. `int("12")` donne l'entier `12` (`int`) ; `str(12)` donne le texte `"12"` (`str`) ; `float("3.5")` donne le flottant `3.5` (`float`) ; `int(3.9)` donne l'entier `3` (`int`) : la partie après la virgule est **coupée**, pas arrondie.

V Un peu de calculs

Un ordinateur, c'est d'abord une grosse calculette. Python connaît les opérations habituelles :

Symbole	Opération	Exemple
`+`	addition	`3 + 4` donne `7`
`-`	soustraction	`10 - 6` donne `4`
`*`	multiplication	`3 * 5` donne `15`
`/`	division	`7 / 2` donne `3.5`
`//`	division entière (quotient)	`7 // 2` donne `3`
`%`	modulo (reste de la division)	`7 % 2` donne `1`
`**`	puissance	`2 ** 3` donne `8`

Les deux lignes `//` et `%` méritent une mention spéciale. Elles répondent à la question de l'école primaire : « dans 7, combien de fois 2, et combien reste-t-il ? » Réponse : 3 fois, et il reste 1. Le modulo `%` est étonnamment utile (par exemple, un nombre est pair si `n % 2` vaut 0).

Règle 4

Deux règles à retenir.

- La division `/` donne **toujours** un `float`, même quand elle « tombe juste » : `6 / 2` donne `3.0` et non `3`. Dès qu'un `float` entre dans un calcul, le résultat est un `float` : `3 * 1.0` donne `3.0`.
- Python respecte les **priorités opératoires** des maths : d'abord les parenthèses, puis `**`, puis `*`, `/`, `//`, `%`, enfin `+` et `-`. À priorité égale, on calcule de gauche à droite : `2 + 3 * 4` donne `14`, `(2 + 3) * 4` donne `20`.

On peut calculer avec des nombres, mais aussi (et surtout) avec des **variables** :

Exemple

```
1 a = 6
2 b = 4
3 somme = a + b
4 produit = a * b
```

Après exécution, `somme` vaut 10 et `produit` vaut 24.

1

A. Le grand classique : `a = a + 1`

Voici une ligne qui déroute tout le monde au début :

```
a = 11
a = a + 1
```

En maths, « $a = a + 1$ » est impossible (aucun nombre n'est égal à lui-même plus un). Mais souvenez-vous : en Python, `=` ne veut pas dire « est égal à », il veut dire « **reçoit** ». La règle pour comprendre :

Règle 5

On lit une affectation de *droite à gauche*.
On calcule d'abord tout ce qui est à droite du `=`, *puis* on range le résultat dans la variable de gauche.

Déroulons `a = a + 1` : à droite, `a + 1` vaut $11 + 1 = 12$. On range donc 12 dans `a`. Résultat : `a` vaut maintenant 12. On vient d'**augmenter** `a` de 1 (on dit « incrémenter »). Cette technique est partout en programmation, par exemple pour compter des points dans un jeu.

Passerelle avec Scratch

Encore une vieille connaissance ! Le bloc Scratch
ajouter (1) à (a) fait exactement ce que fait `a = a + 1`.

B. Fonctions mathématiques : le module `math`

Pour des calculs plus avancés (racine carrée, cosinus…), il faut charger une boîte à outils supplémentaire, le **module** `math`. On l'importe en **tout début de programme** :

```
import math

racine = math.sqrt(16)       # racine carree de 16 -> 4.0
puissance = math.pow(2, 5)   # 2 puissance 5 -> 32.0
```

Quelques fonctions utiles : `math.sqrt(x)` (racine carrée), `math.cos(x)`, `math.sin(x)` (l'angle `x` est en radians, une unité que vous verrez en maths plus tard), `math.pi` (la constante π). La liste complète est dans la documentation officielle : `https://docs.python.org/fr/3/library/math.html`.

Remarque Pour une simple puissance, l'opérateur `**` suffit et est plus court : `2 ** 5` donne directement `32` (un entier), là où `math.pow(2, 5)` donne `32.0` (un flottant).

Exemple — *L'aire du disque*
Écrire un programme qui, à partir d'une variable `rayon`, calcule et affiche l'aire d'un disque (rappel : aire $= \pi \times r^2$). Utiliser `math.pi`.

▶ **Correction.** On importe le module `math` pour disposer de π :

```
import math

rayon = 3
aire = math.pi * rayon ** 2
print(aire)
```

Pour un rayon de 3, le programme affiche `28.274333882308138`.

▶ **Exercices d'application :** exercices 7 à 14 (les calculs)

VI Chaînes de caractères, entrées et sorties

On a déjà croisé les chaînes de caractères (le texte entre guillemets). Voyons ce qu'on peut en faire, et surtout comment **dialoguer** avec l'utilisateur.

A. Coller des chaînes : la concaténation

Le signe `+` ne sert pas qu'à additionner : avec du texte, il sert à **coller bout à bout** deux chaînes. On appelle ça la **concaténation**.

Exemple — *Concaténation*

```
debut = "Bonjour "
fin = "le monde !"
message = debut + fin
print(message)    # affiche : Bonjour le monde !
```

Remarque L'espace après « Bonjour » est important : sans lui, on obtiendrait « Bonjourle monde ! ». La machine colle *exactement* ce qu'on lui donne. On peut aussi *répéter* une chaîne avec `*` : `"ha" * 3` donne `"hahaha"`.

B. Mélanger texte et nombres

Que se passe-t-il si on essaie de coller du texte et un nombre ?

Exemple — *Un mélange interdit*

```
age = 15
message = "J'ai " + age + " ans"    # ERREUR !
```

Python renvoie une erreur : il refuse de coller du texte (`str`) et un nombre (`int`). C'est comme demander « combien font une pomme plus le mot chat ? ».

Deux solutions élégantes :

Solution 1 — convertir avec `str` :

```
age = 15
message = "J'ai " + str(age) + " ans"
```

Solution 2 — les f-strings (la plus pratique) : on place un `f` juste avant le guillemet ouvrant, et on glisse les variables entre **accolades** `{ }` :

```
age = 15
message = f"J'ai {age} ans"
```

C'est plus lisible, et on peut mettre plusieurs variables dans la même chaîne. Les f-strings deviendront vite votre réflexe.

C. Parler à l'utilisateur : `input`

Jusqu'ici, nos programmes récitaient un texte tout prêt. Pour les rendre vivants, on veut pouvoir **poser une question** et **récupérer la réponse** tapée au clavier. C'est le rôle de `input(...)` :

```
prenom = input("Comment t'appelles-tu ? ")
print(f"Bonjour {prenom} !")
```

Passerelle avec Scratch

Souvenir de Scratch : le bloc **demander (…) et attendre** affichait une question et rangeait la réponse dans le bloc **réponse**. En Python, `input("...")` fait les deux d'un coup : il pose la question *et* renvoie ce que l'utilisateur a tapé.

1

Règle 6

Piège à connaître : `input` renvoie **toujours du texte** (`str`), même si l'utilisateur tape un nombre ! Pour calculer avec, il faut le **convertir** :

```
1 age = int(input("Quel est ton age ? "))
2 print(f"L'an prochain tu auras {age + 1} ans.")
```

Sans le `int(...)`, la ligne `age + 1` planterait (on ne peut pas ajouter 1 à du texte).

Exemple — *La somme dialoguée*

Écrire un programme qui demande deux nombres à l'utilisateur, puis affiche leur somme sous la forme « La somme vaut … ». (Attention à convertir les entrées avec `int` !)

▶ **Correction.** Chaque saisie est convertie en entier dès la lecture :

```
1 a = int(input("Premier nombre ? "))
2 b = int(input("Second nombre ? "))
3 print(f"La somme vaut {a + b}")
```

Sans les `int(...)`, `a` et `b` seraient du texte : en tapant `3` et `4`, on obtiendrait `34` (concaténation) au lieu de `7`.

▶ **Exercices d'application :** exercices 15 à 21 (chaînes, entrées/sorties, types)

VII Les fonctions

A. L'idée : une petite machine à transformer

Quand un programme grandit, on a besoin de l'organiser en **morceaux réutilisables** : les **fonctions**. Une fonction, c'est un mini-programme qui a un nom, à qui on peut donner des valeurs d'entrée, et qui (souvent) renvoie un résultat.

L'idée est très proche de la fonction mathématique :

$$\text{valeur d'entrée (le « paramètre »)} \longrightarrow \boxed{\text{fonction}} \longrightarrow \text{valeur de sortie (renvoyée par « return »)}$$

En maths, si $f(x) = 3x + 2$, alors pour $x = 4$ on obtient $y = 14$. Codons exactement cette fonction en Python :

```
1 def ma_fonction(x):
2     y = 3 * x + 2
3     return y
```

Décortiquons cette syntaxe, ligne par ligne :

- `def` annonce qu'on **définit** une fonction (*define*) ;
- `ma_fonction` est le nom qu'on lui donne ;
- `(x)` est le **paramètre** : la valeur d'entrée ;
- les **deux-points** : ouvrent le « corps » de la fonction ;
- `return` indique la valeur **renvoyée**.

B. L'indentation : le décalage sacré

Vous avez remarqué le **décalage** des lignes à l'intérieur de la fonction ? Ce décalage s'appelle l'**indentation**. Il n'est pas décoratif : en Python, c'est lui qui indique quelles lignes appartiennent à la fonction.

Règle 7

L'indentation définit les « blocs » de code. Les lignes décalées (4 espaces, ou une tabulation) forment le **corps** de la fonction. Dès qu'une ligne revient à la marge de gauche, elle est *hors* de la fonction.

C'est l'équivalent, en Scratch, du fait qu'un bloc soit *à l'intérieur* d'un autre bloc qui l'enveloppe. Sauf qu'ici, l'emboîtement se voit au décalage. **Attention** : Python est intraitable là-dessus. Une indentation qui n'est pas régulière, et c'est l'erreur `IndentationError`. Choisissez les 4 espaces (le standard) et n'en changez plus.

C. Utiliser (« appeler ») une fonction

Définir une fonction ne la fait pas travailler : c'est comme acheter un robot mixeur et le laisser dans son carton. Il faut **l'appeler** en écrivant son nom suivi de la valeur à lui donner :

Exemple

```
1 def ma_fonction(x):
2     return 3 * x + 2
3
4 solution = ma_fonction(4)
5 print(solution)   # affiche 14
```

Au moment de l'exécution, `ma_fonction(4)` est *remplacé* par la valeur renvoyée, c'est-à-dire 14.



D. Plusieurs paramètres, ou aucun

Une fonction peut recevoir plusieurs paramètres, séparés par des virgules. Les paramètres peuvent aussi être du texte :

Exemple

```
1 def dit_bonjour(nom, age):
2     return f"Bonjour {nom}, tu as {age} ans."
3
4 print(dit_bonjour("Toto", 14))
```

Affiche : `Bonjour Toto, tu as 14 ans.`

1

E. Le grand piège : `return` n'est pas `print` !

Voici la confusion la plus fréquente de l'année. Retenez bien la différence :

Règle 8

• `print(...)` **affiche** quelque chose à l'écran (pour l'humain). Ça *ne* renvoie *rien* d'utilisable par le programme.
• `return ...` **renvoie** une valeur au programme (pour la machine). On peut la ranger dans une variable et la réutiliser.

Une fonction qui utilise `print` mais pas `return` *montre* un résultat mais ne le *rend* pas : impossible de le récupérer pour continuer un calcul. Dans certains langages, une telle fonction « qui ne renvoie rien » s'appelle une **procédure**.

Exemple — *return vs print*

```
1 def carre_return(x):
2     return x * x
3
4 def carre_print(x):
5     print(x * x)
6
7 resultat = carre_return(5)    # resultat vaut 25, on peut s'en servir
8 resultat = carre_print(5)     # affiche 25, mais resultat vaut... rien (None) !
```

Exemple — *A ou B ?*

Expliquer avec ses propres mots la différence entre la fonction `A` et la fonction `B`. Laquelle permet de récupérer le résultat pour l'utiliser ensuite ?

```
1 def A(x):
2     return x + 1
3
4 def B(x):
5     print(x + 1)
```

▶ **Correction.** `A` **renvoie** `x + 1` : on peut récupérer ce résultat, par exemple `r = A(3)` range `4` dans `r`. `B` se contente d'**afficher** `x + 1` : l'écran montre `4`, mais `s = B(3)` range `None` (« rien ») dans `s`. Seule la fonction `A` permet de réutiliser le résultat.

▶ **Exercices d'application :** exercices 22 à 29 (les fonctions)

VIII Booléens et comparaisons

A. Vrai ou faux : le type booléen

Si je vous dis « 4 est égal à 5 », vous répondez « faux ». Si je dis « 7 est égal à 7 », vous répondez « vrai ». Ces réponses « vrai / faux » sont ce qu'on appelle des **booléens**. En Python, il n'y a que deux booléens : `True` (vrai) et `False` (faux), avec une majuscule.

Une **comparaison** produit un booléen. Le test d'égalité s'écrit avec un **double** signe égal `==` :

Exemple

Tapez dans la console :

```
1 4 == 5      # renvoie False
2 7 == 7      # renvoie True
```

Règle 9

Ne confondez jamais = et == !
• **= affecte** une valeur : `age = 15` range 15 dans `age`.
• **== compare** : `age == 15` demande « est-ce que `age` vaut 15 ? » et renvoie `True` ou `False`.
C'est l'erreur la plus courante du monde entier. Vous voilà prévenus.

Les opérateurs de comparaison :

Opérateur	Sens	Opérateur	Sens
`==`	égal à	`!=`	différent de
`<`	strictement inférieur	`>`	strictement supérieur
`<=`	inférieur ou égal	`>=`	supérieur ou égal

B. Combiner des conditions : `and`, `or`, `not`

On peut relier plusieurs comparaisons avec les **opérateurs logiques** : `and` (« et »), `or` (« ou ») et `not` (« non »).

- `A and B` est vrai **seulement si** `A` *et* `B` sont vrais tous les deux ;
- `A or B` est vrai **dès que** `A` *ou* `B` (ou les deux) est vrai ;
- `not A` inverse : vrai devient faux, et inversement.

On résume tout dans des **tables de vérité** :

Le « et » (`and`)

A	B	A and B
V	V	**V**
V	F	F
F	V	F
F	F	F

Le « ou » (`or`)

A	B	A or B
V	V	**V**
V	F	**V**
F	V	**V**
F	F	F

Remarque Le « ou » de l'informatique est *inclusif* : « fromage ou dessert » à la cantine version Python, ça voudrait dire qu'on a le droit aux deux. Dommage que la vraie cantine utilise le « ou » exclusif…

Exemple — *Vrai ou faux ?*
Sans machine, donner `True` ou `False` pour chaque expression, avec `a = 4` et `b = 7` :

```
1 a == b
2 a != b
3 a < b
4 a >= 4
```

▶ **Correction.** Avec `a = 4` et `b = 7` : `a == b` vaut `False` ; `a != b` vaut `True` ; `a < b` vaut `True` ; `a >= 4` vaut `True` (4 est bien supérieur *ou égal* à 4).

1

Exemple — *Combinaisons logiques*

Sans machine, évaluer ces expressions :

```
(3 > 2) and (5 == 5)
(3 > 2) and (5 == 6)
(3 > 2) or (5 == 6)
not (3 > 2)
```

▶ **Correction.** `True` (les deux sont vraies) ; `False` (`5 == 6` est faux, donc le « et » est faux) ; `True` (il suffit que `3 > 2` soit vrai) ; `False` (`not` inverse `True`).

Exemple — *Traduire une phrase en condition*

Écrire l'expression Python qui teste si une variable `note` est comprise **entre** 0 **et** 20 (bornes incluses). Indice : il faut relier deux comparaisons avec `and`.

▶ **Correction.** `note >= 0 and note <= 20`. (Python accepte aussi l'écriture raccourcie `0 <= note <= 20`.)

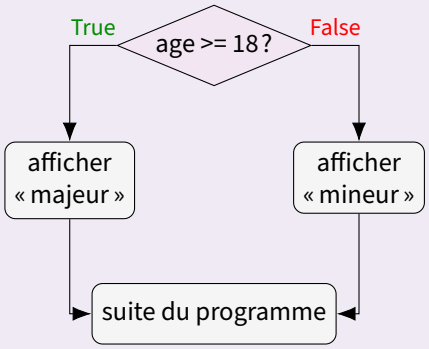
IX Les instructions conditionnelles

A. « Si… alors… sinon… »

Un programme intéressant doit pouvoir **prendre des décisions** : faire une chose dans un cas, une autre dans l'autre. C'est le rôle du `if` (« si ») et du `else` (« sinon ») :

```
if age >= 18:
    print("Tu es majeur.")
else:
    print("Tu es mineur.")
```

Si la condition `age >= 18` est `True`, le bloc du `if` s'exécute (et celui du `else` est ignoré). Sinon, c'est l'inverse. Remarquez encore l'**indentation** et les **deux-points** : ce sont eux qui délimitent chaque bloc.



Passerelle avec Scratch

C'est le bloc Scratch



que vous connaissez. En Python, `si` devient `if` et `sinon` devient `else`.

B. Plusieurs cas : `elif`

Parfois il y a plus de deux possibilités. On utilise alors `elif` (contraction de « *else if* », « sinon si ») autant de fois que nécessaire :

Exemple — *Les mentions*

```
note = 13
if note >= 16:
    print("Tres bien")
elif note >= 12:
    print("Bien")
elif note >= 10:
    print("Passable")
else:
    print("Insuffisant")
```

Python teste les conditions **dans l'ordre** et s'arrête à la première qui est vraie.

Exemple — *La facture du magasin*

Pour un magasin, écrire une fonction `facture` qui prend en paramètres le prix unitaire d'un article et le nombre d'articles achetés, et renvoie le montant à payer. Règle commerciale : si le client achète au moins 5 articles, il bénéficie de 5 % de réduction sur le total.

▶ **Correction.** On calcule le total, puis on applique la réduction seulement si la condition est vraie :

```
def facture(prix_unitaire, nombre):
    total = prix_unitaire * nombre
    if nombre >= 5:
        total = total * 0.95    # 5 % de reduction
    return total
```

Par exemple `facture(10, 4)` renvoie `40` et `facture(10, 5)` renvoie `47.5`.

▶ **Exercices d'application :** exercices 30 à 38 (booléens et instructions conditionnelles)

X Les boucles

Dernière grande notion, et non des moindres : les **boucles**. Une boucle permet de **répéter** des instructions sans les réécrire dix fois. C'est LA raison pour laquelle les ordinateurs sont si puissants : ils ne se lassent jamais de répéter.

Le programme officiel distingue deux familles de boucles : les boucles **bornées** (on sait à l'avance combien de tours) et les boucles **non bornées** (on tourne jusqu'à ce qu'une condition soit remplie).

A. La boucle bornée : `for`

Quand on connaît le **nombre de répétitions** à l'avance, on utilise `for` avec `range` :

Exemple

```
for i in range(5):
    print("Bonjour")
```

Affiche « Bonjour » exactement 5 fois.

`range(5)` fabrique la suite de nombres $0, 1, 2, 3, 4$ (cinq nombres, en commençant à 0 — une habitude déroutante mais universelle en informatique). À chaque tour, la variable `i` prend la valeur suivante. On peut d'ailleurs s'en servir :

Exemple

```
for i in range(1, 6):
    print(i)
```

`range(1, 6)` va de 1 **inclus** à 6 **exclu**. Le programme affiche donc $1, 2, 3, 4, 5$.

1

Passerelle avec Scratch

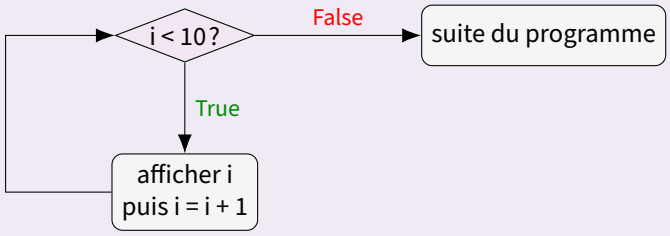
C'est le bloc répéter (5) fois de Scratch. Simple, non ?

B. La boucle non bornée : `while`

Quand on **ne sait pas** à l'avance combien de tours seront nécessaires, mais qu'on sait *à quelle condition* s'arrêter, on utilise `while` (« tant que ») :

```
1 i = 0
2 while i < 10:
3     print(i)
4     i = i + 1
5 print("Termine !")
```

Tant que la condition `i < 10` est vraie, le bloc indenté se répète. À chaque tour, on augmente `i` de 1 ; quand `i` atteint 10, la condition devient fausse et on sort de la boucle.



Règle 10

Danger : la boucle infinie ! Si la condition d'un `while` ne devient *jamais* fausse, le programme tourne pour l'éternité (ou jusqu'à ce qu'on l'arrête). Dans l'exemple ci-dessus, si on oublie la ligne `i = i + 1`, alors `i` reste à 0, la condition `i < 10` reste vraie… et Python affiche 0 jusqu'à la fin des temps. Pour arrêter un programme emballé dans Spyder : le bouton **stop** rouge (ou Ctrl + C dans la console).

Passerelle avec Scratch

Cela ressemble au bloc répéter jusqu'à ⟨ ⟩ de Scratch — avec une petite subtilité : Scratch répète *jusqu'à ce que* la condition soit vraie, alors que `while` répète *tant que* la condition est vraie. C'est la même idée vue à l'envers.

C. `for` ou `while` : lequel choisir ?

Le bon réflexe : si vous pouvez compléter la phrase « je vais répéter *tant de fois* », prenez une boucle `for`. Si vous ne pouvez compléter que « je répète *tant que…* » ou « *jusqu'à ce que…* », prenez un `while`.

Exemple — *Somme des entiers de 1 à n*

Écrire une fonction `somme` qui prend en paramètre un entier `n` et renvoie la somme $1+2+\cdots+n$, en utilisant une boucle `for`.

▶ **Correction.** On part d'une variable `s` qui vaut 0, puis, à chaque tour, on lui ajoute la valeur de `i` (c'est le grand classique `a = a + 1`, mais avec `i` à la place de `1`) :

```
1 def somme(n):
2     s = 0
3     for i in range(1, n + 1):
4         s = s + i
```

```
    return s
```

Attention à `range(1, n + 1)` : la borne de droite est exclue, il faut donc écrire `n + 1` pour que `i` prenne aussi la valeur `n`. Par exemple `somme(4)` renvoie $1+2+3+4=10$.

Exemple — *Table de multiplication*
Écrire une fonction `table` qui prend en paramètre un entier et affiche sa table de multiplication de 1 à 10. Par exemple `table(3)` affiche « $1\times 3=3$ », « $2\times 3=6$ », …, « $10\times 3=30$ ».

▶ **Correction.** La variable `i` parcourt les entiers de 1 à 10 :

```
def table(n):
    for i in range(1, 11):
        print(f"{i} x {n} = {i * n}")
```

Exemple — *Mystère mathématique*
Sans machine, quelle est la valeur finale de `x` ? Vérifier ensuite.

```
x = 1
n = 5
while n > 1:
    x = x * n
    n = n - 1
```

(Petit indice : le résultat porte un nom connu en maths…)

▶ **Correction.** Suivons les variables tour après tour :

	départ	tour 1	tour 2	tour 3	tour 4
`x`	1	5	20	60	120
`n`	5	4	3	2	1

Quand `n` vaut 1, la condition `n > 1` est fausse et la boucle s'arrête : `x` vaut $120=5\times4\times3\times2\times1$. C'est la **factorielle** de 5, notée 5!.

▶ **Exercices d'application :** exercices 39 à 47 (les boucles), défis 48 à 51, exercice 52 (critiquer une réponse d'IA)

XI Mini-projets de synthèse

Vous avez maintenant toutes les briques : variables, types, calculs, chaînes, fonctions, conditions et boucles. Assemblons-les dans de vrais petits programmes.

Exemple — *Le quiz*
Écrire un programme qui pose 3 questions de culture générale à l'utilisateur (avec `input`), compte le nombre de bonnes réponses dans une variable `score`, et affiche le score final sous la forme « Vous avez *score*/3 ».

▶ **Correction.** Une variable `score` compte les bonnes réponses :

```
score = 0
reponse = input("Quelle est la capitale de l'Italie ? ")
if reponse == "Rome":
    score = score + 1
reponse = input("Combien font 7 x 8 ? ")
if reponse == "56":
    score = score + 1
reponse = input("Quelle planete est la plus proche du Soleil ? ")
if reponse == "Mercure":
    score = score + 1
print(f"Vous avez {score}/3")
```

Remarque : on compare `reponse` au *texte* `"56"`, puisque `input` renvoie toujours du texte.

Exemple — *Deviner le nombre*

L'ordinateur « pense » à un nombre entre 1 et 100 ; l'utilisateur doit le deviner. Après chaque proposition, le programme répond « trop grand » ou « trop petit », jusqu'à ce que l'utilisateur trouve. On tire un nombre au hasard avec le module `random` :

```
import random
secret = random.randint(1, 100)   # un entier au hasard entre 1 et 100
```

(Indice : c'est un cas typique de boucle `while` — on ne sait pas combien d'essais il faudra.)

▶ **Correction.** On redemande un nombre **tant que** la proposition est différente du secret :

```
import random
secret = random.randint(1, 100)

proposition = int(input("Votre proposition ? "))
while proposition != secret:
    if proposition > secret:
        print("trop grand")
    else:
        print("trop petit")
    proposition = int(input("Votre proposition ? "))
print("Bravo, vous avez trouve !")
```

Exemple — *Le distributeur poli*

Écrire une fonction `distributeur` qui prend en paramètre le montant retiré et affiche un message adapté : refuse si le montant est négatif ou nul, prévient si le montant dépasse 300 (plafond), et confirme le retrait sinon.

▶ **Correction.** Trois cas, donc un `if`, un `elif` et un `else` :

```
def distributeur(montant):
    if montant <= 0:
        print("Retrait refuse : montant invalide.")
    elif montant > 300:
        print("Retrait refuse : le plafond est de 300 euros.")
    else:
        print(f"Retrait de {montant} euros confirme.")
```

Exemple — *Le thermomètre*

Écrire une fonction `celsius_vers_fahrenheit` qui convertit une température (rappel : $F = C \times 1{,}8 + 32$). Puis, à l'aide d'une boucle, afficher la table de conversion de 0 à 100 °C, de 10 en 10.

▶ **Correction.** La fonction traduit la formule ; la boucle fait 11 tours et, au tour `i`, calcule la température `c = 10 * i`, qui prend donc les valeurs $0, 10, 20, \ldots, 100$:

```
def celsius_vers_fahrenheit(c):
    return c * 1.8 + 32

for i in range(11):
    c = 10 * i
    print(f"{c} C = {celsius_vers_fahrenheit(c)} F")
```

Le programme affiche `0 C = 32.0 F`, `10 C = 50.0 F`, …, jusqu'à `100 C = 212.0 F`.

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Déjà vu :** les blocs de Scratch, au collège ; variables, conditions et boucles y étaient déjà, seule l'écriture change.
- **À venir :** le complément *Dessiner avec la tortue*, juste après, met en images boucles et fonctions ; au chapitre *Photographie numérique*, deux boucles `for` parcourront les **pixels** d'une image, et au chapitre *Informatique embarquée et objets connectés*, une boucle `while` et un test `if` piloteront un thermostat.
- **Plus loin :** en spécialité NSI de Première, ces bases s'enrichissent de nouveaux types (**listes**, **dictionnaires**) pour traiter beaucoup de données.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Je veux...	**J'écris...**
afficher un message	`print("Bonjour")`
ranger une valeur dans une variable	`age = 15`
lire une saisie au clavier	`nom = input("Nom ? ")`
convertir une saisie en nombre	`n = int(input("? "))`
coller du texte et une variable	`f"J'ai {age} ans"`
connaître le type	`type(age)`
prendre une décision	`if ... : / elif ... : / else :`
répéter un nombre connu de fois	`for i in range(n):`
répéter tant qu'une condition tient	`while condition:`
définir une fonction	`def f(x): ... return ...`

Retour à la situation de départ

1. Dans une **variable** (§ III) : le prénom, lu au clavier par `input`, est rangé sous un nom, puis recollé dans le message (§ VI).
2. `input` renvoie toujours du **texte** : le `+` colle alors les deux chaînes (**concaténation**) et donne `"34"`. Il fallait **convertir** chaque saisie avec `int` (§ IV et VI).
3. Une condition `if`/`else` choisit la réponse (§ IX), et une boucle `while` recommence *tant que* la proposition diffère du secret, sans connaître à l'avance le nombre de tours (§ X) ; c'est le mini-projet « Deviner le nombre » (§ XI).

Les messages d'erreur les plus fréquents (et ce qu'ils veulent dire)

Un message d'erreur n'est pas une insulte : c'est la machine qui essaie de vous aider. Apprenez à les lire, ils vous font gagner un temps fou.

Erreur	Traduction en clair
`SyntaxError`	« Votre phrase est mal construite. » Souvent un `:` oublié, une parenthèse ou un guillemet non fermé.
`NameError`	« Je ne connais pas ce nom. » Une variable utilisée avant d'être créée, ou une faute de frappe dans son nom.
`IndentationError`	« Votre décalage est bancal. » Vérifiez vos espaces en début de ligne (toujours 4 espaces).
`TypeError`	« Vous mélangez des choux et des carottes. » Par exemple additionner du texte et un nombre (pensez à `str(...)` ou `int(...)`).

Voilà, vous avez fait le tour des bases de Python. Le reste n'est qu'entraînement : programmer, ça s'apprend comme le vélo — en tombant un peu, puis en pédalant beaucoup. Bon code !

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ saisir, exécuter et corriger un programme dans l'environnement de travail ;
- ☐ déclarer et utiliser une **variable** ;
- ☐ reconnaître les **types** de base : entier, flottant, chaîne, booléen ;
- ☐ écrire un **calcul** avec les opérateurs (`+ - * / // % **`) ;
- ☐ manipuler des **chaînes de caractères**, afficher avec `print` et saisir avec `input` ;
- ☐ définir et appeler une **fonction** avec des paramètres et une valeur de retour ;
- ☐ écrire une **condition** (`if` / `elif` / `else`) avec des comparaisons et des booléens ;
- ☐ écrire une **boucle** (`for`, `while`) pour répéter des instructions.

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Sauf mention [sur papier], les exercices se font **sur machine** (Spyder ou Basthon) : on écrit dans l'éditeur, on exécute, on vérifie dans la console.
- ▶ *signale les exercices où l'on écrit un vrai programme à tester.*
- Avancez dans l'ordre : chaque section suit la progression du cours.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Premiers pas : afficher ↑cours

★ **Exercice 1** *Bonjour* ▶ corrigé p. 214 • essentiel

▶ Écrire un programme qui affiche : *Bonjour, je m'appelle ...* (en y mettant son prénom).

▷ coup de pouce

★ **Exercice 2** *Le menu de la cantine* ▶ corrigé p. 214

▶ Écrire un programme qui affiche le menu du jour **sur trois lignes** : l'entrée, le plat, puis le dessert (par exemple *Entrée : salade niçoise*). ▷ coup de pouce

Variables ↑cours

★ **Exercice 3** *Deux variables* ▶ corrigé p. 214 • essentiel

▶ Créer une variable `sport` contenant son sport préféré (du texte) et une variable `heures` contenant le nombre d'heures qu'on y consacre par semaine (un nombre), puis les afficher.

▷ coup de pouce

★ **Exercice 4** *Valeur finale* ▶ corrigé p. 214 • essentiel

[sur papier] Sans machine, donner la valeur finale de `a` *et* de `b`, puis vérifier. ▷ coup de pouce

```
1 a = 5
2 b = a + 1
3 a = 10
```

★ **Exercice 5** *Des noms parlants* ▶ corrigé p. 214

[sur papier] Ce programme calcule le prix de 3 places de cinéma à 12 €, mais ses noms de variables ne veulent rien dire :

```
1 x = 12
2 y = 3
3 z = x * y
```

1. Proposer des noms **parlants** (et valides) pour `x`, `y` et `z`.

2. Un camarade propose `prix place`, `3places` et `total€`. Pourquoi Python les refuse-t-il ?

▷ coup de pouce

★★ **Exercice 6** *L'échange* ▶ corrigé p. 215 • essentiel

▶ On a `a = 1` et `b = 2`. Écrire les instructions qui **échangent** leurs valeurs (à la fin, `a` vaut 2 et `b` vaut 1).

▷ coup de pouce

Calculs ↑ cours

★ **Exercice 7** *Somme et produit* ▶ corrigé p. 215

▶ Ranger deux nombres dans `a` et `b`, puis afficher leur somme *et* leur produit. ▷ coup de pouce

★ **Exercice 8** *Le trajet* ▶ corrigé p. 215 • essentiel

▶ À partir de deux variables `vitesse` (en km/h) et `duree` (en heures), calculer et afficher la distance parcourue. Test : 90 km/h pendant 2,5 h donnent 225 km. ▷ coup de pouce

★★ **Exercice 9** *La tirelire* ▶ corrigé p. 215 • essentiel

▶ La variable `tirelire` de Léa vaut 20. Sa grand-mère **double** son contenu, puis Léa dépense 15 € au cinéma, puis elle ajoute 8 € d'argent de poche. En modifiant `tirelire` à chaque étape, afficher le contenu final (attendu : 33). ▷ coup de pouce

★★ **Exercice 10** *Priorités de calcul* ▶ corrigé p. 215

[sur papier] Python respecte les priorités opératoires des maths. Sans machine, donner la valeur *et* le type de chaque résultat, puis vérifier.

```
2 + 3 * 4
(2 + 3) * 4
10 - 6 / 2
2 * 3 ** 2
20 // 3 % 2
```

▷ coup de pouce

★★ **Exercice 11** *La note d'une application* ▶ corrigé p. 215

▶ Sur une boutique d'applications, un jeu a reçu 12 avis à 5 étoiles, 5 avis à 4 étoiles et 3 avis à 1 étoile. Ranger ces nombres d'avis dans des variables, puis calculer et afficher la note moyenne du jeu (attendu : 4,15). ▷ coup de pouce

★★ **Exercice 12** *Le forfait de données* ▶ corrigé p. 216

▶ Un forfait mobile comprend 80 Go de données par mois (Go : gigaoctet). Vous en avez déjà utilisé 25 %. Écrire un programme qui calcule et affiche le nombre de gigaoctets qu'il vous reste.

▷ coup de pouce

★★ **Exercice 13** *Quel jour serons-nous ?* ▶ corrigé p. 216 • essentiel

▶ On numérote les jours de la semaine : lundi = 0, mardi = 1, …, dimanche = 6. Aujourd'hui, c'est mercredi (jour 2). Écrire un programme qui calcule et affiche :

1. le nombre de semaines complètes contenues dans 100 jours ;
2. le numéro du jour de la semaine qu'il sera dans 100 jours.

▷ coup de pouce

★★★ **Exercice 14** *Le rendu de monnaie* ▶ corrigé p. 216 ≈ 15 min

▶ Écrire un programme qui, à partir d'une somme en euros entiers (variable `montant`), affiche comment la payer avec le **moins de coupures possible** en billets de 10, pièces de 2 et pièces de 1. Par exemple pour 27 € : 2 billets de 10, 3 pièces de 2, 1 pièce de 1 ($20 + 6 + 1 = 27$).

▷ coup de pouce ▷ début de solution

Chaînes de caractères et dialogue ↑ cours

★ **Exercice 15** *Texte ou calcul ?* ▶ corrigé p. 216 • essentiel

▶ Écrire un programme qui affiche **exactement** les trois lignes ci-dessous. La deuxième doit être *calculée* par Python (interdit d'écrire `17` soi-même !).

```
12 + 5
17
Bonjour Monaco
```

▷ coup de pouce

★ **Exercice 16** *Nom de code* ▶ corrigé p. 216 • essentiel

▶ Demander à l'utilisateur son prénom, puis sa ville, et afficher à l'aide d'une f-string : « Agent *prénom*, votre mission vous attend à *ville*. » ▷ coup de pouce

★★ **Exercice 17** *Le type d'un résultat* ▶ corrigé p. 216

[sur papier] Sans machine, donner la valeur et le type (`int`, `float` ou `str`) du **résultat** de chaque expression, puis vérifier avec `type(...)` : `7 + 2` ; `7 / 7` ; `"7" + "2"` ; `3 * 1.0` ; `int("5") + 1` ; `str(4) + "0"`. ▷ coup de pouce

★★ **Exercice 18** *Le temps d'écran* ▶ corrigé p. 217 • essentiel

▶ Demander à l'utilisateur combien d'heures par jour il passe devant un écran, puis afficher combien d'heures cela représente sur une année (365 jours), et combien de **jours entiers** cela fait. (Attention : `input` renvoie du texte, il faut le convertir avec `int` !) ▷ coup de pouce

★★ **Exercice 19** *Le temps de téléchargement* ▶ corrigé p. 217

▶ Demander la taille d'un fichier en mégaoctets (Mo), puis le débit de la connexion en mégaoctets par seconde (Mo/s : chaque seconde, on reçoit ce nombre de mégaoctets). Ce sont des nombres **à virgule**, à taper avec un point : convertir avec `float`. Afficher ensuite en une seule f-string : « Un fichier de T Mo se télécharge en D secondes. » Test : 45 Mo à 2,5 Mo/s donnent 18 secondes. ▷ coup de pouce

★★ **Exercice 20** *L'écho* ▶ corrigé p. 217

▶ Demander un mot puis un nombre entier `n` à l'utilisateur, et afficher le mot répété `n` fois collé (par exemple `"go"` et 3 donnent `gogogo`). ▷ coup de pouce

★★ **Exercice 21** *Le ticket de caisse* ▶ corrigé p. 217

▶ Demander le nom d'un article, son prix unitaire (nombre à virgule) et la quantité achetée (entier), puis afficher un ticket encadré par des lignes de tirets, par exemple : ▷ coup de pouce

```
--------------------
3 x croissant
Prix unitaire : 1.5
TOTAL : 4.5 euros
--------------------
```

1

Les fonctions ↑cours

★ **Exercice 22** *Les codes PIN* ▶ corrigé p. 217 • essentiel

▶ Un code PIN est formé de chiffres (chacun de 0 à 9). Avec 1 chiffre, il y a 10 codes possibles ; avec 2 chiffres, $10 \times 10 = 100$; chaque chiffre de plus multiplie le nombre de codes par 10. Écrire une fonction `nb_codes(n)` qui renvoie le nombre de codes possibles à `n` chiffres. Tester avec `nb_codes(4)` (attendu : 10 000). ▷ coup de pouce

★ **Exercice 23** *L'aire du triangle* ▶ corrigé p. 218

▶ Écrire une fonction `aire_triangle(base, hauteur)` qui renvoie l'aire d'un triangle (rappel : $\frac{\text{base} \times \text{hauteur}}{2}$). ▷ coup de pouce

★★ **Exercice 24** *Une note sur 20* ▶ corrigé p. 218

▶ Un contrôle est noté sur un total qui n'est pas toujours 20. Écrire une fonction `note_sur_20(points, total)` qui renvoie la note ramenée sur 20. Vérifier que `note_sur_20(12, 15)` vaut 16. ▷ coup de pouce

★★ **Exercice 25** *Retrouver le poids d'origine* ▶ corrigé p. 218

▶ Une messagerie compresse les photos avant de les envoyer : la photo envoyée pèse 75 % du poids de la photo d'origine. Écrire une fonction `poids_origine(poids_envoye)` qui renvoie le poids de la photo d'origine. Vérifier que `poids_origine(3)` vaut 4 (en Mo). (Attention : ajouter 25 % au poids envoyé ne donne *pas* le bon résultat !) ▷ coup de pouce

★★ **Exercice 26** *Le poids d'une photo* ▶ corrigé p. 218 • essentiel

▶ Une photo numérique non compressée utilise 3 octets par pixel (un pour le rouge, un pour le vert, un pour le bleu). Écrire une fonction `poids_photo(largeur, hauteur)` qui renvoie le poids de l'image en octets, puis calculer celui d'une photo de 4000×3000 pixels. Combien de mégaoctets cela fait-il (1 Mo = 1 000 000 octets) ? ▷ coup de pouce

★★ **Exercice 27** *Tout en secondes* ▶ corrigé p. 218

▶ Écrire une fonction `en_secondes(h, m, s)` qui prend une durée en heures, minutes et secondes et renvoie cette durée convertie en secondes. Tester avec `en_secondes(1, 30, 15)` (on doit obtenir 5415). ▷ coup de pouce

★★ **Exercice 28** *Deviner l'affichage (fonctions)* ▶ corrigé p. 218 • essentiel

[sur papier] Sans machine, écrire ce qu'affiche ce programme, puis vérifier sur machine.

```
1  def f(x):
2      return 2 * x + 1
3
4  def g(x):
5      return x * x
6
7  print(f(3))
8  print(g(f(1)))
9  print(f(g(2)) + 1)
```

▷ coup de pouce

★★★ **Exercice 29** *Le coût d'un trajet* ▶ corrigé p. 218 ≈ 15 min

▶ Une voiture consomme `conso` litres d'essence aux 100 km.

1. Écrire une fonction `litres(distance, conso)` qui renvoie le nombre de litres consommés pour parcourir `distance` kilomètres.
2. Écrire une fonction `cout_trajet(distance, conso, prix_litre)` qui renvoie le coût du trajet en euros **en appelant** `litres`.
3. Combien coûte un trajet de 800 km avec une voiture qui consomme 6 L aux 100 km, si l'essence coûte 2 € le litre ?

▷ coup de pouce ▷ début de solution

Les conditions ↑ cours

★ **Exercice 30** *Le code secret* ▶ corrigé p. 219 • essentiel

▶ Écrire une fonction `verifier(code)` qui renvoie `"Acces autorise"` si le code reçu vaut 2468, et `"Acces refuse"` sinon. ▷ coup de pouce

★★ **Exercice 31** *Pair ?* ▶ corrigé p. 219

▶ Écrire une fonction `est_pair(n)` qui renvoie `True` si `n` est pair et `False` sinon. ▷ coup de pouce

★★ **Exercice 32** *Le signe* ▶ corrigé p. 219

▶ Écrire une fonction `signe(x)` qui renvoie `"negatif"`, `"nul"` ou `"positif"` selon le nombre reçu (utiliser `elif`).

▷ coup de pouce

★★ **Exercice 33** *Le découpage en paquets* ▶ corrigé p. 219

▶ Sur un réseau, un fichier voyage découpé en **paquets** de 1500 octets au plus : tous les paquets sont pleins, sauf peut-être le dernier. Écrire un programme qui demande la taille d'un fichier (en octets, un entier), puis affiche « Il faut n paquets. » (avec la vraie valeur, à l'aide d'une f-string). Tester avec 3000 (2 paquets) et 3001 (3 paquets). ▷ coup de pouce

★★ **Exercice 34** *Valeur absolue maison* ▶ corrigé p. 219

▶ Écrire une fonction `valeur_absolue(x)` qui renvoie la valeur absolue de `x` **sans** utiliser la fonction `abs` de Python (se servir d'un `if`). ▷ coup de pouce

★★ **Exercice 35** *L'icône de batterie* ▶ corrigé p. 219 • essentiel

▶ Un téléphone affiche une icône selon le niveau de batterie (en %). Écrire une fonction `icone(niveau)` qui renvoie `"critique"` si le niveau est inférieur à 10, `"faible"` s'il est inférieur à 30, `"moyenne"` s'il est inférieur à 70, et `"pleine"` sinon. Tester avec 5, 50 et 100. ▷ coup de pouce

★★★ **Exercice 36** *La meilleure antenne* ▶ corrigé p. 220 ≈ 15 min

▶ Un téléphone capte trois antennes-relais et se connecte à celle dont le signal est le plus fort. Écrire une fonction `meilleur_signal(a, b, c)` qui renvoie le plus grand des trois niveaux de signal reçus (sans utiliser la fonction `max` de Python). Tester avec `meilleur_signal(40, 75, 62)` (attendu : 75). ▷ coup de pouce ▷ début de solution

★★★ **Exercice 37** *Année bissextile* ▶ corrigé p. 220 ≈ 15 min

▶ Écrire une fonction `bissextile(annee)` qui renvoie `True` si l'année est bissextile. Règle : une année est bissextile si elle est divisible par 4, *sauf* les années de siècle (divisibles par 100) qui doivent en plus être divisibles par 400. Tester avec 2024 (oui), 1900 (non) et 2000 (oui).

▷ coup de pouce ▷ début de solution

★★ **Exercice 38** *Une couleur valide ?* ▶ corrigé p. 220

▶ Sur un écran, une couleur est codée par trois nombres : sa part de rouge, de vert et de bleu. Chacun doit être compris entre 0 et 255 (bornes incluses). Écrire une fonction `couleur_valide(rouge, vert, bleu)` qui renvoie `True` si les trois nombres sont valides, et `False` sinon. Tester avec `couleur_valide(255, 128, 0)` (`True`) et `couleur_valide(300, 0, 0)` (`False`).

▷ coup de pouce

Les boucles ↑ cours

★ **Exercice 39** *De 1 à 10* ▶ corrigé p. 221 • essentiel

▶ Écrire un programme qui affiche tous les entiers de 1 à 10 (boucle `for`).

▷ coup de pouce

★ **Exercice 40** *Trace de boucle* ▶ corrigé p. 221 • essentiel

[sur papier] Sans machine, écrire ce qu'affiche ce programme, puis vérifier.

```
1 for i in range(4):
2     print(i * i)
```

▷ coup de pouce

★★ **Exercice 41** *Un message qui se propage* ▶ corrigé p. 221 • essentiel

▶ Une personne publie un message (étape 0). À l'étape 1, ses 3 amis le reçoivent ; à l'étape 2, chacun d'eux le relaie à 3 nouveaux amis, soit 9 personnes ; et ainsi de suite (on suppose que personne ne le reçoit deux fois) : à l'étape `i`, 3^i nouvelles personnes reçoivent le message. Écrire une fonction `diffusion(n)` qui renvoie le nombre total de personnes touchées de l'étape 0 à l'étape `n` (l'auteur compris). Vérifier que `diffusion(3)` vaut 40, puis calculer `diffusion(10)`. ▷ coup de pouce

★★ **Exercice 42** *Les puissances de 2* ▶ corrigé p. 221

▶ Écrire une fonction `puissances_de_2(n)` qui affiche $2^0, 2^1, 2^2, \ldots, 2^n$, une valeur par ligne. Que vaut 2^{10} ? (Ce nombre est partout en informatique !)

▷ coup de pouce

★★ **Exercice 43** *Les écouteurs* ▶ corrigé p. 221 • essentiel

▶ Léa part de 0 € et économise 15 € par semaine. Avec une boucle `while`, calculer et afficher au bout de combien de semaines elle aura **au moins** 200 € pour s'offrir des écouteurs. ▷ coup de pouce

★★ **Exercice 44** *Les multiples de 7* ▶ corrigé p. 222

▶ Écrire un programme qui **compte** (sans les afficher) combien il y a de multiples de 7 entre 1 et 1000, puis affiche ce nombre.

▷ coup de pouce

★★★ **Exercice 45** *Les abonnés* ▶ corrigé p. 222 ≈ 15 min

▶ Le compte d'une créatrice de vidéos gagne 3 % d'abonnés par mois : chaque mois, son nombre d'abonnés est multiplié par 1,03. Écrire une fonction `abonnes(depart, n)` qui renvoie le nombre d'abonnés au bout de `n` mois, à l'aide d'une boucle `for` (sans utiliser `**`). Vérifier que `abonnes(1000, 10)` vaut environ 1343,92 (soit environ 1344 abonnés).

▷ coup de pouce ▷ début de solution

★★★ **Exercice 46** *Combien de chiffres ?* ▶ corrigé p. 222 ≈ 15 min

▶ Écrire une fonction `nb_chiffres(n)` qui renvoie le nombre de chiffres d'un entier positif. Par exemple `nb_chiffres(2048)` vaut 4.

▷ coup de pouce ▷ début de solution

★★★ **Exercice 47** *Nombre premier* ▶ corrigé p. 222 ≈ 20 min

▶ Écrire une fonction `est_premier(n)` qui renvoie `True` si `n` est un nombre premier (divisible seulement par 1 et par lui-même), et `False` sinon. ▷ coup de pouce ▷ début de solution

Petits défis de synthèse ↑ cours

★★ **Exercice 48** *Fizz* ▶ corrigé p. 222 ≈ 10 min

▶ Afficher les entiers de 1 à 30, mais remplacer par le mot « Fizz » ceux qui sont des multiples de 3. ▷ coup de pouce

★★★ **Exercice 49** *L'ordinateur devine* ▶ corrigé p. 223 ≈ 30 min

▶ Cette fois, c'est **l'utilisateur** qui pense à un nombre entre 1 et 100, et l'ordinateur qui cherche. Il propose toujours le milieu de l'intervalle encore possible (variables `bas` et `haut`) ; l'utilisateur répond `+` (c'est plus grand), `-` (c'est plus petit) ou `=` (trouvé), et il resserre l'intervalle. Afficher à la fin le nombre d'essais. Combien lui en faut-il au maximum ? ▷ coup de pouce ▷ début de solution

★★★ **Exercice 50** *Moyenne d'une série de notes* ▶ corrigé p. 223 ≈ 20 min

▶ L'utilisateur saisit des notes une par une ; il tape -1 pour terminer. Afficher alors la moyenne des notes saisies. ▷ coup de pouce ▷ début de solution

★★★ **Exercice 51** *Le PGCD* ▶ corrigé p. 223 ≈ 15 min

▶ Écrire une fonction `pgcd(a, b)` qui renvoie le plus grand diviseur commun de deux entiers, par la méthode des soustractions successives : tant que les deux nombres sont différents, on remplace le plus grand par sa différence avec le plus petit. Tester avec `pgcd(24, 36)` (attendu : 12). ▷ coup de pouce ▷ début de solution

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 52** *Une puissance calculée par un assistant* ▶ corrigé p. 223 • essentiel

▶ Un élève a demandé à un assistant d'IA : « Écris une fonction `puissance(x, n)` qui renvoie `x` multiplié `n` fois par lui-même, avec une boucle `for` et sans utiliser l'opérateur `**`. » Voici la réponse obtenue :

```
1  def puissance(x, n):
2      resultat = 1
3      for i in range(n + 1):
4          resultat = resultat * x
5      return resultat
6
7  print(puissance(2, 3))    # affiche 8
```

1. La réponse est-elle correcte ? Recopier la fonction, l'exécuter et comparer ce qui s'affiche avec le commentaire de l'assistant. Tester aussi `puissance(5, 2)` et `puissance(3, 0)`.
2. Localiser l'erreur et corriger la fonction (une seule ligne à modifier). ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relisez le tout premier programme du cours (« Hello World ») : il suffit d'en changer le texte. ↑énoncé

Ex. 2 – piste Une instruction `print` affiche une ligne : combien en faut-il pour trois lignes ? ↑énoncé

Ex. 3 – piste Du texte s'écrit entre guillemets, un nombre sans guillemets. Pour afficher la valeur d'une variable, on écrit son nom *sans* guillemets dans `print`. ↑énoncé

Ex. 4 – piste Lisez les lignes une par une, dans l'ordre : au moment où l'on calcule `b`, combien vaut `a` ? La troisième ligne revient-elle en arrière ? ↑énoncé

Ex. 5 q. 2 – piste Relisez dans le cours les règles pour nommer une variable : espaces, chiffre au début, caractères spéciaux. ↑énoncé

Ex. 6 – piste Si l'on écrit directement `a = b`, l'ancienne valeur de `a` est perdue : rangez-la d'abord dans une troisième variable `temporaire`. ↑énoncé

Ex. 7 – piste Deux lignes pour ranger les nombres (`a = ...`), puis un `print` par résultat. La multiplication s'écrit avec le symbole `*`. ↑énoncé

Ex. 8 – piste Distance = vitesse × durée. Attention : en Python, 2,5 s'écrit avec un point. ↑énoncé

Ex. 9 – piste Chaque étape s'écrit `tirelire = ...` en partant de l'ancienne valeur de `tirelire`. ↑

Ex. 10 – piste Comme en maths : d'abord les parenthèses, puis `**`, puis `*`, `/`, `//`, `%` (de gauche à droite), enfin `+` et `-`. Pensez aussi au type : `/` donne toujours un `float`. ↑énoncé

Ex. 11 – piste Note moyenne = (nombre total d'étoiles données) divisé par (nombre total d'avis). Les 12 avis à 5 étoiles rapportent à eux seuls 12×5 étoiles. Attention aux parenthèses. ↑

Ex. 12 – piste Calculez d'abord la quantité déjà utilisée, puis retirez-la du forfait de départ. Rappel : prendre 25 % d'un nombre, c'est le multiplier par 25/100. ↑énoncé

Ex. 13 – piste Les jours reviennent tous les 7 jours : `//` donne le nombre de semaines complètes, `%` le nombre de jours qui restent. N'oubliez pas qu'on part du jour 2. ↑énoncé

Ex. 14 – piste Commencez par la plus grosse coupure : `//` donne combien on en utilise, `%` donne ce qui reste à payer avec les coupures plus petites. ↑énoncé

Ex. 14 – début de solution
```
billets10 = montant // 10
reste = montant % 10
```
puis la même chose avec les pièces de 2, en partant de `reste`. ↑énoncé

Ex. 15 – piste Pour la troisième ligne, collez deux morceaux de texte avec `+` (sans oublier l'espace entre les deux mots). ↑énoncé

Ex. 16 – piste Deux `input`, chacun rangé dans sa variable. Dans une f-string (un `f` juste avant le guillemet), chaque variable s'écrit entre accolades. ↑énoncé

Ex. 17 – piste Pour chaque expression, demandez-vous si les valeurs sont des nombres ou du texte (entre guillemets). Relisez dans le cours ce que donne toujours la division `/`. ↑énoncé

Ex. 18 – piste Sur une année : multipliez par 365. Pour les jours entiers, une journée dure 24 heures : quel opérateur donne le quotient entier ? ↑énoncé

Ex. 19 – piste Combien de secondes faut-il pour recevoir 45 Mo si l'on en reçoit 2,5 chaque seconde ? Posez le calcul avec les nombres, puis remplacez-les par les variables. Convertissez chaque `input` avec `float(...)` *avant* de calculer. ↑énoncé

Ex. 20 – piste Avec les chaînes, `*` répète : `"ha" * 2` donne `"haha"`. Le nombre saisi doit d'abord être converti avec `int`. ↑énoncé

Ex. 21 – piste Commencez par les trois `input` (avec les bonnes conversions), calculez le total, puis enchaînez les `print`. Une ligne de tirets peut s'écrire `"-" * 20`. ↑énoncé

Ex. 22 – piste Avec 3 chiffres, on a $10 \times 10 \times 10$ codes. Quel opérateur de Python calcule une puissance ? ↑énoncé

Ex. 23 – piste Le corps de la fonction tient en une ligne : `return` suivi du calcul, écrit avec les noms des paramètres. ↑énoncé

Ex. 24 – piste Règle de trois : 12 points sur 15, combien cela fait-il sur 20 ? Écrivez le calcul avec les nombres, puis remplacez-les par `points` et `total`. ↑énoncé

Ex. 25 – piste Compresser revient à multiplier le poids d'origine par 0,75. Pour revenir en arrière, quelle opération faut-il faire ? ↑énoncé

Ex. 26 – piste Nombre de pixels = largeur × hauteur, et chaque pixel occupe 3 octets. Pour obtenir des mégaoctets, divisez par 1 000 000. ↑énoncé

Ex. 27 – piste Une heure contient 3600 secondes et une minute 60 secondes : convertissez chaque morceau, puis additionnez. ↑énoncé

Ex. 28 – piste Calculez de l'intérieur vers l'extérieur : pour `g(f(1))`, calculez d'abord `f(1)`, puis donnez ce résultat à `g`. ↑énoncé

Ex. 29 – piste Pour 100 km, la voiture consomme `conso` litres : pour 1 km, elle consomme donc `conso / 100` litres. ↑énoncé

Ex. 29 – début de solution
```
def cout_trajet(distance, conso, prix_litre):
    nb_litres = litres(distance, conso)
```
puis on multiplie par le prix d'un litre. ↑énoncé

Ex. 30 – piste Un `if` pour le bon code, un `else` pour tous les autres. Relisez dans le cours quel signe sert à *comparer* deux valeurs. ↑énoncé

Ex. 31 – piste La comparaison `n % 2 == 0` vaut déjà `True` ou `False` : on peut la renvoyer directement. ↑énoncé

Ex. 32 – piste Trois cas, donc : `if` pour le premier, `elif` pour le deuxième, `else` pour le dernier. ↑

Ex. 33 – piste Avec 3001 octets : combien de paquets pleins de 1500 octets ? Reste-t-il des octets à envoyer ? Pensez à `//` et à `%`. ↑énoncé

Ex. 34 – piste Quels sont les deux cas possibles selon le signe de `x` ? Que vaut la valeur absolue dans chacun d'eux ? ↑énoncé

Ex. 35 – piste Testez les seuils dans l'ordre croissant avec `if` / `elif` : dès qu'une condition est vraie, les suivantes ne sont plus regardées. ↑énoncé

Ex. 36 – piste Gardez le plus grand niveau « vu jusqu'ici » dans une variable `grand`, puis comparez-la aux autres niveaux un par un. ↑énoncé

Ex. 36 – début de solution
```
grand = a
if b > grand:
    grand = b
```
puis la même chose avec `c`, et enfin `return grand`. ↑énoncé

Ex. 37 – piste « Divisible par 4 » s'écrit `annee % 4 == 0`. Traitez les cas du plus particulier au plus général : d'abord 400, puis 100, puis 4. ↑énoncé

Ex. 37 – début de solution
```
if annee % 400 == 0:
    return True
elif annee % 100 == 0:
    return False
```
↑énoncé

Ex. 38 – piste Chaque nombre doit être à la fois supérieur ou égal à 0 *et* inférieur ou égal à 255, et cela pour les trois nombres : toutes ces conditions doivent être vraies, pensez à `and`. ↑

Ex. 39 – piste `range(1, 6)` donne $1, 2, 3, 4, 5$: la borne de droite est exclue. Que faut-il écrire pour aller jusqu'à 10 ? ↑énoncé

Ex. 40 – piste Notez d'abord la liste des valeurs que prend `i` avec `range(4)` (on part de 0), puis calculez `i * i` pour chacune. ↑énoncé

Ex. 41 – piste À l'étape `i`, `3 ** i` personnes reçoivent le message. Additionnez-les dans une variable `total` qui commence à 0, en parcourant `range(n + 1)`. ↑énoncé

Ex. 42 – piste La boucle `for i in range(n + 1)` parcourt $0, 1, \ldots, n$; à chaque tour, on affiche `2 ** i`. ↑énoncé

Ex. 43 – piste Deux variables : `economies` et `semaines`. Tant que les économies sont *inférieures* à 200, on ajoute 15 et on compte une semaine de plus. ↑énoncé

Ex. 44 – piste Un compteur commence à 0 ; on parcourt les nombres de 1 à 1000 et on ajoute 1 au compteur lorsque le nombre est multiple de 7 (avec `%`). ↑énoncé

Ex. 45 – piste Une variable `a` part de `depart` ; chaque mois (un tour de boucle), elle est multipliée par 1,03. ↑énoncé

Ex. 45 – début de solution
```
def abonnes(depart, n):
    a = depart
    for mois in range(n):
```
↑énoncé

Ex. 46 – piste Diviser par 10 avec `//` enlève le dernier chiffre (`2048 // 10` vaut `204`). Comptez combien de fois on peut le faire, dans une boucle `while`, avant d'arriver à 0. ↑énoncé

Ex. 46 – début de solution
```
compteur = 0
while n > 0:
    n = n // 10
```
(il reste à augmenter le compteur et à le renvoyer.) ↑énoncé

Ex. 47 – piste `n` est premier s'il vaut au moins 2 et si *aucun* nombre `d` compris entre 2 et `n - 1` ne le divise. Dès qu'on trouve un diviseur, on peut répondre. ↑énoncé

Ex. 47 – début de solution
```
if n < 2:
    return False
for d in range(2, n):
    if n % d == 0:
```
↑énoncé

Ex. 48 – piste Une boucle `for` de 1 à 30 ; à chaque tour, un `if` teste si le nombre est multiple de 3 (avec `%`) ; sinon (`else`), on affiche le nombre. ↑énoncé

Ex. 49 – piste Le nombre proposé est `(bas + haut) // 2`. Si l'utilisateur répond `+`, le nombre cherché est au-dessus de la proposition : que devient alors `bas` ? ↑énoncé

Ex. 49 – début de solution
```
bas = 1
haut = 100
essais = 0
reponse = ""
while reponse != "=":
```
↑énoncé

Ex. 50 – piste Boucle `while` : on ne sait pas combien de notes il y aura. Gardez la `somme` et le `nombre` de notes ; la valeur -1 ne doit pas être comptée. ↑énoncé

Ex. 50 – début de solution
```
somme = 0
nombre = 0
note = float(input("Note : "))
while note != -1:
```
↑énoncé

Ex. 51 – piste Boucle `while a != b` ; à chaque tour, un `if` décide lequel des deux nombres est le plus grand. ↑énoncé

Ex. 51 – début de solution
```
while a != b:
    if a > b:
        a = a - b
```
↑énoncé

Ex. 52 q. 2 – piste Comptez combien de tours fait `range(n + 1)` quand `n` vaut 3. Combien de fois faut-il multiplier par `x` ? ↑énoncé

FICHE DE RÉVISION

L'essentiel

- Un **programme** est une suite d'instructions ; Python est un langage de **haut niveau**, **interprété**. On écrit le programme dans l'**éditeur**, on teste et on inspecte dans la **console**.
- Une **variable** a un **nom** et une **valeur** ; `=` veut dire « **reçoit** » : on calcule à droite, puis on range à gauche.
- Chaque valeur a un **type** (`int`, `float`, `str`, `bool`) qui dit ce qu'on a le droit d'en faire.
- Une **fonction** reçoit des **paramètres** et **renvoie** un résultat avec `return` ; il faut l'**appeler** pour qu'elle travaille.
- `if`/`elif`/`else` pour **décider**, `for` pour répéter un nombre **connu** de fois, `while` pour répéter **tant qu'**une condition est vraie.
- Les **deux-points** `:` ouvrent un bloc et l'**indentation** (4 espaces) dit quelles lignes en font partie.

Types, opérations et conversions

Type	Exemples	Opérateur	Exemple
`int` (entier)	`12`, `-4`, `0`	`+ - *`	`3 * 5` donne `15`
`float` (à virgule)	`5.2` (un **point**, pas `5,2`)	`/`	`7 / 2` donne `3.5` ; `6 / 2` donne `3.0`
`str` (texte)	`"Bonjour"`, `"12"`	`//`	`7 // 2` donne `3` (quotient)
`bool` (booléen)	`True`, `False`	`%`	`7 % 2` donne `1` (reste)
`type(x)` donne le type de `x`		`**`	`2 ** 3` donne `8`

- **Conversions** : `int("42")` donne `42` ; `float("3.14")` donne `3.14` ; `str(42)` donne `"42"` ; `int(3.9)` donne `3` (on **coupe**, on n'arrondit pas).
- **Priorités** comme en maths : `2 + 3 * 4` donne `14`, `(2 + 3) * 4` donne `20`. `/` donne **toujours** un `float`.
- **Avec du texte**, `+` **colle** (concaténation) : `"5" + "2"` donne `"52"` ; `"ha" * 3` donne `"hahaha"`.
- **f-string** : `f"J'ai {age} ans"` glisse la valeur de `age` dans le texte (sinon : `"J'ai " + str(age) + " ans"`).
- **Module `math`** : `import math` en tout début de programme, puis `math.sqrt(16)`, `math.pi`…
- **Comparaisons** (résultat `True`/`False`) : `==` `!=` `<` `>` `<=` `>=` ; on les combine avec `and` (les deux vraies), `or` (au moins une vraie, « ou » **inclusif**), `not` (inverse).

À savoir réécrire

1. Lire au clavier, convertir, afficher

```
prenom = input("Ton prenom ? ")
age = int(input("Ton age ? "))
print(f"Bonjour {prenom} !")
print(f"Tu auras {age + 1} ans.")
```

2. Une fonction qui renvoie un résultat

```
def moyenne(a, b):
    return (a + b) / 2

m = moyenne(12, 15)
print(m)                # affiche 13.5
```

3. Plusieurs cas : `if` / `elif` / `else`

```
def appreciation(note):
    if note >= 14:
        return "Bien"
    elif note >= 10:
        return "Moyen"
    else:
        return "Insuffisant"

print(appreciation(12))    # affiche Moyen
```

4. Boucle bornée : additionner de 1 à 10

```
somme = 0
for i in range(1, 11):
    somme = somme + i
print(somme)            # affiche 55
```

5. Boucle et test : les multiples de 3

```
for i in range(1, 16):
    if i % 3 == 0:
        print(f"{i} est un multiple de 3")
```

6. Boucle non bornée : `while`

```
n = 1
c = 0
while n < 100:
    n = n * 2
    c = c + 1
print(f"{n} atteint en {c} tours")
# affiche : 128 atteint en 7 tours
```

`range(5)` donne $0, 1, 2, 3, 4$ (5 nombres, on commence à 0) ; `range(1, 6)` va de 1 **inclus** à 6 **exclu** ; `range(0, 101, 10)` avance de 10 en 10. Dans un `while`, la variable testée **doit changer** à chaque tour, sinon c'est la **boucle infinie** (arrêt : bouton stop rouge ou `Ctrl + C`).

Méthode : suivre les variables à la main

On note la valeur de chaque variable **après** chaque ligne, en lisant chaque affectation **de droite à gauche** (on calcule avec les valeurs *actuelles*, puis on range).

```
a = 2
b = a + 5
a = a * b
b = b - 3
```

après la ligne	1	2	3	4
`a`	2	2	$2 \times 7 = 14$	14
`b`	—	$2 + 5 = 7$	7	$7 - 3 = 4$

Même méthode pour une boucle : une colonne par **tour** (voir le « mystère » du cours : x finit à $120 = 5!$).

Lire un message d'erreur

Python indique le **numéro de la ligne** et le **nom de l'erreur** : c'est une aide, pas une sanction.

Erreur	Ce qu'elle veut dire	Exemple fautif
`SyntaxError`	phrase mal construite : `:` oublié, parenthèse ou guillemet non fermé	`print("Bonjour)`
`IndentationError`	décalage irrégulier en début de ligne (toujours 4 espaces)	corps d'un `if` non décalé
`NameError`	nom inconnu : variable pas encore créée, faute de frappe, `Print` au lieu de `print`	`prix = 3` puis `print(prx)`
`TypeError`	mélange de types interdit : texte + nombre	`"J'ai " + 15` ⇒ `str(15)` ou f-string

Erreurs à éviter

Confondre `=` (**affecte**) et `==` (**compare**) ; confondre `print` (**affiche** pour l'humain, ne renvoie rien : on récupère `None`) et `return` (**renvoie** une valeur réutilisable) ; oublier qu'`input` renvoie **toujours du texte** (`3` et `4` tapés donnent `"34"` sans `int`) ; écrire `5,2` au lieu de `5.2` ; oublier les `:` ou l'indentation ; croire que `range(1, 6)` contient `6` ; nommer une variable avec une espace ou un chiffre en tête (`prix total`, `2note` sont interdits ; `prenom` plutôt que `prénom`) ; oublier que `Age` et `age` sont deux variables différentes ; définir une fonction sans jamais l'appeler.

Savoir-faire à maîtriser — je sais…

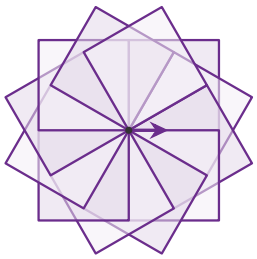
- ☐ saisir, exécuter et corriger un programme dans l'environnement de travail ;
- ☐ déclarer et utiliser une **variable** ;
- ☐ reconnaître les **types** de base : entier, flottant, chaîne, booléen ;
- ☐ écrire un **calcul** avec les opérateurs (`+ - * / // % **`) ;
- ☐ manipuler des **chaînes de caractères**, afficher avec `print` et saisir avec `input` ;
- ☐ définir et appeler une **fonction** avec des paramètres et une valeur de retour ;
- ☐ écrire une **condition** (`if` / `elif` / `else`) avec des comparaisons et des booléens ;
- ☐ écrire une **boucle** (`for`, `while`) pour répéter des instructions.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 224

1. Donner la valeur et le type de `17 // 5`, `17 % 5` et `10 / 2`.
2. Écrire ce qu'affiche une boucle `for i in range(2, 6):` dont le corps (indenté) est `print(i * 3)`.
3. Écrire une fonction `perimetre(longueur, largeur)` qui renvoie le périmètre d'un rectangle.
4. Expliquer pourquoi `n = input("Nombre ? ")` suivi de `print(n + 1)` provoque une erreur, et la corriger.
5. Donner la valeur de `(3 > 2) and (5 == 6)`, puis de `(3 > 2) or (5 == 6)`.

Complément — Dessiner avec la tortue

```
import turtle as t
def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)
for k in range(12):
    carre(60)
    t.right(30)
```



12 carrés, on tourne de 360 / 12 = 30 °

DANS CE CHAPITRE

- Cours 48
- Exercices 53
- Corrigés 225

1

COURS

Programme Notions transversales de programmation : **séquence** d'instructions, **boucle bornée** (`for`), définition et appel de **fonctions** avec **paramètres** ; ici mises en œuvre en dessinant avec le module `turtle`.

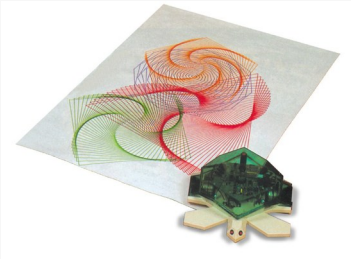
Idée Une « tortue » se déplace sur l'écran en traçant un trait derrière elle. En lui donnant des ordres simples (avance, tourne), on dessine des figures. `turtle` rend **visibles** les boucles et les fonctions : on *voit* la répétition à l'écran.

Objectifs Piloter la tortue (avancer, tourner, lever et baisser le stylo) ; **répéter** un motif avec une boucle `for` ; calculer l'angle d'un **polygone régulier** ($360/n$) ; définir et appeler une **fonction** avec paramètres ; **colorier** une forme ; composer une **rosace**.

Prérequis La boucle `for` et les fonctions (revoyez le cours « Les bases de Python » si besoin).

Pour les curieux — *Un peu d'histoire* (facultatif)

La « tortue » vient du langage **Logo**, créé en 1967 par **Seymour Papert** et Wally Feurzeig. À l'origine, c'était un vrai petit robot en forme de tortue qui traçait au sol ! L'idée de Papert : on apprend le mieux en **fabriquant** des choses. Python a hérité de cette tortue.



Une tortue robot Logo qui trace au sol



Seymour Papert présente Logo (1987)

→ frise chronologique p. 10

I Prendre la tortue en main

Tout programme commence par **importer** le module, et se termine par `turtle.done()` (qui garde la fenêtre du dessin ouverte).

```
1 import turtle
2
3 turtle.forward(100)   # avance de 100 pas (pixels)
4 turtle.left(90)       # tourne de 90 degres vers la gauche
5 turtle.forward(100)
6
7 turtle.done()         # garde la fenetre ouverte
```

Écrire `turtle` devant chaque ordre est un peu long. On peut donner un **surnom** au module (ici `t`), ce qui allège tout le code :

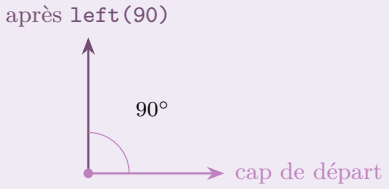
```
1 import turtle as t
2
3 t.forward(100)
4 t.left(90)
5 t.forward(100)
6 t.done()
```

A. Les ordres essentiels

Ordre	Effet
`forward(d)`	avance de `d` pas
`backward(d)`	recule de `d` pas
`left(a)`	tourne de `a` degrés vers la gauche
`right(a)`	tourne de `a` degrés vers la droite
`penup()` / `pendown()`	lever / baisser le stylo (se déplacer sans tracer)
`color("red")`	choisir la couleur du trait
`pensize(n)`	choisir l'épaisseur du trait
`circle(r)`	tracer un cercle de rayon `r`

À retenir — Position et cap de la tortue

La tortue a une **position** et une **direction** (son « cap »). `left` et `right` ne font que changer le cap : le trait n'apparaît qu'avec `forward` (ou `backward`). Les angles sont en degrés.



► **Exercices d'application :** exercices 1 **et** 2 (premiers tracés, sans boucle)

II Répéter : la boucle `for`

Dessiner un carré « à la main », c'est écrire quatre fois la même chose :

```
1  import turtle as t
2  t.forward(100)
3  t.left(90)
4  t.forward(100)
5  t.left(90)
6  t.forward(100)
7  t.left(90)
8  t.forward(100)
9  t.left(90)
10 t.done()
```

… mais **répéter**, c'est justement le rôle d'une boucle `for` ! Le même carré, en bien plus court :

```
1  import turtle as t
2  for cote in range(4):
3      t.forward(100)
4      t.left(90)
5  t.done()
```



carré : 4 côtés, on tourne de 90°

1

Règle 1 — Le secret des polygones réguliers

Pour fermer un polygone régulier à `n` côtés, la tortue doit avoir tourné en tout de 360°. À chaque sommet, elle tourne donc de $\frac{360}{n}$ degrés.

Un hexagone (6 côtés, on tourne de $360/6 = 60°$) devient immédiat :

```
import turtle as t
for cote in range(6):      # hexagone
    t.forward(80)
    t.left(60)
t.done()
```



hexagone

► **Exercices d'application :** exercices 3 **à** 5 (boucle `for` et polygones réguliers)

III Réutiliser : les fonctions

Quand on veut redessiner une même forme plusieurs fois, on en fait une **fonction** avec un **paramètre** pour la régler (ici, la taille du côté).

```
import turtle as t

def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)

carre(50)
carre(100)      # on reutilise, avec une autre taille
t.done()
```

Encore mieux : une seule fonction pour **n'importe quel** polygone régulier, en appliquant la règle des $360/n$ degrés.

```
def polygone(n, cote):
    for i in range(n):
        t.forward(cote)
        t.left(360 / n)

polygone(3, 100)    # triangle
polygone(8, 60)     # octogone
```

► **Exercices d'application :** exercices 6 **et** 7 (une fonction pour le carré, puis pour tout polygone)

IV Colorier une forme

Pour remplir une figure de couleur, on encadre son tracé par `begin_fill()` et `end_fill()`.

```
import turtle as t
t.color("orange")
t.begin_fill()
for i in range(3):
    t.forward(120)
    t.left(120)
t.end_fill()
t.done()
```

▶ **Exercices d'application :** exercice 8 (trois carrés colorés)

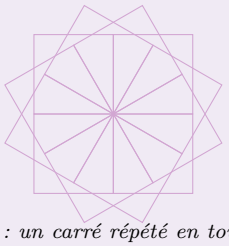
V Le plus joli : les rosaces

Une **rosace** s'obtient en répétant une même figure tout en **tournant** un peu à chaque fois. Ici, 12 carrés, pivotés de $360/12 = 30°$:

```
import turtle as t

def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)

for k in range(12):      # 12 carres...
    carre(100)
    t.left(30)           # ...pivotes de 30 degres
t.done()
```



rosace : un carré répété en tournant

▶ **Exercices d'application :** exercices 9 **et** 10 (les rosaces), défis 11 **à** 13, exercice 14 (critiquer une réponse d'IA)

À retenir — Motif, boucle et rotation

On tient là toute la puissance du chapitre : une **fonction** (le motif), une **boucle** (la répétition) et un petit **tour** à chaque étape suffisent à produire des figures très riches. À vous de jouer sur la feuille d'exercices !

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)
Ce complément réinvestit directement le chapitre *Les bases de la programmation Python* : la **boucle** `for` y répète un motif, et une **fonction** avec **paramètres** le rend réutilisable. À l'origine, la tortue de Logo était un vrai **robot** piloté par programme, dont les moteurs sont des **actionneurs**, comme ceux que l'on rencontrera au chapitre *Informatique embarquée et objets connectés*. Enfin, la fenêtre où elle dessine est un quadrillage de **pixels** : au chapitre *Photographie numérique*, on coloriera directement ces pixels, en codant chaque couleur en **RVB**.

1

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Démarrer	`import turtle as t` au début, `t.done()` à la fin.
Avancer / reculer	`forward(d)` / `backward(d)` : le trait n'apparaît qu'en avançant.
Tourner	`left(a)` / `right(a)` : change seulement le **cap** (angles en degrés).
Répéter un motif	une boucle `for` : `for i in range(n):` puis le motif **indenté**.
Polygone régulier	`n` côtés → tourner de `360 / n` à chaque sommet.
Réutiliser un dessin	une **fonction** avec paramètre(s) : `def carre(cote):` …
Remplir de couleur	`color("red")`, puis `begin_fill()` … `end_fill()`.
Se déplacer sans tracer	`penup()` … `pendown()`.
Rosace	un motif répété en tournant un peu à chaque fois (tours × angle = 360).

Savoir-faire à maîtriser

À la fin de ce complément, je sais…

☐ déplacer et orienter la tortue (avancer, tourner, lever et baisser le crayon) ;

☐ **prédire** le dessin d'un court programme `turtle` ;

☐ utiliser une boucle `for` pour **répéter** un motif ;

☐ calculer l'angle dont tourne la tortue pour un **polygone régulier** ($360/n$) ;

☐ définir et appeler une **fonction** avec des paramètres pour réutiliser une figure ;

☐ **colorier** une forme fermée ;

☐ construire une figure répétitive (**rosace**) en combinant boucle et fonction.

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- ▶ *À programmer et exécuter* sur machine. Commencez toujours par `import turtle as t` et terminez par `t.done()`.
- **Réflexes** : une figure qui se répète → une boucle `for` ; un motif à réutiliser → une **fonction** ; un polygone à `n` côtés → on tourne de `360 / n`.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Premiers tracés ↑cours

★ **Exercice 1** *Le carré* ▶ corrigé p. 225 • essentiel

▶ Dessiner un carré de côté `100`, **sans boucle** : on écrit les quatre « avancer, tourner de `90` » les uns après les autres. ▷ coup de pouce



résultat attendu

★ **Exercice 2** *Le rectangle* ▶ corrigé p. 225

▶ Dessiner un rectangle de `200` de large sur `100` de haut. *(Les quatre côtés ne sont pas tous égaux, mais on tourne toujours de* `90`*.)* ▷ coup de pouce

Répéter : la boucle `for` ↑cours

★ **Exercice 3** *L'escalier* ▶ corrigé p. 226 • essentiel

▶ Dessiner un escalier de **5 marches** : une marche, c'est « avancer, tourner, avancer, tourner ». Utiliser une boucle qui répète 5 fois ce motif. ▷ coup de pouce



résultat attendu

★★ **Exercice 4** *Triangle et hexagone* ▶ corrigé p. 226 • essentiel

▶ Dessiner un **triangle équilatéral** (3 côtés), puis un **hexagone** (6 côtés). Dans chaque cas, de combien de degrés faut-il tourner à chaque sommet ? ▷ coup de pouce



triangle *hexagone*

★★ **Exercice 5** *L'étoile à cinq branches* ▶ corrigé p. 226

▶ Dessiner une étoile à 5 branches. ▷ coup de pouce ▷ début de solution



résultat attendu

Réutiliser : les fonctions ↑ cours

★★ **Exercice 6** *La fonction carré* ▶ corrigé p. 226 • essentiel

▶ Écrire une fonction `carre(cote)` qui dessine un carré, puis l'appeler trois fois pour dessiner trois carrés de tailles différentes (par exemple 50, 100, 150). ▷ coup de pouce

★★ **Exercice 7** *Un polygone au choix* ▶ corrigé p. 227

▶ Écrire une fonction `polygone(n, cote)` qui dessine un polygone régulier à `n` côtés de longueur `cote`. L'utiliser pour dessiner un triangle, un carré, puis un octogone. ▷ coup de pouce

Colorier une forme ↑ cours

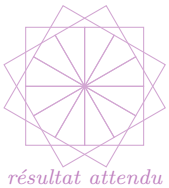
★★ **Exercice 8** *Trois carrés colorés* ▶ corrigé p. 227

▶ Dessiner **trois carrés** côte à côte, remplis de trois couleurs différentes (par exemple rouge, vert, bleu). ▷ coup de pouce

Répéter en tournant : les rosaces ↑ cours

★★ **Exercice 9** *Rosace de carrés* ▶ corrigé p. 227 • essentiel

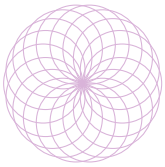
▶ En réutilisant la fonction `carre(cote)`, dessiner une rosace formée de **12 carrés**, chacun pivoté de 30 degrés par rapport au précédent. *(Pourquoi 30 ? Parce que* $12 \times 30 = 360$*.)* ▷ coup de pouce



résultat attendu

★★ **Exercice 10** *Rosace de cercles* ▶ corrigé p. 228

▶ La tortue sait tracer un cercle avec `t.circle(rayon)`. Dessiner une jolie « fleur » en répétant **18 cercles** de rayon 80, en tournant un peu entre chacun. De combien faut-il tourner à chaque fois ? ▷ coup de pouce

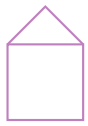


résultat attendu

Défis ↑cours

★★★ **Exercice 11** *La maison* ▶ corrigé p. 228 ≈ 15 min

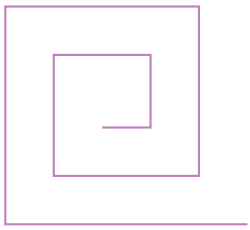
▶ Dessiner une maison : un **carré** pour la façade, surmonté d'un **triangle** pour le toit. *(Il faudra bien placer la tortue et choisir les bons angles pour le toit.)* ▷ coup de pouce ▷ début de solution



résultat attendu

★★★ **Exercice 12** *La spirale carrée* ▶ corrigé p. 228 ≈ 10 min

▶ Dessiner une spirale : on tourne toujours de `90` degrés, mais la longueur du côté **augmente** à chaque tour (par exemple `10, 20, 30, ...`). Tracer une vingtaine de segments. ▷ coup de pouce ▷ début de solution



une spirale carrée

★★★ **Exercice 13** *Du triangle à l'heptagone* ▶ corrigé p. 228 ≈ 20 min

▶ Dessiner, **côte à côte** sur une même ligne, un triangle, un carré, un pentagone, un hexagone et un heptagone (7 côtés), tous de côté `50`, en n'écrivant qu'**une seule** boucle pour enchaîner les cinq figures. ▷ coup de pouce ▷ début de solution



résultat attendu

Critiquer une réponse d'IA ↑cours

★★ **Exercice 14** *Le pentagone de l'assistant* ▶ corrigé p. 229

▶ Un élève a demandé à un assistant d'IA : « Dessine un pentagone régulier de côté `100` avec `turtle`. » Voici la réponse obtenue :

```
1 import turtle as t
2 # Un pentagone a 5 cotes et ses angles valent 108 degres
3 for i in range(5):
4     t.forward(100)
5     t.left(108)
6 t.done()
```

1. La réponse est-elle correcte ? Avant d'exécuter, calculer de combien de degrés la tortue a tourné **en tout** à la fin de la boucle. Puis exécuter le programme : la figure est-elle fermée ?
2. Localiser l'erreur et corriger le programme. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

1

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relisez dans le cours le tableau des instructions de la tortue : laquelle fait avancer, laquelle fait tourner ? Il faut un couple « avancer, tourner » par côté. ↑énoncé

Ex. 2 – piste Faites le tour d'un rectangle dessiné sur une feuille en notant la longueur de chaque côté, dans l'ordre : quelle alternance observez-vous ? ↑énoncé

Ex. 3 – piste Commencez par les quatre lignes d'*une seule* marche et vérifiez qu'à la fin la tortue regarde de nouveau dans la direction de départ. Placez ensuite ces lignes dans la boucle. ↑énoncé

Ex. 4 – piste Pour revenir à son point de départ dans la même direction, la tortue doit avoir tourné de 360 degrés en tout, répartis également entre les sommets. ↑énoncé

Ex. 5 – piste Comme pour un polygone, on répète 5 fois « avancer, tourner ». Mais en traçant l'étoile, la tortue fait *deux* tours complets sur elle-même avant de revenir à son départ : 2×360 degrés en tout. ↑énoncé

Ex. 5 – début de solution On répète 5 fois : avancer, puis tourner de `144` degrés. ↑énoncé

Ex. 6 – piste Placez la boucle du carré sous `def carre(cote):` (en la décalant vers la droite) et remplacez la longueur `100` par `cote`. ↑énoncé

Ex. 7 – piste Partez du carré (4 côtés, on tourne de `90`) : remplacez le nombre de côtés par `n` et l'angle par le calcul qui convient pour `n` côtés. ↑énoncé

Ex. 8 – piste Pour passer d'un carré au suivant **sans tracer de trait**, levez le stylo avec `t.penup()`, avancez avec `t.forward(...)`, puis baissez-le avec `t.pendown()` ; pour remplir, encadrez le tracé par `t.begin_fill()` … `t.end_fill()`. ↑énoncé

Ex. 9 – piste Dans une boucle de 12 tours : on dessine un carré, puis on fait pivoter la tortue avant le carré suivant. Pensez à recopier la définition de `carre` au début du programme. ↑énoncé

Ex. 10 – piste Les 18 cercles doivent se répartir régulièrement sur un tour complet de 360 degrés. ↑

Ex. 11 – piste Dessinez la façade, puis amenez la tortue au coin haut-gauche. Le toit est un triangle équilatéral : réutilisez les angles trouvés à l'exercice « Triangle et hexagone ». ↑énoncé

Ex. 11 – début de solution
```
for i in range(4):
    t.forward(100)
    t.left(90)
t.left(90)
t.forward(100)
```
(la tortue est au coin haut-gauche ; il reste à l'orienter vers le sommet du toit.) ↑énoncé

Ex. 12 – piste Une boucle `for i in range(20)` ; la longueur du côté change à chaque tour, elle peut valoir par exemple `(i + 1) * 10`. ↑énoncé

Ex. 12 – début de solution
```
cote = 10
for i in range(20):
    t.forward(cote)
```
(il reste à tourner, puis à faire grandir `cote`.) ↑énoncé

Ex. 13 – piste Réutilisez la fonction `polygone(n, cote)` de l'exercice « Un polygone au choix » dans une boucle où `n` prend les valeurs 3 à 7. Entre deux figures, déplacez la tortue sans tracer. ↑énoncé

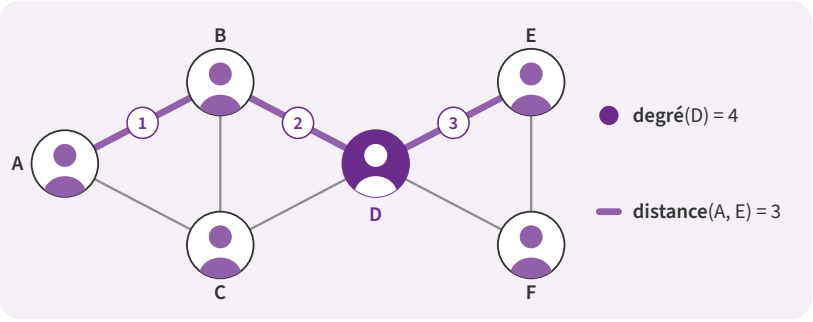
Ex. 13 – début de solution
```
for n in range(3, 8):
    polygone(n, 50)
    t.penup()
```
(il reste à avancer assez loin, puis à reposer le crayon.) ↑énoncé

Ex. 14 q. 2 – piste Pour que la figure se ferme, la somme de toutes les rotations doit valoir 360 degrés. Combien vaut 5×108 ? ↑énoncé

2

CHAPITRE 2

Les réseaux sociaux



DANS CE CHAPITRE

- Cours 58
- Exercices 70
- Débat 77
- Fiche de révision 79
- Corrigés 230

COURS

2

Programme Identité numérique, e-réputation, identification, authentification ; réseaux sociaux existants ; modèle économique ; représentation par un graphe (degré, distance, rayon, diamètre, centre) ; petit monde et six degrés de séparation ; algorithmes de recommandation.

Idée Un réseau social, ce n'est pas seulement une application : c'est un immense **graphe** de relations entre des personnes, exploité par une **entreprise** qui a un modèle économique. Comprendre les deux, c'est reprendre la main.

Objectifs Distinguer les grands réseaux et leurs ordres de grandeur ; protéger sa vie privée ; comprendre *comment ils gagnent de l'argent* ; représenter et mesurer un réseau par un graphe ; comprendre *qui choisit ce que vous voyez*.

Situation de départ

1. Votre application préférée est gratuite, et pourtant l'entreprise qui la possède gagne des milliards. Avec quoi la payez-vous ?
2. Pourquoi votre fil ne vous montre-t-il pas les mêmes vidéos qu'à votre voisin de classe ?
3. Un message passé d'ami en ami doit atteindre un inconnu à l'autre bout du monde. Combien d'intermédiaires faut-il, à votre avis ?

Pour les curieux — *Un peu d'histoire (et un ancêtre surprenant)* (facultatif)

Avant Internet, en **1967**, le psychologue américain **Stanley Milgram** tente une expérience étonnante : faire parvenir une lettre à un inconnu à l'autre bout du pays, uniquement de connaissance en connaissance. Résultat : en moyenne **cinq ou six intermédiaires** suffisent. Le monde est bien plus « petit » qu'on ne le croit… nous y reviendrons.

Il faudra attendre **1997** et le site *SixDegrees.com* (dont le nom rend justement hommage à Milgram !) pour voir le premier réseau social en ligne. Puis tout s'accélère : **2003** MySpace et LinkedIn, **2004** Facebook (créé par **Mark Zuckerberg**, étudiant de 19 ans à Harvard), **2005** YouTube, **2006** Twitter, **2010** Instagram, **2011** Snapchat, **2016** TikTok. En moins de vingt ans, une poignée d'entreprises se sont retrouvées à relier… une bonne partie de l'humanité.



Stanley Milgram (1974)



Mark Zuckerberg (2019)

→ frise chronologique p. 10

I Qu'est-ce qu'un réseau social ?

Définition 1

Un **réseau social** (en ligne) est un service qui permet à des personnes de se créer un **profil**, de se **relier** à d'autres (amis, abonnés, contacts) et de **partager** des contenus (textes, photos, vidéos). Le mot « réseau » n'est pas un hasard : l'ensemble des personnes et des liens forme bel et bien un réseau, que l'on peut dessiner.

Il en existe des dizaines, et ils ne se ressemblent pas : certains servent surtout à partager des **photos** (Instagram), d'autres des **vidéos courtes** (TikTok), des **messages** (X, anciennement Twitter), des **contacts professionnels** (LinkedIn) ou des **messages privés** (WhatsApp, Snapchat). Pour les comparer, on regarde : la nature du contenu, le public visé, et surtout un **ordre de grandeur** du nombre d'utilisateurs.

Document 1 — Combien d'utilisateurs ? Des ordres de grandeur

On ne connaît jamais le chiffre exact (il change tous les jours), mais on peut retenir des **ordres de grandeur** du nombre d'utilisateurs actifs chaque mois (chiffres arrondis, mis à jour en octobre 2026 ; chiffres publiés par les entreprises, 2025-2026, et DataReportal) :

Réseau	Contenu principal	Ordre de grandeur
Facebook	tout (texte, photo, vidéo)	∼ 3 milliards
WhatsApp	messages privés	plus de 3 milliards
Instagram	photos, vidéos courtes	∼ 3 milliards
YouTube	vidéos	∼ 2,5 milliards*
TikTok	vidéos courtes	∼ 1,6 milliard*
LinkedIn	professionnel	∼ 1,3 milliard d'*inscrits***
Snapchat	photos éphémères	près de 1 milliard
X (Twitter)	messages courts	*centaines de millions*

* YouTube et TikTok ne publient pas ce chiffre : on donne le nombre de personnes que leurs publicités peuvent atteindre chaque mois (DataReportal, 2025). ** LinkedIn annonce ses membres inscrits, pas ses utilisateurs actifs, qui sont bien moins nombreux.

Rappel : la population mondiale est d'environ **8,2 milliards** d'humains (ONU, 2024).

Sources : Meta (Instagram : 3 milliards d'utilisateurs actifs mensuels, septembre 2025 ; WhatsApp : plus de 3 milliards, avril 2025 ; Facebook : environ 3 milliards) ; Snap Inc., résultats du 2e trimestre 2026 (971 millions d'utilisateurs actifs mensuels) ; LinkedIn, *About us* (plus de 1,3 milliard de membres, 2026) ; DataReportal (Kepios, We Are Social, Meltwater), *Digital 2025 Global Overview Report*, février 2025 (audiences publicitaires de YouTube et TikTok) ; ONU, *World Population Prospects 2024* (8,2 milliards d'humains en 2024).

▶ Activité 1 — Dresser le portrait des réseaux

À l'aide du Document 1 et de vos propres connaissances :

1. Classer ces réseaux par ordre de grandeur du nombre d'utilisateurs. Que remarque-t-on pour les trois premiers ? À quelle entreprise appartiennent-ils ?
2. Citer deux réseaux dont le contenu principal est la vidéo.
3. Un réseau annonce « 3 milliards d'utilisateurs ». Expliquer pourquoi ce chiffre est difficile à vérifier, et pourquoi on parle plutôt d'un *ordre de grandeur*.

4. Choisir deux réseaux que vous connaissez et rédiger, pour chacun, une phrase qui le distingue des autres (public, type de contenu, façon de s'en servir).

► **Exercices d'application :** exercices 1 **et** 2 (reconnaître et comparer des réseaux)

II Identité numérique, vie privée et sécurité

Sur un réseau social, vous laissez une trace de vous-même : c'est votre **identité numérique**.

Définition 2

L'**identité numérique** est l'ensemble des informations qui vous représentent en ligne : profil, photos, publications, commentaires, « j'aime », mais aussi les traces laissées sans y penser (heure de connexion, position, recherches).

La **e-réputation** est l'image que ces informations donnent de vous aux autres.

Une confusion fréquente : « s'identifier » et « s'authentifier », ce n'est pas la même chose.

Règle 1

S'identifier, c'est *annoncer qui l'on est* (donner son identifiant, par exemple son adresse e-mail).
S'authentifier, c'est *prouver* que c'est bien vous (donner votre mot de passe, un code reçu par SMS, une empreinte).

Un identifiant peut être connu de tous ; le moyen d'authentification, lui, doit rester **secret**.

Exemple Sur le portail du lycée, votre **identifiant** est votre nom d'élève (les professeurs le connaissent). Votre **mot de passe**, lui, ne doit être connu que de vous : c'est ce qui *authentifie* que la personne qui se connecte est bien vous.

Document 2 — « Ce que je publie ne me regarde plus vraiment »

Trois idées à garder en tête :

- **Rien ne s'efface vraiment.** Une photo supprimée a pu être copiée, enregistrée (capture d'écran), rediffusée. On parle de *permanence* des contenus.
- **Public par défaut.** Sur beaucoup de réseaux, un compte est *visible de tous* tant qu'on n'a pas réglé la confidentialité.
- **Le futur vous lira.** Un recruteur, une université… peuvent chercher votre nom. Votre e-réputation d'aujourd'hui vous suit demain.

Définition 3 — Données personnelles

Une **donnée personnelle** est une information qui permet d'identifier une personne, directement (nom, photo) ou indirectement (adresse e-mail, numéro de téléphone, localisation). Dans l'Union européenne, leur collecte et leur utilisation sont encadrées par le **RGPD** (*Règlement général sur la protection des données*, règlement UE 2016/679).

Règle 2 — Vos droits sur vos données

- **Âge** : en France, c'est à partir de **15 ans** que l'on peut consentir seul à l'utilisation de ses données par un service en ligne (loi Informatique et libertés) ; en dessous, l'accord des parents (titulaires de l'autorité parentale) est aussi nécessaire.
- **Droits** : droit d'**accès** (savoir quelles données un service détient sur vous et en obtenir une copie), de **rectification** et d'**effacement**.
- **Recours** : si vous résidez en France (ou dans l'UE), vos droits relèvent du RGPD et de la loi Informatique et libertés ; l'autorité de contrôle est la **CNIL**.

Et à Monaco ?

Monaco n'est pas membre de l'Union européenne : le RGPD n'y est pas directement applicable (un site monégasque qui s'adresse à des personnes situées dans l'UE doit toutefois aussi le respecter). La protection des données y est assurée par la **loi n° 1.565 du 3 décembre 2024** relative à la protection des données personnelles, qui s'aligne sur les standards européens.

- Ce sont les **mêmes grands droits** : accès (art. 12), rectification (art. 13), effacement (art. 14), opposition (art. 17), portabilité (art. 18).
- **Mineurs** : pour un service en ligne, en dessous de **15 ans**, il faut l'autorisation du ou des titulaires de l'autorité parentale (art. 6).
- **Autorité de contrôle** : l'**APDP** (Autorité de protection des données personnelles, `apdp.mc`), qui a succédé à la CCIN.

▶ Activité 2 — Régler sa confidentialité

1. Voici le menu *Confidentialité* du compte de Léa, 15 ans, sur un réseau social imaginaire : *type de compte* : public ; *qui peut commenter* : tout le monde ; *qui peut vous envoyer un message* : tout le monde ; *ajouter la position aux publications* : activé ; *liste des abonnés* : visible par tous. Repérer trois réglages qui exposent la vie privée de Léa et indiquer, pour chacun, le choix qui la protège.

2. Proposer un mot de passe « solide » et expliquer, en une phrase, ce qui le rend difficile à deviner.

3. Distinguer, dans la liste suivante, ce qui relève de l'*identification* et de l'*authentification* : saisir son adresse e-mail ; entrer un code reçu par SMS ; poser son doigt sur le capteur d'empreinte ; taper son pseudo.

4. Débat éclair (5 minutes) : « Publier une photo d'un camarade sans son accord, est-ce grave ? » Noter deux arguments.

▶ **Exercices d'application :** exercices 3 à 6 (identité numérique, e-réputation, mot de passe, droits sur ses données)

III Comment gagnent-ils de l'argent ?

La plupart des réseaux sociaux sont **gratuits**. Pourtant, ce sont des entreprises qui pèsent des milliards. La question mérite donc d'être posée sérieusement.

Règle 3

La principale ressource des grands réseaux sociaux est la **publicité ciblée**. Le réseau collecte des **données** sur vous (âge, centres d'intérêt, ce sur quoi vous cliquez), puis **loue** aux annonceurs la possibilité de vous montrer *la bonne publicité au bon moment*. Plus vous restez longtemps, plus il peut afficher de publicités : c'est l'**économie de l'attention**.

Exemple Une formule résume bien l'idée : *« Si c'est gratuit, c'est que le produit, c'est vous. »* Plus exactement : le produit vendu aux annonceurs, c'est votre **attention** et vos **données**.

Document 3 — Les revenus d'un géant

Un grand réseau social tire l'essentiel de ses revenus de la publicité. Sur 100 € gagnés, il n'est pas rare que **plus de 95 €** viennent des annonceurs, et à peine quelques euros d'autres sources (abonnements payants, badges, boutique en ligne). Chez Meta (Facebook, Instagram, WhatsApp), la publicité a fourni environ **98 %** du chiffre d'affaires en 2023 (Meta, rapport annuel 2023).

On mesure aussi le « revenu moyen par utilisateur » (en anglais *ARPU*) : le total des revenus divisé par le nombre d'utilisateurs. Pour Facebook, fin 2023, il était d'environ 13 dollars par trimestre en moyenne mondiale, soit de l'ordre de **40 € par utilisateur et par an**, et plus de 60 dollars par trimestre aux États-Unis et au Canada (Meta, résultats du 4e trimestre 2023).

Autrement dit : chaque compte « gratuit » rapporte, en réalité, de l'argent à l'entreprise.

Source : Meta Platforms, rapport annuel 2023 (formulaire 10-K) et résultats du 4e trimestre 2023 (part de la publicité, revenu moyen par utilisateur).

▶ Activité 3 — Suivre l'argent

À l'aide du Document 3 :

1. Citer la principale source de revenus d'un grand réseau social.
2. Expliquer, avec vos mots, ce qu'on vend réellement à l'annonceur.
3. Un réseau compte 2 milliards d'utilisateurs et un revenu moyen de 40 € par utilisateur et par an. Estimer son revenu annuel (ordre de grandeur).
4. « Économie de l'attention » : expliquer pourquoi une application a intérêt à ce que vous y passiez le plus de temps possible.

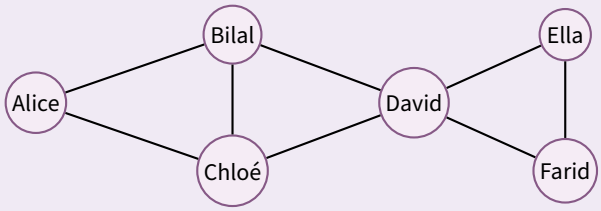
▶ **Exercices d'application :** exercices 7 à 9 (le modèle économique)

IV Un réseau social, c'est un graphe

Pour *mesurer* un réseau, les informaticiens le dessinent sous forme de **graphe**.

Définition 4

Un **graphe** est constitué de **sommets** (ici : les personnes) reliés par des **arêtes** (ici : « est ami avec », ou « suit »). Le **degré** d'un sommet est le nombre d'arêtes qui en partent : c'est, en gros, le nombre d'amis de la personne.



Sur ce petit réseau d'amitiés, on peut déjà lire beaucoup de choses. Par exemple, le degré de David est 4 (il est relié à Bilal, Chloé, Ella et Farid) : c'est la personne la plus « connectée ».

Définition 5

Une **chaîne** entre deux sommets est un chemin qui les relie en suivant les arêtes. Sa **longueur** est le nombre d'arêtes empruntées. La **distance** entre deux personnes est la longueur de la **plus courte** chaîne qui les relie (le « nombre de poignées de main »).

Exemple La distance entre Alice et Ella est **3** : Alice – Bilal – David – Ella (aucun chemin plus court n'existe). Alice et David, eux, sont à distance **2**.

À partir de la distance, on définit trois mesures du programme : le rayon, le diamètre et le centre.

Définition 6

L'**excentricité** d'un sommet est la distance qui le sépare du sommet le *plus éloigné*.
Le **diamètre** du graphe est la **plus grande** excentricité (les deux personnes les plus éloignées du réseau).
Le **rayon** est la **plus petite** excentricité.
Le **centre** est le (ou les) sommet(s) dont l'excentricité est la plus petite : la personne la mieux placée pour joindre tout le monde rapidement.

▶ Activité 4 — Mesurer le réseau des amis

On travaille sur le graphe ci-dessus.

1. Donner le degré de chaque personne. Qui est « l'influenceur » (plus grand degré) ?
2. Déterminer la distance entre Alice et Farid, puis entre Bilal et Ella.
3. Compléter le tableau des excentricités :

Alice	Bilal	Chloé	David	Ella	Farid
?	?	?	?	?	?

4. En déduire le **diamètre**, le **rayon** et le **centre** du réseau.

5. Ella et Farid décident de devenir amis avec Alice. Redessiner (ou décrire) le graphe et dire si le diamètre change.

► **Exercices d'application :** exercices 10 à 13 (le graphe : degrés, distances, diamètre)

V Le petit monde et les six degrés de séparation

Revenons à l'histoire du début du cours. L'idée qu'on puisse relier deux inconnus par une courte chaîne d'amis a un nom : le **petit monde**.

Document 4 — L'expérience de Milgram (1967)

Stanley Milgram confie des lettres à des habitants du Nebraska. Consigne : faire parvenir la lettre à une personne cible (un agent de change à Boston) **sans envoi direct**, seulement en la transmettant à *une connaissance* que l'on juge plus proche de la cible. À chaque étape, la lettre se rapproche.
Parmi les lettres arrivées à destination, le nombre d'intermédiaires est en moyenne de **5 à 6** (Travers et Milgram, *Sociometry*, 1969). De là est née l'expression « **six degrés de séparation** » : deux personnes au hasard sur Terre seraient reliées par une chaîne d'environ six connaissances.
Et aujourd'hui ? En 2016, une étude menée sur le réseau Facebook (près de 1,6 milliard de comptes à l'époque) a mesuré une distance moyenne d'environ **3,5** (Facebook Research, 2016). Le monde est devenu encore plus petit.

Sources : J. Travers et S. Milgram, « An Experimental Study of the Small World Problem », *Sociometry*, 1969 ; Facebook Research, « Three and a half degrees of separation », février 2016.

Propriété 1

Dans un très grand réseau, la distance moyenne entre deux personnes reste **petite** (quelques unités), même quand le nombre de personnes se compte en milliards. C'est le phénomène du **petit monde**.

► Activité 5 — Combien de degrés nous séparent ?

1. Reformuler « six degrés de séparation » avec vos propres mots.
2. Dans le graphe des amis (section IV), quelle est la plus grande distance entre deux personnes ? Comparer avec « six ».
3. À votre avis, pourquoi la distance moyenne *diminue*-t-elle quand chacun a beaucoup d'amis en ligne ? (Une phrase.)

► **Exercices d'application :** exercice 14 (six degrés de séparation)

VI Qui choisit ce que vous voyez ?

Vous ne voyez jamais *tout* ce qui est publié : un programme trie pour vous.

Définition 7

Un **algorithme de recommandation** est un programme qui **choisit et ordonne** les contenus affichés dans votre fil, en fonction de ce que vous (et des gens qui vous ressemblent) avez déjà regardé, aimé, ou partagé. Son but : vous retenir le plus longtemps possible.

Ce tri a des conséquences qu'il faut connaître :

- **Bulle de filtres** : à force de vous montrer ce qui vous plaît, l'algorithme finit par ne plus vous montrer que *cela*. Vous voyez une version rétrécie du monde.
- **Chambre d'écho** : entouré d'avis semblables au vôtre, vous avez l'impression que « tout le monde pense comme vous ».
- **Désinformation** : une *fausse* information spectaculaire est souvent partagée plus vite qu'une information vraie mais ennuyeuse.

Document 5 — Pourquoi cette vidéo, et pas une autre ?

Pour ordonner votre fil, un algorithme observe des dizaines de signaux : le temps passé sur chaque vidéo, les « j'aime », les partages, les comptes que vous suivez, l'heure, votre pays, l'appareil utilisé… Il en déduit ce qui a le plus de chances de vous faire *rester*. Personne n'a « choisi à la main » votre fil : c'est un calcul, refait à chaque seconde, pour des milliards de personnes en même temps.

▶ Activité 6 — Enquêter sur son fil

1. Décrire, en deux ou trois phrases, comment un réseau décide de l'ordre des contenus de votre fil.
2. Donner un exemple concret de « bulle de filtres » que vous avez déjà remarqué (ou imaginé).
3. Proposer deux gestes simples pour *sortir* de sa bulle (suivre des comptes différents, vérifier une information avant de la partager…).

▶ **Exercices d'application :** exercices 15 **et** 16 (qui choisit ce que je vois ? les amis de mes amis)

VII Bien vivre avec les réseaux sociaux

Les réseaux sont des outils formidables (s'informer, créer, garder le lien, se mobiliser). Mais, comme tout outil puissant, ils demandent quelques précautions de citoyen.

Règle 4

Le cyberharcèlement (insultes, menaces, moqueries répétées en ligne) est **interdit par la loi** et lourdement puni. Face à une situation de harcèlement : ne pas répondre, **garder les preuves** (captures d'écran), **bloquer**, **signaler**, et **en parler** à un adulte de confiance. À Monaco, la ligne d'écoute contre le harcèlement mise en place par le Gouvernement princier (**98 98 96 96**, 24 h/24, 7 j/7) aide aussi les victimes de harcèlement en ligne. Le service français **3018** répond par tchat (application *3018*) ; son numéro court n'est pas garanti depuis une ligne monégasque.

▶ Activité 7 — Ma charte des réseaux

En binôme, rédiger une **charte de cinq règles** « pour bien vivre sur les réseaux » (une pour la vie privée, une pour l'esprit critique, une contre le harcèlement, deux au choix). Ces règles serviront de base à l'exposé vidéo (projet noté, dont le sujet est distribué en classe).

▶ **Exercices d'application :** exercice 17 (citoyen des réseaux)

VIII L'envers de l'écran : l'impact environnemental

Un fil infini, des vidéos qui s'enchaînent toutes seules… Tout cela circule dans des **réseaux**, est stocké dans des **centres de données** et s'affiche sur des **appareils** qu'il a fallu fabriquer. Le numérique n'est pas « immatériel » : il a une empreinte environnementale.

Document 6 — Stocker et diffuser des vidéos

Unité. On mesure les émissions de gaz à effet de serre en **CO_2e** (*équivalent CO_2*) ; **1 Mt** = 1 million de tonnes. Les chiffres ci-dessous sont des **estimations** : on retient des ordres de grandeur.

- **La vidéo domine le trafic.** Elle représente la majorité des données qui circulent sur Internet : environ les **deux tiers** du trafic mondial en 2022, selon une estimation de l'équipementier de réseaux Sandvine. En 2024, avec une autre méthode de classement (une partie des vidéos est désormais comptée avec les réseaux sociaux), le seul *streaming* vidéo à la demande (Netflix, Disney+…) représentait encore plus de la **moitié** du trafic reçu par les abonnés.
  Sources : Sandvine, Global Internet Phenomena Report*, janvier 2023 (données 2022) et mars 2024.*
- **Stocker.** Une vidéo publiée n'est pas rangée une seule fois : la plateforme la conserve sur ses serveurs en **plusieurs définitions** (pour s'adapter à chaque écran et à chaque connexion), souvent recopiée dans plusieurs **centres de données** proches des utilisateurs, et cela tant qu'elle n'est pas supprimée.
- **Diffuser.** Chaque visionnage renvoie la vidéo à travers les réseaux : une vidéo vue un million de fois est transportée un million de fois. Une heure de vidéo échange jusqu'à **1 Go** de données en définition standard, jusqu'à **3 Go** en haute définition et jusqu'à **7 Go** en 4K.
  Source : Netflix, centre d'aide, page sur la consommation de données, consultée en 2026.
- **Combien de CO_2 ? Une estimation discutée.** En 2019, The Shift Project estimait que la vidéo en ligne émettait plus de 300 Mt CO_2e par an (près de 1 % des émissions mondiales). L'Agence internationale de l'énergie (AIE) a jugé ce type de calcul très surestimé : selon elle, une heure de vidéo en streaming émettait en moyenne environ **36 g**

de CO_2 en 2019. Les résultats changent beaucoup selon les hypothèses (appareil, réseau, électricité du pays) ; ce qui est sûr, c'est que plus on regarde, et en haute définition, plus on échange de données.

Sources : The Shift Project, « Climat : l'insoutenable usage de la vidéo en ligne », juillet 2019 ; AIE, « The carbon footprint of streaming video : fact-checking the headlines », décembre 2020.

- **Et le numérique en général ?** Selon les études et ce qu'elles comptent, il pèse environ **2 à 4 %** des émissions mondiales (1,7 % pour le seul secteur des technologies de l'information selon l'UIT et la Banque mondiale, données 2022 ; près de 4 % selon The Shift Project, 2019). Pour la France et le calcul complet de l'empreinte d'un smartphone (environ 80 kg CO_2e, presque tout à la fabrication), on y reviendra au chapitre *Informatique embarquée et objets connectés*.

Source : UIT et Banque mondiale, Measuring the Emissions and Energy Footprint of the ICT Sector*, 2024.*

▶ Activité 8 — Sobriété numérique

À l'aide du Document 6 :

1. Expliquer pourquoi une vidéo très regardée « pèse » bien plus que la taille de son fichier (penser au **stockage** et à la **diffusion**).
2. En vous appuyant sur le chiffre de 80 kg CO_2e du Document 6 (presque tout dû à la **fabrication**), expliquer pourquoi **garder son téléphone plus longtemps** (le faire réparer, choisir un appareil reconditionné) est l'un des gestes les plus efficaces.
3. La **lecture automatique** et la **qualité vidéo maximale** sont souvent activées par défaut. Expliquer pourquoi elles augmentent la quantité de données échangées, puis proposer deux réglages pour la réduire.
4. Faire le lien avec l'**économie de l'attention** (section III) : pourquoi un réseau social n'a-t-il pas intérêt à proposer lui-même de désactiver le **fil infini** ou la lecture automatique ?

▶ **Exercices d'application :** exercices 18 **et** 19 (le coût d'un smartphone, haute ou basse définition), exercice 20 (critiquer une réponse d'IA), exercice 21 (Python au service du thème)

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Juste après :** une photo publiée peut trahir, par des informations cachées dans son fichier, le lieu où elle a été prise (chapitre *Photographie numérique*).
- **Plus tard :** le même **graphe** servira à calculer un itinéraire (chapitre *Localisation, cartographie et mobilité*), et vos publications, des **données personnelles**, seront rangées dans d'immenses **tables** (chapitre *Les données structurées et leur traitement*).
- **Plus loin :** en spécialité NSI de Terminale, on parcourt des graphes et on y calcule des distances par programme.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Réseau social	profils + liens + partage ; comparés par contenu, public, *ordre de grandeur* d'abonnés.
Ordres de grandeur	les plus gros : *milliards* d'utilisateurs (population mondiale : ~ 8 milliards).
Identité numérique	tout ce qui vous représente en ligne ; forge votre *e-réputation* (durable).
Identifier / authentifier	annoncer qui l'on est / *prouver* que c'est bien soi (secret).
Modèle économique	*publicité ciblée* + données ; « si c'est gratuit, le produit c'est vous ».
Graphe	sommets (personnes) + arêtes (liens) ; *degré* = nombre d'amis.
Distance	longueur de la plus courte chaîne (nombre de « poignées de main »).
Diamètre / rayon / centre	plus grande / plus petite excentricité / sommet(s) le(s) mieux placé(s).
Petit monde	Milgram (1967) → « six degrés » ; sur Facebook ($\sim$2016) : $\approx 3{,}5$.
Recommandation	un algorithme trie votre fil ; risques : bulle de filtres, désinformation.
Données personnelles	RGPD ; seul dès **15 ans** ; accès, rectification, effacement ; CNIL (France), APDP et loi n° 1.565 (Monaco).
Citoyenneté	cyberharcèlement interdit ; garder les preuves, bloquer, signaler, en parler (à Monaco : **98 98 96 96**).
Impact environnemental	numérique ≈ 2 à 4 % des GES mondiaux (estimations) ; vidéo = majorité du trafic : stockage + diffusion ; garder ses appareils.

Retour à la situation de départ

1. Avec vos **données** et votre **attention** : le réseau vend aux annonceurs de la **publicité ciblée** (§ III). « Si c'est gratuit, le produit, c'est vous. »
2. Un **algorithme de recommandation** choisit et ordonne votre fil d'après ce que vous avez regardé, aimé ou partagé (§ VI) : chacun reçoit le sien, au risque de s'enfermer dans une **bulle de filtres**.
3. Très peu : dans le **graphe** d'un réseau (§ IV), la **distance** moyenne entre deux personnes reste de quelques unités, même parmi des milliards. C'est le **petit monde** : « six degrés » pour Milgram, environ 3,5 sur Facebook en 2016 (§ V).

Débat argumenté

Un **débat**, proposé en fin de chapitre, accompagne ce thème : « Faut-il interdire les réseaux sociaux aux moins de 15 ans, avec une vérification d'âge obligatoire ? » (documents, rôles, tableau d'arguments et trace écrite, environ 50 min).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ définir un **réseau social** et citer ses principaux usages ;

☐ protéger mon **identité numérique** et régler mes paramètres de confidentialité ;

☐ citer mes **droits** sur mes données personnelles (RGPD en France, loi n° 1.565 à Monaco) ;

☐ expliquer le **modèle économique** (données personnelles, publicité ciblée) ;

☐ modéliser un réseau social par un **graphe** (sommets = comptes, arêtes = relations) et lire le **degré** d'un sommet ;

☐ calculer une **distance**, l'**excentricité** d'un sommet, puis le **diamètre**, le **rayon** et le **centre** d'un graphe ;

☐ expliquer l'idée des « **six degrés de séparation** » (petit monde) ;

☐ expliquer qu'un **algorithme** choisit et ordonne ce qui est affiché (fil, recommandations) ;

☐ citer des ordres de grandeur de l'**impact environnemental** du numérique et des gestes de sobriété ;

☐ adopter de bonnes pratiques (temps d'écran, esprit critique, lutte contre le harcèlement).

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- [sur papier] *signale les exercices à faire sur papier* ; ▶ *ceux qui demandent une petite recherche ou une manipulation.*
- Vocabulaire du graphe : **sommet**, **arête**, **degré**, **distance**, **excentricité**, **diamètre**, **rayon**, **centre**.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Reconnaître et comparer les réseaux ↑cours

★ **Exercice 1** *Le bon réseau* ▶ corrigé p. 232 • essentiel

[sur papier] Associer chaque réseau à son contenu principal : *Instagram, TikTok, LinkedIn, WhatsApp, YouTube* ↔ *vidéos, photos, vidéos courtes, messages privés, contacts professionnels.*

▷ coup de pouce

★ **Exercice 2** *Ordres de grandeur* ▶ corrigé p. 232

[sur papier] On rappelle que la population mondiale est d'environ 8 milliards d'humains.

1. Un réseau annonce « 3 milliards d'utilisateurs actifs ». Quelle proportion de l'humanité cela représente-t-il, environ ?
2. Pourquoi parle-t-on d'un **ordre de grandeur** plutôt que d'un chiffre exact ?

▷ coup de pouce

Identité numérique, sécurité ↑cours

★ **Exercice 3** *Identifier ou authentifier ?* ▶ corrigé p. 232 • essentiel

[sur papier] Pour chaque action, dire s'il s'agit d'**identification** (annoncer qui l'on est) ou d'**authentification** (prouver que c'est bien soi) :

a. saisir son identifiant de l'ENT (*prenom.nom*) ;
b. taper son mot de passe ;
c. déverrouiller son téléphone par reconnaissance du visage ;
d. saisir son numéro de téléphone sur la page de connexion d'une messagerie ;
e. valider, sur son téléphone, la notification « Est-ce bien vous qui essayez de vous connecter ? ».

▷ coup de pouce

★ **Exercice 4** *E-réputation* ▶ corrigé p. 232

[sur papier] « Rien ne s'efface vraiment sur Internet. » Expliquer cette phrase, puis donner un exemple de situation où votre e-réputation d'aujourd'hui pourrait vous suivre plus tard.

▷ coup de pouce

★★ **Exercice 5** *Un bon mot de passe* ▶ corrigé p. 232

▶ Proposer deux mots de passe : un **faible** et un **solide**. Expliquer, pour chacun, ce qui le rend facile ou difficile à deviner (longueur, variété des caractères, mot du dictionnaire, information personnelle…).

▷ coup de pouce

★★ **Exercice 6** *Mes données, mes droits* ▶ corrigé p. 233 • essentiel

[sur papier] Inès, 16 ans, habite à Monaco. Elle a un compte sur une application de partage de photos.

1. Citer trois **données personnelles** que cette application détient sans doute sur elle.
2. Inès veut savoir quelles données l'application conserve sur elle, puis faire supprimer une ancienne photo. Quel droit exerce-t-elle dans chaque cas ?
3. L'application ne lui répond pas. À quelle autorité peut-elle s'adresser ? Et si elle habitait à Nice ?
4. Sa petite sœur, qui a 13 ans, peut-elle ouvrir seule un compte sur cette application ?

▷ coup de pouce

Le modèle économique ↑ cours

★ **Exercice 7** *« Si c'est gratuit… »* ▶ corrigé p. 233 • essentiel

[sur papier] La plupart des réseaux sont gratuits mais très riches.

1. Quelle est leur principale source de revenus ?
2. Compléter : « Si c'est gratuit, c'est que le ……, c'est …… ». Qu'est-ce qu'on vend réellement à l'annonceur ?

▷ coup de pouce

★★ **Exercice 8** *Suivre l'argent* ▶ corrigé p. 233

[sur papier] Un réseau de partage de vidéos compte 500 millions d'utilisateurs et rapporte en moyenne 12 € par utilisateur et par an.

1. Estimer son **revenu annuel** (ordre de grandeur).
2. L'application affiche une publicité toutes les 5 minutes de visionnage. Combien de publicités voit un utilisateur qui la regarde 30 minutes par jour ? Et 1 heure par jour ? En déduire, avec l'idée d'« économie de l'attention », pourquoi l'application a intérêt à vous garder le plus longtemps possible.

▷ coup de pouce

★★ **Exercice 9** *Ce que la publicité sait de vous* ▶ corrigé p. 233

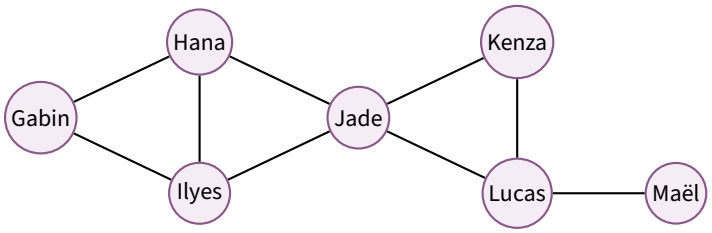
[sur papier] En une semaine, Hugo a aimé une douzaine de vidéos de football, cherché « crampons pas chers », publié une photo localisée au stade Louis-II et regardé jusqu'au bout trois vidéos de recettes de pâtes.

1. Quelles informations le réseau peut-il en **déduire** sur Hugo ?
2. Proposer deux publicités qu'il risque de voir dans son fil la semaine suivante.
3. Expliquer pourquoi ces informations ont de la valeur pour le réseau.

▷ coup de pouce

Un réseau social, c'est un graphe ↑cours

Pour les exercices 10 et 11, on étudie le réseau d'amitiés suivant, différent de celui du cours (une arête = « est ami avec ») :



★★ **Exercice 10** *Degrés et distances* ▶ corrigé p. 233 • essentiel

[sur papier]

1. Donner le **degré** de chaque personne. Qui est le plus « connecté » ?
2. Déterminer la **distance** entre Gabin et Kenza, puis entre Kenza et Maël (donner un plus court chemin).

▷ coup de pouce

★★★ **Exercice 11** *Diamètre, rayon, centre* ▶ corrigé p. 234 ≈ 20 min

[sur papier]

1. Rappeler ce qu'est l'**excentricité** d'un sommet, puis calculer celle de chaque personne.
2. En déduire le **diamètre**, le **rayon** et le **centre** du réseau.
3. Maël devient aussi ami avec Kenza. Le diamètre change-t-il ? Justifier.

▷ coup de pouce ▷ début de solution

★★ **Exercice 12** *Dessiner un réseau* ▶ corrigé p. 234 • essentiel

[sur papier] Dans un club, Léo est ami avec Mia et avec Noé ; Mia est aussi amie avec Noé et avec Paul ; Paul est ami avec Rose. Personne d'autre n'est ami.

1. Dessiner le graphe de ce réseau.
2. Donner le degré de chaque personne.
3. Déterminer la distance entre Léo et Rose, puis entre Noé et Paul.

▷ coup de pouce

★★★ **Exercice 13** *Fabriquer un diamètre* ▶ corrigé p. 234 ≈ 15 min

[sur papier] On dispose de cinq personnes. Dans chaque réseau demandé, on doit pouvoir aller de n'importe qui à n'importe qui en suivant les arêtes.

1. Dessiner un réseau de ces cinq personnes dont le **diamètre** vaut 4. Combien d'arêtes compte-t-il ?
2. Dessiner un réseau de ces cinq personnes dont le diamètre vaut 1. Combien d'arêtes compte-t-il ?
3. Quel est le **centre** du réseau de la question 1 ?

▷ coup de pouce ▷ début de solution

Petit monde et recommandation ↑ cours

★★ **Exercice 14** *Six degrés de séparation* ▶ corrigé p. 234

[sur papier]

1. Expliquer, avec vos propres mots, l'expression « six degrés de séparation » (expérience de Milgram, 1967).
2. Sur Facebook (vers 2016), la distance moyenne mesurée était d'environ 3,5. Que peut-on en conclure sur l'évolution du « petit monde » ?

▷ coup de pouce

★ **Exercice 15** *Qui choisit ce que je vois ?* ▶ corrigé p. 235 • essentiel

[sur papier]

1. Qu'est-ce qu'un **algorithme de recommandation** ? Quel est son but ?
2. Expliquer ce qu'est une **bulle de filtres**, et proposer un geste simple pour en sortir.

▷ coup de pouce

★★★ **Exercice 16** *Les amis de mes amis* ▶ corrigé p. 235 ≈ 15 min

[sur papier] Un réseau propose à chacun des « amis suggérés » : les personnes situées à **distance** 2, en commençant par celles avec qui l'on a le plus d'**amis en commun**. On reprend le graphe d'amitiés de la section IV du cours.

1. Quelles personnes sont à distance 2 d'Alice ? Combien d'amis en commun a-t-elle avec chacune ? Qui lui sera suggéré en premier ?
2. Même question pour Ella.
3. Expliquer pourquoi ce type de suggestion a tendance à refermer chaque groupe sur lui-même (faire le lien avec la bulle de filtres).

▷ coup de pouce ▷ début de solution

★ **Exercice 17** *Citoyen des réseaux* ▶ corrigé p. 235

[sur papier] Face à une situation de cyberharcèlement, citer trois bons réflexes et donner un numéro d'aide joignable à Monaco. ▷ coup de pouce

Impact environnemental ↑ cours

★★ **Exercice 18** *Le coût d'un smartphone* ▶ corrigé p. 235

[sur papier] Sami filme beaucoup de vidéos en 4K pour les publier sur un réseau social. Son téléphone est presque plein : il ne lui reste que 20 Go libres. On suppose qu'une minute de vidéo en 4K occupe environ 400 Mo (1 Go = 1 000 Mo).

1. Combien de minutes de vidéo en 4K Sami peut-il encore filmer ?
2. Sami pense acheter un téléphone neuf avec plus de mémoire. D'après le Document 6 du cours (on y reviendra au thème *Objets connectés*), un smartphone émet environ 80 kg CO_2e sur sa vie, presque tout à sa **fabrication**. Proposer deux solutions plutôt que d'acheter un téléphone neuf.
3. Sami publie une vidéo de 50 Mo, qui est vue 1 million de fois. Quelle quantité de données le réseau social doit-il transporter pour ces visionnages ? Donner le résultat en téraoctets (1 To = 10^6 Mo).
4. Même après avoir libéré son téléphone, Sami garde ses vidéos « dans le nuage ». Où sont-elles stockées en réalité, et pourquoi ce stockage a-t-il aussi un coût environnemental ?

▷ coup de pouce

★★ **Exercice 19** *Haute ou basse définition ?* ▶ corrigé p. 235

[sur papier] On admet qu'une heure de vidéo en ligne échange environ 3 Go de données en **haute définition** et environ 1 Go en **définition standard** (ordres de grandeur, voir le document du cours). Lina regarde 2 heures de vidéos par jour sur son téléphone.

1. Quelle quantité de données échange-t-elle en une semaine en haute définition ? En définition standard ?
2. Combien de fois moins de données consomme-t-elle en définition standard ?
3. Sur un petit écran de téléphone, la différence de qualité se voit peu. Quel conseil donner à Lina ? Pourquoi est-ce bon pour l'environnement ?

▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 20** *Une distance calculée par un assistant* ▶ corrigé p. 236

[sur papier] Un élève a décrit à un assistant d'IA le graphe d'amitiés de la section IV du cours et lui a demandé : « Quelle est la distance entre Alice et Farid ? » Voici la réponse obtenue :

> *Alice et Farid ne sont pas amis directement. Pour aller de l'un à l'autre, il faut passer par David, le sommet le plus connecté du réseau (degré 4). Un plus court chemin est Alice – Bilal – David – Farid (on peut aussi passer par Chloé à la place de Bilal). Ce chemin passe par quatre personnes : la distance entre Alice et Farid est donc de 4. À titre de comparaison, Ella et Farid sont amis, donc à distance 1.*

1. La réponse est-elle correcte ? Vérifier sur le graphe et relire la définition de la distance dans le cours.
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Python au service du thème ↑ cours

★★ **Exercice 21** *Python au service du thème — l'économie de l'attention* ▶ corrigé p. 236

▶ En Python (Spyder ou Basthon). Un réseau social affiche une publicité toutes les 4 minutes de visionnage : un utilisateur qui regarde 10 minutes voit donc 2 publicités (la troisième n'arrive qu'au bout de 12 minutes).

1. Écrire une fonction `revenu(utilisateurs, revenu_par_utilisateur)` qui renvoie le revenu annuel du réseau, en euros. La tester avec les données de l'exercice « Suivre l'argent » : on doit retrouver 6 milliards d'euros.
2. Écrire une fonction `nb_pubs(minutes)` qui renvoie le nombre de publicités vues en `minutes` minutes de visionnage. Que doivent renvoyer `nb_pubs(10)`, `nb_pubs(30)` et `nb_pubs(60)` ? Vérifier sur machine.
3. À l'aide d'une boucle `for`, afficher le nombre de publicités vues **en un an** (365 jours) par un utilisateur qui regarde 30, 60, 90, puis 120 minutes par jour. Expliquer, avec l'économie de l'attention, pourquoi le réseau cherche à vous garder le plus longtemps possible.

▷ coup de pouce ▷ début de solution

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relisez la colonne « Contenu principal » du tableau des ordres de grandeur, dans le cours. ↑énoncé

Ex. 2 – piste Une proportion s'obtient en divisant la partie par le tout. Pour la question 2 : ce chiffre est-il le même d'un jour à l'autre, et qui le publie ? ↑énoncé

Ex. 3 – piste Pour chaque action, demandez-vous : est-ce que je dis *qui* je suis, ou est-ce que je *prouve* que c'est bien moi (par quelque chose que je sais, que je possède ou que je suis) ? ↑énoncé

Ex. 4 – piste Une fois une photo ou un message publié, qui d'autre que vous peut y avoir accès ? Que peut-on en faire avant que vous ne le supprimiez ? ↑énoncé

Ex. 5 – piste Pensez à ce qu'un pirate essaierait en premier : des mots courants, des prénoms, des dates. Qu'est-ce qui rend un mot de passe long à deviner, même pour un ordinateur ? ↑

Ex. 6 – piste Relisez dans le cours l'encadré « Vos droits sur vos données » et l'encadré « Et à Monaco ? » : on y trouve les droits, l'âge à partir duquel on décide seul, et l'autorité de chaque pays. ↑énoncé

Ex. 7 – piste Relisez dans le cours la partie « Comment gagnent-ils de l'argent ? » : qui paie le réseau, et en échange de quoi ? ↑énoncé

Ex. 8 – piste Revenu total = nombre d'utilisateurs × revenu moyen par utilisateur ; gardez le mot « millions » dans le calcul, puis rappelez-vous que 1 000 millions font 1 milliard. ↑énoncé

Ex. 9 – piste Pour chaque action d'Hugo, demandez-vous ce qu'elle révèle : un goût, un besoin d'achat, un lieu de vie ? ↑énoncé

Ex. 10 – piste Le degré d'un sommet est le nombre d'arêtes qui en partent. Pour une distance, cherchez le chemin qui emprunte le moins d'arêtes possible. ↑énoncé

Ex. 11 – piste L'excentricité d'une personne : calculez sa distance à chacune des six autres et gardez la plus grande. ↑énoncé

Ex. 11 – début de solution Excentricité de Gabin : distances à Hana 1, à Ilyes 1, à Jade 2, à Kenza 3, à Lucas 3, à Maël 4 ; la plus grande vaut 4. Faites de même pour les six autres personnes, puis comparez les sept excentricités. ↑énoncé

Ex. 12 – piste Placez d'abord les cinq sommets, puis tracez une arête pour chaque relation « est ami avec » de l'énoncé (attention, il y en a cinq). ↑énoncé

Ex. 13 – piste Diamètre 4 : deux personnes doivent être séparées par 4 arêtes, avec seulement cinq personnes en tout. Diamètre 1 : que signifie « être à distance 1 de tout le monde » ? ↑

Ex. 13 – début de solution Question 1 : placez les cinq personnes « en file indienne », chacune amie seulement avec ses voisines. Question 2 : chacun est ami avec les quatre autres ; attention à ne pas compter deux fois la même arête. ↑énoncé

Ex. 14 – piste Comparez 3,5 avec les « cinq ou six intermédiaires » de l'expérience de Milgram. Les chaînes de connaissances se sont-elles allongées ou raccourcies, et pourquoi ? ↑énoncé

Ex. 15 – piste Relisez la partie du cours « Qui choisit ce que vous voyez ? ». Pour le but de l'algorithme, repensez au modèle économique : plus vous restez, plus… ↑énoncé

Ex. 16 – piste Être à distance 2, c'est ne pas être ami direct, mais être ami d'un de ses amis. Pour les amis en commun, écrivez la liste des amis de chacun et cherchez les noms présents dans les deux listes. ↑énoncé

Ex. 16 – début de solution Amis d'Alice : Bilal et Chloé. Amis de Bilal : Alice, Chloé, David ; amis de Chloé : Alice, Bilal, David. Dans ces listes, qui n'est ni Alice, ni déjà son ami ? ↑énoncé

Ex. 17 – piste Relisez la partie du cours « Bien vivre avec les réseaux sociaux » : elle décrit quoi faire (et ne pas faire) face au harcèlement. ↑énoncé

Ex. 18 – piste Question 1 : convertissez d'abord les 20 Go en mégaoctets. Question 3 : chaque visionnage renvoie toute la vidéo ; multipliez la taille du fichier par le nombre de vues. ↑

Ex. 19 – piste Calculez d'abord le nombre d'heures de vidéo regardées en une semaine, puis multipliez par la quantité de données échangée en une heure. ↑énoncé

Ex. 20 q. 2 – piste Recomptez sur le graphe en suivant mot à mot la définition de la longueur d'une chaîne donnée dans le cours. ↑énoncé

Ex. 21 – piste Question 2 : combien de « tranches » complètes de 4 minutes y a-t-il dans `minutes` ? Cherchez, dans le chapitre *Les bases de Python*, l'opérateur qui donne le quotient d'une division entière. ↑énoncé

Ex. 21 – début de solution Question 3 : avec `for i in range(1, 5):`, la variable `i` prend les valeurs 1, 2, 3, 4 ; la durée quotidienne vaut alors `30 * i`. Il reste à multiplier le nombre de publicités d'une journée par 365. ↑énoncé

DÉBAT

DÉBAT Faut-il interdire les réseaux sociaux aux moins de 15 ans, avec une vérification d'âge obligatoire ?

▶ corrigé p. 237

âge minimal et vérification de l'âge

Durée : 50 min

Consignes

- En classe entière. Les rôles sont tirés au sort : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement.
- Chaque argument s'appuie sur un **document** ci-dessous ou sur le **cours**. Préparation sans IA ni recherche en ligne.
- Avant de commencer, notez sur votre cahier votre position de départ : oui, non ou je ne sais pas.

Rappel du cours. En France comme à Monaco, avant **15 ans**, l'accord des parents est nécessaire pour s'inscrire à un service en ligne. Les grands réseaux vivent de la **publicité ciblée** et de l'**économie de l'attention** ; leurs **algorithmes de recommandation** choisissent ce que vous voyez. Vous avez, pour la plupart, environ 15 ans : ce débat vous concerne directement. Interdire, est-ce protéger, ou priver ?

Les deux camps

Groupe POUR — oui : pas de compte sur les réseaux sociaux avant 15 ans, et l'âge doit être vraiment vérifié.
Groupe CONTRE — non : ni interdiction générale, ni vérification d'âge obligatoire.

Les documents

Document 1 — Ce que disent les règles

Les grands réseaux sociaux fixent souvent un âge minimal de **13 ans**, contrôlé par une simple date de naissance déclarée. En France comme à Monaco, avant **15 ans**, un service en ligne doit avoir l'accord d'un parent pour utiliser vos données. Une loi française de 2023 impose de vérifier l'âge, mais son application a été retardée ; en 2026, une interdiction avant 15 ans a été censurée par le Conseil constitutionnel, qui l'a jugée disproportionnée.

À retenir : des règles existent, mais l'âge déclaré n'est presque jamais vérifié.

Source : RGPD, article 8 ; loi française n° 2023-566 du 7 juillet 2023 ; loi monégasque n° 1.565 du 3 décembre 2024, article 6 ; Conseil constitutionnel, décision n° 2026-911 DC du 14 août 2026.

Document 2 — Dans la vraie vie

Selon une étude de la CNIL publiée en 2021, la **première inscription** sur un réseau social a lieu en moyenne vers **8 ans et demi**. En avril 2024, une commission d'experts (médecins, chercheurs) a recommandé au président de la République française de ne pas donner accès aux réseaux sociaux avant 15 ans (sommeil, harcèlement). D'autres spécialistes rappellent que les réseaux sont aussi un lieu d'amitié, de création et d'information.

À retenir : on s'inscrit tôt, avec des risques mais aussi des bénéfices.

Source : CNIL, sondage réalisé avec l'IFOP (février 2020), résultats publiés en janvier 2021 ; rapport de la commission d'experts « Enfants et écrans. À la recherche du temps perdu », avril 2024.

2

Document 3 — L'Australie passe à l'acte

En Australie, depuis le 10 décembre 2025, une loi votée en 2024 interdit d'avoir un compte sur les principaux réseaux sociaux avant **16 ans**, **même avec l'accord des parents**. Ce sont les plateformes qui doivent empêcher les plus jeunes de s'inscrire, sous peine d'amendes pouvant atteindre 49,5 millions de dollars australiens. Ses opposants craignent des contournements (fausse date de naissance, compte d'un adulte) ou un repli vers des espaces moins surveillés.

À retenir : l'interdiction existe déjà dans un pays, à la charge des plateformes.

Source : Online Safety Amendment (Social Media Minimum Age) Act 2024, *Australie.*

Document 4 — Vérifier l'âge : à quel prix pour la vie privée ?

Vérifier vraiment un âge suppose de le **prouver** : copie de pièce d'identité, carte bancaire, organisme qui certifie l'âge… Tous ces moyens collectent des **données personnelles**, et concernent tout le monde, adultes compris. En 2022, la CNIL a recommandé le « **double anonymat** » : un tiers certifie votre âge sans savoir sur quel site vous allez, et le site reçoit seulement la réponse « plus de 15 ans : oui ou non », sans connaître votre identité.

À retenir : vérifier l'âge de chacun, c'est collecter des données sur tous.

Source : CNIL, analyse des dispositifs de vérification de l'âge en ligne, juillet 2022.

Rôles, déroulé minuté, règles de prise de parole et trace écrite : voir la page « Méthode : débattre » p. 7.

Tableau d'arguments

Recopier ce tableau sur le cahier et le compléter pendant le débat.

Argument POUR (oui)	Doc.	Argument CONTRE (non)	Doc.
…		…	

Trace écrite finale (individuelle)

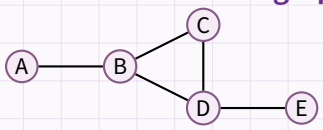
Sur le cahier, notez votre position après le débat (oui, non, je ne sais toujours pas) et si vous avez changé d'avis. Puis, en trois à cinq lignes, justifiez-la avec vos **deux meilleurs arguments**, dont **un argument de l'autre camp** que vous trouvez solide (avec le document qui l'appuie).

FICHE DE RÉVISION

L'essentiel

Réseau social	profils + liens + partage ; les plus gros : quelques *milliards* d'utilisateurs (un *ordre de grandeur*).
Identité numérique	tout ce qui vous représente en ligne, durable et souvent public par défaut ; elle forge votre **e-réputation**.
Identifier / authentifier	annoncer qui l'on est (identifiant) / *prouver* que c'est bien soi (mot de passe, code SMS : **secret**).
Données personnelles	droits d'**accès**, de **rectification**, d'**effacement** ; seul dès **15 ans** ; **RGPD** et **CNIL** (France), **loi n° 1.565** et **APDP** (Monaco).
Modèle économique	**publicité ciblée** : le réseau vend votre **attention** et vos **données** (économie de l'attention).
Recommandation	un **algorithme** choisit et ordonne votre fil pour vous retenir ; risque : **bulle de filtres**.

Méthode : mesurer un graphe



sommet	A	B	C	D	E
degré	1	3	2	3	1
excentricité	3	2	2	2	3

- **Degré** : nombre d'arêtes qui partent du sommet (le nombre d'amis).
- **Distance** : longueur de la **plus courte** chaîne (nombre d'arêtes). Ici A et E sont à distance **3** : A–B–D–E.
- **Excentricité** d'un sommet : sa distance au sommet le *plus éloigné*.
- **Diamètre** = plus grande excentricité = **3** ; **rayon** = plus petite = **2** ; **centre** = sommet(s) d'excentricité minimale : **B, C, D**.

Repères et erreurs à éviter

Milgram (1967) : 5 à 6 intermédiaires → « **six degrés de séparation** » ; Facebook (2016) : distance moyenne ≈ **3,5**. Meta : publicité ≈ **98 %** du chiffre d'affaires (2023). Numérique ≈ **2 à 4 %** des émissions mondiales ; vidéo = majorité du trafic ; garder ses appareils. Cyberharcèlement **interdit** : preuves, bloquer, signaler, en parler (Monaco : **98 98 96 96**). *Pièges :* prendre une chaîne quelconque pour la distance (c'est la **plus courte**) ; compter les sommets au lieu des **arêtes** ; confondre diamètre (plus *grande* excentricité) et rayon (plus *petite*) ; croire que le centre est le sommet de plus grand degré.

Savoir-faire à maîtriser — je sais…

- ☐ définir un **réseau social** et citer ses principaux usages ;
- ☐ protéger mon **identité numérique** et régler mes paramètres de confidentialité ;
- ☐ citer mes **droits** sur mes données personnelles (RGPD en France, loi n° 1.565 à Monaco) ;
- ☐ expliquer le **modèle économique** (données personnelles, publicité ciblée) ;
- ☐ modéliser un réseau social par un **graphe** (sommets = comptes, arêtes = relations) et lire le **degré** d'un sommet ;
- ☐ calculer une **distance**, l'**excentricité** d'un sommet, puis le **diamètre**, le **rayon** et le **centre** d'un graphe ;
- ☐ expliquer l'idée des « **six degrés de séparation** » (petit monde) ;
- ☐ expliquer qu'un **algorithme** choisit et ordonne ce qui est affiché (fil, recommandations) ;
- ☐ citer des ordres de grandeur de l'**impact environnemental** du numérique et des gestes de sobriété ;
- ☐ adopter de bonnes pratiques (temps d'écran, esprit critique, lutte contre le harcèlement).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 238

1. Classer en identification ou authentification : taper son pseudo ; entrer un code reçu par SMS.
2. Expliquer « si c'est gratuit, c'est que le produit, c'est vous ».
3. Dans le graphe de la fiche, ajouter l'arête A–E : donner le degré de A et la distance A–E.
4. Citer un risque des algorithmes de recommandation.

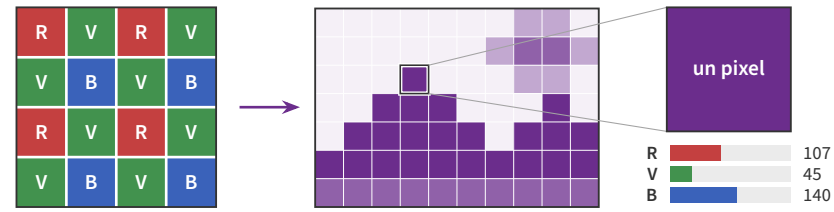
Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 238

1. Donner le type de `"12"` et celui de `12` (ch. 1).
2. Donner la valeur de `x` après `x = 5` puis `x = x + 1` (ch. 1).

3

CHAPITRE 3

La photographie numérique



le capteur (filtre de Bayer) l'image : des pixels

DANS CE CHAPITRE

- Cours 82
- Exercices 90
- Projets et débat 95
- Fiche de révision 105
- Corrigés 239

COURS

3

Programme Du capteur au pixel ; codage des couleurs (RVB) ; définition, résolution et taille d'un fichier ; traitement d'image *par programme* ; métadonnées EXIF ; rôle des algorithmes dans les appareils.

Idée Une photo numérique n'est qu'un immense tableau de **nombres**. Comprendre ces nombres, c'est pouvoir *lire*, *fabriquer* et *transformer* une image **soi-même**, avec quelques lignes de Python.

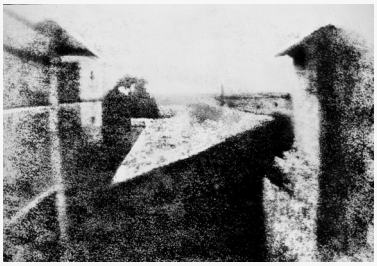
Objectifs Expliquer comment un capteur forme une image ; coder une couleur en RVB ; calculer définition, résolution et poids ; modifier une image pixel par pixel ; lire les métadonnées et en mesurer l'enjeu pour la vie privée.

Situation de départ

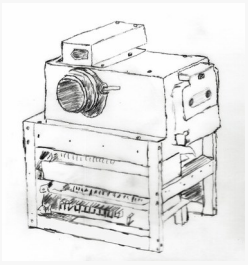
1. Votre téléphone prend des photos de « 12 millions de pixels ». Qu'est-ce qu'un pixel, et pourquoi une photo pèse-t-elle plusieurs mégaoctets ?
2. Vous publiez une photo prise dans votre chambre. Un inconnu peut-il savoir où et quand elle a été prise, alors que rien ne se voit sur l'image ?
3. Comment une application met-elle votre photo en noir et blanc en une fraction de seconde ?

Pour les curieux — *Un peu d'histoire* (facultatif)

La toute première photographie est prise vers **1826** par le Français **Nicéphore Niépce** (« Point de vue du Gras ») : il a fallu *plusieurs heures* de pose ! En **1888**, **George Eastman** lance l'appareil **Kodak** avec un slogan resté célèbre : « *Vous appuyez sur le bouton, nous faisons le reste.* » Mais la vraie révolution est de **1975** : l'ingénieur **Steven Sasson**, chez Kodak, construit le **premier appareil photo numérique**. Il pesait environ 3,6 kg, enregistrait sur une cassette en **23 secondes**… une image de 100 × 100 pixels en noir et blanc ! (IEEE, 2022) Aujourd'hui, le capteur qui tient dans votre téléphone compte plus de **mille fois** plus de pixels (souvent 12 millions ou davantage, contre 10 000), en couleurs.



« Point de vue du Gras » (Niépce, vers 1826)



Le premier appareil numérique (Sasson, 1975)

Sources : Harry Ransom Center (université du Texas à Austin), qui conserve le « Point de vue du Gras » de Nicéphore Niépce (vers 1826–1827), `hrc.utexas.edu` ; George Eastman Museum (Rochester), histoire de l'appareil Kodak de 1888 et de son slogan ; IEEE, *Milestone* « Handheld Digital Camera, 1975 » (appareil de Steven Sasson : environ 3,6 kg, image de 100 × 100 pixels enregistrée en 23 s), inaugurée en 2022, `ethw.org`.

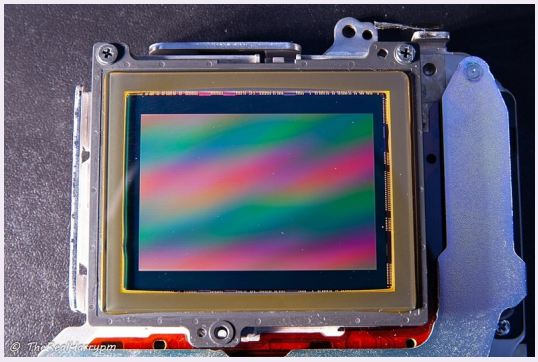
→ frise chronologique p. 10

I De la lumière à l'image : le capteur

Autrefois, l'appareil photo contenait une **pellicule** : une surface sensible à la lumière, qu'il fallait ensuite développer chimiquement en laboratoire. Aujourd'hui, la pellicule est remplacée par un **capteur** électronique.

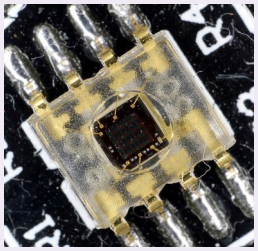
Définition 1

Un **capteur** est une surface couverte de millions de petites cellules, les **photosites**. Chaque photosite mesure la **quantité de lumière** qu'il reçoit et la transforme en un **nombre** (c'est la *numérisation*) : plus il reçoit de lumière, plus le nombre est grand.

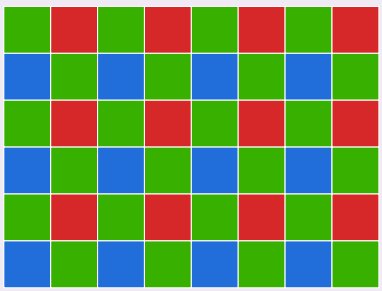


Le capteur d'un appareil photo hybride, démonté (un capteur « CMOS », du nom de la technologie de fabrication utilisée par presque tous les appareils actuels) : la surface rectangulaire centrale porte des dizaines de millions de photosites.

Seul, un photosite ne distingue pas les couleurs : on obtiendrait une image en **niveaux de gris**. Pour capter la couleur, on place devant les photosites un **filtre de Bayer** : des petits filtres rouges, verts et bleus. Fait amusant, il y a **deux fois plus de filtres verts** que de rouges ou de bleus, car notre œil est plus sensible au vert.



Un capteur vu de très près : la mosaïque du filtre coloré



Filtre de Bayer : 1 photosite rouge, 1 bleu et *2 verts* par groupe de quatre.

▶ Activité 1 — Expliquer le capteur

Rédiger, en quelques lignes, un résumé qui explique à un camarade comment un capteur d'appareil photo fabrique une image (mots à réutiliser : *photosite*, *lumière*, *nombre*, *filtre de Bayer*).

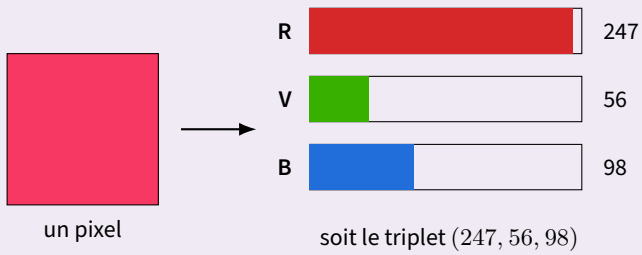
▶ **Exercices d'application :** exercice 1 (le capteur et la formation de l'image)

II Le pixel et le codage des couleurs (RVB)

L'image finale est un quadrillage de points colorés : les **pixels** (de l'anglais *picture element*).

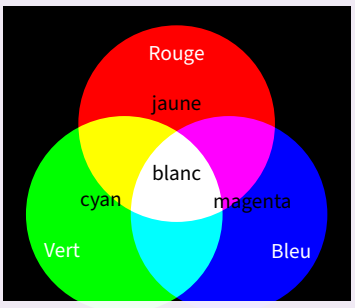
Définition 2

Chaque **pixel** porte une couleur codée par **trois nombres** entre **0 et 255** : la quantité de **R**ouge, de **V**ert et de **B**leu (système **RVB**, ou *RGB* en anglais). Exemple : (247, 56, 98). C'est la **synthèse additive** : en mélangeant de la lumière rouge, verte et bleue, on obtient toutes les autres couleurs.



Exemple

Quelques couleurs utiles : rouge = (255,0,0) vert = (0,255,0) bleu = (0,0,255) **noir** = (0,0,0) **blanc** = (255,255,255) jaune = (255,255,0). Quand les trois canaux sont *égaux*, on obtient un **gris**.



La synthèse additive : rouge + vert = jaune, les trois ensemble = blanc.

▶ Activité 2 — Jouer avec les couleurs

1. Avec 256 valeurs possibles par canal (de 0 à 255) et 3 canaux, calculer le nombre total de couleurs différentes.
2. Donner le code RVB pour obtenir : du **noir**, du **blanc**, du **jaune** (rouge + vert), du **violet** (rouge + bleu).
3. Que voit-on à l'écran avec le code (125,125,125) ? Et avec (30,30,30) ?

★ Projet — Dessiner des drapeaux avec Python

Maintenant que vous savez coder une couleur, vous allez **fabriquer** des images de zéro : des drapeaux (Monaco, France, puis d'autres), en respectant les **proportions** et les **couleurs officielles**.
→ voir le projet *« Dessiner des drapeaux avec Python »*, en fin de chapitre.

► **Exercices d'application :** exercices 2 à 4 (le pixel et le codage RVB)

III Définition, résolution et poids d'une image

Définition 3

La **définition** d'une image est son nombre de pixels : largeur × hauteur. Une image 800×600 contient $800 \times 600 = 480\,000$ pixels.

La **résolution** est le nombre de pixels par unité de longueur, souvent en **ppp** (pixels par pouce, ou *dpi*). Un pouce vaut 2,54 cm. Plus la résolution est élevée, plus l'image est nette à l'impression.

Règle 1

Une image en couleurs « brute » stocke 3 nombres par pixel (R, V, B), chacun sur **1 octet** (un **octet** est la petite unité de mémoire qui permet de ranger un nombre entier de 0 à 255). Son **poids** vaut donc environ :

$$\text{poids} = \text{largeur} \times \text{hauteur} \times 3 \text{ octets}.$$

(Les formats comme le JPEG *compressent* ensuite l'image pour réduire ce poids.)

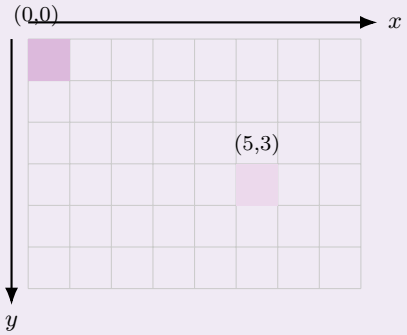
► Activité 3 — Calculer sur les images

1. Une photo de définition 1200×900 est tirée au format 20×15 (en cm). Calculer sa résolution en ppp (rappel : 1 pouce = 2,54 cm).
2. Pour une belle impression, il faut environ 300 ppp. Calculer la définition minimale d'une photo imprimée en 15×10 cm.
3. Calculer le poids « brut » d'une photo de définition 4000×3000 (en méga-octets ; 1 Mo $\approx 10^6$ octets).

► **Exercices d'application :** exercices 5 à 9 (définition, résolution, poids)

IV Modifier une image par programme (Python + PIL)

Une image étant un tableau de nombres, on peut la **transformer** avec un programme. On utilise la bibliothèque **PIL**. Chaque pixel est repéré par des coordonnées (x, y) : (0,0) est en **haut à gauche**.



Le pixel (0,0) est en haut à gauche ; x va vers la droite, y vers le bas.

Les programmes de ce chapitre utilisent l'image `terre.jpg` (500 × 500 pixels) : c'est la célèbre photo de la Terre « *The Blue Marble* » prise par l'équipage d'Apollo 17 en 1972, dans le domaine public. Elle est à télécharger ici : `https://github.com/MathThibaud/nsi-fichiers-eleves/tree/main/seconde/03-photo-image-terre` ; à défaut, n'importe quelle photo `.jpg` renommée `terre.jpg` convient. Elle doit être rangée dans le même dossier que le programme.

La ligne `from PIL import Image` charge l'outil `Image` de la bibliothèque **PIL**, un peu comme `import turtle as t` chargeait la tortue ; on écrit ensuite `Image.open(...)` pour ouvrir une image.

Lire un pixel avec `getpixel`, le **modifier** avec `putpixel` :

```
from PIL import Image
img = Image.open("terre.jpg")        # l'image doit etre dans le meme dossier
img = img.convert("RGB")             # 3 canaux R, V, B (utile pour un PNG)

r, v, b = img.getpixel((100, 250))   # lire le pixel (100, 250)
print("rouge :", r, "vert :", v, "bleu :", b)

img.putpixel((250, 250), (255, 0, 0)) # colorier ce pixel en rouge
img.show()                            # afficher l'image
```

Pour parcourir *toute* l'image, on utilise **deux boucles `for`** imbriquées :

```
from PIL import Image
img = Image.open("terre.jpg").convert("RGB")
largeur, hauteur = img.size          # taille de l'image, en pixels

for y in range(hauteur):
    for x in range(largeur):
        r, v, b = img.getpixel((x, y))
        img.putpixel((x, y), (v, b, r))   # on decale les canaux
img.show()
```

▶ Activité 4 — Ses premières retouches

Saisir et tester les programmes ci-dessus dans Spyder (ou Basthon), avec l'image `terre.jpg`. Puis :

1. Modifier le premier programme pour afficher les canaux du pixel (250,300).
2. Expliquer, en une phrase, l'effet du programme à deux boucles (le décalage des canaux).

3. Après une petite recherche sur le **négatif** d'une image, écrire un programme qui *inverse* chaque canal (indice : remplacer r par $255 - r$, etc.).

4. Après une recherche sur les **niveaux de gris**, écrire un programme qui rend l'image grise (indice : remplacer R, V et B par leur *moyenne* $(r + v + b)//3$).

Avec un `if`, on peut ne modifier *que certains* pixels (par exemple selon leur position ou leur couleur) : c'est la base de la retouche « localisée ».

★ Projet — Retoucher un tableau célèbre

Vous appliquerez ces techniques à de vraies œuvres d'art (du domaine public) : changer la couleur du ciel de *La Nuit étoilée* de Van Gogh, ou celle d'un vêtement dans un portrait.
→ voir le projet *« Retoucher un tableau célèbre en RVB »*, en fin de chapitre.

► **Exercices d'application :** exercices 10 à 15 (modifier une image en Python)

V Les métadonnées (EXIF) et la vie privée

Un fichier photo ne contient pas *que* l'image. Il embarque aussi des informations sur la prise de vue : ce sont les **métadonnées EXIF**.

Définition 4

Les **métadonnées EXIF** sont des données *cachées* rangées dans le fichier : date et heure, marque et modèle de l'appareil, réglages, définition… et, si l'appareil possède un GPS (tous les smartphones), les **coordonnées du lieu** de la prise de vue.

On peut les lire avec un petit programme Python :

```
from PIL import Image, ExifTags
img = Image.open("photo.jpg")
print("definition :", img.size)              # (largeur, hauteur)
exif = img.getexif()                         # metadonnees : numero -> valeur
for numero in exif:
    nom = ExifTags.TAGS.get(numero, numero)  # nom lisible de la metadonnee
    print(nom, ":", exif[numero])
```

Ce programme s'utilise tel quel, sans chercher à en comprendre chaque ligne (la ligne `ExifTags.TAGS.get(...)` va chercher le nom de la métadonnée dans une table fournie par PIL). Pour le GPS, il n'affiche qu'un simple nombre (par exemple `GPSInfo : 70`) : les coordonnées sont rangées à part, dans un sous-dossier des métadonnées ; les logiciels de photo et les sites de lecture EXIF les affichent en clair, comme dans le tableau ci-dessous.

Chaque métadonnée est rangée sous un **numéro** (par exemple 271 pour le fabricant, 272 pour le modèle, 306 pour la date) ; `ExifTags.TAGS` donne le **nom** qui correspond à chaque numéro (`Make`, `Model`, `DateTime`…).

Métadonnée EXIF	Valeur (exemple fictif)
Date et heure de la prise de vue	2026:05:14 16:42:07
Fabricant / modèle	Smartphone (modèle quelconque)
Définition	4032×3024
Temps de pose / sensibilité	1/120 s / ISO 100
Latitude GPS	$43° 43′ 50″$ N
Longitude GPS	$7° 25′ 31″$ E
Altitude GPS	12 m

Extrait de métadonnées EXIF (valeurs fictives). En rouge : les coordonnées GPS, qui suffisent à situer la prise de vue sur une carte.

Règle 2

Publier une photo prise au smartphone peut **révéler sans le vouloir** où et quand elle a été prise (domicile, école…). Beaucoup de réseaux sociaux retirent l'EXIF à la publication, mais **pas tous** : dans le doute, on peut *effacer les métadonnées* avant de partager.

▶ Activité 5 — Enquêter sur une photo

1. Sur une photo fournie par le professeur, retrouver à l'aide du programme : la **date** de prise de vue, le **fabricant** de l'appareil, la **définition**.
2. Expliquer en quoi la présence de coordonnées GPS dans une photo peut poser un **problème de vie privée**.

▶ **Exercices d'application :** exercices 16 **et** 17 (métadonnées EXIF et vie privée), exercice 18 (critiquer une réponse d'IA)

VI Les algorithmes dans nos appareils

La photo « parfaite » du smartphone n'est pas seulement captée : elle est **calculée**. Des algorithmes interviennent après la prise de vue :

- **mise au point** et **exposition** automatiques ;
- **HDR** : combinaison de plusieurs clichés pour équilibrer ombres et lumières ;
- **débruitage**, accentuation, **filtres** et embellissement automatique ;
- détection de visages, *portraits* avec flou d'arrière-plan.

Règle 3

Une image numérique est **facile à modifier** : une retouche peut être invisible. Une photo n'est donc plus une **preuve** en soi. Avec l'IA (images générées, *deepfakes*), il faut plus que jamais garder un **esprit critique** : d'où vient l'image ? qui l'a publiée ?

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Déjà vu :** une photo est un tableau de nombres, que l'on parcourt et modifie avec les **boucles** et les **conditions** du chapitre *Les bases de la programmation Python*.
- **À venir :** les coordonnées des métadonnées viennent du récepteur **GPS** (chapitre *Localisation, cartographie et mobilité*), et le capteur d'image rejoindra les autres **capteurs** au chapitre *Informatique embarquée et objets connectés*.
- **Plus loin :** chaque valeur de 0 à 255 tient sur un octet, soit 8 **bits** (des 0 et des 1) ; l'écriture binaire s'étudie en spécialité NSI de Première.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Capteur / photosite	mesure la lumière → un nombre ; filtre de Bayer (2× plus de vert).
Pixel	un point de l'image, codé en **RVB** : 3 nombres de 0 à 255.
Synthèse additive	R + V + B → toutes les couleurs ; 3 canaux égaux = gris.
Définition	nombre de pixels = largeur × hauteur.
Résolution	pixels par pouce (ppp / dpi) ; 1 pouce = 2,54 cm.
Poids brut	largeur × hauteur ×3 octets (le JPEG compresse).
Coordonnées	pixel (x,y) ; (0,0) en haut à gauche.
PIL	`getpixel` (lire), `putpixel` (écrire), 2 boucles `for` pour tout parcourir.
Négatif / gris	$255 - c$ pour chaque canal ; moyenne $(r+v+b)//3$ pour le gris.
EXIF	métadonnées cachées (date, appareil, **GPS**) → enjeu de vie privée.
Algorithmes	HDR, filtres, retouche ; une photo n'est plus une preuve → esprit critique.

Retour à la situation de départ

1. Un **pixel** est un point de l'image, codé par 3 nombres **RVB** de 0 à 255 (§ II). À un octet par nombre, 12 millions de pixels (la **définition**) font environ 36 Mo « bruts », que le JPEG **compresse** en quelques Mo (§ III).
2. Oui, souvent : les **métadonnées EXIF** cachées dans le fichier donnent la date, l'appareil et, sur un smartphone, les **coordonnées GPS** du lieu (§ V). Tous les réseaux ne les retirent pas : dans le doute, on les efface avant de partager.
3. Une image est un tableau de nombres : un programme parcourt chaque pixel avec deux boucles `for` et remplace R, V et B par leur moyenne $(r+v+b)//3$ (§ IV) ; trois canaux égaux donnent un **gris** (§ II).

Débat argumenté

Un **débat**, proposé en fin de chapitre, accompagne ce thème : « Faut-il obliger à signaler toute photo retouchée, dans la publicité comme sur les réseaux sociaux ? » (documents, rôles, tableau d'arguments et trace écrite, environ 50 min).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ expliquer comment un **capteur** transforme la lumière en image numérique ;
- ☐ décrire une image par ses **pixels** et coder une couleur en **RVB** (3 valeurs de 0 à 255) ;
- ☐ distinguer **définition**, **résolution** et **poids** d'une image, et estimer un poids ;
- ☐ modifier une image par un programme **Python** (parcourir et changer les pixels avec la bibliothèque PIL) ;
- ☐ expliquer ce que sont les métadonnées **EXIF** et le risque qu'elles posent pour la vie privée ;
- ☐ citer des traitements automatiques (algorithmes) réalisés par un appareil photo ou un smartphone.

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- [sur papier] *signale les exercices à faire sur papier* ; ▶ *ceux où l'on écrit et teste un vrai programme* (Spyder ou Basthon, avec la bibliothèque PIL).
- Rappel utile : une couleur = trois nombres **RVB** entre 0 et 255 ; le pixel (0,0) est en **haut à gauche**.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Le capteur et la formation de l'image ↑cours

★ **Exercice 1** *Vrai ou faux* ▶ corrigé p. 240 • essentiel

[sur papier] Répondre par **vrai** ou **faux**, et corriger les phrases fausses.

a. Un photosite mesure une couleur et la transforme directement en trois nombres.
b. Il y a autant de filtres rouges que de filtres verts sur un capteur.
c. Sans filtre de couleur, un capteur donnerait une image en niveaux de gris.
d. Plus un photosite reçoit de lumière, plus le nombre qu'il produit est grand.

▷ coup de pouce

Le pixel et le codage RVB ↑cours

★ **Exercice 2** *Nommer la couleur* ▶ corrigé p. 240 • essentiel

[sur papier] Donner la couleur obtenue par chacun des codes RVB suivants :

(255,0,0) (0,0,0) (255,255,255) (0,255,0) (255,255,0) (40,40,40). ▷ coup de pouce

★ **Exercice 3** *Coder une couleur* ▶ corrigé p. 240

[sur papier] Donner un code RVB plausible pour : un **gris clair**, du **cyan** (vert + bleu), un **bleu très sombre**, un **vert moyen** (le vert seul, à moitié de sa force), de l'**orange** (beaucoup de rouge, un peu moins de vert), un **rose pâle** (du rouge éclairci par du blanc). ▷ coup de pouce

★★ **Exercice 4** *Combien de couleurs ?* ▶ corrigé p. 240 • essentiel

[sur papier] Un ancien format d'image codait chaque canal (R, V, B) par une valeur entière de 0 à 15 seulement, soit 16 possibilités.

1. Combien de couleurs différentes peut-on coder en tout ? Donner le calcul.
2. Combien de nuances de **gris** peut-on coder (les trois canaux égaux) ?

▷ coup de pouce

Définition, résolution et poids ↑cours

★★ **Exercice 5** *Compter les pixels* ▶ corrigé p. 241 • essentiel

[sur papier] Une image a pour définition 1920×1080.

1. Combien de pixels contient-elle ? Donner un ordre de grandeur en millions.
2. Calculer son **poids brut** (3 octets par pixel), en méga-octets (1 Mo $\approx 10^6$ octets).

▷ coup de pouce

★★ **Exercice 6** *Doubler la définition* ▶ corrigé p. 241

[sur papier] L'image A a pour définition 800×600 ; l'image B, la même scène photographiée en « double définition », mesure 1600×1200.

1. Combien de pixels contient chaque image ?
2. Combien de fois plus de pixels l'image B contient-elle ? Est-ce « deux fois plus » ?
3. Calculer le poids brut de l'image B en Mo (3 octets par pixel).

▷ coup de pouce

★★ **Exercice 7** *Résolution d'une impression* ▶ corrigé p. 241 • essentiel

[sur papier] On imprime une photo de définition 1500×1000 sur du papier de $12{,}7 \times 8{,}47$ cm.

1. Convertir la largeur en **pouces** (1 pouce = 2,54 cm).
2. En déduire la **résolution** de l'impression en ppp. Le résultat convient-il pour une « belle » impression (environ 300 ppp) ?

▷ coup de pouce

★★★ **Exercice 8** *Imprimer en grand* ▶ corrigé p. 241 ≈ 15 min

[sur papier] Un appareil photo donne des images de 4000×3000 pixels.

1. Quelle est la plus grande taille d'impression (en cm) possible pour une « belle » impression à 300 ppp ?
2. On veut en faire une affiche de 60 cm de large. Quelle sera la résolution de l'impression ? Est-ce une « belle » impression ? Pourquoi est-ce malgré tout acceptable pour une affiche ?

▷ coup de pouce ▷ début de solution

★★ **Exercice 9** *Compression* ▶ corrigé p. 241

[sur papier] Une photo 4000×3000 pèse, en réalité, 4 Mo sur la carte mémoire, alors que son poids « brut » est bien plus grand. Calculer le poids brut, puis expliquer d'où vient la différence.

▷ coup de pouce

Modifier une image par programme (PIL) ↑cours

★★ **Exercice 10** *Lire un pixel* ▶ corrigé p. 241

▶ À l'aide de l'image `terre.jpg` du cours (« The Blue Marble »), écrire un programme qui affiche les trois canaux R, V, B du pixel de coordonnées $(120, 200)$. ▷ coup de pouce

★★ **Exercice 11** *Que fait ce programme ?* ▶ corrigé p. 241

[sur papier] On considère le programme suivant, appliqué à une image de 500 × 500 :

```
from PIL import Image
img = Image.open("terre.jpg")
for y in range(500):
    for x in range(500):
        r, v, b = img.getpixel((x, y))
        img.putpixel((x, y), (0, v, b))
img.show()
```

1. Que fait ce programme pour chaque pixel ?
2. Quel sera l'effet visible sur l'image ? Justifier.

▷ coup de pouce

★★ **Exercice 12** *Le négatif* ▶ corrigé p. 242 • essentiel

▶ Écrire un programme qui transforme en **négatif** (chaque canal c est remplacé par $255 - c$) seulement la **moitié gauche** de `terre.jpg` ; la moitié droite reste intacte. Tester, puis afficher le résultat.

▷ coup de pouce

★★ **Exercice 13** *Éclaircir l'image* ▶ corrigé p. 242

▶ Écrire un programme qui **éclaircit** `terre.jpg` en ajoutant 60 à chacun des trois canaux de chaque pixel. Attention, un canal ne peut pas dépasser 255 : on écrira par exemple `min(255, r + 60)`, qui vaut le plus petit des deux nombres. Que deviennent les zones déjà très claires ?

▷ coup de pouce

★★★ **Exercice 14** *Niveaux de gris* ▶ corrigé p. 242 ≈ 15 min

▶ Notre œil est plus sensible au vert (c'est pourquoi le filtre de Bayer compte deux fois plus de filtres verts). Écrire un programme qui rend l'image **grise** en en tenant compte : remplacer R, V et B par $m = (3r + 6v + b)$ // 10. Vérifier qu'une image grise a bien ses trois canaux égaux. Calculer enfin à la main m pour le pixel $(100, 200, 50)$, et le comparer à la simple moyenne $(r + v + b)$ // 3 vue dans le cours.

▷ coup de pouce ▷ début de solution

★★★ **Exercice 15** *Le miroir* ▶ corrigé p. 243 ≈ 20 min

▶ Écrire un programme qui retourne `terre.jpg` comme dans un **miroir** : la colonne de gauche passe à droite, et inversement. On ne crée pas de nouvelle image : on **échange** les pixels deux à deux.

▷ coup de pouce ▷ début de solution

Métadonnées et vie privée ↑ cours

★ **Exercice 16** *Les données cachées* ▶ corrigé p. 243 • essentiel

[sur papier]

1. Qu'appelle-t-on les **métadonnées EXIF** d'une photo ? Citer trois informations qu'elles peuvent contenir.
2. En quoi la présence de coordonnées **GPS** dans une photo publiée peut-elle poser un problème de vie privée ?
3. Proposer une précaution simple avant de partager une photo prise au smartphone.

▷ coup de pouce

★ **Exercice 17** *Une photo est-elle une preuve ?* ▶ corrigé p. 243

[sur papier] « Puisque c'est une photo, c'est forcément vrai. » Discuter cette affirmation en une ou deux phrases (on pourra parler de retouche, de filtres et d'images générées par IA). ▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 18** *Poids et résolution selon un assistant* ▶ corrigé p. 243

[sur papier] Un élève a demandé à un assistant d'IA : « Mon téléphone prend des photos de 6000×4000 pixels. Combien pèsent-elles, et puis-je les imprimer en 30×20 cm ? » Voici la réponse obtenue :

> *Votre photo contient* $6000 \times 4000 = 24\,000\,000$ *pixels, soit 24 mégapixels. Chaque pixel est codé par trois nombres (rouge, vert, bleu) sur un octet chacun, donc 3 octets par pixel : le poids brut est* $24\,000\,000 \times 3 = 72\,000\,000$ *octets, soit 72 Mo. En pratique, le fichier JPEG est compressé et pèse plutôt 5 à 10 Mo. Pour l'impression, aucune inquiétude : sur une largeur de 30 cm, la résolution de votre photo est de 6000 ppp, bien au-dessus des 300 ppp recommandés.*

1. La réponse est-elle correcte ? Refaire chaque calcul (rappel : 1 pouce = 2,54 cm).
2. Localiser l'erreur, nommer la notion confondue et donner la valeur juste. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Coups de pouce

après avoir vraiment cherché : la ***piste*** *d'abord, le* ***début de solution*** *si besoin.*

Ex. 1 – piste Relisez la partie du cours sur le capteur et le schéma du filtre de Bayer : que mesure un photosite à lui seul ? Comptez les filtres de chaque couleur dans un groupe de quatre. ↑

Ex. 2 – piste Pour chaque code, repérez les canaux « allumés » (proches de 255) et les canaux « éteints » (0), puis aidez-vous du schéma de la synthèse additive du cours. ↑ énoncé

Ex. 3 – piste Partez des couleurs de l'exemple du cours et du schéma de la synthèse additive. Pour un gris, que doivent avoir en commun les trois nombres ? Pour une couleur sombre, les nombres sont-ils petits ou grands ? Pour éclaircir une couleur, on lui « ajoute du blanc » : que deviennent les canaux qui valaient 0 ? ↑ énoncé

Ex. 4 – piste Pour chaque valeur du rouge, on peut choisir n'importe quelle valeur du vert, puis n'importe quelle valeur du bleu : les possibilités se multiplient. Pour un gris, une fois le rouge choisi, reste-t-il un choix pour le vert et le bleu ? ↑ énoncé

Ex. 5 – piste Nombre de pixels = largeur × hauteur. Poids brut = nombre de pixels ×3 octets ; divisez ensuite par 10^6 pour obtenir des Mo. ↑ énoncé

Ex. 6 – piste Calculez le nombre de pixels de chaque image (largeur × hauteur) *avant* de comparer : la largeur double, mais la hauteur aussi. ↑ énoncé

Ex. 7 – piste Largeur en pouces = largeur en cm divisée par 2,54. La résolution est un nombre de pixels *par pouce* : divisez le nombre de pixels de la largeur par la largeur en pouces. ↑

Ex. 8 – piste À 300 ppp, chaque pouce imprimé utilise 300 pixels : combien de pouces peut-on remplir avec les 4000 pixels de la largeur ? ↑ énoncé

Ex. 8 – début de solution Largeur : $4000/300 \approx 13{,}3$ pouces, à convertir en cm (multiplier par 2,54). Faites de même pour la hauteur. Pour l'affiche, convertissez d'abord 60 cm en pouces. ↑ énoncé

Ex. 9 – piste Calculez le poids brut comme à l'exercice « Compter les pixels », puis comparez-le aux 4 Mo. Quel traitement le format JPEG fait-il subir à l'image ? ↑ énoncé

Ex. 10 – piste Repartez du premier programme du cours : `Image.open`, puis `getpixel((x, y))`, qui renvoie les trois canaux du pixel. ↑énoncé

Ex. 11 – piste Comparez le triplet lu, `(r, v, b)`, et le triplet écrit : quel canal a changé, et quelle valeur prend-il ? ↑énoncé

Ex. 12 – piste Reprenez le programme à deux boucles du cours (celui qui décale les canaux). Deux changements : la ligne `putpixel`, et les valeurs que prend `x` (jusqu'où aller pour rester dans la moitié gauche ?). ↑énoncé

Ex. 13 – piste Même structure que le négatif : seul le calcul de la nouvelle couleur change, canal par canal. ↑énoncé

Ex. 14 – piste Deux boucles, comme pour le négatif. Dans la boucle, calculez d'abord `m`, puis écrivez un pixel dont les trois canaux valent `m`. ↑énoncé

Ex. 14 – début de solution Dans les deux boucles :

```
r, v, b = img.getpixel((x, y))
m = (3 * r + 6 * v + b) // 10
```

(il reste à écrire le pixel gris avec `putpixel`.) ↑énoncé

Ex. 15 – piste Le pixel de la colonne `x` échange sa place avec celui de la colonne `largeur - 1 - x`, sur la même ligne. Ne parcourez que la moitié gauche de l'image, sinon chaque échange serait aussitôt défait. ↑énoncé

Ex. 15 – début de solution

```
for y in range(hauteur):
    for x in range(largeur // 2):
        gauche = img.getpixel((x, y))
        droite = img.getpixel((largeur - 1 - x, y))
```

↑énoncé

Ex. 16 – piste Relisez dans le cours la définition des métadonnées EXIF, le tableau d'exemple et la règle qui le suit. ↑énoncé

Ex. 17 – piste Pensez à tout ce qui peut modifier une image entre la prise de vue et sa publication, et à la partie du cours sur les images *calculées* par les smartphones. ↑énoncé

Ex. 18 q. 2 – piste Relisez la définition de la résolution dans le cours : en quelle unité s'exprime-t-elle ? Convertissez 30 cm en pouces. ↑énoncé

PROJETS ET DÉBAT

PROJET 1 Dessiner des drapeaux avec Python

Idée Jusqu'ici, on *modifiait* des images. Ici, on va en **fabriquer** à partir de rien : des drapeaux.

Objectif Réinvestir le codage **RVB** en respectant les **proportions** et les **couleurs officielles** de chaque drapeau.

Outil la bibliothèque `PIL` (`Image.new`), les **boucles** du cours, et le codage **RVB** des couleurs.

IA



Autorisé : demander une explication (le rôle d'une fonction de `PIL`, le sens d'un message d'erreur). **Interdit** : faire écrire le programme par une IA. Vous devez pouvoir expliquer chaque ligne de votre code.

1 Créer une image et la remplir

On sait déjà *lire* et *modifier* les pixels d'une image avec `getpixel`, `putpixel` et une double boucle `for`. Pour dessiner un drapeau, il suffit en plus de créer une image **vierge** avec `Image.new`, puis de **colorier chaque pixel** en **RVB** :

```
from PIL import Image

largeur, hauteur = 500, 400
img = Image.new("RGB", (largeur, hauteur))      # image vierge (noire) largeur x hauteur

for y in range(hauteur):
    for x in range(largeur):
        img.putpixel((x, y), (255, 0, 0))       # (R, V, B) : ici, tout en rouge
```

Toute l'idée est là : dans la boucle, on **teste la position** du pixel (x, y) pour choisir sa couleur **RVB**. Rappel : le point (0,0) est en **haut à gauche**, x va vers la droite, y vers le bas.

> **Remarque** Les **dimensions** (proportions) et les **codes couleur** exacts (souvent en RVB) de chaque drapeau se trouvent sur la **Wikipédia anglaise** (*en.wikipedia.org*), aux rubriques « *Proportion* » et « *Colours* ». Prenez l'habitude d'aller les y chercher **dès** Monaco et la France — et pensez à **citer vos sources**.

2 Exemple 1 : le drapeau de Monaco

Le drapeau monégasque a des **proportions 4 :5** (4 de haut pour 5 de large) et se compose de **deux bandes horizontales** égales : **rouge** en haut, **blanc** en bas. On choisit donc une image de 500 × 400 (largeur × hauteur, soit bien 5 : 4). Pour chaque pixel, on regarde s'il est dans la **moitié haute** ($y <$ hauteur/2) ou basse :

```
from PIL import Image

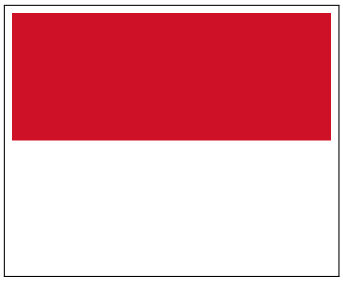
largeur, hauteur = 500, 400
img = Image.new("RGB", (largeur, hauteur))

rouge = (206, 17, 38)                       # rouge de Monaco
blanc = (255, 255, 255)

for y in range(hauteur):
    for x in range(largeur):
```

```
        if y < hauteur // 2:
            img.putpixel((x, y), rouge)      # bande du haut
        else:
            img.putpixel((x, y), blanc)      # bande du bas

img.save("monaco.png")
img.show()
```



3 Exemple 2 : le drapeau de la France

Le drapeau français a des **proportions 2 :3** et se compose de **trois bandes verticales** égales : **bleu**, **blanc**, **rouge**. On prend une image de 600×400 (soit $3 : 2$). Cette fois, on teste la position `x` du pixel (trois zones) :

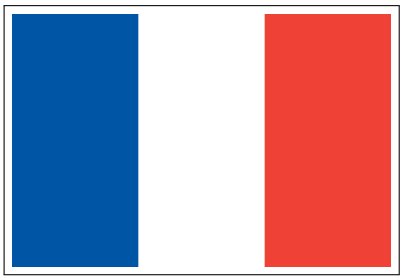
```
from PIL import Image

largeur, hauteur = 600, 400
img = Image.new("RGB", (largeur, hauteur))

bleu  = (0, 85, 164)
blanc = (255, 255, 255)
rouge = (239, 65, 53)

for y in range(hauteur):
    for x in range(largeur):
        if x < largeur // 3:
            img.putpixel((x, y), bleu)       # bande de gauche
        elif x < 2 * largeur // 3:
            img.putpixel((x, y), blanc)      # bande du milieu
        else:
            img.putpixel((x, y), rouge)      # bande de droite

img.save("france.png")
img.show()
```



▶ Activité 6 — Prendre le programme en main

Saisir et tester les deux programmes ci-dessus. Puis répondre :

1. Dans le programme de la France, à quoi sert `largeur // 3` ? Pourquoi trois zones ?

2. Modifier le drapeau de Monaco pour que la bande du haut soit **bleue** au lieu de rouge.
3. Que faudrait-il changer pour obtenir un drapeau **deux fois plus grand** ?

4 À vous de jouer !

▶ Activité 7 — Des drapeaux à trois bandes

En vous inspirant du drapeau français (trois bandes **verticales**) ou en passant à des bandes **horizontales**, dessiner les drapeaux suivants en **respectant proportions et couleurs** (dimensions et codes couleur exacts sur la **version anglaise de Wikipédia**).

Drapeau	Description	Proportions
Italie	3 bandes verticales : vert, blanc, rouge	2 :3
Belgique	3 bandes verticales : noir, jaune, rouge	13 :15
Allemagne	3 bandes horizontales : noir, rouge, or (jaune)	3 :5
Pays-Bas	3 bandes horizontales : rouge, blanc, bleu	2 :3

Astuce : pour des bandes **horizontales**, on teste la position `y` du pixel (`y < hauteur // 3`, etc.) au lieu de `x`.

Défi 1 : le disque du Japon (à compléter)

Pour le Japon, **pas question** d'utiliser une fonction toute faite comme `ellipse` : ce serait tricher ! On va dessiner le disque **nous-mêmes**, avec l'**équation du cercle**. Un pixel (x, y) se trouve **à l'intérieur** du disque de centre (x_c, y_c) et de rayon r lorsque :

$$(x - x_c)^2 + (y - y_c)^2 \leqslant r^2.$$

Le drapeau du Japon a des proportions 2 : 3, un fond blanc et un disque rouge **centré** dont le diamètre vaut 3/5 de la hauteur. **Compléter** la ligne du `if` en traduisant l'équation ci-dessus en Python.

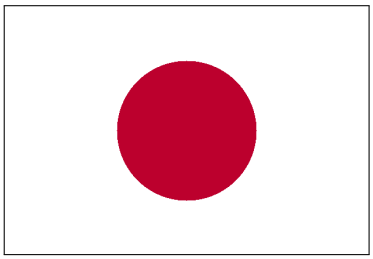
```
from PIL import Image

largeur, hauteur = 600, 400              # proportions 2:3
img = Image.new("RGB", (largeur, hauteur), (255, 255, 255))

xc, yc = largeur // 2, hauteur // 2      # centre du drapeau
r = int(0.3 * hauteur)                   # rayon = 3/5 de la demi-hauteur ; int garde la
                                         # partie entiere
rouge = (188, 0, 45)

for y in range(hauteur):
    for x in range(largeur):
        if ..................................... :   # A COMPLETER : dans le disque ?
            img.putpixel((x, y), rouge)

img.save("japon.png")
img.show()
```



Le résultat attendu.

▶ Activité 8 — Le disque du Japon

1. Compléter puis tester le programme.
2. Expliquer avec vos mots ce que teste votre condition.
3. Modifier le rayon pour obtenir un disque **deux fois plus petit**.

△ L'axe des y est à l'envers !

L'origine (0,0) est en **haut à gauche** et l'axe des y est orienté **vers le bas** : un y *plus grand* correspond à un point *plus bas* sur l'image. C'est l'**inverse** du repère habituel en mathématiques ! Attention, donc, en écrivant vos **inégalités** : « vers le haut » signifie un y **plus petit**, « vers le bas » un y **plus grand**. Le mieux est de **tester** votre condition sur un pixel dont vous connaissez déjà la couleur (par exemple un coin).

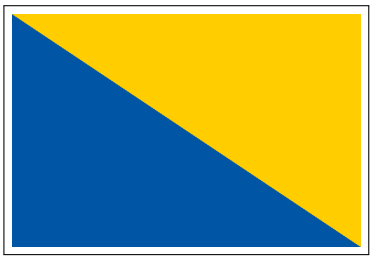
Défi 2 : une diagonale (à compléter)

Passons du cercle à une **droite**. On coupe le drapeau en deux triangles par la **diagonale** qui va du coin (0,0) (haut-gauche) au coin (largeur, hauteur) (bas-droit). L'équation de cette droite est :

$$y = \frac{\text{hauteur}}{\text{largeur}} \times x.$$

Un pixel est **en dessous** de la diagonale (donc dans le triangle du bas) lorsque son y est *plus grand* que celui de la droite, c'est-à-dire lorsque $y > \frac{\text{hauteur}}{\text{largeur}} \times x$ (attention à l'axe des y !). **Compléter** la condition.

```
1  from PIL import Image
2
3  largeur, hauteur = 600, 400
4  img = Image.new("RGB", (largeur, hauteur))
5
6  bleu  = (0, 85, 164)
7  jaune = (255, 205, 0)
8
9  for y in range(hauteur):
10     for x in range(largeur):
11         if ..........................................  :   # A COMPLETER : sous la diagonale ?
12             img.putpixel((x, y), bleu)     # triangle du bas
13         else:
14             img.putpixel((x, y), jaune)    # triangle du haut
15
16 img.save("diagonale.png")
17 img.show()
```



Le résultat attendu.

▶ Activité 9 — La diagonale

1. Compléter puis tester le programme.
2. Échanger les deux triangles (bleu en haut, jaune en bas). Que faut-il changer ?
3. **Défi** : obtenir l'*autre* diagonale (du coin haut-droit au coin bas-gauche).

Défi 3 : des drapeaux plus difficiles

Vous avez maintenant les deux outils : le **cercle** et la **droite**. On peut alors s'attaquer à des drapeaux qui mêlent bandes, triangles et bandes obliques. Le principe ne change pas : on parcourt tous les pixels et on teste une **condition**.

▶ Activité 10 — À vous de relever le défi

1. **Le Bénin** : une bande **verticale** verte à gauche, et à droite **deux bandes horizontales** (jaune en haut, rouge en bas). Réalisable uniquement avec des *rectangles* : à vous de trouver comment les combiner.
2. **La Tchéquie** : moitié haute blanche, moitié basse rouge, et un **triangle bleu** à gauche pointant vers le centre. Le triangle se dessine en testant, pour chaque pixel, s'il est *du bon côté* de **deux** droites.
3. **Les Seychelles** : cinq **bandes obliques** (bleu, jaune, rouge, blanc, vert) qui partent toutes du coin **en bas à gauche**. Les droites de séparation visent les **tiers** des bords haut et droit (*pas* des angles égaux !) : pour un pixel, on compare sa *pente* (hauteur $- y)/x$ à celle de ces droites (attention : sur le bord gauche, $x = 0$ et on ne peut pas diviser par 0 : traiter ces pixels à part).

▶ Activité 11 — Votre drapeau

1. Choisir un drapeau (pays, région, club…), rechercher ses **dimensions** et ses **couleurs** exactes, puis le reproduire le plus fidèlement possible.
2. **Bonus** : expliquer pourquoi certains drapeaux (avec une *étoile*, un *blason*…) sont presque impossibles à dessiner ainsi.

Grille d'évaluation

La **moitié des points** récompense vos **initiatives** : allez au-delà de ce qui est demandé, et gardez une trace de ce que vous avez cherché et essayé.

Critère	Points
Le socle	
Les quatre drapeaux à trois bandes sont dessinés, avec les proportions et les couleurs de la Wikipédia anglaise	/3
Le drapeau du Japon est complet : disque centré, de la bonne taille, condition expliquée	/2
La diagonale est obtenue, et vous savez échanger les deux triangles	/2
Votre code est lisible : noms de variables parlants, commentaires sur les lignes importantes	/1
Vous expliquez (à l'oral ou par écrit) le rôle de chaque ligne d'un de vos programmes	/2
Vos initiatives	
Vous avez dessiné des drapeaux supplémentaires de votre choix, sources (dimensions, couleurs) citées	/3
Vous avez réussi un ou plusieurs défis : autre diagonale, Bénin, Tchéquie, Seychelles…	/2
Vous avez inventé un effet ou une variante (dégradé, contour, drapeau « délavé », mélange de formes…)	/2
Vous avez cherché par vous-même une fonction ou une notion (documentation, tutoriel) et vous savez l'expliquer ; les formes demandées restent dessinées pixel par pixel	/2
Soin et originalité de l'ensemble (rendu, présentation des images)	/1
Total	**/20**

Crédits. Les images de drapeaux de ce projet sont produites par les programmes Python qui les accompagnent (bibliothèque `PIL`). Les drapeaux nationaux ne sont pas protégés par le droit d'auteur ; couleurs et proportions d'après la Wikipédia anglaise.

PROJET 2 Retoucher un tableau célèbre en RVB

Idée Appliquer vos programmes de traitement d'image à de **vraies œuvres d'art**, pour les transformer comme un artiste numérique.

Objectif Parcourir tous les pixels, **modifier les canaux** R, V, B, et recolorer une **zone choisie** avec une condition.

Fichiers `nuit_etoilee.jpg` et `blue_boy.jpg` sont fournis (œuvres du **domaine public**). À placer dans le même dossier que vos programmes.

IA



Autorisé : demander une explication (le rôle d'une fonction de `PIL`, le sens d'un message d'erreur). **Interdit** : faire écrire le programme par une IA. Vous devez pouvoir expliquer chaque ligne de votre code.

> **Remarque** Ces deux tableaux — *La Nuit étoilée* de Vincent **van Gogh** (1889) et *The Blue Boy* (*Le Garçon en bleu*) de Thomas **Gainsborough** (vers 1770) — sont dans le **domaine public** : on peut donc les utiliser et les modifier librement.

1 Exemple 1 : recolorer tout un tableau

On parcourt **tous les pixels** et on **échange les canaux** rouge et bleu. Le ciel bleu de Van Gogh devient chaud, et les étoiles jaunes deviennent bleutées.

```
from PIL import Image

img = Image.open("nuit_etoilee.jpg")
largeur, hauteur = img.size          # recupere la taille automatiquement !

for y in range(hauteur):
    for x in range(largeur):
        r, v, b = img.getpixel((x, y))
        img.putpixel((x, y), (b, v, r))    # on echange R et B
img.save("nuit_modifiee.jpg")
img.show()
```

(Le parcours de tous les pixels peut prendre quelques secondes : c'est normal.)



Original



Canaux R et B échangés

▶ Activité 12 — Premiers essais sur La Nuit étoilée

1. Saisir et tester le programme ci-dessus. Que remarque-t-on pour le ciel ? pour les étoiles ?
2. À quoi sert la ligne `largeur, hauteur = img.size` ?
3. Écrire un programme qui donne le **négatif** du tableau (remplacer chaque canal c par $255 - c$).
4. Écrire un programme qui transforme le tableau en **niveaux de gris** (chaque canal remplacé par la moyenne $(r + v + b)//3$).

2 Exemple 2 : changer la couleur d'un vêtement

Ici, on ne modifie **qu'une partie** de l'image, grâce à une condition `if`. Objectif : transformer le **costume bleu** du garçon en **costume rouge**, sans toucher au visage ni au décor. On repère les pixels où le **bleu domine**.

```
from PIL import Image

img = Image.open("blue_boy.jpg")
largeur, hauteur = img.size

for y in range(hauteur):
    for x in range(largeur):
        r, v, b = img.getpixel((x, y))
```

```
        # le bleu domine (le costume) -> on repeint en rouge
        if b > r + 15 and b > v + 5 and b > 60:
            gris = (r + v + b) // 3          # on garde la luminosite (les plis)
            rouge = gris + 50
            if rouge > 255:                  # on ne depasse pas 255
                rouge = 255
            img.putpixel((x, y), (rouge, gris // 3, gris // 4))
img.save("garcon_modifie.jpg")
img.show()
```



Original



Costume recoloré

▶ Activité 13 — Améliorer et personnaliser

1. Le repérage est **imparfait** (certaines zones d'ombre sont oubliées). Modifier les valeurs de la condition pour **mieux** recolorer le costume.
2. Changer la couleur finale : essayer un costume **vert**, puis **violet**.
3. En gardant la luminosité `gris`, expliquer pourquoi les **plis** du tissu restent visibles après recoloration.

3 Le projet

▶ Activité 14 — Votre œuvre numérique

Choisir **une œuvre du domaine public** (l'une des deux fournies, ou une autre trouvée sur un site d'images libres de droit) et réaliser une **transformation personnelle** qui combine au moins :

- une transformation **globale** (négatif, niveaux de gris, échange de canaux…) ;
- une modification **localisée** (changer la couleur d'un objet, d'un ciel, d'un vêtement).

Vous rendrez : l'**image de départ**, votre **image transformée**, votre **programme Python**, et **deux ou trois phrases** expliquant ce que fait votre programme et d'où vient l'œuvre (**source** et licence).

Grille d'évaluation

Critère	Points
Le programme fonctionne et parcourt correctement l'image	/6
Transformation globale réussie	/4
Modification localisée (avec condition `if`) réussie	/5
Explications claires + source de l'œuvre citée	/3
Originalité et soin du rendu	/2
Total	**/20**

Les versions modifiées des deux tableaux sont produites par les programmes Python de ce projet.

DÉBAT Faut-il obliger à signaler toute photo retouchée, dans la publicité comme sur les réseaux sociaux ?

▶ corrigé p. 244

retouche et transparence

Durée : 50 min

Consignes

- En classe entière. Les rôles sont tirés au sort : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement.
- Chaque argument s'appuie sur un **document** ci-dessous ou sur le **cours**.  Préparation sans IA ni recherche en ligne.
- Avant de commencer, notez sur votre cahier votre position de départ : oui, non ou je ne sais pas.

Rappel du cours. Une image numérique est un tableau de **pixels**, chacun codé par trois nombres (**RVB**) : la modifier, c'est changer ces nombres, et quelques lignes de Python y suffisent. Les smartphones eux-mêmes retouchent chaque photo grâce à des **algorithmes** (HDR, débruitage, embellissement). Une retouche peut être invisible : une photo n'est donc plus une **preuve** en soi. Faut-il prévenir celui qui regarde ?

Les deux camps

Groupe POUR — oui : toute photo retouchée diffusée en public (publicité, réseaux sociaux) doit être signalée.
Groupe CONTRE — non : une telle obligation générale serait inutile ou impossible ; on peut la limiter à certains cas.

Les documents

Document 1 — La mention « Photographie retouchée » (2017)

Depuis le 1er octobre 2017, en France, une photo à usage commercial (publicité, affiche, catalogue) sur laquelle la **silhouette** d'un mannequin a été affinée ou épaissie par logiciel doit porter la mention « **Photographie retouchée** », sous peine d'une amende de 37 500 €. Le but est la **santé publique** : limiter les images de corps impossibles à atteindre. Une retouche de la lumière, des couleurs ou de la peau n'est pas concernée.

À retenir : la loi vise la silhouette retouchée, et seulement dans un usage commercial.

Source : code de la santé publique, article L2133-2 ; décret n° 2017-738 du 4 mai 2017.

Document 2 — Les influenceurs aussi (2023)

Depuis la loi française du 9 juin 2023, un **influenceur** qui fait la promotion d'un produit doit ajouter la mention **« Image retouchée »** à ses photos et vidéos si sa silhouette a été affinée ou épaissie, ou son visage modifié ; un filtre qui transforme le visage est concerné. En revanche, une photo personnelle publiée sans partenariat commercial n'est soumise à aucune obligation.

À retenir : sur les réseaux, l'obligation ne concerne que la promotion commerciale.

Source : loi n° 2023-451 du 9 juin 2023 visant à encadrer l'influence commerciale et à lutter contre les dérives des influenceurs sur les réseaux sociaux.

Document 3 — Toute photo est déjà retouchée ?

Avant même que vous y touchiez, votre smartphone a transformé la photo : il combine plusieurs clichés (HDR), réduit le bruit, accentue les contours, parfois lisse la peau. Les photographes ont toujours ajusté la lumière, le contraste ou le cadrage, déjà dans la chambre noire. Pour beaucoup de photographes et d'artistes, une mention sur toute image modifiée n'aurait pas de sens : présente sur presque toutes les photos, elle ne serait plus lue.

À retenir : la vraie question est : à partir de quand une retouche trompe-t-elle ?

Source : d'après le cours (« Les algorithmes dans nos appareils »).

Document 4 — Photo de presse : interdit de toucher au contenu

Pour une photo de presse, l'enjeu est la **confiance**. Le grand concours de photojournalisme World Press Photo interdit d'ajouter, de retirer ou de déplacer un élément de l'image ; seuls des réglages raisonnables de lumière, de contraste et de couleur sont admis, et des photos ont déjà été disqualifiées pour avoir enfreint ces règles. Les **métadonnées EXIF** d'un fichier peuvent indiquer le logiciel de retouche… mais elles s'effacent facilement.

À retenir : la presse distingue régler l'image et changer ce qu'elle montre.

Source : World Press Photo, règles d'inscription et code éthique du concours.

Rôles, déroulé minuté, règles de prise de parole et trace écrite : voir la page « Méthode : débattre » p. 7.

Tableau d'arguments

Recopier ce tableau sur le cahier et le compléter pendant le débat.

Argument POUR (oui)	Doc.	Argument CONTRE (non)	Doc.
…		…	

Trace écrite finale (individuelle)

Sur le cahier, notez votre position après le débat (oui, non, je ne sais toujours pas) et si vous avez changé d'avis. Puis, en trois à cinq lignes, justifiez-la avec vos **deux meilleurs arguments**, dont **un argument de l'autre camp** que vous trouvez solide (avec le document qui l'appuie).

FICHE DE RÉVISION

L'essentiel

- Lumière → **photosites** (+ filtre de Bayer) → nombres → **pixels** : une photo est un **tableau de nombres**.
- Chaque **pixel** est codé en **RVB** : 3 nombres de `0` à `255`, chacun sur **1 octet**.
- Une image se **transforme par programme** (Python + PIL) ; la photo du smartphone est déjà **calculée** (HDR, filtres…).
- Le fichier contient aussi des **métadonnées EXIF** (date, appareil, parfois **GPS**) qui peuvent **révéler où et quand** la photo a été prise : dans le doute, les effacer avant de partager.
- Une retouche peut être invisible : une photo n'est plus une **preuve** (IA, *deepfakes*) ⇒ esprit critique. Repères : **1826**, première photo (Niépce) ; **1975**, premier appareil numérique (Sasson).

Notions clés

Notion	En bref
Photosite, Bayer	plus de lumière ⇒ plus grand nombre ; filtre : 1 R, 1 B et **2 V** par groupe de 4 (œil sensible au vert).
Synthèse additive	R + V = jaune ; R + B = violet ; les trois ensemble = blanc ; **trois canaux égaux = gris**.
Couleurs usuelles	noir (0,0,0), blanc (255,255,255), rouge (255,0,0), vert (0,255,0), bleu (0,0,255).
PIL	`getpixel` lit, `putpixel` écrit ; (0,0) en **haut à gauche** ; 2 boucles `for` pour tout parcourir.

Méthodes à savoir refaire

- **Nombre de couleurs** : 256 valeurs par canal, 3 canaux ⇒ $256 \times 256 \times 256 = 16\,777\,216$ couleurs.
- **Définition** (nombre de pixels) = largeur × hauteur : $4000 \times 3000 = 12\,000\,000$ pixels.
- **Poids brut** = largeur × hauteur ×3 octets : $12\,000\,000 \times 3 = 36\,000\,000$ octets ≈ 36 Mo (1 Mo $\approx 10^6$ octets).
- **Résolution** (ppp) = pixels ÷ pouces (1 pouce = 2,54 cm) : 1500 pixels sur 12,7 cm = 5 pouces ⇒ 300 ppp.
- **Lire un programme PIL** : la double boucle traite largeur × hauteur pixels ; $255 - c$ par canal = **négatif** ; $(r+v+b)//3$ partout = **gris**.

Erreurs à éviter

Confondre **définition** (nombre de pixels) et **résolution** (pixels par pouce) ; oublier le ×3 du poids ou la conversion cm → pouces ; ne pas reconnaître un **gris** (canaux égaux) ; placer (0,0) en bas à gauche.

Savoir-faire à maîtriser — je sais…

- ☐ expliquer comment un **capteur** transforme la lumière en image numérique ;
- ☐ décrire une image par ses **pixels** et coder une couleur en **RVB** (3 valeurs de 0 à 255) ;
- ☐ distinguer **définition**, **résolution** et **poids** d'une image, et estimer un poids ;
- ☐ modifier une image par un programme **Python** (parcourir et changer les pixels avec la bibliothèque PIL) ;
- ☐ expliquer ce que sont les métadonnées **EXIF** et le risque qu'elles posent pour la vie privée ;
- ☐ citer des traitements automatiques (algorithmes) réalisés par un appareil photo ou un smartphone.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 245

1. Donner le code RVB du jaune, puis dire ce qu'affiche (200,200,200).
2. Calculer le poids brut, en Mo, d'une image 2000 × 1500.
3. Une image de 1000 pixels de large est imprimée sur 25,4 cm. Calculer sa résolution.
4. Le pixel (10,5) vaut (200,50,90). Donner sa valeur après passage en négatif.
5. Citer deux métadonnées EXIF et expliquer le risque lié au GPS.

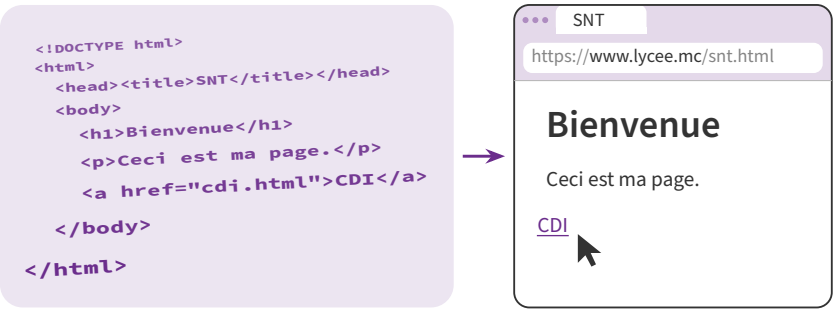
Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 245

1. Définir le degré d'un sommet d'un graphe (ch. 2).
2. Écrire une boucle qui affiche les entiers de 0 à 255 (ch. 1).

4

CHAPITRE 4

Le Web



DANS CE CHAPITRE

- Cours 108
- Exercices 119
- Débat 126
- Fiche de révision 128
- Corrigés 246

COURS

Programme Repères historiques du Web ; distinguer *Web* et *Internet* ; le modèle **client/serveur** et le protocole **HTTP** ; les **URL** et les hyperliens ; le couple **HTML/CSS** ; moteurs de recherche et **indexation** ; sécurité et traces (HTTPS, cookies).

Idée Une page web est un simple **fichier texte** : du **HTML** pour le *contenu*, du **CSS** pour l'*apparence*. En comprenant ces deux langages, vous passez de *consommateur* à *auteur* du Web.

Objectifs Situer l'invention du Web ; ne plus confondre Web et Internet ; lire une URL et un échange client/serveur ; écrire une page en HTML ; la mettre en forme en CSS ; comprendre comment un moteur de recherche classe ses réponses.

Situation de départ

1. Vous tapez l'adresse d'un site et la page s'affiche. Qui vous l'a envoyée, et sous quelle forme ? Et que veut dire « Erreur 404 » quand cela échoue ?
2. Pourquoi tel site arrive-t-il en tête de vos recherches, et pas un autre ?
3. Après avoir regardé une paire de baskets sur un site, vous la voyez en publicité partout ailleurs. Comment ces sites vous reconnaissent-ils ?

Pour les curieux — *Un peu d'histoire* (facultatif)

L'idée d'un texte « augmenté » de renvois — l'**hypertexte** — est imaginée dès **1965** par l'Américain **Ted Nelson**. Mais c'est en **1989** au **CERN** (près de Genève) que le Britannique **Tim Berners-Lee**, avec le Belge **Robert Cailliau**, invente le **World Wide Web** : relier des documents *de machine à machine* par des **hyperliens**. Il écrit le premier **navigateur** et met en ligne la **première page web** de l'histoire, dont le CERN a restauré une copie, consultable aujourd'hui. Fait remarquable : il choisit de ne rien breveter — le Web est **libre et gratuit**.

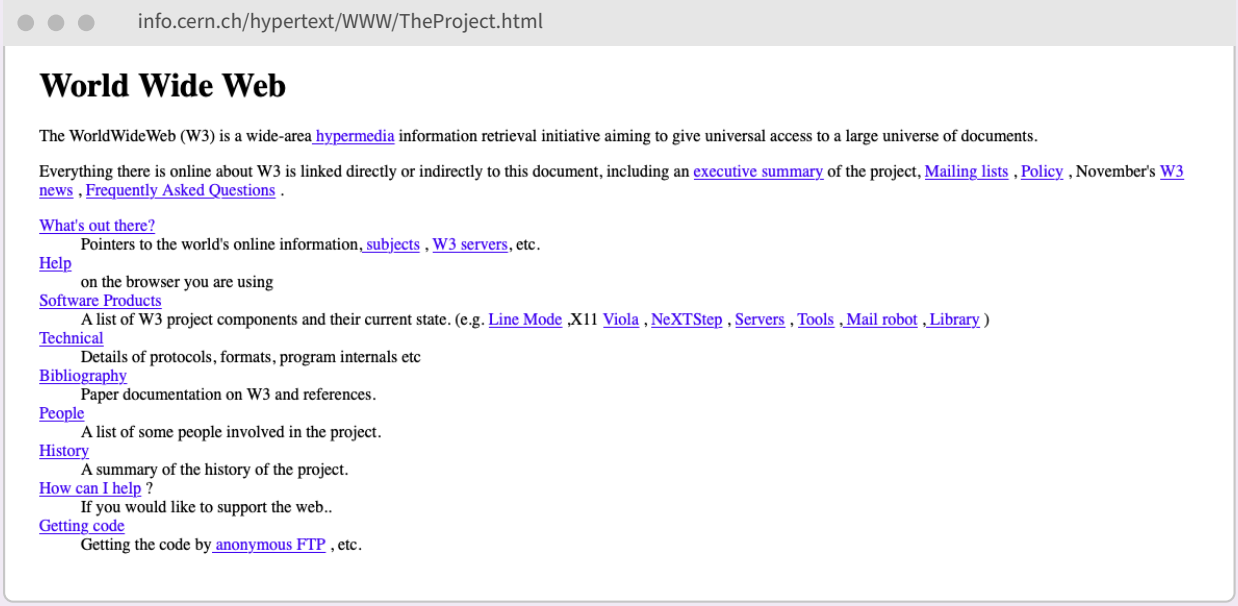


Le premier serveur web, un ordinateur NeXT



Tim Berners-Lee (2009)

→ frise chronologique p. 10



La toute première page web (1991), dans la copie restaurée par le CERN et consultable sur `info.cern.ch` *: les hyperliens y sont déjà soulignés.*

Règle 1 — Repères historiques (les grandes étapes)

- **1965** — Ted Nelson invente le concept d'**hypertexte**.
- **1989** — naissance du Web au **CERN** (Tim Berners-Lee).
- **1993** — le Web tombe dans le **domaine public** ; le navigateur **Mosaic** le rend populaire hors des laboratoires.
- **1994** — création du **W3C**, l'organisme qui **normalise** le Web.
- **1995** — des pages *interactives* (**JavaScript**) et *dynamiques* (**PHP**) deviennent possibles.
- **2001** — standardisation des pages grâce au **DOM**.
- **2010** — explosion du Web **mobile** (smartphones, applications).

Sources : ministère de l'Éducation nationale, programme de SNT de seconde, *Bulletin officiel* spécial n° 1 du 22 janvier 2019 (repères historiques du Web) ; CERN, « La naissance du Web » et première page web reconstituée, `home.cern`, `info.cern.ch` ; W3C, « About W3C » (création du consortium en 1994), `w3.org`.

I Le Web n'est pas Internet

C'est la confusion la plus fréquente. Il faut la lever tout de suite.

Définition 1

Internet est le *réseau* physique mondial : des millions d'ordinateurs reliés par des câbles, de la fibre et des ondes, qui échangent des données grâce au protocole **IP**.

Le **Web** (ou « toile ») est *l'un des services* qui circulent sur Internet : l'ensemble des **pages** reliées par des **hyperliens**, consultées avec un **navigateur**.

Autrement dit, le Web *s'appuie* sur Internet, mais ce n'est pas la même chose. Sur Internet circulent aussi les **e-mails** (protocole SMTP), les transferts de fichiers (FTP), les appels vidéo, les jeux en ligne… Le Web repose sur **trois technologies** inventées par Berners-Lee : le protocole **HTTP**, les adresses **URL** et le langage **HTML**.

Règle 2 — À retenir

Internet = **le réseau** (la route). Web = **un service** qui roule dessus (les pages). Une bonne image : Internet est le *réseau routier*, le Web est *l'ensemble des magasins* qu'on visite en l'empruntant.

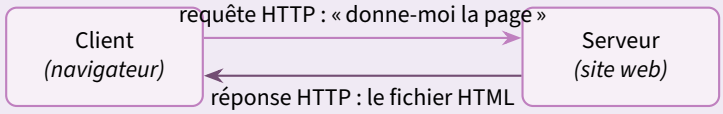
► **Exercices d'application :** exercices 1 **et** 2 (le Web n'est pas Internet)

II Le modèle client / serveur et le protocole HTTP

Quand vous consultez un site, deux ordinateurs dialoguent.

Définition 2

Le **client** est votre navigateur (Firefox, Chrome…) : il **demande** une page.
Le **serveur** est un ordinateur, allumé en permanence, qui **stocke** le site et **répond** en envoyant la page. Ce dialogue suit des règles précises : le protocole **HTTP** (*HyperText Transfer Protocol*).



Le navigateur envoie une **requête**, le serveur renvoie une **réponse** (la page, une image…) accompagnée d'un **code** qui dit si la requête a **réussi** ou **échoué** :

Code	Signification
200	tout va bien, voici la page *(réussite)*
403	accès interdit *(échec : non autorisé)*
404	page non trouvée *(échec : la ressource n'existe pas)*
500	erreur du serveur *(échec : le serveur a planté)*

Règle 3 — HTTP et HTTPS

HTTPS est la version **sécurisée** de HTTP : les données échangées sont **chiffrées**. Le petit **cadenas** dans la barre d'adresse indique que la connexion est protégée — indispensable avant de saisir un mot de passe ou un numéro de carte.

A. Décomposer une requête HTTP

Une requête HTTP est un petit message texte. Elle précise une **méthode** (l'action voulue) et la **ressource** demandée sur un **hôte** (le serveur) :

```
GET /snt/web.html HTTP/1.1        <- methode + ressource demandee
Host: www.lycee.mc                <- le serveur vise
```

La méthode `GET` veut dire « *donne-moi* cette ressource ». On peut **passer des informations au serveur** en les ajoutant à la fin de l'URL, après un point d'interrogation `?` : ce sont les **paramètres**. Chaque paramètre s'écrit `nom=valeur`, et on les sépare par une esperluette `&` :

```
/recherche?ville=Monaco&jour=lundi
```

Ici, deux paramètres sont **passés** au serveur : `ville` vaut `Monaco` et `jour` vaut `lundi`. C'est ainsi qu'un formulaire de recherche transmet ce que vous avez tapé.

Règle 4 — Anatomie d'un échange

Requête (client → serveur) : *méthode* + *ressource* + éventuels *paramètres* après `?`. **Réponse** (serveur → client) : un *code* (200, 404…) + le *contenu* (la page HTML).

B. Chez le client, chez le serveur

Le serveur peut renvoyer une page toute prête, ou la **fabriquer sur mesure** en fonction des paramètres reçus (une page *dynamique*). La page reçue peut aussi contenir du **code exécuté par le client** — le plus souvent du **JavaScript** — qui anime la page directement dans votre navigateur (menus, animations, vérification d'un formulaire).

Règle 5 — Deux endroits où « ça calcule »

Côté serveur : la page est choisie ou construite avant l'envoi (souvent en PHP).
Côté client : le navigateur exécute le **JavaScript** contenu dans la page.
On peut voir le code d'une page reçue par un **clic droit** → « **Afficher le code source** ».

► **Exercices d'application :** exercices 3 à 7 (client/serveur ; la requête HTTP)

III L'adresse d'une page : l'URL

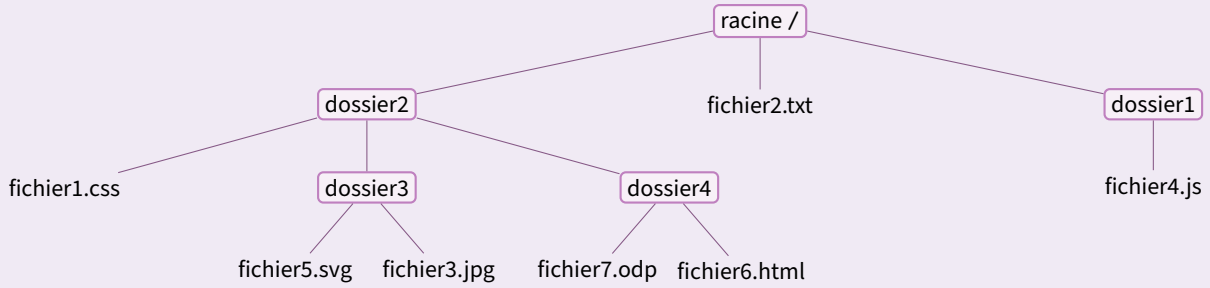
Pour demander une page, le client doit connaître son adresse : son **URL**.

Définition 3

Une **URL** (*Uniform Resource Locator*) est l'adresse unique d'une ressource (page, image, fichier) sur le Web. Elle se lit de gauche à droite :

`https://www.lycee.mc/cours/snt/web.html`
↑ protocole ↑ nom du serveur (domaine) ↑ chemin du fichier

Les fichiers d'un site sont rangés dans des dossiers, comme sur votre ordinateur : c'est une structure en **arborescence** (un arbre à l'envers dont la racine est notée `/`).



Les **dossiers** sont encadrés, les **fichiers** sont les feuilles de l'arbre.

Règle 6 — Chemin absolu, chemin relatif

Un **chemin absolu** part de la **racine** `/` : `/dossier2/dossier3/fichier3.jpg`.
Un **chemin relatif** part du **dossier courant**, celui de la page qui fait le lien (pas de `/` au début) : depuis `dossier2`, on écrit `dossier3/fichier3.jpg`. On **remonte** d'un niveau avec `..` : depuis `dossier4`, on écrit `../dossier3/fichier5.svg`.

Dans une page HTML (§ IV), ces chemins s'écrivent dans deux **attributs** : `src` pour le fichier d'une **image**, `href` pour la destination d'un **lien**. Par exemple, dans une page rangée dans `dossier4`, `<img src="../dossier3/fichier3.jpg" alt="photo" />` affiche l'image `fichier3.jpg`, et `<a href="fichier6.html">Suite</a>` crée un lien vers la page `fichier6.html`, rangée dans le même dossier.

► **Exercices d'application :** exercices 8 à 10 (l'URL et les chemins)

IV HTML : le contenu de la page

Nous arrivons au cœur du chapitre : le couple **HTML** + **CSS**.

Définition 4

HTML (*HyperText Markup Language*) est le langage qui décrit le **contenu** et la **structure** d'une page : titres, paragraphes, images, liens… Ce n'est **pas** un langage de programmation (pas de boucles ni de conditions) : c'est un langage de **balisage**.

En HTML, on entoure le contenu de **balises**. Une balise **ouvrante** `<p>` et une balise **fermante** `</p>` délimitent un **élément** :

```
1 <h1>Hello World ! Ceci est un titre</h1>
2 <p>Ceci est un <strong>paragraphe</strong>. Avez-vous bien compris ?</p>
```

Rendu dans le navigateur

Hello World ! Ceci est un titre

Ceci est un **paragraphe**. Avez-vous bien compris ?

Chaque balise a un **sens** (on parle de *sémantique*) : `<h1>` est un titre important (`<h2>`, `<h3>`… des sous-titres), `<p>` un paragraphe, `<strong>` un mot important, `<img>` une image, `<a>` un lien. Voici les balises à connaître :

Balise	Rôle	Exemple
`<h1>` à `<h6>`	titres, du plus important au moins important	`<h2>Horaires</h2>`
`<p>`	un paragraphe	`<p>Bonjour</p>`
`<strong>`, `<em>`	mot important (gras), mise en valeur (italique)	`<em>vraiment</em>`
` `	un retour à la ligne (se ferme seule)	`ligne 1 ligne 2`
`<ul>`, `<ol>`, `<li>`	liste à puces, liste numérotée, un élément de la liste	`<ul><li>pain</li></ul>`
`<img />`	une image (se ferme seule) : `src` donne le fichier, `alt` un texte qui la décrit	`<img src="chat.jpg" alt="un chat" />`
`<a>`	un lien : `href` donne la destination	`<a href="page2.html">Suite</a>`
`<table>`, `<tr>`, `<th>`, `<td>`	un tableau, une ligne, une case d'en-tête, une case	voir l'exemple ci-dessous
`<div>`	un bloc qui regroupe des éléments (pour les mettre en forme ensemble)	`<div>...</div>`

Le texte de l'attribut `alt` s'affiche si l'image ne se charge pas, et il est lu à voix haute aux personnes aveugles : on ne l'oublie jamais. Voici ces balises à l'œuvre :

```
<h2>Le club</h2>
<img src="images/logo.png" alt="Logo du club" />
<ol>
  <li>Echauffement</li>
  <li>Match</li>
</ol>
<table>
  <tr> <th>Jour</th> <th>Heure</th> </tr>
  <tr> <td>Mercredi</td> <td>14 h</td> </tr>
</table>
<a href="contact.html">Nous contacter</a>
```

Rendu dans le navigateur

Le club

logo du club

1. Échauffement
2. Match

Jour	Heure
Mercredi	14 h

Nous contacter

Règle 7 — Les deux règles d'or des balises

1. **Toute balise ouverte doit être refermée.**
2. **Les balises s'imbriquent proprement** (« premier ouvert, dernier fermé »). `<p><strong>...</strong></p>` est correct ; `<p><strong>...</p></strong>` est **interdit**.

Une balise peut porter des **attributs**, dans la balise ouvrante. Les deux plus utiles sont `id` (un identifiant *unique* dans la page) et `class` (une catégorie, partageable) :

```
<h2 class="chapitre">Un sous-titre</h2>
<p id="intro">Le paragraphe d'introduction.</p>
```

A. La structure complète d'une page HTML

Une vraie page web suit toujours le même squelette :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Ma premiere page</title>
  </head>
  <body>
    <h1>Bienvenue</h1>
    <p>Ceci est ma page web.</p>
  </body>
</html>
```

Le `<head>` contient les informations *invisibles* (titre de l'onglet, encodage) ; le `<body>` contient tout ce qui s'**affiche** dans la page.

▶ **Exercices d'application :** exercices 11 à 14 (lire et écrire du HTML)

V CSS : l'apparence de la page

Le HTML dit *ce qu'il y a* ; le **CSS** dit *à quoi ça ressemble.*

Définition 5

CSS (*Cascading Style Sheets*, « feuilles de style en cascade ») est le langage qui gère l'**apparence** : couleurs, polices, tailles, alignements, marges, dispositions…

Une règle CSS **cible** des éléments (le *sélecteur*) et leur applique des **propriétés** :

```css
h1 {
  text-align: center;
  color: white;
  background-color: crimson;
}
h2 {
  font-family: Verdana;
  font-style: italic;
  color: green;
}
```

Rendu dans le navigateur

Ceci est un titre

Ceci est un sous-titre

Les propriétés les plus utiles sont les suivantes :

Propriété	Effet	Exemple
`color`	couleur du texte	`color: red;`
`background-color`	couleur du fond	`background-color: black;`
`font-family`	police de caractères	`font-family: Verdana;`
`font-size`	taille du texte	`font-size: 20px;`
`font-style`	texte normal ou en italique	`font-style: italic;`
`text-align`	alignement : `left`, `center`, `right`	`text-align: center;`
`border`	bordure (épaisseur, style, couleur)	`border: 2px solid blue;`
`margin`	marge *autour* de l'élément	`margin: 10px;`
`padding`	marge *à l'intérieur* de l'élément	`padding: 8px;`
`width`	largeur (par exemple d'une image)	`width: 300px;`

Une couleur s'écrit par son **nom** anglais (`red`, `crimson`, `teal`) ou par son code **RVB** en hexadécimal (`#e63946`), le même codage RVB que pour les pixels d'une photo. Chaque propriété se termine par un **point-virgule**.

Règle 8 — Les trois sélecteurs à connaître

On peut cibler les éléments d'une page de trois façons différentes :

On cible…	En HTML	En CSS
toutes les balises d'un type	`<p>`	`p { ...}`
un élément par son **id** (unique)	`id="intro"`	`#intro { ...}`
tous les éléments d'une **classe**	`class="alerte"`	`.alerte { ...}`

En CSS, l'`id` se cible avec un **dièse** `#`, la `class` avec un **point** `.`. C'est ainsi qu'on met en forme *un* paragraphe précis sans toucher aux autres.

Règle 9 — Où écrire le CSS ? Une feuille de style reliée à la page

On écrit les règles CSS dans un fichier à part, par exemple `style.css`, rangé à côté des pages. Chaque page HTML le **relie** par une balise `<link>` placée dans le `<head>` :

```html
<head>
  <meta charset="utf-8" />
  <title>Mon site</title>
  <link rel="stylesheet" href="style.css" />
</head>
```

Une seule feuille de style peut ainsi habiller **toutes** les pages d'un site : on change une couleur dans `style.css`, et tout le site change. (Sur un éditeur en ligne comme `jsfiddle.net`, la zone « CSS » joue le rôle de ce fichier.)

▶ Activité 1 — Ma première page

Sur un éditeur en ligne (par exemple `jsfiddle.net` ou `codepen.io`) :

1. Recopier le squelette HTML complet, avec un `<h1>`, deux `<p>` et une image.
2. Ajouter du CSS pour centrer le titre, le colorer, et changer la police des paragraphes.
3. Donner à un seul paragraphe l'`id="important"` et le colorer en rouge *via* `#important`.

▶ **Exercices d'application :** exercices 15 à 17 (mettre en forme avec CSS)

VI Les hyperliens : la « toile »

Ce qui fait la **toile**, ce sont les liens d'une page vers une autre. On les crée avec la balise `<a>` et son attribut `href` (l'URL de destination) :

```html
<a href="https://www.gouv.mc">Visiter le site du gouvernement</a>
```

Entre les pages d'un même site, on utilise des **chemins relatifs** (§ III) : un menu de navigation n'est qu'une liste de liens, recopiée sur chaque page.

```html
<ul>
  <li><a href="index.html">Accueil</a></li>
  <li><a href="pages/contact.html">Contact</a></li>
</ul>
```

En suivant les liens de page en page, on « navigue » : c'est la **navigation hypertexte**, l'idée fondatrice de Berners-Lee.

VII À qui appartient une page ? Notions juridiques

Publier n'est pas voler : sur le Web comme ailleurs, ce que l'on trouve **appartient à quelqu'un**.

Définition 6

Le **droit d'auteur** protège automatiquement toute création (texte, photo, musique, code) : son auteur décide qui peut l'utiliser. Une **licence** est l'autorisation qui accompagne une œuvre et fixe le **droit d'usage** : ce qu'on a le droit d'en faire (copier, modifier, diffuser, vendre…).

Règle 10 — Copier n'est pas toujours permis

Récupérer une image ou un texte trouvé sur le Web pour le republier **sans autorisation** peut être **illégal**. Pour réutiliser sereinement, on cherche des œuvres sous **licence libre** (par exemple **Creative Commons**, ou les **logiciels libres**) qui autorisent d'avance certains usages — souvent à condition de **citer l'auteur**.

Un fichier numérique se copie à l'infini pour presque rien : cela ne veut pas dire qu'il est **sans valeur**. Un logiciel, une base de données, une photo ont une **valeur** (de travail, d'usage, commerciale). C'est pourquoi le Web a fait naître à la fois un immense **commerce en ligne** et un mouvement de **partage** (Wikipédia, logiciels libres) fondé, lui aussi, sur des licences précises.

► **Exercices d'application :** exercice 18 (droit d'auteur : ai-je le droit ?)

VIII Trouver une page : les moteurs de recherche

Il existe des milliards de pages. Pour s'y retrouver, on utilise un **moteur de recherche** (Google, Bing, Qwant…).

Définition 7

Un **moteur de recherche** explore le Web en permanence grâce à des robots (les *crawlers*) qui suivent les liens, et range les pages trouvées dans un immense **index** (comme l'index d'un livre). Quand vous tapez une requête, le moteur ne cherche pas « dans le Web » mais **dans son index**, déjà prêt.

Règle 11 — L'ordre des résultats n'est pas neutre

Le moteur **classe** les pages par **pertinence**, à l'aide d'un algorithme. Un critère historique (le *PageRank* de Google) : une page est jugée importante si **beaucoup d'autres pages importantes pointent vers elle**. S'ajoutent la présence des mots-clés, la langue, la localisation… Les premiers résultats peuvent aussi être des **publicités** (signalées par la mention « Sponsorisé »).

► Activité 2 — Aiguiser son esprit critique

1. Faire une même recherche sur deux moteurs différents et comparer les premiers résultats.
2. Repérer, sur une page de résultats, ce qui est une **publicité** et ce qui ne l'est pas.
3. Expliquer pourquoi « être en première page » est un enjeu commercial énorme.

► **Exercices d'application :** exercices 19 **et** 20 (les moteurs de recherche)

IX Sécurité et confidentialité : régler son navigateur

Naviguer n'est jamais tout à fait anonyme, et laisser un site exécuter du code sur sa machine comporte des **risques**. Heureusement, le navigateur se **règle**.

Règle 12 — Cookies et traces

Un **cookie** est un petit fichier qu'un site dépose dans votre navigateur pour vous **reconnaître** (rester connecté, retenir un panier…) — mais aussi pour **vous suivre** d'un site à l'autre à des fins publicitaires. Chaque visite laisse des **traces** : **historique**, adresse **IP** (l'adresse de votre appareil sur le réseau, qu'on étudiera au chapitre *Internet*), cookies.

Règle 13 — Les réglages importants du navigateur

Dans les *paramètres* de son navigateur, on peut :

- **gérer les cookies** : les refuser (surtout les cookies « tiers », qui pistent) ;
- **effacer l'historique** de navigation, ou activer la **navigation privée** pour ne rien enregistrer ;
- **contrôler les autorisations** d'un site (caméra, micro, position) et **interdire** l'exécution de certains programmes ;
- préférer les sites en **HTTPS** (cadenas) et bloquer les traceurs.

Règle 14 — Attention à l'hameçonnage (*phishing*)

De **fausses pages**, imitant une vraie (banque, réseau social), cherchent à voler vos identifiants : c'est l'**hameçonnage**. Un **lien peut cacher une adresse différente** de son texte : avant de cliquer, **survolez le lien** pour lire la *vraie* URL, et vérifiez le nom de domaine. En cas de doute, on ne clique pas et on ne saisit aucune information personnelle.

► **Exercices d'application :** exercice 21 (régler et sécuriser son navigateur), exercice 22 (critiquer une réponse d'IA), exercice 23 (Python au service du thème)

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Déjà vu :** les hyperliens forment un **graphe**, comme les amitiés d'un **réseau social** (c'est sur ce graphe que travaille le PageRank) ; en CSS, une couleur se code en **RVB**, comme un pixel (chapitre *Photographie numérique*).
- **À venir :** au chapitre *Internet*, on verra comment le navigateur trouve l'adresse du serveur et comment la page voyage en petits morceaux, les **paquets**.
- **Plus loin :** en spécialité NSI de Première, on étudie de plus près les formulaires et le **JavaScript**.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Web ≠ Internet	Internet = le réseau ; Web = un service (les pages) qui roule dessus.
Inventeur	Tim Berners-Lee, au CERN, 1989–1991 ; W3C en 1994.
Client / serveur	le navigateur *demande* (requête HTTP), le serveur *répond* (la page).
HTTP / HTTPS	protocole du Web ; HTTPS = version chiffrée (cadenas).
Requête HTTP	méthode (`GET`) + ressource ; **paramètres** après `?` : `nom=valeur&...`.
Codes de réponse	200 (ok), 403 (interdit), 404 (introuvable), 500 (erreur serveur).
Client / serveur (2)	serveur : PHP (dynamique) ; client : **JavaScript** ; clic droit → code source.
URL	adresse d'une page : protocole + domaine + chemin ; absolu (`/`) ou relatif (`..`).
HTML	le **contenu** : langage de *balises* (`<h1>`, `<p>`, `<a>`...).
Règles des balises	toute balise ouverte est refermée ; imbrication propre.
CSS	l'**apparence** : sélecteur + propriétés (couleur, police, alignement...).
Sélecteurs	`p` (balise), `#id` (unique), `.classe` (partagée).
Hyperlien	`<a href="...">` ; c'est ce qui fait la « toile ».
Droit d'auteur	toute création a un auteur ; une **licence** fixe le droit d'usage (ex. Creative Commons).
Moteur de recherche	robots + *index* ; résultats **classés** (PageRank), pubs incluses.
Sécurité navigateur	régler cookies/historique/navigation privée ; se méfier de l'**hameçonnage**.

Retour à la situation de départ

1. Un **serveur**, qui répond à la **requête HTTP** de votre navigateur, le **client** (§ II) : il envoie un simple fichier texte écrit en **HTML** (§ IV), mis en forme par du **CSS** (§ V). Le code **404** signifie « page non trouvée » (§ II).
2. Le **moteur de recherche** cherche dans son **index** et **classe** les pages avec un algorithme : une page est jugée importante si beaucoup de pages importantes pointent vers elle (PageRank). Les premiers résultats peuvent aussi être des **publicités** (§ VIII).
3. Grâce aux **cookies** déposés dans votre navigateur, en particulier les cookies « tiers », qui vous suivent d'un site à l'autre ; on peut les refuser dans les réglages du navigateur (§ IX).

Débat argumenté

Un **débat**, proposé en fin de chapitre, accompagne ce thème : « Un moteur de recherche doit-il rester libre de classer ses résultats comme il le veut ? » (documents, rôles, tableau d'arguments et trace écrite, environ 50 min).

Savoir-faire à maîtriser

À la fin du chapitre, je sais...

- ☐ distinguer le **Web** d'**Internet** ;
- ☐ décrire l'échange **client / serveur** et le rôle du protocole **HTTP** ;
- ☐ décomposer une **URL** (protocole, nom de domaine, chemin) ;
- ☐ écrire une page simple en **HTML** (balises de structure) et la mettre en forme avec **CSS** ;
- ☐ créer un **hyperlien** d'une page vers une autre ;
- ☐ expliquer le fonctionnement d'un **moteur de recherche** (indexation puis classement) ;
- ☐ régler mon navigateur pour préserver ma sécurité et ma vie privée (HTTPS, cookies).

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- [sur papier] *à faire sur papier* ; ▶ *à réaliser sur un éditeur en ligne* (`jsfiddle.net` ou `codepen.io`).
- Les tableaux de balises HTML et de propriétés CSS du cours (§ IV et V) peuvent être consultés pour tous les exercices.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Le Web et Internet ↑cours

★ **Exercice 1** *Vrai ou faux* ▶ corrigé p. 247 • essentiel

[sur papier] Répondre et corriger les phrases fausses.

a. Le Web et Internet, c'est la même chose.
b. Un e-mail est transporté par le Web.
c. Le Web repose sur trois technologies : HTTP, les URL et le HTML.
d. Le Web a été inventé par Tim Berners-Lee, au CERN.

▷ coup de pouce

★ **Exercice 2** *Web ou Internet ?* ▶ corrigé p. 247

[sur papier] Ranger dans deux colonnes ce qui relève du **Web** et ce qui relève d'**Internet** (mais pas du Web) : une page consultée dans un navigateur ; un e-mail ; un lien hypertexte ; un transfert de fichier par FTP ; le protocole HTTP. ▷ coup de pouce

La requête HTTP, client et serveur ↑cours

★★ **Exercice 3** *Décomposer une requête* ▶ corrigé p. 247 • essentiel

[sur papier] On tape dans le navigateur l'adresse : `https://www.meteo.mc/prevoir?ville=Nice&jour=mardi`.

1. Quelle **ressource** est demandée au serveur (avant le `?`) ?
2. Combien de **paramètres** sont passés au serveur ? Donner le **nom** et la **valeur** de chacun.
3. Que signifie la méthode `GET` d'une requête HTTP ?

▷ coup de pouce

★ **Exercice 4** *Réussite ou échec ?* ▶ corrigé p. 247

[sur papier] Associer chaque **code de réponse HTTP** à sa signification :

200 **403** **404** **500**

Significations : page non trouvée • tout va bien • erreur du serveur • accès interdit. ▷ coup de pouce

★★ **Exercice 5** *Client ou serveur ?* ▶ corrigé p. 247

[sur papier]

1. Un langage s'exécute *dans le navigateur* pour animer la page : lequel, **JavaScript** ou **PHP** ?
2. Comment fait-on, dans un navigateur, pour **voir le code source** d'une page reçue ?
3. « Le serveur construit une page différente selon les paramètres reçus. » Comment appelle-t-on une telle page ?

▷ coup de pouce

★★ **Exercice 6** *Suivre une requête* ▶ corrigé p. 248

[sur papier] Vous tapez `https://www.lycee.mc/cours.html` dans la barre d'adresse. Le navigateur retrouve d'abord l'adresse du serveur `www.lycee.mc` sur le réseau (on verra comment au chapitre *Internet*). Remettre ensuite les étapes suivantes dans l'ordre :

A. Le navigateur affiche la page.
B. Le serveur trouve le fichier `cours.html` et renvoie une réponse avec le code `200`.
C. Le navigateur envoie au serveur la requête `GET /cours.html`.

Puis : quel code le serveur aurait-il renvoyé si le fichier `cours.html` n'existait pas ? ▷ coup de pouce

★★★ **Exercice 7** *Le formulaire de recherche* ▶ corrigé p. 248 ≈ 15 min

[sur papier] Sur le site d'une médiathèque, on cherche « Harry Potter » au format « livre ». Après validation du formulaire, la barre d'adresse affiche (le `+` remplace l'espace) :

```
https://www.mediatheque.mc/chercher?titre=Harry+Potter&format=livre
```

1. Quelle ressource est demandée ? Quels paramètres sont passés, avec quelles valeurs ?
2. Écrire l'adresse obtenue si l'on cherche « Astérix » au format « BD ».
3. La page de résultats est-elle une page toute prête ou une page dynamique ? Justifier.
4. Pourquoi un site ne doit-il jamais transmettre un mot de passe de cette façon ?

▷ coup de pouce ▷ début de solution

L'URL et les chemins ↑ cours

★ **Exercice 8** *Décomposer une URL* ▶ corrigé p. 248 • essentiel

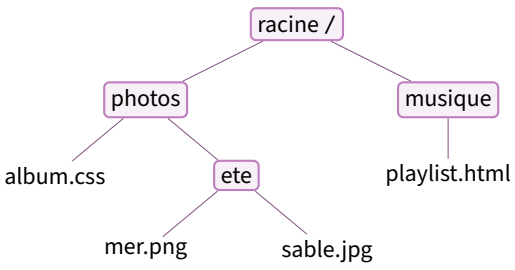
[sur papier] On considère l'URL : `https://www.lycee.mc/cours/snt/web.html`.

1. Quel est le **protocole** ? Que garantit-il de plus que `http` ?
2. Quel est le **nom du serveur** (domaine) ?
3. Quel est le **chemin** du fichier demandé ?

▷ coup de pouce

★★ **Exercice 9** *Chemins absolu et relatif* ▶ corrigé p. 248

[sur papier] On dispose de l'arborescence suivante (la racine est `/`) :



1. Donner le **chemin absolu** du fichier `sable.jpg`.
2. Le fichier `album.css` (dans `photos`) veut utiliser `sable.jpg`. Donner le **chemin relatif** correspondant.
3. La page `playlist.html` (dans `musique`) veut afficher `mer.png`. Donner le chemin relatif correspondant.

▷ coup de pouce

★★★ **Exercice 10** *Le site déménage* ▶ corrigé p. 248 ≈ 20 min

[sur papier] Le site d'un club est rangé ainsi : à la racine, le fichier `index.html` et deux dossiers, `images` (qui contient `logo.png`) et `pages` (qui contient `contact.html`).

1. Dessiner l'arborescence de ce site.
2. Dans `index.html`, le logo s'affiche avec `<img src="images/logo.png" alt="Logo du club" />`. On copie cette ligne telle quelle dans `contact.html` : le logo ne s'affiche plus. Pourquoi ? Corriger le chemin.
3. Écrire, dans `contact.html`, un lien qui ramène à `index.html`.
4. On renomme le dossier `images` en `img`. Quels fichiers faut-il modifier, et comment ?

▷ coup de pouce ▷ début de solution

Lire du HTML ↑ cours

★ **Exercice 11** *Que voit-on ?* ▶ corrigé p. 249 • essentiel

[sur papier] Décrire ce que le navigateur affiche pour ce code :

```
1 <h1>Recette</h1>
2 <p>Il faut <strong>trois</strong> oeufs.</p>
3 <ul>
4   <li>farine</li>
5   <li>sucre</li>
6 </ul>
```

▷ coup de pouce

★★ **Exercice 12** *Corriger les erreurs* ▶ corrigé p. 249

[sur papier] Ce code contient **deux** erreurs d'imbrication ou de fermeture. Les trouver et les corriger.

```
1 <p>Bonjour <strong>tout le monde</p></strong>
2 <h2>Chapitre 1
```

▷ coup de pouce

★★ **Exercice 13** *Du rendu au code* ▶ corrigé p. 249

[sur papier] Écrire le code HTML (contenu du `<body>` seulement) qui produit l'affichage suivant : un grand titre « Mon club de foot » ; un paragraphe « Entraînement le mercredi. » où le mot *mercredi* est mis en valeur (en gras) ; une liste à puces de deux éléments, « gardiens » et « joueurs » ; enfin un lien « Le site du club » vers `https://www.monclub.mc`.

▷ coup de pouce

Écrire du HTML et du CSS ↑cours

★★ **Exercice 14** *Ma page de présentation* ▶corrigé p. 249 • essentiel

▶ Écrire une page HTML **complète** (squelette compris) qui contient : un titre `<h1>` avec votre prénom, un paragraphe de présentation, une **liste** de trois loisirs, et un **lien** vers un site de votre choix. ▷coup de pouce ▷début de solution

★★ **Exercice 15** *Un peu de style* ▶corrigé p. 250

▶ Reprendre la page précédente et ajouter du CSS pour : centrer le titre et lui donner un fond coloré ; mettre les paragraphes dans la police `Verdana` ; ajouter en bas de page une ligne de signature (par exemple « Page réalisée par . . . ») et la colorer en gris, *via* un `id`. ▷coup de pouce

★★ **Exercice 16** *Le bon sélecteur* ▶corrigé p. 250 • essentiel

[sur papier] On considère le code suivant :

```html
<h1 id="haut">Horaires</h1>
<p class="info">Ouvert du lundi au vendredi.</p>
<p>Ferme le dimanche.</p>
<p class="info" id="fin">Contact : 93 00 00 00</p>
```

```css
p { font-family: Verdana; }
.info { color: green; }
#fin { text-align: right; }
```

1. Quels éléments sont écrits en Verdana ? Lesquels sont en vert ? Lequel est aligné à droite ?
2. Écrire une règle CSS qui met **uniquement** le titre en bleu, de deux façons différentes.

▷coup de pouce

★★ **Exercice 17** *Prévoir le rendu* ▶corrigé p. 250

[sur papier] Sans machine, décrire précisément le rendu (couleurs, alignement, style) de :

```html
<h1 class="titre">Actualites</h1>
<p class="titre">Bienvenue</p>
```

```css
.titre {
  text-align: center;
  color: darkblue;
}
```

▷coup de pouce

Droit d'auteur ↑cours

★★ **Exercice 18** *Ai-je le droit ?* ▶corrigé p. 250

[sur papier]

1. Vous trouvez une belle photo sur le Web et voulez la publier sur le site du lycée. Est-ce automatiquement autorisé ? Pourquoi ?
2. Qu'est-ce qu'une **licence** ? Que permet une licence **Creative Commons** ?
3. « Un fichier se copie gratuitement, donc il n'a aucune valeur. » Discuter cette phrase en une ou deux phrases.

▷coup de pouce

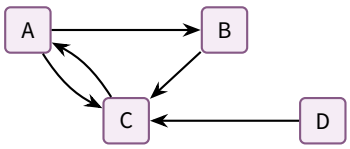
Trouver une page ↑cours

★ **Exercice 19** *Le moteur de recherche* ▶corrigé p. 251 • essentiel
[sur papier]

1. Qu'est-ce que l'**index** d'un moteur de recherche ? Quand vous tapez une requête, le moteur cherche-t-il « dans le Web » ou « dans son index » ?
2. Citer un critère qui aide un moteur à **classer** les pages (par exemple le PageRank).
3. Comment reconnaît-on, en haut d'une page de résultats, ce qui est une **publicité** ?

▷coup de pouce

★★★ **Exercice 20** *Le graphe des liens* ▶corrigé p. 251 ≈15 min
[sur papier] Quatre pages A, B, C et D se font des liens (une flèche de A vers B signifie : « la page A contient un lien vers la page B ») :



1. Pour chaque page, compter le nombre de liens qui pointent **vers** elle.
2. D'après l'idée du PageRank, quelle page sera jugée la plus importante ?
3. A et B reçoivent chacune un seul lien. Laquelle des deux devrait être mieux classée ? Justifier.
4. Un site crée des centaines de fausses pages qui pointent toutes vers lui. Que cherche-t-il à faire ? Pourquoi les moteurs de recherche luttent-ils contre cela ?

▷coup de pouce ▷début de solution

Sécurité et confidentialité ↑cours

★ **Exercice 21** *Régler et sécuriser son navigateur* ▶corrigé p. 251
[sur papier]

1. Citer trois réglages de sécurité/confidentialité que l'on peut modifier dans un navigateur.
2. Qu'est-ce que l'**hameçonnage** (*phishing*) ?
3. Avant de cliquer sur un lien reçu par message, quel **geste simple** permet de vérifier vers quelle adresse il mène réellement ?

▷coup de pouce

Critiquer une réponse d'IA ↑cours

★★ **Exercice 22** *Le cadenas rend-il anonyme ?* ▶corrigé p. 251
[sur papier] Un élève a demandé à un assistant d'IA : « Quand je vois le cadenas HTTPS, est-ce que je suis anonyme sur le site ? » Voici la réponse obtenue :

> *Le cadenas signifie que le site utilise HTTPS, la version sécurisée de HTTP : les données échangées entre votre navigateur et le serveur sont chiffrées. Personne ne peut donc lire au passage votre mot de passe ou votre numéro de carte, ce qui est indispensable avant de les saisir. Et comme la connexion est chiffrée, le site ne peut pas vous suivre : avec HTTPS, les cookies sont désactivés et votre navigation est totalement anonyme. Le cadenas ne garantit pas, en revanche, que le site est honnête : une page d'hameçonnage peut très bien être en HTTPS.*

1. La réponse est-elle correcte ? Relire la section « Sécurité et confidentialité » du cours et comparer phrase par phrase.

2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Python au service du thème ↑ cours

★★ **Exercice 23** *Python au service du thème — fabriquer du HTML* ▶ corrigé p. 252

▶ En Python (Spyder ou Basthon). Un serveur peut **fabriquer** une page sur mesure avant de l'envoyer (page *dynamique*) : c'est alors un programme qui écrit le code HTML. On le fait ici en Python, en collant des chaînes de caractères (concaténation).

1. Écrire une fonction `balise(nom, contenu)` qui renvoie le code d'un élément HTML : `balise("h1", "Mon club")` doit renvoyer `"<h1>Mon club</h1>"`. La tester aussi avec `balise("strong", "mercredi")`.
2. Sans machine, écrire ce qu'affiche le programme suivant, puis vérifier. Que montrerait un navigateur qui recevrait ce code HTML ?

```
1 print("<ul>")
2 for i in range(1, 4):
3     print(balise("li", f"Photo {i}"))
4 print("</ul>")
```

3. Écrire une fonction `url(protocole, serveur, chemin)` qui reconstitue une URL complète. La tester avec les trois morceaux trouvés à l'exercice « Décomposer une URL » : on doit retrouver l'URL complète de cet exercice.

▷ coup de pouce ▷ début de solution

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relisez la section « Le Web n'est pas Internet » : le Web est-il le réseau lui-même, ou un service qui circule sur ce réseau ? ↑ énoncé

Ex. 2 – piste Pour chaque élément, demandez-vous s'il fait partie des trois technologies du Web (pages HTML, adresses URL, protocole HTTP) ou s'il utilise le réseau autrement. ↑ énoncé

Ex. 3 – piste Tout ce qui suit le `?` est formé de paramètres `nom=valeur`, séparés par `&`. ↑ énoncé

Ex. 4 – piste Relisez le tableau des codes de réponse dans la section du cours sur le protocole HTTP. ↑

Ex. 5 – piste Relisez l'encadré « Deux endroits où ça calcule » du cours : que fait le serveur, que fait le navigateur ? ↑ énoncé

Ex. 6 – piste Dans un échange HTTP, c'est toujours le client qui parle le premier. ↑ énoncé

Ex. 7 – piste Repérez le `?` et le `&` pour découper l'adresse. Pour la question 4 : où s'affichent les paramètres, et où restent-ils enregistrés ensuite ? ↑ énoncé

Ex. 7 – début de solution Seules les valeurs changent : `titre=...` et `format=...`. Le serveur peut-il avoir préparé à l'avance une page pour chaque titre possible ? ↑ énoncé

Ex. 8 – piste Une URL se lit de gauche à droite : ce qui précède `://`, puis le nom du serveur jusqu'au premier `/` qui suit, puis le reste. ↑ énoncé

Ex. 9 – piste Chemin absolu : on part de la racine `/` et on descend dossier par dossier. Chemin relatif : on part du dossier où se trouve le fichier qui fait le lien, sans `/` au début ; `..` fait remonter d'un niveau. ↑ énoncé

Ex. 10 – piste Un chemin relatif se lit à partir du dossier où se trouve la page qui l'utilise. Où se trouve `contact.html` ? Faut-il d'abord remonter ? ↑ énoncé

Ex. 10 – début de solution Depuis le dossier `pages`, `..` ramène à la racine du site, où se trouvent `index.html` et le dossier `images`. ↑ énoncé

Ex. 11 – piste Relisez dans le cours le rôle de chaque balise : `<h1>`, `<p>`, `<strong>`, `<ul>` et `<li>`. ↑

Ex. 12 – piste Relisez les deux règles d'or des balises : toute balise ouverte est-elle refermée ? Laquelle a été ouverte en dernier ? ↑énoncé

Ex. 13 – piste Une balise par élément : `<h1>` pour le titre, `<p>` et `<strong>` pour le paragraphe, `<ul>` et `<li>` pour la liste, `<a href="...">` pour le lien. ↑énoncé

Ex. 14 – piste Partez du squelette du cours : `<head>` pour ce qui est invisible (titre de l'onglet), `<body>` pour tout ce qui s'affiche. ↑énoncé

Ex. 14 – début de solution
```
<!DOCTYPE html>
<html>
   <head>
      <meta charset="utf-8" />
      <title>Ma page</title>
   </head>
   <body> ...
```
↑énoncé

Ex. 15 – piste Une règle CSS : un sélecteur, puis des propriétés entre accolades (`text-align`, `background-color`, `font-family`, `color`). Pour la signature : donnez-lui un `id` dans le HTML, puis ciblez-le en CSS avec `#`. ↑énoncé

Ex. 16 – piste Pour chaque règle, lisez le sélecteur : `p` vise une balise, `.info` une classe (le point), `#fin` un identifiant (le dièse). Cherchez ensuite les éléments concernés dans le HTML. ↑

Ex. 17 – piste Quels éléments du code HTML portent la classe `titre` ? La règle `.titre` s'applique à chacun d'eux. ↑énoncé

Ex. 18 – piste Relisez la section « À qui appartient une page ? » : qui décide de l'usage d'une œuvre, et comment l'auteur peut-il autoriser certains usages à l'avance ? ↑énoncé

Ex. 19 – piste Relisez la définition du moteur de recherche et la règle « L'ordre des résultats n'est pas neutre », dans le cours. ↑énoncé

Ex. 20 – piste Relisez la règle du cours : une page est importante si *beaucoup* d'autres pages *importantes* pointent vers elle. Il y a donc deux choses à regarder : le nombre de liens reçus, et l'importance de la page qui envoie le lien. ↑énoncé

Ex. 20 – début de solution Liens reçus : A en reçoit un (de C), B un (de A). Qui, de C ou de A, est la page la plus importante du graphe ? ↑énoncé

Ex. 21 – piste Relisez la section « Sécurité et confidentialité » du cours. Pour la question 3 : le texte affiché d'un lien est-il forcément l'adresse vers laquelle il mène ? ↑énoncé

Ex. 22 q. 2 – piste Relisez l'encadré du cours sur les cookies et les traces de navigation : que dit-il de l'anonymat ? ↑énoncé

Ex. 23 – piste Écrivez d'abord sur papier le résultat attendu, `<h1>Mon club</h1>`, et repérez les morceaux qui changent d'un appel à l'autre (le nom de la balise, le contenu) et ceux qui ne changent jamais (les chevrons, la barre oblique) : il reste à les coller avec `+`. ↑

Ex. 23 – début de solution
```
def balise(nom, contenu):
    return "<" + nom + ">" + contenu + ...
```
(il reste la balise fermante.) Pour `url`, n'oubliez ni `"://"` ni la barre `"/"` entre le serveur et le chemin. ↑énoncé

DÉBAT

DÉBAT Un moteur de recherche doit-il rester libre de classer ses résultats comme il le veut ?

▶ corrigé p. 252

moteurs de recherche, référencement et accès à l'information

Durée : 50 min

Consignes

- En classe entière. Les rôles sont tirés au sort : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement.
- Chaque argument s'appuie sur un **document** ci-dessous ou sur le **cours**. Préparation sans IA ni recherche en ligne.
- Avant de commencer, notez sur votre cahier votre position de départ : oui, non ou je ne sais pas.

Rappel du cours. Un **moteur de recherche** ne cherche pas « dans le Web » mais dans son **index**, construit par des robots qui suivent les liens. Il **classe** les pages par pertinence grâce à un **algorithme** (le *PageRank*, les mots-clés, la langue, la localisation…) et les premiers résultats peuvent être des **publicités**. Presque personne ne va voir la deuxième page de résultats : être classé en tête, c'est exister. Mais qui doit fixer les règles de ce classement ?

Les deux camps

Groupe POUR — oui : c'est le service du moteur, il doit rester libre de son classement.
Groupe CONTRE — non : la loi doit fixer des règles au classement des résultats.

Les documents

Document 1 — Google Shopping : 2,42 milliards d'euros d'amende

Le 27 juin 2017, la Commission européenne a infligé à Google une amende de **2,42 milliards d'euros**. Google, en **position dominante** dans la recherche en ligne en Europe, en avait abusé : son propre comparateur de prix (Google Shopping) apparaissait en haut des résultats, et les comparateurs concurrents plus bas, là où presque personne ne clique. La Cour de justice de l'Union européenne a confirmé l'amende en 2024.

À retenir : en Europe, un moteur dominant ne peut pas favoriser ses propres services.

Source : Commission européenne, décision du 27 juin 2017 (affaire AT.39740) ; Cour de justice de l'Union européenne, arrêt du 10 septembre 2024.

Document 2 — De nouvelles règles européennes

Depuis mars 2024, le règlement européen sur les marchés numériques (*DMA*) interdit aux plus grandes plateformes de **mieux classer leurs propres services** que ceux des concurrents. Le règlement sur les services numériques (*DSA*) oblige les très grands moteurs à proposer des résultats qui **ne reposent pas sur le profil** de l'utilisateur. Un règlement de 2019 leur impose aussi de rendre publics les **principaux paramètres** de leur classement.

À retenir : ces règles encadrent la façon de classer, pas l'ordre des résultats.

Source : règlements (UE) 2022/1925 (DMA), 2022/2065 (DSA, article 38) et 2019/1150 (article 5).

Document 3 — Le grand jeu du référencement

Être en tête des résultats vaut de l'or. Des entreprises paient des spécialistes du **référencement** pour améliorer la place de leurs pages, et achètent des **annonces** affichées au-dessus des résultats avec la mention « Sponsorisé ». Le moteur, lui, garde une partie de ses critères **secrète** : si la recette exacte était connue, les tricheurs passeraient devant. D'autres moteurs (Bing, Qwant, Ecosia, DuckDuckGo…) classent les pages autrement.

À retenir : le classement est un enjeu commercial, et sa recette reste en partie secrète.

Source : d'après le cours (« L'ordre des résultats n'est pas neutre »).

Document 4 — Le droit au déréférencement

Le 13 mai 2014, la Cour de justice de l'Union européenne a jugé qu'un particulier peut demander à un moteur de ne plus afficher, quand on tape **son nom**, des liens vers des informations dépassées ou inexactes le concernant. La page reste en ligne, mais disparaît des résultats. Le RGPD a confirmé ce droit ; à Monaco, la loi n° 1.565 prévoit aussi un droit à l'effacement. Chaque demande est examinée par le **moteur lui-même**.

À retenir : le moteur doit parfois retirer un résultat, et c'est lui qui tranche.

Source : Cour de justice de l'Union européenne, arrêt du 13 mai 2014, Google Spain (C-131/12) ; RGPD, article 17 ; loi monégasque n° 1.565 du 3 décembre 2024, article 14.

Rôles, déroulé minuté, règles de prise de parole et trace écrite : voir la page « Méthode : débattre » p. 7.

Tableau d'arguments

Recopier ce tableau sur le cahier et le compléter pendant le débat.

Argument POUR (oui)	Doc.	Argument CONTRE (non)	Doc.
…		…	

Trace écrite finale (individuelle)

Sur le cahier, notez votre position après le débat (oui, non, je ne sais toujours pas) et si vous avez changé d'avis. Puis, en trois à cinq lignes, justifiez-la avec vos **deux meilleurs arguments**, dont **un argument de l'autre camp** que vous trouvez solide (avec le document qui l'appuie).

FICHE DE RÉVISION

L'essentiel

- **Internet** = le **réseau** ; le **Web** = **un service** d'Internet : des pages reliées par des **hyperliens**.
- **Client** (navigateur) → **requête HTTP** → **serveur** → **réponse** (code + page) ; **HTTPS** = **chiffré**.
- **HTML** décrit le **contenu** (balises) ; **CSS** décrit l'**apparence**.
- Un **moteur de recherche** cherche dans son **index**, puis **classe** les pages : l'ordre n'est pas neutre.

Méthode 1 — Décomposer une URL et la requête

`https://www.lycee.mc/meteo?ville=Monaco&jour=lundi`
↑ protocole ↑ domaine (serveur) ↑ ressource (chemin) ↑ paramètres

Après le `?` : paramètres `nom=valeur` séparés par `&` ; ici **2** : `ville` vaut `Monaco`, `jour` vaut `lundi`. Codes : **200** ok, **403** interdit, **404** introuvable, **500** erreur serveur. PHP s'exécute sur le **serveur**, JavaScript chez le **client**.

Méthode 2 — Lire du HTML, cibler en CSS

```html
<h1 id="titre">Blog</h1>
<p class="info">Bon <strong>match</strong></p>
<a href="page2.html">Suite</a>
```

```css
#titre { color: red; }
.info { font-style: italic; }
p { font-family: Verdana; }
```

Rendu : titre « Blog » rouge ; paragraphe italique en Verdana, « match » en gras ; lien « Suite ». Sélecteurs : `p` (toutes les `<p>`), `#id` (unique), `.classe` (partagée). **Deux règles d'or** : toute balise ouverte est refermée ; les balises s'imbriquent proprement (`<p><strong>...</strong></p>`).

Repères, enjeux et erreurs à éviter

1989 : Tim Berners-Lee invente le Web au **CERN** ; **1994** : **W3C**. **Droit d'auteur** : toute création est protégée ; on ne republie qu'avec l'accord de l'auteur ou une **licence** (Creative Commons). **Sécurité** : régler cookies, historique, navigation privée ; **hameçonnage** = fausse page voleuse d'identifiants ⇒ survoler le lien.

Erreurs : confondre Web et Internet (l'e-mail circule sur Internet, hors du Web) ; croire que le cadenas garantit un site honnête (il garantit le chiffrement) ; confondre `#` (id) et `.` (classe) ; fermer les balises dans le désordre.

Savoir-faire à maîtriser — je sais…

- ☐ distinguer le **Web** d'**Internet** ;
- ☐ décrire l'échange **client / serveur** et le rôle du protocole **HTTP** ;
- ☐ décomposer une **URL** (protocole, nom de domaine, chemin) ;
- ☐ écrire une page simple en **HTML** (balises de structure) et la mettre en forme avec **CSS** ;
- ☐ créer un **hyperlien** d'une page vers une autre ;
- ☐ expliquer le fonctionnement d'un **moteur de recherche** (indexation puis classement) ;
- ☐ régler mon navigateur pour préserver ma sécurité et ma vie privée (HTTPS, cookies).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 253

1. Citer un service d'Internet qui ne fait pas partie du Web.
2. Donner domaine, ressource et paramètres de `https://www.bus.mc/horaires?ligne=2&arret=Casino`.
3. Corriger `<p>Bonne <strong>lecture</p></strong>`.
4. Écrire la règle CSS qui met en vert tous les éléments de classe `alerte`.
5. Expliquer ce que signifie le code de réponse 404.

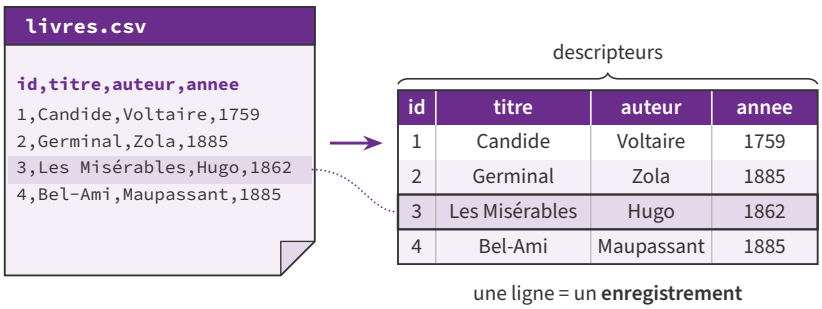
Et avant ? — *questions sur les chapitres précédents.* ▶ réponses p. 253

1. Donner le code RVB du blanc (ch. 3).
2. Donner ce qu'affiche `print("www." + "lycee.mc")` (ch. 1).

5

CHAPITRE 5

Les données structurées et leur traitement



DANS CE CHAPITRE

- Cours 130
- Exercices 139
- TP et débat 145
- Fiche de révision 152
- Corrigés 255

COURS

Programme Donnée et **donnée personnelle** ; **métadonnées** ; principaux **formats** et représentations (formats ouverts, **CSV**) ; **table** de données (descripteur, enregistrement) ; opérations de **recherche**, **filtre** et **tri** sur une ou plusieurs tables ; données **ouvertes** (*open data*) et **dans le nuage**.

Idée Le monde est noyé sous les **données**. Bien rangées dans une **table** (lignes et colonnes), elles deviennent une mine que l'on peut *fouiller* : chercher, filtrer, trier, croiser. Ce chapitre vous transforme en **data-détective**.

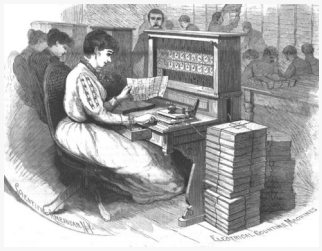
Objectifs Distinguer donnée, donnée personnelle et métadonnée ; lire une table (descripteur, enregistrement) ; comprendre le format **CSV** ; **rechercher**, **filtrer**, **trier** et **croiser** des tables ; utiliser un site d'*open data* ; mesurer les enjeux (vie privée, nuage, *big data*).

Situation de départ

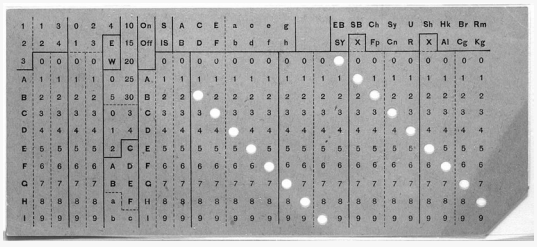
1. Sur un site de vente, vous cochez « moins de 30 € » et « taille M », puis « prix croissant ». Que fait le site, au juste, de la liste de ses articles ?
2. Vos photos sont « dans le nuage ». Où sont-elles vraiment, et qui peut les voir ?
3. Une application vous demande votre âge, votre position et vos contacts. A-t-elle le droit de tout garder et d'en faire ce qu'elle veut ?

Pour les curieux — *Un peu d'histoire* (facultatif)

Ranger des données par machine est *plus vieux* que l'ordinateur ! En **1890**, les États-Unis doivent recenser près de 63 millions d'habitants ; le recensement précédent, celui de 1880, avait demandé **sept ans** de dépouillement à la main. L'ingénieur **Herman Hollerith** invente une machine qui lit des **cartes perforées** — une carte par personne, un trou par caractéristique : c'est déjà « une fiche par individu, les mêmes rubriques pour tous », l'idée même d'une **table**. Le recensement de 1890 est bouclé avec **plusieurs mois d'avance** (U.S. Census Bureau). La société de Hollerith deviendra, par fusions, une certaine… **IBM**.



Les machines de Hollerith en 1890



Une carte perforée : une carte par personne

Source : U.S. Census Bureau, pages d'histoire « Herman Hollerith » et « Tabulation and Processing » (recensements de 1880 et 1890), `census.gov`.

→ frise chronologique p. 10

I Une donnée, c'est quoi au juste ?

Un texto, une note, une température, un « j'aime », une position GPS… tout cela, ce sont des **données**.

Définition 1

Une **donnée** est une information élémentaire enregistrée (un nombre, un mot, une date, une image…), que l'on peut stocker et traiter par une machine.

Certaines données sont banales ; d'autres vous concernent *directement* et méritent une protection particulière.

Définition 2

Une **donnée personnelle** est une information qui permet d'**identifier une personne**, directement (nom, photo, numéro de téléphone) ou indirectement (adresse IP, position, identifiant, plaque d'immatriculation…).

Exemple *« La température était de 21 °C »* est une donnée, mais **pas** personnelle. *« Léa, née le 3 mai 2009 à Monaco »* contient des données **personnelles**.

A. Les métadonnées : les données sur les données

Définition 3

Une **métadonnée** est une donnée qui **décrit une autre donnée**. Elle ne fait pas partie du contenu, mais l'accompagne : pour une photo, sa **date**, l'**appareil** utilisé, le lieu (**GPS**) ; pour un fichier, son **nom**, sa **taille**, sa date de **modification**.

Méthode — Retrouver les métadonnées d'un fichier

On peut **retrouver les métadonnées** d'un fichier personnel : un clic droit » « Propriétés » (ou « Informations ») révèle taille, dates, format… Une photo prise au smartphone embarque même, souvent, les **coordonnées GPS** du lieu de la prise de vue (chapitre *Photographie numérique*) : une métadonnée qui peut, sans qu'on y pense, **trahir où l'on habite**.

▶ Activité 1 — Chasse aux métadonnées

Sur un fichier de votre espace personnel (une photo, un document), afficher les **propriétés** et relever trois métadonnées : la **taille**, la **date** de dernière modification, et le **format** (l'extension). Lesquelles sont des données *personnelles* ?

▶ **Exercices d'application :** exercices 1 **et** 2 (donnée personnelle ? donnée ou métadonnée ?)

II Ranger l'information : la table de données

Pour *exploiter* des données, on les range dans un **tableau** à deux dimensions : une **table**. Voici notre fil rouge, une petite table de jeux vidéo (les **ventes** sont en millions d'exemplaires, arrondies, d'après les chiffres publiés par les éditeurs en 2023 ; elles ont augmenté depuis) :

id	titre	studio	annee	genre	pegi	ventes
1	Minecraft	2	2011	bac à sable	7	300
2	GTA V	3	2013	action	18	190
3	Wii Sports	1	2006	sport	7	83
4	Mario Kart 8	1	2014	course	3	60
5	Red Dead Redemption 2	3	2018	action	18	55
6	Animal Crossing	1	2020	simulation	3	45

Source : chiffres de ventes arrondis (2023), communiqués des éditeurs (Microsoft/Mojang, Take-Two/Rockstar Games, Nintendo).

Définition 4

Dans une **table de données** :

- chaque **colonne** est un **descripteur** (ou *attribut*) : ce qu'on renseigne pour chacun (ici `titre`, `annee`, `genre`…) ;
- chaque **ligne** est un **enregistrement** (une *fiche*, un *individu*) : elle donne **une valeur pour chaque descripteur**.

Ainsi, la table ci-dessus a **7 descripteurs** et **6 enregistrements**. La ligne d'`id 1` est *un* enregistrement ; sa valeur pour le descripteur `genre` est `bac à sable`.

À retenir — Pourquoi un descripteur « id » ?

Deux jeux pourraient porter le même titre, ou sortir la même année. Pour **distinguer sans ambiguïté** chaque enregistrement, on ajoute souvent un descripteur `id` : un numéro **unique**. On dit que c'est un **identifiant**.

▶ Activité 2 — Lire une table

À l'aide de la table *jeu* :

1. Combien y a-t-il de descripteurs ? d'enregistrements ?
2. Quelle est la valeur du descripteur `ventes` pour *Mario Kart 8* ?
3. Citer un descripteur dont les valeurs sont des **nombres**, et un dont les valeurs sont du **texte**.

▶ **Exercices d'application :** exercices 3 **et** 4 (vocabulaire de la table ; concevoir une table)

III Le format CSV : du texte, et rien que du texte

Comment *stocker* une table dans un fichier que *tous* les logiciels sauront lire ? Avec le format **CSV**.

Définition 5

Un fichier **CSV** (*Comma-Separated Values*, « valeurs séparées par des virgules ») est un simple fichier **texte** qui représente une table :

- la **première ligne** donne les descripteurs (l'*en-tête*) ;
- **chaque ligne suivante** est un enregistrement ;
- sur une ligne, les valeurs sont séparées par un **séparateur** (souvent la virgule `,` ou le point-virgule `;`).

Les trois premières lignes de notre table, en CSV :

```
id,titre,studio,annee,genre,pegi,ventes
1,Minecraft,2,2011,bac a sable,7,300
2,GTA V,3,2013,action,18,190
```

Règle 1 — Format ouvert / format fermé

Le CSV est un **format ouvert** : sa structure est publique, n'importe quel logiciel (tableur, éditeur de texte, navigateur, programme) peut le lire, aujourd'hui comme dans trente ans, sans dépendre d'une marque. À l'inverse, un **format fermé** (propriétaire) est contrôlé par un éditeur, qui seul en connaît tous les détails : l'ancien format `.xls` d'Excel, par exemple. Son successeur, le `.xlsx`, est aujourd'hui une norme publiée, mais il reste bien plus complexe qu'un simple fichier texte. Les **données ouvertes** (*open data*) sont presque toujours publiées en formats ouverts.

Document 1 — Quand un mauvais format fait « perdre » 16 000 malades

En **octobre 2020**, l'agence de santé publique anglaise a « perdu » **15 841** résultats de tests COVID-19. La cause ? Les laboratoires envoyaient leurs résultats en fichiers **CSV** ; pour les rassembler, l'agence les importait automatiquement dans un vieux format de tableur, le `.xls`, **limité à 65 536 lignes**. Chaque cas occupant plusieurs lignes, un fichier ne pouvait contenir qu'environ 1 400 cas. Une fois cette limite atteinte, les cas suivants n'étaient signalés par *aucune* erreur… ils étaient **silencieusement ignorés**. Résultat : près de **48 000** personnes ayant croisé un cas positif n'ont pas été prévenues à temps (Public Health England et BBC News, octobre 2020). Le fichier **CSV** d'origine, lui, n'avait pas de plafond : un texte peut avoir *autant de lignes qu'on veut. La morale : le bon format de données peut avoir des conséquences très réelles.*

Sources : Public Health England, communiqué du 5 octobre 2020 (15 841 cas non signalés) ; BBC News, « Excel : Why using Microsoft's tool caused Covid-19 results to be lost », 5 octobre 2020.

À retenir — L'*open data* : les données publiques, à tous

Depuis **2011**, l'État français ouvre ses données sur `data.gouv.fr` (portail géré par *Etalab*, le service de l'État chargé d'ouvrir les données publiques) ; la Principauté de Monaco fait de même sur son portail **open data**. Horaires de bus, qualité de l'air, résultats d'élections, prénoms donnés chaque année… des milliers de tables sont **librement téléchargeables**, le plus souvent en CSV. Fouiller ces données est devenu un vrai pouvoir citoyen — et le métier des **data-journalistes**.

Source : Etalab, `data.gouv.fr`, page « À propos » (ouverture du portail en 2011).

► **Exercices d'application :** exercices 5 à 7 (lire et écrire du CSV)

5

IV Fouiller une table : rechercher, filtrer, trier

C'est le cœur du chapitre — et du métier de data-détective. Trois opérations reviennent sans cesse.

Règle 2 — Les trois opérations de base

- **Rechercher** : trouver *l'*enregistrement (ou les) qui correspond à une valeur précise.
  « Quel est le genre de Minecraft ? »
- **Filtrer** : ne garder que les enregistrements qui vérifient un **critère**.
  « Tous les jeux de genre `action`*. »* → GTA V, Red Dead Redemption 2.
- **Trier** : ranger les enregistrements selon les valeurs d'**un** descripteur, en ordre croissant ou décroissant.
  « Les jeux du plus vendu au moins vendu. » → Minecraft, GTA V, Wii Sports…

Exemple — *Filtrer puis trier* *« Les jeux tout public (*`pegi` $\leqslant$ *7), du plus récent au plus ancien. »*
On **filtre** d'abord (`pegi` vaut 3 ou 7) : Minecraft, Wii Sports, Mario Kart 8, Animal Crossing. Puis on **trie** par `annee` décroissante : Animal Crossing (2020), Mario Kart 8 (2014), Minecraft (2011), Wii Sports (2006).

À retenir — Un critère peut en combiner plusieurs

On combine des conditions avec **ET** (les deux à la fois) ou **OU** (au moins une). *« Les jeux* `action` ***ET*** *sortis après 2015 »* ne garde que Red Dead Redemption 2. C'est le rôle de `and` et de `or` en Python (chapitre *Les bases de Python*).

A. Croiser deux tables

Notre table range le studio sous forme d'un **numéro** (2, 3…), pas d'un nom. Les noms sont dans une **seconde** table, *studio* :

id	nom	pays
1	Nintendo	Japon
2	Mojang	Suède
3	Rockstar Games	États-Unis

Règle 3 — Croiser (ou fusionner) deux tables

Deux tables partagent souvent un descripteur commun (ici, le **numéro de studio**). **Croiser** les tables, c'est **compléter** chaque enregistrement de l'une par les informations de l'autre **qui a la même valeur** pour ce descripteur.

Exemple Minecraft a `studio` = `2` ; dans la table *studio*, l'`id` `2` est **Mojang** (**Suède**). En croisant les deux tables, on apprend donc que *Minecraft est édité par Mojang, un studio suédois* — une information qu'**aucune** des deux tables ne donnait à elle seule.

▶ Activité 3 — Data-détective : premières fouilles

Sur les tables *jeu* et *studio* :

1. **Rechercher** : en quelle année est sorti *Red Dead Redemption 2* ?
2. **Filtrer** : donner les titres des jeux **déconseillés aux mineurs** (`pegi` = 18).
3. **Trier** : ranger tous les jeux par `annee` **croissante**.
4. **Croiser** : dans quel **pays** est édité *Mario Kart 8* ?

▶ **Exercices d'application :** exercices 8 à 13 (rechercher, filtrer, trier, calculer, croiser)

V L'outil du quotidien : le tableur

Pour réaliser ces opérations *sans programmer*, l'outil roi est le **tableur** (Excel, LibreOffice Calc, Google Sheets…). On y ouvre un CSV, et chaque descripteur devient une **colonne** (`A`, `B`, `C`…), chaque enregistrement une **ligne** numérotée.

Règle 4 — Ce que le tableur sait faire d'un clic

- **Trier** une colonne (croissant / décroissant) : bouton *Trier*.
- **Filtrer** : le *filtre automatique* n'affiche que les lignes qui vérifient un critère.
- **Calculer** sur une colonne avec des **formules** : `=SOMME(...)`, `=MOYENNE(...)`, `=MAX(...)`…
- **Représenter** les données par un **graphique** (barres, camembert…).

Remarque Le tableur reste **le** logiciel le plus utilisé au monde pour manipuler des données. Les **TP tableur** proposés en fin de chapitre en font pratiquer les gestes de base (formules, recopie, calculs sur une colonne) ; les mêmes gestes servent ensuite à trier et filtrer une vraie table, par exemple `jeux.csv` ouvert dans le tableur (menu *Données* : *Trier*, *Filtre automatique*). *(Attention toutefois au piège du §III : un tableur a des **limites** cachées que le CSV n'a pas.)*

▶ **Exercices d'application :** exercices 14 et 15 (fouiller au tableur ; lire des formules)

VI Données personnelles, nuage et *big data*

A. Vos données valent de l'or : le RGPD

Chaque site, chaque application **collecte** des données sur vous. La loi encadre cette collecte.

Règle 5 — Le RGPD

Le **RGPD** (Règlement Général sur la Protection des Données, 2018) protège vos données personnelles dans l'Union européenne. Il impose notamment : le **consentement** (on doit vous demander votre accord), un **but précis** (on ne collecte que le nécessaire), et des **droits** pour vous : accéder à vos données, les corriger, et **les faire effacer** (« droit à l'oubli »).
Vos droits. Si vous résidez en France (ou dans l'Union européenne), vos droits relèvent du **RGPD** et de la loi *Informatique et libertés* ; l'autorité de contrôle est la **CNIL**. En France, en dessous de **15 ans**, l'inscription à un service en ligne demande l'accord des parents.

Et à Monaco ?

Monaco n'est **pas** membre de l'Union européenne : la loi n° 1.565 (2024) y garantit les mêmes grands droits, sous le contrôle de l'**APDP** (voir l'encadré « Et à Monaco ? » du chapitre *Les réseaux sociaux*).

B. Les données dans le nuage (*cloud*)

Définition 6

Stocker ses données **dans le nuage** (*cloud*), c'est les enregistrer non pas sur son appareil, mais sur des **serveurs distants**, accessibles par Internet (photos synchronisées, documents partagés, sauvegardes…).

À retenir — Le nuage : pratique… mais

Le nuage offre l'**accès partout** et le **partage** facile. En contrepartie : vos données sont **chez quelqu'un d'autre** (une entreprise, souvent à l'étranger), il faut une **connexion**, et la question se pose : *qui* peut les lire, et *où* sont-elles vraiment stockées ?

C. Le *big data* : le déluge de données

Règle 6 — Les « 3 V » du *big data*

On parle de **big data** (mégadonnées) quand les données deviennent trop volumineuses pour les outils classiques. On les caractérise par **trois V** : le **Volume** (des quantités colossales), la **Vélocité** (elles arrivent en continu, très vite) et la **Variété** (textes, images, capteurs, clics…). Bien exploitées, elles nourrissent les recommandations, les prévisions, l'intelligence artificielle.

▶ Activité 4 — Explorer un vrai jeu de données ouvert

Sur `data.gouv.fr` (ou le portail *open data* de Monaco) :

1. Chercher un jeu de données qui vous intéresse (transports, environnement, culture…) et noter son **titre** et son **format** (CSV ?).
2. Combien a-t-il de **descripteurs** (colonnes) ? Citez-en trois.
3. Formuler **une** question à laquelle ces données permettraient de répondre par un **tri** ou un **filtre**.

▶ **Exercices d'application :** exercices 16 **et** 17 (open data ; mes données, mes droits), défi 18 (data-détective sur l'open data), exercice 19 (critiquer une réponse d'IA), exercice 20 (Python au service du thème)

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Déjà vu :** les **métadonnées EXIF** d'une photo (chapitre *Photographie numérique*) forment une petite fiche de descripteurs ; un critère combiné s'écrit avec `and` et `or` en Python (chapitre *Les bases de la programmation Python*).
- **À venir :** au chapitre *Localisation, cartographie et mobilité*, une **trame NMEA** se lira comme une ligne de **CSV** ; au dernier chapitre, les mesures des **capteurs** d'objets connectés viendront grossir le **nuage** et le *big data*.
- **Plus loin :** en spécialité NSI de Première, on lit des fichiers CSV en Python et on filtre une table avec une boucle `for` et un test `if`.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Donnée	une information élémentaire enregistrée (nombre, mot, date…).
Donnée personnelle	permet d'**identifier** une personne (directement ou non).
Métadonnée	une donnée qui **décrit** une autre donnée (date, taille, GPS d'une photo…).
Table	lignes et colonnes ; **descripteur** = colonne, **enregistrement** = ligne.
CSV	fichier *texte*, format **ouvert** ; 1re ligne = en-tête ; séparateur `,` ou `;`.
Format ouvert / fermé	public et pérenne (CSV) / contrôlé par un éditeur (ancien `.xls`).
Rechercher / filtrer / trier	trouver / ne garder que… / ranger selon un descripteur.
Croiser deux tables	les relier par un descripteur commun.
Tableur	l'outil pour trier, filtrer, calculer, représenter (Excel, Calc, Sheets).
Open data	données publiques librement réutilisables (`data.gouv.fr`…).
RGPD	protège les données personnelles (UE) : consentement, droits, droit à l'oubli (CNIL).
Monaco	loi n° 1.565 (2024) : les mêmes grands droits ; autorité : APDP.
Nuage (*cloud*)	données sur des **serveurs distants**, accessibles par Internet.
Big data	les « 3 V » : **V**olume, **V**élocité, **V**ariété.

Retour à la situation de départ

1. Ses articles sont rangés dans une **table** : une ligne par article (un **enregistrement**), une colonne par caractéristique (un **descripteur**) (§ II). Le site **filtre** avec un critère combiné par ET (prix et taille), puis **trie** selon le descripteur prix (§ IV).
2. Sur des **serveurs distants**, accessibles par Internet, chez une entreprise souvent à l'étranger (§ VI) : c'est pratique, mais il faut une connexion, et il faut se demander qui peut les lire.
3. Non : ce sont des **données personnelles** (§ I). Le **RGPD** (UE) et la loi n° 1.565 (Monaco) imposent votre **consentement** et un **but précis**, et vous donnent le droit d'accéder à vos données, de les corriger et de les faire effacer (§ VI).

Débat argumenté

Un **débat**, proposé en fin de chapitre, accompagne ce thème : « Faut-il ouvrir à tous le plus possible de données publiques, même quand elles concernent des personnes ? » (documents, rôles, tableau d'arguments et trace écrite, environ 50 min).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ distinguer une **donnée**, un **descripteur** (attribut) et un **enregistrement** (ligne) ;

☐ retrouver les **métadonnées** d'un fichier personnel et les distinguer des données ;

☐ lire une table de données : repérer les descripteurs et compter les enregistrements ;

☐ reconnaître le format **CSV** (du texte, un séparateur, une ligne d'en-tête) et l'ouvrir dans un tableur ;

☐ utiliser un site de **données ouvertes** (*open data*) pour sélectionner et récupérer des données ;

☐ **rechercher**, **filtrer** et **trier** des enregistrements selon un ou plusieurs critères ;

☐ réaliser un tri croissant ou décroissant sur une colonne ;

☐ citer les enjeux liés aux **données personnelles**, au *cloud* et au *big data* (les 3 V).

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- [sur papier] *à faire sur papier* ; [au tableur] *à réaliser au tableur* (LibreOffice Calc, Excel ou Google Sheets).
- Plusieurs exercices utilisent la table *jeu* (ventes en millions d'exemplaires) et la table *studio* ci-dessous.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Table *jeu* :

id	titre	studio	annee	genre	pegi	ventes
1	Minecraft	2	2011	bac à sable	7	300
2	GTA V	3	2013	action	18	190
3	Wii Sports	1	2006	sport	7	83
4	Mario Kart 8	1	2014	course	3	60
5	Red Dead Redemption 2	3	2018	action	18	55
6	Animal Crossing	1	2020	simulation	3	45

Table *studio* :

id	nom	pays
1	Nintendo	Japon
2	Mojang	Suède
3	Rockstar Games	États-Unis

Données, données personnelles, métadonnées ↑cours

★ **Exercice 1** *Donnée personnelle ou pas ?* ▶ corrigé p. 256 • essentiel

[sur papier] Pour chaque information, dire si c'est une **donnée personnelle** (elle permet d'identifier une personne) ou non. Justifier en trois mots.

a. l'adresse e-mail d'un élève ;
b. la température qu'il fait à Monaco aujourd'hui ;
c. le numéro de téléphone de votre voisin ;
d. le nombre d'habitants de la France ;
e. la position GPS enregistrée dans votre dernière photo.

▷ coup de pouce

★ **Exercice 2** *Donnée ou métadonnée ?* ▶ corrigé p. 256

[sur papier] Vous prenez une photo. Classer en deux colonnes (**donnée** / **métadonnée**) : les pixels de l'image ; la date de la prise de vue ; le modèle de l'appareil ; le contenu visible (un chat) ; les coordonnées GPS ; le poids du fichier. ▷ coup de pouce

Lire une table ↑cours

★ **Exercice 3** *Vocabulaire de la table* ▶corrigé p. 256 • essentiel

[sur papier] À l'aide de la table *jeu* :

1. On complète la table *jeu* : on ajoute un descripteur `plateforme`, puis trois nouveaux jeux. Combien la table a-t-elle alors de **descripteurs** ? d'**enregistrements** ?
2. Citer tous les descripteurs de la table ainsi complétée.
3. Donner la valeur du descripteur `genre` pour l'enregistrement d'`id 5`.
4. Pourquoi le descripteur `titre` ferait-il un moins bon **identifiant** que `id` ? (une phrase)

▷coup de pouce

★★ **Exercice 4** *Concevoir une table* ▶corrigé p. 256

[sur papier] Le CDI du lycée veut enregistrer ses **prêts de livres** dans une table.

1. Proposer au moins **quatre descripteurs** pour cette table.
2. Lequel servira d'**identifiant** ? Pourquoi ne pas utiliser le nom de l'élève ?
3. Écrire deux enregistrements possibles (on pourra les présenter sous forme de tableau).

▷coup de pouce

Le format CSV ↑cours

★★ **Exercice 5** *Lire du CSV* ▶corrigé p. 257 • essentiel

[sur papier] On donne le début d'un fichier CSV public (jeux de société d'une médiathèque) :

```
titre;editeur;duree;age_min
Les Aventuriers du Rail;Days of Wonder;45;8
Dixit;Libellud;30;8
7 Wonders;Repos Prod;30;10
```

1. Quel est le **séparateur** utilisé ?
2. Combien y a-t-il de **descripteurs** ? d'**enregistrements** ?
3. Donner la valeur du descripteur `duree` pour l'enregistrement *Dixit*.
4. Ce fichier est-il en format **ouvert** ou **fermé** ? Citer un avantage de ce choix.

▷coup de pouce

★★ **Exercice 6** *Écrire du CSV* ▶corrigé p. 257

[sur papier] Écrire les **trois premières lignes** (en-tête comprise) du fichier CSV correspondant à la table *studio*, en utilisant la **virgule** comme séparateur. ▷coup de pouce

★★ **Exercice 7** *Un CSV mal formé* ▶corrigé p. 257

[sur papier] Ce fichier CSV contient **trois** erreurs qui gêneront un tableur. Les trouver, puis recopier le fichier corrigé (on inventera la valeur manquante).

```
nom;ville;age
Lina;Monaco;15
Hugo,Nice,16
Zoe;Menton
Tom;Beausoleil;quinze
```

▷coup de pouce

Rechercher, filtrer, trier ↑cours

★ **Exercice 8** *Rechercher* ▶corrigé p. 257

[sur papier] À l'aide de la table *jeu* :

1. Quel est le **genre** de *Mario Kart 8* ?
2. Quel jeu s'est vendu à **300** millions d'exemplaires ?

▷coup de pouce

★★ **Exercice 9** *Filtrer* ▶corrigé p. 257 • essentiel

[sur papier] Donner la liste des **titres** des jeux qui vérifient chaque critère :

1. genre `action` ;
2. `pegi` $\leqslant 7$ (tout public) ;
3. sortis **après 2013 ET** vendus à plus de 50 millions.

▷coup de pouce

★★ **Exercice 10** *Trier* ▶corrigé p. 258 • essentiel

[sur papier]

1. Ranger les jeux par `ventes` **décroissantes** (donner la liste des titres).
2. Ranger les jeux par `titre`, dans l'ordre **alphabétique**.
3. **Filtrer puis trier** : les jeux `pegi` $= 18$, du plus récent au plus ancien.

▷coup de pouce

★★ **Exercice 11** *Calculer sur une colonne* ▶corrigé p. 258

[sur papier] À l'aide de la table *jeu* :

1. Combien de jeux ont un `pegi` de 18 ?
2. Calculer la **moyenne** des ventes des jeux de genre `action`.
3. Quelle est l'année de sortie du jeu le plus ancien ? De quel jeu s'agit-il ?

▷coup de pouce

★★★ **Exercice 12** *Croiser deux tables* ▶corrigé p. 258 ≈15 min

[sur papier] En utilisant les tables *jeu* **et** *studio* (reliées par le numéro de studio) :

1. Quel **studio** (nom) a édité *Minecraft* ? Dans quel **pays** ?
2. Donner les titres de tous les jeux édités par un studio **japonais**.
3. *Défi :* quel **pays** totalise le plus grand nombre de `ventes` cumulées ?

▷coup de pouce ▷début de solution

★★★ **Exercice 13** *Pourquoi deux tables ?* ▶corrigé p. 258 ≈10 min

[sur papier] Un élève propose de supprimer la table *studio* : dans la table *jeu*, il remplacerait le numéro de studio par deux descripteurs, `nom_studio` et `pays_studio`.

1. Combien de fois faudrait-il alors écrire « Nintendo » et « Japon » ?
2. Le studio Rockstar Games change de nom. Combien de cases faut-il modifier avec une seule table ? Et avec les deux tables actuelles ?
3. Que risque-t-on si l'on oublie de modifier l'une des cases ?
4. On ajoute un nouveau jeu de Mojang. Que suffit-il d'écrire, pour le studio, dans la table *jeu* actuelle ?

▷coup de pouce ▷début de solution

Au tableur ↑cours

★★ **Exercice 14** *Fouiller au tableur* ▶corrigé p. 258 • essentiel

[au tableur] On fournit le fichier `jeux.csv` (la table *jeu*, séparateur `;`, encodage UTF-8). L'ouvrir dans un tableur, puis :

1. **Trier** le tableau par `ventes` décroissantes. Quel jeu apparaît en tête ?
2. Activer le **filtre automatique** et n'afficher que les jeux de genre `action`.
3. Dans une cellule vide, écrire la formule `=SOMME(...)` qui calcule le **total** des ventes, puis `=MOYENNE(...)` pour la moyenne.
4. *Bonus :* créer un **graphique en barres** des ventes par jeu.

▷coup de pouce

★★ **Exercice 15** *Lire des formules* ▶corrigé p. 259

[sur papier] La table *jeu* est recopiée dans un tableur : les en-têtes sont en ligne 1, les six enregistrements en lignes 2 à 7, et les descripteurs dans les colonnes A (`id`) à G (`ventes`), dans l'ordre de la table.

1. Quelle valeur affichent les formules `=MAX(G2:G7)`, `=MIN(D2:D7)` et `=SOMME(G3:G4)` ?
2. Écrire la formule qui calcule la moyenne des `pegi`, puis donner son résultat (arrondi au dixième).

▷coup de pouce

Open data, vie privée et nuage ↑cours

★ **Exercice 16** *Les données ouvertes* ▶corrigé p. 259

[sur papier]

1. Qu'est-ce que l'*open data* ? Citer un site où en trouver.
2. Pourquoi les données ouvertes sont-elles souvent publiées en **CSV** plutôt qu'en `.xlsx` ?

▷coup de pouce

★ **Exercice 17** *Mes données, mes droits* ▶corrigé p. 259 • essentiel

[sur papier]

1. Que protège le **RGPD** ? Citer un droit qu'il vous accorde sur vos données.
2. Stocker ses photos « dans le nuage » : donner un **avantage** et un **inconvénient**.
3. Les « 3 V » du **big data** : les nommer.
4. Monaco n'est pas dans l'Union européenne. Quelle **loi** protège les données personnelles d'un lycéen qui réside à Monaco, et quelle **autorité** contrôle son application ?

▷coup de pouce

★★★ **Exercice 18** *Défi — data-détective sur l'open data* ▶corrigé p. 259 ≈30 min

[au tableur] Sur `data.gouv.fr` (ou le portail open data de Monaco), télécharger un jeu de données en CSV qui vous intéresse, l'ouvrir au tableur, et **répondre à une question** de votre choix en utilisant un **tri** ou un **filtre**. Rédiger : la question posée, l'opération faite, la réponse trouvée.

▷coup de pouce ▷début de solution

Critiquer une réponse d'IA ↑cours

★★ **Exercice 19** *Le jeu le plus récent, selon un assistant* ▶ corrigé p. 259

[sur papier] Un élève a donné la table *jeu* et la table *studio* à un assistant d'IA et lui a demandé : « Quel est le jeu le plus récent, et quel studio l'a édité ? » Voici la réponse obtenue :

> *Pour trouver le jeu le plus récent, on trie la table jeu selon le descripteur* `ventes`*, en ordre décroissant : le premier enregistrement est alors Minecraft. Pour connaître son studio, on lit sa valeur pour le descripteur* `studio`*, qui vaut 2, et on croise avec la table studio : l'enregistrement d'identifiant 2 est Mojang, en Suède. Le jeu le plus récent est donc Minecraft, édité par Mojang.*

1. La réponse est-elle correcte ? Refaire l'opération sur la table et comparer.
2. Localiser l'erreur et donner la bonne réponse. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Python au service du thème ↑cours

★★ **Exercice 20** *Python au service du thème — filtrer avec une fonction* ▶ corrigé p. 260

[sur papier] Un tableur filtre une table ligne par ligne : pour chaque enregistrement, le critère est vrai ou faux. En Python, un critère s'écrit donc comme une fonction qui renvoie un **booléen**. On prend celui du cours : « `action` **ET** sorti après 2015 ».

1. Compléter la fonction suivante, qui renvoie `True` si le jeu vérifie le critère et `False` sinon. ▷ coup de pouce

```
1 def garder(genre, annee):
2     return ...
```

2. Sans machine, écrire ce qu'affichent les trois lignes suivantes, puis vérifier sur machine. Que deviendraient ces trois affichages si l'on remplaçait `and` par `or` dans la fonction ?

```
1 print(garder("action", 2013))       # GTA V
2 print(garder("action", 2018))       # Red Dead Redemption 2
3 print(garder("simulation", 2020))   # Animal Crossing
```

3. Écrire une fonction `succes(annee, ventes)` qui traduit le critère 3 de l'exercice « Filtrer » (sortis après 2013 **et** vendus à plus de 50 millions). La tester sur *Mario Kart 8*, *GTA V* et *Animal Crossing*.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Pour chaque information, demandez-vous : en la lisant, pourrait-on retrouver une personne précise, directement ou en la recoupant avec d'autres informations ? ↑énoncé

Ex. 2 – piste Relisez la définition d'une métadonnée dans le cours : est-ce le contenu de la photo lui-même, ou une information qui le *décrit* ? ↑énoncé

Ex. 3 – piste Un descripteur est une colonne, un enregistrement est une ligne (la ligne d'en-tête n'en est pas un). Pour la question 1, comptez d'abord sur la table actuelle, puis ajoutez ce qui est nouveau. Pour la question 4, relisez l'encadré du cours « Pourquoi un descripteur id ? ». ↑énoncé

Ex. 4 – piste Un descripteur est une colonne, un enregistrement est une ligne (ici, un prêt). Un identifiant doit être *unique* : deux élèves peuvent-ils porter le même nom ? Un même élève peut-il emprunter plusieurs fois ? ↑énoncé

Ex. 5 – piste La première ligne est l'en-tête : elle donne les descripteurs, ce n'est pas un enregistrement. Le séparateur est le caractère qui revient entre deux valeurs. ↑énoncé

Ex. 6 – piste Une ligne d'en-tête avec les descripteurs, puis une ligne par enregistrement ; sur chaque ligne, les valeurs se suivent dans le même ordre, séparées par des virgules. ↑énoncé

Ex. 7 – piste Comparez chaque ligne à l'en-tête : même séparateur ? Même nombre de valeurs ? Dans une même colonne, des valeurs de même nature ? ↑énoncé

Ex. 8 – piste Repérez d'abord la bonne ligne (celle du jeu, ou celle qui contient la valeur cherchée), puis lisez la valeur dans la bonne colonne. ↑énoncé

Ex. 9 – piste Pour le critère 3, un jeu n'est retenu que s'il vérifie les *deux* conditions à la fois : vérifiez-les l'une après l'autre, jeu par jeu. ↑énoncé

Ex. 10 – piste Question 3 : on ne garde d'abord que les jeux de `pegi` 18, puis on range ces seuls jeux selon l'année. ↑énoncé

Ex. 11 – piste Filtrez d'abord les enregistrements concernés, puis faites le calcul sur eux seuls. Une moyenne est une somme divisée par le nombre de valeurs. ↑énoncé

Ex. 12 – piste Le numéro de la colonne `studio` de la table *jeu* est l'`id` d'une ligne de la table *studio* : c'est lui qui relie les deux tables. ↑énoncé

Ex. 12 – début de solution Question 3 : additionnez les ventes des jeux de chaque studio (studio 1 : *Wii Sports*, *Mario Kart 8*, *Animal Crossing* ; puis studio 2, puis studio 3), puis cherchez le pays de chaque studio. ↑énoncé

Ex. 13 – piste Imaginez la grande table : sur chaque ligne de jeu, on recopierait le nom et le pays du studio. Combien de jeux chaque studio a-t-il édités ? ↑énoncé

Ex. 13 – début de solution Avec deux tables, « Nintendo » n'est écrit qu'*une* fois, dans la table *studio* ; la table *jeu* ne contient que son numéro, 1. Comptez de la même façon pour Rockstar Games. ↑énoncé

Ex. 14 – piste Dans une formule, une plage de cellules s'écrit de la première à la dernière cellule, séparées par deux-points (par exemple `B2:B10`). Repérez d'abord la colonne et les lignes des ventes. ↑énoncé

Ex. 15 – piste Écrivez la lettre de chaque colonne au-dessus de la table (A pour `id`, B pour `titre`…) et le numéro de ligne devant chaque enregistrement (la ligne 1 est l'en-tête). ↑énoncé

Ex. 16 – piste Relisez l'encadré du cours sur l'*open data* et la règle « Format ouvert / format fermé ». ↑

Ex. 17 – piste Toutes les réponses sont dans la dernière section du cours : la règle sur le RGPD, l'encadré sur Monaco, la partie sur le nuage et celle sur le *big data*. ↑énoncé

Ex. 18 – piste Choisissez un jeu de données de petite taille (quelques centaines de lignes) et une question à laquelle un tri ou un filtre répond directement (« quel est le plus grand… ? », « combien de… ? »). ↑énoncé

Ex. 18 – début de solution Exemple de démarche : « Quelle commune des Alpes-Maritimes compte le plus d'habitants ? » → filtre sur le département, tri décroissant sur la population, lecture de la première ligne. ↑énoncé

Ex. 19 q. 2 – piste Relisez la question posée : quel descripteur faut-il trier pour trouver le jeu « le plus récent » ? Comparez avec ce qu'a fait l'assistant. ↑énoncé

Ex. 20 q. 1 – piste Une fonction peut renvoyer directement le résultat d'une comparaison : `return pegi == 18` renvoie `True` ou `False`. Relisez dans le chapitre *Les bases de Python* comment on *compare* deux valeurs (et pas comment on les affecte), puis comment on relie deux conditions. ↑énoncé

TP ET DÉBAT

TP 1 Tableur 1 : notes, fréquences et graphiques

Excel, le grand oublié des programmes — I

Fichiers à télécharger : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/seconde/05-tp-tableur-1` — **archive .zip**

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Pour commencer

Récupérer le fichier `tableur1.xlsx` (à télécharger : `https://github.com/MathThibaud/nsi-fichiers-eleves/tree/main/seconde/05-tp-tableur-1`). Il contient **quatre feuilles** nommées « `Exercice 1` », « `Exercice 2` », « `Exercice 3` » et « `Exercice 4` » (onglets en bas de la fenêtre). Les **données de départ** y sont déjà saisies ; les cellules sur **fond jaune** sont **à compléter**.

Exercice 1 — *Notes d'un contrôle (feuille `Exercice 1`)*

On a relevé, dans l'ordre croissant, les notes obtenues à un contrôle de mathématiques par les élèves de la classe de 2de 11 (cellules `C2:O3`).

1. **Moyenne d'une série de données.** En `B5`, saisir la formule `=MOYENNE(C2:O3)`. *(Utiliser le menu des fonctions, choisir `MOYENNE`, puis sélectionner la plage `C2` à `O3`.)*
2. **Ligne des effectifs.** La ligne `8` donne les différentes notes. Compléter la ligne des **effectifs** (`C9:O9`) : pour chaque note, le nombre d'élèves l'ayant obtenue.
3. **Fréquences (décimales, puis en pourcentage).**
   (a) En `P9`, calculer l'**effectif total** avec `=SOMME(C9:O9)`.
   (b) En `C10`, calculer la fréquence de la première note avec `=C9/$P9`.
   (c) Recopier `C10` vers `D10:O10` avec la **poignée de recopie** (la petite croix en bas à droite de la cellule). *Le symbole `$` **fixe** la colonne `P` lors de la recopie.*
   (d) En `C11`, saisir `=C10`. Puis, par un clic droit » « Format de cellule » » onglet `Nombre`, choisir **Pourcentage**, `0` décimale. Recopier vers `D11:O11`.
   (e) Compléter `P10` et `P11` avec `=SOMME(C10:O10)` et `=SOMME(C11:O11)`. Que doivent valoir ces totaux ?
4. **Effectifs cumulés croissants (E.C.C.).**
   (a) En `C12`, saisir `=C9`.
   (b) En `D12`, saisir `=C12+D9`, puis recopier vers `E12:O12`.
5. **Moyenne pondérée par les effectifs.** En `B14`, saisir la formule `=SOMMEPROD(C8:O8;C9:O9)/P9`.
   Comparer avec la moyenne trouvée à la question **1** : que remarque-t-on ?

Exercice 2 *Fréquences et F.C.C. (feuille `Exercice 2`)*

La feuille `Exercice 2` donne les effectifs du lycée par **année de naissance**. Compléter, à l'aide de formules, la ligne des **fréquences** et celle des **fréquences cumulées croissantes** (F.C.C.).

Exercice 3 *Graphiques (feuille `Exercice 3`)*

La feuille `Exercice 3` donne les effectifs du lycée par **niveau** et par **dominante**.

1. Sélectionner les cellules `A1:D2`, puis *Insertion » Graphique » Secteurs* pour construire un **diagramme circulaire** des élèves de Seconde par dominante.
2. Sélectionner `A1:D4`, puis *Insertion » Graphique » Colonnes* pour construire un **histogramme** des effectifs par niveau et par dominante. Soigner le titre et la légende.

Exercice 4 *Moyennes (feuille `Exercice 4`)*

La feuille `Exercice 4` donne les salaires d'une entreprise (colonne `A`).

1. En `D2`, faire calculer le **nombre d'employés** (fonction `NB`).
2. En `D3`, faire calculer la **masse salariale**, c'est-à-dire le montant total des salaires (fonction `SOMME`).
3. En `D4`, faire calculer le **salaire moyen** (fonction `MOYENNE`).

TP 2 Tableur 2 : suites de nombres et conjecture

Excel, le grand oublié des programmes — II

Fichiers à télécharger : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/seconde/05-tp-tableur-2` — archive .zip

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Pour commencer

Récupérer le fichier `tableur2.xlsx` (à télécharger : `https://github.com/MathThibaud/nsi-fichiers-eleves/tree/main/seconde/05-tp-tableur-2`). Il contient les feuilles « `Exercice 1` » à « `Exercice 4` » et un « `Bonus` ». Les valeurs de départ y sont déjà en place (cellules sur fond jaune à compléter).

Exercice 1 *Une suite de nombres, pas de 1 (feuille `Exercice 1`)*

Pour créer une suite de 30 nombres consécutifs dans la colonne `A` : la cellule `A1` contient déjà le premier nombre, `5`. En `A2`, saisir la formule `=A1+1`, puis, à l'aide de la **poignée de recopie**, créer la suite de 30 nombres dans la colonne `A`. On dit que le **pas** entre deux nombres est de `1`.

Exercice 2 *Un pas réglable (feuille `Exercice 2`)*

On veut compter de `8` en `8` à partir de `5`. Le **pas** est en `C1` (`8`) et le premier nombre en `A1` (`5`). En `A2`, saisir `=A1+C$1`, puis recopier vers le bas. *Le symbole `$` **fixe** la ligne `1` lors de la recopie.*

Exercice 3 *Somme d'une suite : émettre une conjecture (feuille `Exercice 3`)*

On compte de `3` en `3` à partir de `5` (pas en `C1`, premier terme en `A1`). En `A2`, saisir `=A1+C$1` et recopier pour obtenir 30 termes.

1. En `C4`, le **nombre de termes** : `=NB(A1:A30)`.
2. En `C7`, la plus petite valeur : `=MIN(A1:A30)`. En `C8`, la plus grande : `=MAX(A1:A30)`.
3. En `C9`, la **somme des deux valeurs extrêmes**. En `C12`, la **somme de tous les termes** : `=SOMME(A1:A30)`.
4. **Conjecture.** En faisant varier les données, trouver la relation entre la **somme des termes**, le **nombre de termes** et les **valeurs extrêmes**. Noter votre idée.

Exercice 4 *Contrôler la conjecture : départ aléatoire (feuille `Exercice 4`)*

On fait varier automatiquement le premier terme.

1. En `B1`, un décimal aléatoire entre 0 et 1 : `=ALEA()`. En `B2` : `=B1*100`. En `B3`, un **entier** aléatoire entre 0 et 99 : `=ENT(B2)` (partie entière).
2. Le pas est en `E1` (`3`). En `C1`, recopier le nombre aléatoire : `=B3`. Créer dans la colonne `C` une suite de 30 termes de pas 3.
3. Compléter `E4`, `E7`, `E8`, `E9` et `E12` (comme en `Exercice 3`, mais sur la colonne `C`). **Vérifier** la conjecture. On peut appuyer sur `F9` pour retirer un nouveau nombre aléatoire.

Exercice 5 *BONUS : départ et pas quelconques (feuille `Bonus`)*

Faire varier automatiquement le premier terme **et** le pas.

1. Obtenir en `B3` un entier aléatoire entre 0 et 99. Choisir un pas entier quelconque en `E1`.
2. Dans la colonne `C`, créer une suite d'autant de termes que voulu, de pas celui de `E1`, à partir de `B3`.
3. Compléter `E2`, `E7`, `E8`, `E9` et `E12` en prenant pour plage la **colonne `C` entière** : `C:C`. Vérifier la conjecture.

TP 3 Tableur 3 : représenter et résumer une série

Excel, le grand oublié des programmes — III

Fichiers à télécharger : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/seconde/05-tp-tableur-3` — **archive .zip**

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Pour commencer

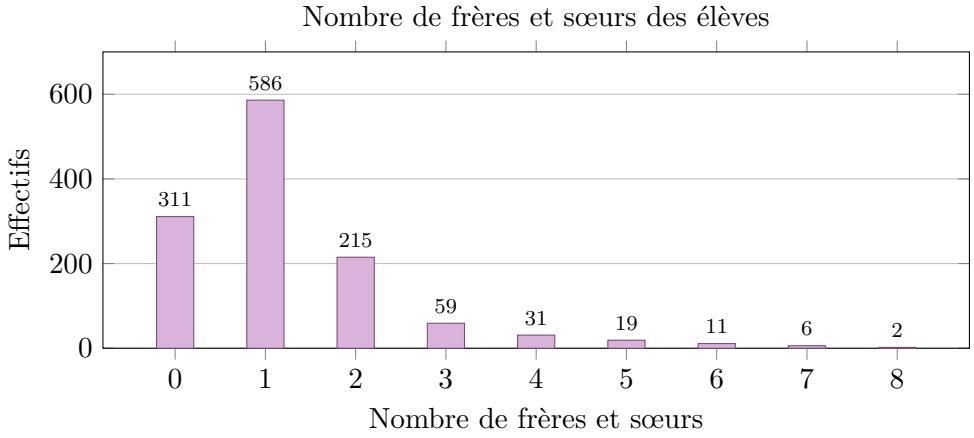
Récupérer le fichier `tableur3.xlsx` (à télécharger : `https://github.com/MathThibaud/nsi-fichiers-eleves/tree/main/seconde/05-tp-tableur-3`). Il contient quatre feuilles (« `Exercice 1` » à « `Exercice 4` ») dont les données sont déjà saisies.

Exercice 1 *Représentation de séries statistiques (feuille* `Exercice 1`*)*

1er cas : série statistique discrète. Nombre de frères et sœurs parmi les 1240 élèves d'un lycée :

Nombre de frères et sœurs	0	1	2	3	4	5	6	7	8
Effectifs	311	586	215	59	31	19	11	6	2

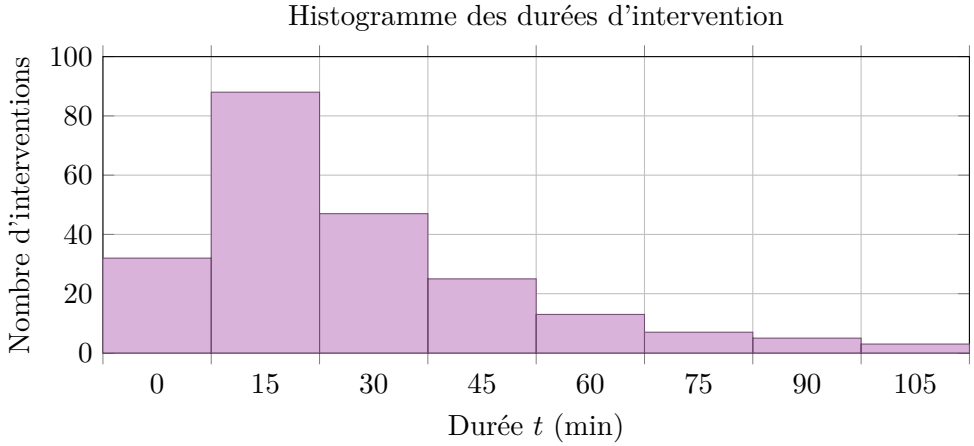
Diagramme en barres : via *Insertion » Graphique*, représenter cette série par un diagramme en barres (soigner le titre et la légende). Résultat attendu :

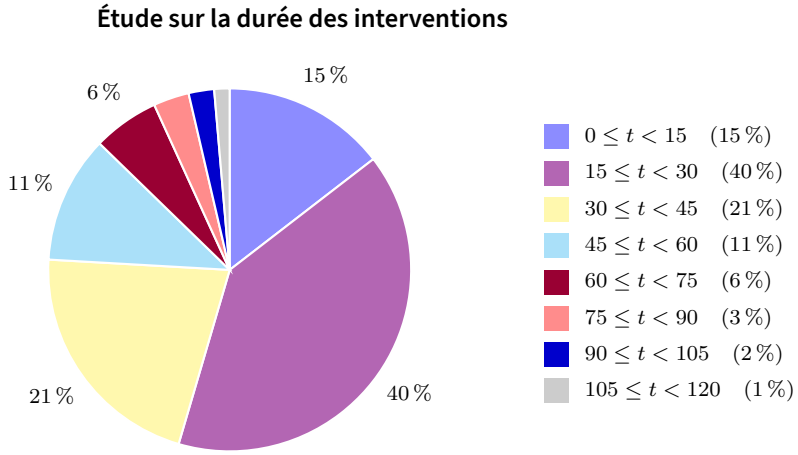


2e cas : série à classes. Durée des interventions :

Durée (min)	$0 \leq t < 15$	$15 \leq t < 30$	$30 \leq t < 45$	$45 \leq t < 60$	$60 \leq t < 75$	$75 \leq t < 90$	$90 \leq t < 105$	$105 \leq t < 120$
Nombre d'interventions	32	88	47	25	13	7	5	3

Histogramme, puis **diagramme circulaire** de cette série (via *Insertion » Graphique*). Résultats attendus :





Pourcentages arrondis à l'unité (total des effectifs : 220).

Exercice 2 *Caractéristiques de position d'une série en liste (feuille* `Exercice 2`*)*

Taille (cm) de 10 nourrissons (colonne `A`) : 49 ; 50,5 ; 50 ; 48,5 ; 52 ; 50 ; 49,5 ; 51 ; 49,5 ; 50,5.

- (a) Ordonner les tailles par ordre croissant (*Données » Trier*).
- (b) En `C1` : `=MEDIANE(A1:A10)`.
- (c) En `C2` : `=QUARTILE(A1:A10;1)`. En `C3` : `=QUARTILE(A1:A10;3)`.
- (d) En `C4` : `=MOYENNE(A1:A10)`.
- (e) En `C5` : calculer l'**étendue** (`=MAX(A1:A10)-MIN(A1:A10)`).

Exercice 3 *Caractéristiques de position dans un tableau (feuille* `Exercice 3`*)*

Tailles de 885 nourrissons :

Taille t	47,5	48	48,5	49	49,5	50	50,5	51	51,5	52	52,5	53	TOTAL
Effectif n	55	77	105	115	132	154	66	77	55	25	13	11	
$t \times n$													
E. C. C.	55												

- (a) Compléter la ligne des produits $t \times n$ (par exemple `=B3*B4`, à recopier).
- (b) Calculer la moyenne à l'aide de $\dfrac{\sum(t \times n)}{\sum n}$.
- (c) Calculer l'étendue.
- (d) Trouver médiane, premier et troisième quartile par lecture de la ligne des E.C.C. *Méthode : avec un effectif total N, le premier quartile est la première valeur dont l'E.C.C. atteint (ou dépasse) $N/4$; la médiane, la première dont l'E.C.C. atteint $N/2$; le troisième quartile, la première dont l'E.C.C. atteint $3N/4$.*

Exercice 4 *Caractéristiques de position pour données groupées (feuille* `Exercice 4`*)*

Tailles de 640 nourrissons :

Taille t	[45 ;46[	[46 ;47[	[47 ;48[	[48 ;49[	[49 ;50[	[50 ;51[	[51 ;52[	[52 ;53[	[53 ;54[	[54 ;55[	TOTAL
Effectif n	10	25	35	65	128	190	95	45	32	15	
Fréquence	1,56%										
F. C. C.	1,56%										
Centre de classe	45,5										

- (a) Compléter les fréquences, les centres de classe et les F.C.C.
- (b) Localiser médiane, premier et troisième quartile *(même méthode, avec les F.C.C. : première classe dont la F.C.C. atteint* 25 %*,* 50 %*, puis* 75 %*).*
- (c) Calculer la moyenne avec `=SOMMEPROD(centres;effectifs)/total`.

5

DÉBAT Faut-il ouvrir à tous le plus possible de données publiques, même quand elles concernent des personnes ?

▶ corrigé p. 260

open data contre vie privée

Durée : 50 min

Consignes

- En classe entière. Les rôles sont tirés au sort : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement.
- Chaque argument s'appuie sur un **document** ci-dessous ou sur le **cours**. Préparation sans IA ni recherche en ligne.
- Avant de commencer, notez sur votre cahier votre position de départ : oui, non ou je ne sais pas.

Rappel du cours. Une **donnée personnelle** permet d'identifier une personne, directement ou indirectement. Les **données ouvertes** (*open data*) sont des tables publiques librement téléchargeables, le plus souvent en **CSV**. **Croiser deux tables** permet de trouver des informations qu'aucune ne donne seule. Ouvrir les données, c'est donner du pouvoir aux citoyens… mais à qui exactement, et pour en faire quoi ?

Les deux camps

Groupe POUR — oui : il faut publier le plus possible de données publiques, en les protégeant au mieux.
Groupe CONTRE — non : dès que des personnes sont concernées, il faut limiter fortement ce que l'on publie.

Les documents

Document 1 — L'ouverture par défaut

Depuis 2011, l'État français publie ses données sur `data.gouv.fr` ; Monaco a aussi son portail *open data*. En France, une loi de 2016 a fait de l'ouverture **la règle** : les administrations de plus de 50 agents et les collectivités de plus de 3 500 habitants doivent publier leurs principales bases de données dans un format ouvert, après en avoir retiré, en principe, les **données personnelles**.

À retenir : en France, publier les données publiques est la règle, pas l'exception.

Source : loi n° 2016-1321 du 7 octobre 2016 pour une République numérique ; `data.gouv.fr`.

Document 2 — À quoi servent les données ouvertes ?

Horaires des bus, qualité de l'air, dépenses des communes : grâce aux données ouvertes, des développeurs créent des applications de transport, des journalistes vérifient l'usage de l'argent public, des chercheurs étudient la pollution. Pendant l'épidémie de COVID-19, des bénévoles ont créé, à partir des données de Santé publique France, des sites de suivi des cas très consultés. Payées par les contribuables, ces données doivent, pour beaucoup, revenir à tous.

À retenir : les données ouvertes servent à créer des services et à contrôler l'action publique.

Source : `data.gouv.fr` *; Santé publique France, données COVID-19 publiées en* open data *(2020–2022).*

Document 3 — Le prix de la maison du voisin

Depuis 2019, la base publique « Demandes de valeurs foncières » (DVF) donne, pour chaque vente immobilière récente, la **date**, le **prix**, la **surface** et la **localisation** du bien, sans le nom du vendeur ni de l'acheteur. Très utile pour connaître les prix d'un quartier… mais, quand on connaît ses voisins, on devine combien chacun a payé. Les conditions d'utilisation interdisent de chercher à réidentifier les personnes.

À retenir : même sans nom, une table peut en dire beaucoup sur des personnes.

Source : `data.gouv.fr`*, jeu de données « Demandes de valeurs foncières » (Direction générale des Finances publiques).*

Document 4 — « Anonyme » ne veut pas dire « introuvable »

En 2000, la chercheuse Latanya Sweeney a montré qu'environ **87 %** des Américains pouvaient être identifiés avec trois informations : **code postal**, **date de naissance** et **sexe**. Retirer la colonne « nom » ne suffit donc pas : en croisant la table avec une autre, on retrouve qui est qui. Pour le RGPD, une donnée **pseudonymisée** (nom remplacé par un numéro) reste personnelle. C'est pourquoi l'INSEE regroupe les prénoms trop rares de son fichier.

À retenir : retirer ou remplacer le nom ne rend pas une donnée anonyme.

Source : L. Sweeney, « Simple Demographics Often Identify People Uniquely *», 2000 ; RGPD, considérant 26 ; INSEE, fichier des prénoms.*

Rôles, déroulé minuté, règles de prise de parole et trace écrite : voir la page « Méthode : débattre » p. 7.

Tableau d'arguments

Recopier ce tableau sur le cahier et le compléter pendant le débat.

Argument POUR (oui)	Doc.	Argument CONTRE (non)	Doc.
…		…	

Trace écrite finale (individuelle)

Sur le cahier, notez votre position après le débat (oui, non, je ne sais toujours pas) et si vous avez changé d'avis. Puis, en trois à cinq lignes, justifiez-la avec vos **deux meilleurs arguments**, dont **un argument de l'autre camp** que vous trouvez solide (avec le document qui l'appuie).

FICHE DE RÉVISION

Les notions clés

Donnée personnelle	identifie une personne, directement (nom, photo) ou non (adresse IP, position, plaque…).
Métadonnée	décrit une autre donnée : date, taille, GPS d'une photo… (peut trahir où l'on habite).
Open data	données publiques librement téléchargeables, souvent en CSV (`data.gouv.fr`, portail de Monaco).
Nuage (*cloud*)	données sur des **serveurs distants** : accès partout, partage, mais chez quelqu'un d'autre.
Big data	« 3 V » : **Volume**, **Vélocité**, **Variété**.

Une table et son fichier CSV

id	titre	annee	genre	pegi
1	Minecraft	2011	bac à sable	7
2	GTA V	2013	action	18
3	Wii Sports	2006	sport	7
4	Mario Kart 8	2014	course	3

```
id,titre,annee,genre,pegi
1,Minecraft,2011,bac a sable,7
2,GTA V,2013,action,18
...
```

5 descripteurs (colonnes), **4 enregistrements** (lignes ; l'en-tête n'en est pas un) ; `id` est un **identifiant** (valeur unique). Le CSV, simple **texte**, est un format **ouvert** ; l'ancien `.xls` est **fermé**.

Méthode : fouiller la table

- **Rechercher** : genre de Minecraft → `bac à sable`.
- **Filtrer** (`pegi` $\leqslant 7$) → Minecraft, Wii Sports, Mario Kart 8 ; critères combinés par **ET** / **OU**.
- **Trier** ce résultat par `annee` **décroissante** → Mario Kart 8, Minecraft, Wii Sports.
- **Au tableur** : bouton *Trier*, *filtre automatique*, formules `=SOMME(...)`, `=MOYENNE(...)`, `=MAX(...)`, graphiques.

Repères, enjeux et erreurs à éviter

2011 : ouverture de `data.gouv.fr`. **2020** : 15 841 résultats COVID « perdus » en Angleterre, l'ancien `.xls` étant limité à 65 536 lignes. **RGPD** (2018, UE) : consentement, droits d'accès, de correction et d'effacement (« droit à l'oubli »), CNIL. **Monaco** : loi n° 1.565 (2024), mêmes grands droits, autorité : **APDP**. *Erreurs :* compter l'en-tête comme un enregistrement ; inverser colonne et ligne ; confondre **filtrer** (garder certaines lignes) et **trier** (les ranger toutes).

Savoir-faire à maîtriser — je sais…

- ☐ distinguer une **donnée**, un **descripteur** (attribut) et un **enregistrement** (ligne) ;
- ☐ retrouver les **métadonnées** d'un fichier personnel et les distinguer des données ;
- ☐ lire une table de données : repérer les descripteurs et compter les enregistrements ;
- ☐ reconnaître le format **CSV** (du texte, un séparateur, une ligne d'en-tête) et l'ouvrir dans un tableur ;
- ☐ utiliser un site de **données ouvertes** (*open data*) pour sélectionner et récupérer des données ;
- ☐ **rechercher**, **filtrer** et **trier** des enregistrements selon un ou plusieurs critères ;
- ☐ réaliser un tri croissant ou décroissant sur une colonne ;
- ☐ citer les enjeux liés aux **données personnelles**, au *cloud* et au *big data* (les 3 V).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 261

1. Donner le séparateur et le nombre de descripteurs de l'en-tête `nom;classe;masse_kg`.
2. Dans la table ci-dessus, filtrer les jeux de `pegi` 18, puis trier tous les jeux par `annee` croissante.
3. Expliquer pourquoi une plaque d'immatriculation est une donnée personnelle.
4. Citer deux métadonnées d'une photo prise au smartphone.

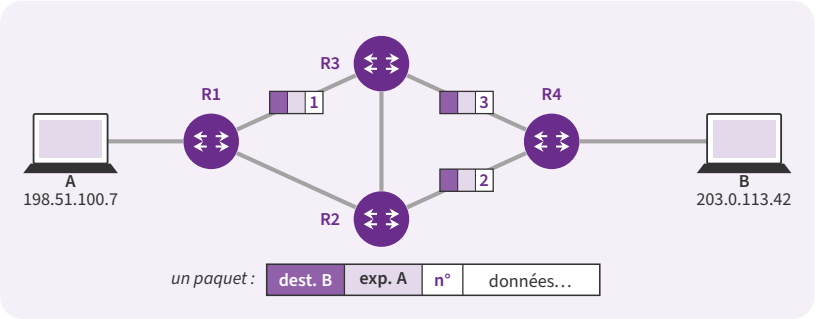
Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 261

1. Dire quel langage décrit l'apparence d'une page web (ch. 4).
2. Citer deux droits sur ses données personnelles (ch. 2).

6

CHAPITRE 6

Internet



DANS CE CHAPITRE

- Cours 154
- Exercices 162
- TP et débat 168
- Fiche de révision 172
- Corrigés 262

COURS

Programme Repères historiques ; protocole **TCP/IP** (paquets, routage) ; adresses symboliques et serveurs **DNS** ; réseaux **pair-à-pair** ; indépendance d'Internet vis-à-vis du réseau physique ; ordres de grandeur du trafic.

Idée Internet, c'est le **réseau** : des millions de machines qui s'échangent des **paquets** de données, de proche en proche, sans chef d'orchestre central. Une idée simple, robuste… et devenue vitale.

Objectifs Situer la naissance d'Internet ; distinguer les rôles d'**IP** et de **TCP** ; comprendre le **routage** des paquets et ses limites ; passer d'une adresse symbolique à une adresse **IP** (DNS) ; connaître les grands types de réseaux physiques et l'ordre de grandeur du trafic.

Situation de départ

1. Votre photo de 6 Mo part chez un ami, à Montréal. Part-elle d'un seul bloc ? Par où passe-t-elle ? Et que se passe-t-il si un câble est coupé en route ?
2. Vous tapez `wikipedia.fr` dans la barre d'adresse. Comment votre ordinateur sait-il à quelle machine s'adresser ?
3. En rentrant chez vous, votre téléphone passe de la 4G au Wi-Fi sans couper la musique en cours. Comment Internet peut-il fonctionner sur des supports aussi différents ?

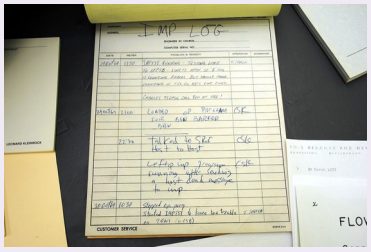
Remarque **Internet n'est pas le Web !** On l'a vu au chapitre *Le Web* : **Internet** est le *réseau* (les câbles, la fibre, les ondes, les machines) ; le **Web** n'est que *l'un des services* qui circulent dessus, à côté des e-mails, des appels vidéo, des jeux en ligne… Ce chapitre ouvre la boîte : *comment* l'information voyage-t-elle d'un bout à l'autre du monde ?

I Une belle histoire : d'ARPANET à Internet

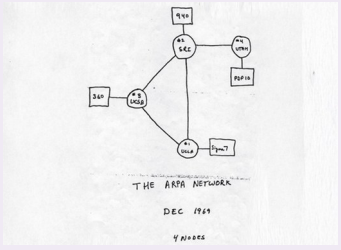
A. Le premier message : « LO… »

Nous sommes le **29 octobre 1969**. Dans un laboratoire de l'université de Californie (UCLA), un étudiant, **Charley Kline**, tente d'envoyer le tout premier message d'un ordinateur à un autre, situé à **560 km** de là, au *Stanford Research Institute* (SRI), plus au nord en Californie. Le mot à transmettre pour se connecter était simple : `LOGIN`. Il tape `L`… `O`… et au moment du `G`, la machine distante **plante** ! Le tout premier message de l'histoire d'Internet fut donc simplement « `LO` » — comme dans l'expression anglaise « *lo and behold* » (« et voilà »).

Une heure plus tard, après réparation, `LOGIN` passait en entier. Le réseau **ARPANET** était né : l'ancêtre direct d'Internet.



Le cahier de bord du 29 octobre 1969 : « LO »



ARPANET en 1969 : quatre ordinateurs reliés



Vinton Cerf, co-inventeur de TCP/IP

B. Les grandes étapes

Règle 1 — Repères historiques

- **années 1950–60** — premiers réseaux d'ordinateurs, mais *propriétaires* (liés à un constructeur ou à l'opérateur téléphonique).
- **1969** — premier message sur **ARPANET** (le fameux « `LO` »), aux États-Unis ; en **1971**, la France lance son propre réseau, **Cyclades**.
- **1974** — **Vinton Cerf** et **Robert Kahn** inventent le protocole **TCP/IP**, la « langue commune » qui reliera *tous* les réseaux.
- **1983** — tous les réseaux basculent sur TCP/IP : c'est la **naissance d'Internet**. La même année apparaît le **DNS** (les adresses en toutes lettres).
- **1989–1991** — Tim Berners-Lee invente le **Web**, *par-dessus* Internet (première page en ligne en 1991).
- **aujourd'hui** — des **milliards** de machines et d'objets connectés.

Sources : UCLA, Samueli School of Engineering, pages sur le premier message d'ARPANET (29 octobre 1969, laboratoire de Leonard Kleinrock) ; Inria, pages historiques consacrées au réseau Cyclades (projet lancé en 1971), `inria.fr` ; V. Cerf et R. Kahn, « A Protocol for Packet Network Intercommunication », *IEEE Transactions on Communications*, mai 1974.

À retenir — Un réseau sans centre

Personne ne « possède » Internet. Il n'a **pas de centre**, pas de chef. C'est justement ce qui le rend si **robuste** : si une partie tombe en panne, l'information trouve un autre chemin.

Pour les curieux — *Un mythe tenace* (facultatif)

On raconte souvent qu'ARPANET devait survivre à une guerre nucléaire : c'est en partie un mythe. L'idée d'un réseau qui *survit* à la destruction d'une partie de ses nœuds vient des travaux de Paul Baran (RAND Corporation, années 1960) ; ARPANET, lui, servait d'abord à partager des ordinateurs entre universités.

► **Exercices d'application :** exercices 1 **et** 2 (un peu d'histoire : d'ARPANET à Internet)

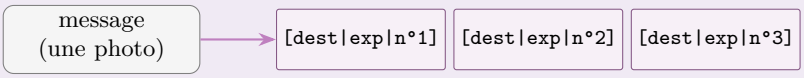
II Voyager en petits morceaux : les paquets

Quand vous envoyez une photo, elle ne part pas d'un seul bloc. Internet la **découpe** en petits morceaux de taille limitée : les **paquets**.

Définition 1

Un **paquet** est un petit bloc de données de **taille limitée** (au plus 1 500 octets environ sur un réseau Ethernet ou Wi-Fi). Il contient une partie du message, plus une « étiquette » : l'**adresse de l'expéditeur**, l'**adresse du destinataire**, et un **numéro** (pour remettre les morceaux dans l'ordre à l'arrivée).

Chaque paquet fait son propre chemin, indépendamment des autres : c'est la **commutation par paquets**, le principe de base d'Internet.



découpé en paquets qui voyagent *séparément*...

Règle 2 — Une force du système

Le format des paquets est **uniforme** : peu importe qu'ils transportent du texte, une image, du son ou une vidéo. Chaque paquet peut même emprunter un **chemin différent** ! Ils seront **réassemblés dans l'ordre** à l'arrivée grâce à leur numéro.

► **Exercices d'application :** exercices 3 **à** 5 (les paquets)

III Deux protocoles complémentaires : IP et TCP

Pour que tout cela fonctionne, les machines parlent la même « langue » : le protocole **TCP/IP**. En réalité, ce sont **deux** protocoles qui se partagent le travail.

Protocole	Son rôle
IP *(Internet Protocol)*	**adresser** et **acheminer** chaque paquet : donner une adresse à chaque machine et faire avancer les paquets de routeur en routeur.
TCP *(Transmission Control Protocol)*	**fiabiliser** la communication : vérifier que *tous* les paquets arrivent, **redemander** ceux qui manquent, et les **remettre dans l'ordre**.

Règle 3 — Fiable, mais pas « à l'heure »

TCP garantit que tout paquet **finira** par arriver (sauf panne matérielle). En revanche, ni IP ni TCP ne garantissent *quand* : il n'y a **aucune garantie de temps**. C'est pourquoi une vidéo en direct peut « saccader » : une image qui arrive trop tard ne sert plus à rien, et le film se fige un instant en l'attendant.

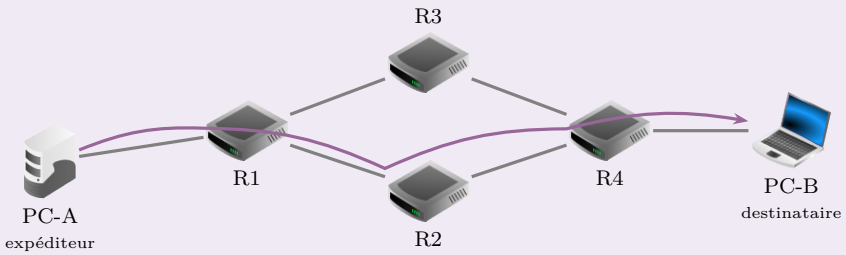
► **Exercices d'application :** exercices 6 **et** 7 (IP et TCP)

IV L'algorithme-roi : le routage

Comment un paquet trouve-t-il son chemin ? Grâce à des machines spécialisées, les **routeurs**.

Définition 2

Un **routeur** est un ordinateur qui **aiguille** les paquets. Il ne connaît pas tout le réseau : il tient seulement une **carte locale** de ses voisins, et décide, pour chaque paquet, **la prochaine étape**. De proche en proche, le paquet progresse jusqu'à destination.



un chemin possible : A → R1 → R2 → R4 → B

Règle 4 — Les limites du routage

Un paquet peut se **perdre** : panne d'un routeur, ou **destruction volontaire**. En effet, chaque paquet possède un compteur, le **nombre maximal de routeurs** qu'il peut traverser (le *TTL*). À chaque étape, le compteur diminue ; s'il atteint **0**, le paquet est **détruit** pour ne pas tourner en rond éternellement et encombrer le réseau. C'est alors **TCP** qui s'en aperçoit et **redemande** le paquet manquant.

▶ Activité 1 — Le routage débranché

En classe, chaque élève joue un **routeur** et ne connaît que ses **voisins directs**. Un « paquet » (un papier avec un destinataire) doit traverser la salle de main en main. On observe : il n'existe **aucune carte globale**, et pourtant le paquet arrive. On « détruit » un routeur (un élève s'assoit) : le paquet trouve un **autre chemin**.

▶ **Exercices d'application :** exercices 8 à 10 (le routage ; le compteur TTL)

V Les adresses : IP et DNS

A. L'adresse IP

Définition 3

Chaque machine connectée possède une **adresse IP** : son « numéro » sur le réseau. En version **IPv4**, c'est une suite de **quatre nombres** de **0 à 255**, séparés par des points.

172	·	217	·	22	·	14	= une adresse IPv4
0 à 255		0 à 255		0 à 255		0 à 255	

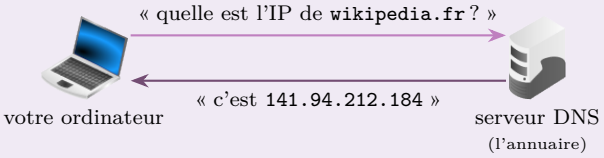
Il y a environ **4 milliards** d'adresses IPv4 possibles… ce qui ne suffit plus ! On déploie donc **IPv6**, avec des adresses bien plus longues (assez pour des milliards de milliards d'objets connectés).

B. Le DNS : l'annuaire d'Internet

Personne ne retient `172.217.22.14`. On tape `google.fr`. Le passage de l'un à l'autre est assuré par le **DNS**.

Définition 4

Une **adresse symbolique** (comme `wikipedia.fr`) est l'adresse « en toutes lettres », facile à retenir. Le **DNS** (*Domain Name System*) est l'annuaire géant d'Internet qui **traduit** une adresse symbolique en **adresse IP** numérique.



Le DNS n'est pas une seule machine, mais un **immense ensemble d'ordinateurs** répartis dans le monde et sans cesse mis à jour.

▶ Activité 2 — Retrouver une adresse IP

Dans un terminal, taper `ping wikipedia.fr` (ou `nslookup wikipedia.fr`) : on voit apparaître l'**adresse IP** correspondant au nom (le 28/09/2026, on obtenait par exemple `141.94.212.184`). Ce résultat peut **varier dans le temps et selon le lieu** : noter l'adresse effectivement obtenue. Refaire avec le site du lycée. Deux noms différents peuvent-ils mener à la même IP ?

▶ Exercices d'application : exercices 11 à 15 (adresses IP et DNS)

VI Internet est indépendant du réseau physique

Point fort de génie : TCP/IP est **logiciel**. Il ne vit pas dans les câbles, mais **dans chaque machine**. Du coup, Internet peut circuler sur **n'importe quel** support physique.

Réseau physique	Filaire ?	Débit	Usage
Fibre optique	filaire	très élevé	épine dorsale d'Internet, box récentes
Ethernet (câble RJ45)	filaire	élevé	réseau local (salle info)
ADSL (ligne téléphone)	filaire	faible/moyen	ancien accès Internet, en voie d'obsolescence
Wi-Fi	sans fil	moyen/élevé	à la maison, au lycée, portée courte
4G / 5G	sans fil	élevé	téléphone mobile, grande portée
Bluetooth	sans fil	faible	entre appareils proches (écouteurs…)

Grâce à cette indépendance, votre téléphone passe **sans coupure** de la 4G au Wi-Fi, ou d'antenne en antenne quand vous voyagez.

Définition 5

Le **débit** d'une liaison est la quantité de données qu'elle transmet en une seconde. Il se mesure en **bits par seconde** (bit/s). Un **bit** est la plus petite donnée possible, un `0` ou un `1`, et 1 octet = 8 bits. On utilise surtout le **mégabit par seconde** (1 Mbit/s = 10^6 bit/s) et le **gigabit par seconde** (1 Gbit/s = 10^9 bit/s). Ordres de grandeur : une fibre optique atteint souvent 1 Gbit/s, une vieille ligne ADSL quelques Mbit/s.

Méthode — Calculer une durée de téléchargement

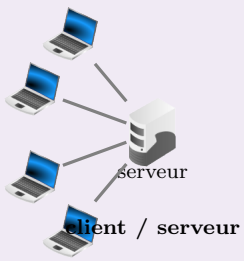
Attention aux unités : les débits s'expriment en **bits** par seconde, mais la taille des fichiers en **octets**. Pour savoir combien de temps prend un téléchargement, on convertit d'abord la taille du fichier en bits (on multiplie par 8), puis on la divise par le débit.

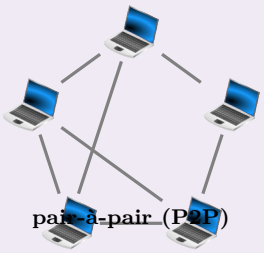
VII Les réseaux pair-à-pair (P2P)

D'habitude, un **serveur** central distribue à des **clients** (le modèle du Web). Mais il existe une autre organisation :

Définition 6

Dans un réseau **pair-à-pair** (*peer-to-peer*, P2P), il n'y a plus de serveur central : **chaque machine est à la fois émetteur et récepteur**, et partage directement avec les autres.





Règle 5 — Intérêts… et dérives

Intérêt : très **robuste** (pas de point unique de panne) et efficace pour **partager de gros fichiers** (chacun aide à distribuer) ; utilisé pour la visioconférence, certaines mises à jour, la blockchain… **Dérive** : le P2P sert aussi au **téléchargement illégal** d'œuvres protégées (musique, films) : un **usage illicite**, puni par la loi.

► **Exercices d'application :** exercices 16 **et** 17 (réseaux physiques ; pair-à-pair)

VIII Ordres de grandeur : le trafic explose

Internet a fait disparaître le télégramme, le télex, une partie du courrier postal, et bientôt le téléphone fixe (remplacé par la **voix sur IP**). Le volume de données échangées est **vertigineux**.

Année	Trafic mondial (ordre de grandeur, par an)
2010	$\sim$ 0,2 Zo (Cisco, 2011)
2016	$\sim$ 1 Zo (Cisco, 2017)
2021	$\sim$ 3,3 Zo ($3{,}3 \times 10^{21}$ octets ; prévision Cisco, 2017)
2024	$\sim$ 7 Zo (estimation UIT, 2024)

Ces valeurs viennent d'organismes différents, qui ne mesurent pas exactement la même chose : on compare des **ordres de grandeur**. Pour 2024, l'UIT estime environ 6 Zo sur les réseaux fixes et 1,3 Zo sur les réseaux mobiles.

Sources : Cisco, *Visual Networking Index (VNI) – Forecast and Methodology*, éditions 2011 (trafic 2010) et 2017 (trafic 2016, prévision 2021) ; UIT (Union internationale des télécommunications), *Measuring digital development – Facts and Figures 2024*, novembre 2024 (trafic fixe $\approx$ 6 Zo, mobile $\approx$ 1,3 Zo, estimations), `itu.int`.

Rappel des unités : 1 **ko** $= 10^3$ o, 1 **Mo** $= 10^6$, 1 **Go** $= 10^9$, 1 **To** $= 10^{12}$, 1 **Po** $= 10^{15}$, 1 **Eo** $= 10^{18}$, 1 **Zo** (zetta-octet) $= 10^{21}$ octets.

Règle 6 — Deux enjeux de société

Le déni de service (attaque DDoS) : saturer un site de messages pour le rendre indisponible.
La neutralité du Net : le principe, présent dès l'origine, selon lequel les routeurs transmettent **tous** les paquets de la même façon, *sans* favoriser un contenu ou un service. Elle est régulièrement remise en cause par des intérêts commerciaux.

► **Exercices d'application :** exercices 18 **et** 19 (ordres de grandeur ; temps d'un téléchargement), exercice 20 (critiquer une réponse d'IA), exercice 21 (Python au service du thème)

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Déjà vu :** le **Web** n'est qu'un des services d'Internet (une requête HTTP part en paquets, après une question au **DNS**) ; un réseau de routeurs se représente par un **graphe**, comme les amitiés d'un **réseau social**.
- **À venir :** au chapitre *Localisation, cartographie et mobilité*, le réseau routier sera lui aussi un graphe ; les **objets connectés** du dernier chapitre reçoivent une adresse IP, d'où le besoin d'**IPv6**.
- **Plus loin :** en spécialité NSI de Première, on étudie de plus près les protocoles qui fiabilisent la transmission, comme le **protocole du bit alterné**.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Internet ≠ Web	Internet = le réseau ; le Web = un service qui roule dessus.
Histoire	ARPANET 1969 (« LO »), Cyclades 1971, TCP/IP 1974, Internet né en 1983.
Paquet	petit bloc : données + adresse expéditeur + destinataire + numéro.
IP	**adresse** et **achemine** les paquets (adresse IPv4 : 4 nombres 0–255).
TCP	**fiabilise** : redemande les paquets perdus, les remet dans l'ordre.
Pas de garantie de temps	fiable mais pas « à l'heure » → le streaming peut saccader.
Routeur / routage	aiguille chaque paquet *une étape* à la fois, sans carte globale.
TTL	nombre max de routeurs ; à 0, le paquet est détruit (TCP le redemande).
DNS	annuaire : adresse symbolique (`wikipedia.fr`) → adresse IP.
Réseau physique	indépendant : fibre, Ethernet, Wi-Fi, 4G/5G, ADSL, Bluetooth.
Pair-à-pair (P2P)	pas de serveur central ; robuste ; aussi usages illicites.
Trafic	ordres de grandeur : en zetta-octets/an (10^{21}) ; neutralité du Net.

Retour à la situation de départ

1. Non : elle est découpée en environ 4 000 **paquets** d'au plus 1 500 octets (§ II), qui avancent de **routeur** en routeur, chacun par son propre chemin (§ IV). Si un câble est coupé, les paquets passent ailleurs, et **TCP** redemande ceux qui se sont perdus (§ III).
2. Il interroge le **DNS**, l'annuaire d'Internet, qui traduit cette **adresse symbolique** en **adresse IP**, le « numéro » de la machine sur le réseau (§ V).
3. TCP/IP est un **logiciel**, présent dans chaque machine et non dans les câbles : Internet est **indépendant du réseau physique** et circule aussi bien sur la 4G que sur le Wi-Fi ou la fibre (§ VI).

Débat argumenté

Un **débat**, proposé en fin de chapitre, accompagne ce thème : « Les fournisseurs d'accès doivent-ils traiter tous les paquets de la même façon ? » (documents, rôles, tableau d'arguments et trace écrite, environ 50 min).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ raconter en quelques mots l'histoire d'Internet (d'**ARPANET** à Internet) ;
- ☐ expliquer la **commutation par paquets** ;
- ☐ distinguer les rôles des protocoles **IP** et **TCP** ;
- ☐ expliquer le principe du **routage** (un paquet passe de routeur en routeur) ;
- ☐ distinguer une **adresse IP** d'un **nom de domaine** et expliquer le rôle du **DNS** ;
- ☐ expliquer qu'Internet est **indépendant** du support physique ;
- ☐ décrire un réseau **pair-à-pair** (P2P) et un de ses usages.

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- [sur papier] *à faire sur papier* ; ▶ *à faire sur machine* (terminal, navigateur).
- Un **TP sur le simulateur Filius**, proposé en fin de chapitre, complète ces exercices.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Un peu d'histoire ↑cours

★ **Exercice 1** *Remettre dans l'ordre* ▶ corrigé p. 262

[sur papier] Ranger ces événements du plus ancien au plus récent, avec leur date : *invention du Web* • *premier message sur ARPANET* • *naissance d'Internet (bascule TCP/IP)* • *invention du protocole TCP/IP*. ▷ coup de pouce

★ **Exercice 2** *« LO… »* ▶ corrigé p. 262

[sur papier] En 1969, quel mot voulait-on transmettre pour le premier message d'ARPANET ? Que s'est-il passé, et quelles ont donc été les deux premières lettres réellement reçues ? ▷ coup de pouce

Les paquets ↑cours

★ **Exercice 3** *Pourquoi découper ?* ▶ corrigé p. 262 • essentiel

[sur papier]

1. Qu'appelle-t-on un **paquet** ? Citer les trois informations de son « étiquette ».
2. À quoi sert le **numéro** porté par chaque paquet ?

▷ coup de pouce

★★ **Exercice 4** *Arrivés dans le désordre* ▶ corrigé p. 263

[sur papier] Un message de 4 paquets arrive dans l'ordre suivant : n°3, n°1, n°4, n°2.

1. Est-ce un problème ? Comment la machine réceptrice reconstitue-t-elle le message ?
2. Quel protocole (IP ou TCP) est responsable de cette remise en ordre ?

▷ coup de pouce

★★★ **Exercice 5** *Découper une photo* ▶ corrigé p. 263 ≈ 10 min

[sur papier] On envoie une photo de 3 Mo (3×10^6 octets). On suppose que chaque paquet transporte 1500 octets de la photo, et que son étiquette ajoute 40 octets (on simplifie : en réalité, étiquette comprise, un paquet ne dépasse pas 1500 octets environ).

1. En combien de paquets la photo est-elle découpée ?
2. Combien d'octets d'étiquettes faut-il envoyer en plus de la photo ? Est-ce beaucoup par rapport à la photo ?

3. En route, 1 % des paquets se perdent. Combien de paquets faut-il renvoyer ? Quel protocole s'en charge ?

▷ coup de pouce ▷ début de solution

IP et TCP ↑ cours

★★ **Exercice 6** *Qui fait quoi ?* ▶ corrigé p. 263 • essentiel

[sur papier] Associer chaque tâche au bon protocole (**IP** ou **TCP**) :

a. donner une adresse à chaque machine ;
b. redemander un paquet perdu ;
c. acheminer un paquet de routeur en routeur ;
d. remettre les paquets dans l'ordre.

▷ coup de pouce

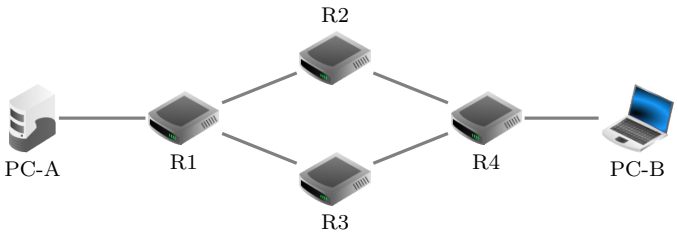
★★ **Exercice 7** *Fiable mais pas à l'heure* ▶ corrigé p. 263

[sur papier] « Internet garantit que mes paquets arriveront, mais pas *quand*. » Expliquer cette phrase, puis dire pourquoi cela peut gêner une **visioconférence** en direct. ▷ coup de pouce

Le routage ↑ cours

★★ **Exercice 8** *Suivre un paquet* ▶ corrigé p. 263 • essentiel

[sur papier] On considère le réseau de routeurs suivant (PC-A veut joindre PC-B) :



1. Donner **deux** chemins possibles de PC-A vers PC-B.
2. Le routeur R1 connaît-il tout le réseau ? Que décide-t-il exactement pour un paquet ?
3. Le câble entre R2 et R4 tombe en panne. Le paquet peut-il quand même arriver ? Comment ?

▷ coup de pouce

★★ **Exercice 9** *Le compteur TTL* ▶ corrigé p. 263 • essentiel

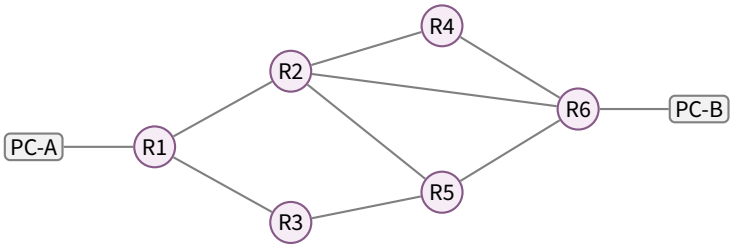
[sur papier] Chaque paquet a un compteur « nombre maximal de routeurs » (TTL), qui diminue de 1 à chaque routeur.

1. À quoi sert ce compteur ? Que devient un paquet dont le compteur atteint 0 ?
2. Un paquet part avec un TTL de 3 et doit traverser 5 routeurs. Arrive-t-il ? Que fait alors TCP ?

▷ coup de pouce

★★★ **Exercice 10** *Le chemin le plus court* ▶ corrigé p. 264 ≈ 15 min

[sur papier] Dans le réseau ci-dessous, PC-A envoie des paquets à PC-B. Chaque routeur traversé **ralentit** un peu le paquet.



1. Quel chemin traverse le **moins** de routeurs ? Combien en traverse-t-il ?
2. Le câble R2–R6 tombe en panne. Quel est maintenant le plus petit nombre de routeurs à traverser ? Donner **tous** les chemins qui réalisent ce minimum.
3. Pourquoi cet exemple montre-t-il qu'Internet est **robuste** ?

▷ coup de pouce ▷ début de solution

Adresses IP et DNS ↑ cours

★ **Exercice 11** *Est-ce une adresse IPv4 valide ?* ▶ corrigé p. 264 • essentiel

[sur papier] Pour chacune, dire si c'est une adresse IPv4 **valide** et justifier : `192.168.0.1` • `10.0.300.5` • `172.16.4` • `8.8.8.8`.

▷ coup de pouce

★★ **Exercice 12** *Combien d'adresses IPv4 ?* ▶ corrigé p. 264

[sur papier] Une adresse IPv4 est formée de 4 nombres, chacun compris entre 0 et 255.

1. Combien de valeurs différentes chaque nombre peut-il prendre ? En déduire le nombre total d'adresses IPv4 possibles (donner le calcul, puis un ordre de grandeur).
2. Il y a environ 8 milliards d'humains, et beaucoup possèdent plusieurs appareils connectés (téléphone, ordinateur, montre, télévision…). Ces adresses suffisent-elles ? Quelle solution le cours présente-t-il ?

▷ coup de pouce

★ **Exercice 13** *Le rôle du DNS* ▶ corrigé p. 264 • essentiel

[sur papier]

1. Que fait le **DNS** ? Dans quel sens traduit-il : de `wikipedia.fr` vers `141.94.212.184` (adresse relevée le 28/09/2026 ; elle peut varier dans le temps et selon le lieu), ou l'inverse ?
2. Pourquoi utilise-t-on des adresses symboliques plutôt que de taper directement l'adresse IP ?

▷ coup de pouce

★★ **Exercice 14** *Sur machine* ▶ corrigé p. 264

▶ Dans un terminal, taper `nslookup www.gouv.mc` (le site du Gouvernement princier). Relever l'**adresse IP** affichée. Recommencer avec un deuxième site, `www.service-public.fr`, puis taper `ping www.service-public.fr` : retrouve-t-on la même adresse qu'avec `nslookup` ? *(But : voir le DNS à l'œuvre.)*

▷ coup de pouce

★★ **Exercice 15** *Quand le DNS ne répond plus* ▶ corrigé p. 265 • essentiel

[sur papier] Un matin, le serveur DNS de votre box tombe en panne ; tout le reste fonctionne.

1. Vous tapez `wikipedia.fr` dans le navigateur. Que se passe-t-il ? Pourquoi ?
2. Votre voisin, qui connaît l'adresse IP du site, la tape directement dans la barre d'adresse. La page peut-elle s'afficher ? Justifier.
3. En déduire pourquoi on dit que le DNS est l'« annuaire » d'Internet.

▷ coup de pouce

Réseaux physiques, P2P et trafic ↑cours

★ **Exercice 16** *Filaire ou sans fil ?* ▶ corrigé p. 265

[sur papier] Classer en deux colonnes (**filaire** / **sans fil**) : Wi-Fi, fibre optique, 4G, Ethernet, Bluetooth, ADSL. Lequel offre le **plus gros débit** ? ▷ coup de pouce

★ **Exercice 17** *Client/serveur ou pair-à-pair ?* ▶ corrigé p. 265

[sur papier]

1. Quelle est la différence entre un modèle **client/serveur** et un modèle **pair-à-pair** ?
2. Citer un **intérêt** du pair-à-pair, et un **usage illicite** qu'on peut en faire.

▷ coup de pouce

★★ **Exercice 18** *Ordres de grandeur* ▶ corrigé p. 265

[sur papier] On rappelle : 1 Go $= 10^9$ o, 1 To $= 10^{12}$ o, 1 Po $= 10^{15}$ o, 1 Zo $= 10^{21}$ o.

1. Combien de **Go** y a-t-il dans 1 **To** ? dans 1 **Po** ?
2. En 2024, le trafic mondial annuel sur Internet est estimé à environ 7 Zo (UIT, 2024). Écrire ce nombre en octets (puissance de 10), et dire à quel préfixe (Zo) cela correspond.

▷ coup de pouce

★★★ **Exercice 19** *Le temps d'un téléchargement* ▶ corrigé p. 265 ≈ 15 min

[sur papier] Rappel du cours : les débits se mesurent en **bits par seconde** ; 1 octet = 8 bits et 1 Mbit/s $= 10^6$ bits par seconde. On télécharge un film de 4 Go (4×10^9 octets).

1. Combien de **bits** faut-il recevoir ?
2. Combien de temps dure le téléchargement avec la fibre à 100 Mbit/s ? Avec une vieille ligne ADSL à 8 Mbit/s ? Donner les résultats en secondes, puis en minutes.
3. Pourquoi le temps réel est-il souvent un peu plus long ? (Penser aux paquets.)

▷ coup de pouce ▷ début de solution

Critiquer une réponse d'IA ↑cours

★★ **Exercice 20** *Un paquet perdu, selon un assistant* ▶ corrigé p. 266

[sur papier] Un élève a demandé à un assistant d'IA : « Que se passe-t-il quand un paquet se perd sur Internet ? » Voici la réponse obtenue :

> *Sur Internet, un message est découpé en paquets qui voyagent séparément, de routeur en routeur, et peuvent emprunter des chemins différents. Chaque paquet porte un compteur (le TTL) qui diminue à chaque routeur traversé ; s'il tombe à zéro, le paquet est détruit, ce qui évite qu'il tourne en rond pour toujours. Un paquet peut donc être perdu. C'est alors le protocole IP qui s'en aperçoit et redemande le paquet manquant à l'expéditeur, puis remet les paquets dans le bon ordre. Le message finit toujours par arriver complet, mais sans garantie de délai.*

1. La réponse est-elle correcte ? Relire le tableau « IP et TCP » du cours et vérifier, pour chaque tâche citée, quel protocole en est chargé.
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Python au service du thème ↑ cours

★★ **Exercice 21** *Python au service du thème — paquets et durée d'envoi* ▶ corrigé p. 266

▶ En Python (Spyder ou Basthon). Une machine découpe chaque fichier en morceaux de 1500 octets ; le dernier morceau peut être plus petit, mais il occupe quand même un paquet.

1. Sans machine : combien de paquets faut-il pour un fichier de 4500 octets ? de 4501 octets ? de 700 octets ?
2. Compléter la fonction suivante, qui renvoie le nombre de paquets nécessaires pour un fichier de `taille` octets, puis vérifier les réponses de la question 1.

```
def nb_paquets(taille):
    n = taille // 1500
    if ...:
        n = ...
    return n
```

3. Chaque paquet porte en plus une étiquette de 40 octets. Écrire une fonction `duree(taille, debit)` qui renvoie la durée d'envoi en secondes, étiquettes comprises, d'un fichier de `taille` octets à `debit` Mbit/s (1 octet = 8 bits). L'utiliser pour une photo de 4 500 000 octets à 20 Mbit/s. ▷ coup de pouce ▷ début de solution

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relisez l'encadré « Repères historiques » au début du cours : chaque événement y figure avec sa date. ↑ énoncé

Ex. 2 – piste Relisez le récit « Le premier message », au début du cours. ↑ énoncé

Ex. 3 – piste Relisez la définition d'un paquet : que trouve-t-on sur son étiquette, en plus des données transportées ? Pensez à ce qui arrive si les paquets prennent des chemins différents. ↑ énoncé

Ex. 4 – piste Relisez la définition d'un paquet : que contient son étiquette, en plus des deux adresses ? ↑

Ex. 5 – piste Nombre de paquets = taille de la photo divisée par la taille d'un morceau. Chaque paquet porte sa propre étiquette. ↑ énoncé

Ex. 5 – début de solution $3 \times 10^6 \div 1500 = 3\,000\,000 \div 1500$: simplifiez en divisant d'abord par 100 en haut et en bas. Pour la question 2, multipliez le nombre de paquets par 40. ↑ énoncé

Ex. 6 – piste Relisez le tableau « IP et TCP » du cours : l'un des protocoles s'occupe de chaque paquet pris séparément, l'autre du message dans son ensemble. ↑ énoncé

Ex. 7 – piste TCP redemande les paquets perdus : cela prend du temps. Dans une visioconférence, une image qui arrive avec une seconde de retard sert-elle encore ? ↑ énoncé

Ex. 8 – piste Un routeur ne connaît que ses voisins directs. Pour la panne, cherchez sur le dessin un chemin qui n'emprunte pas le câble R2–R4. ↑ énoncé

Ex. 9 – piste Faites diminuer le compteur routeur après routeur : 3, 2, 1… Que se passe-t-il quand il atteint 0, et combien de routeurs reste-t-il alors à traverser ? ↑ énoncé

Ex. 10 – piste Tous les chemins commencent par R1 et finissent par R6. Cherchez d'abord s'il existe un chemin avec un seul routeur entre R1 et R6, puis avec deux. ↑ énoncé

Ex. 10 – début de solution Après la panne, les voisins de R6 sont R4 et R5. Pour chacun, cherchez comment l'atteindre depuis R1 en passant par un seul autre routeur. ↑énoncé

Ex. 11 – piste Pour chaque adresse, vérifiez deux choses : combien y a-t-il de nombres ? Chacun est-il compris entre 0 et 255 ? ↑énoncé

Ex. 12 – piste De 0 à 255, il y a 256 valeurs (n'oubliez pas le 0). Chaque nombre se choisit indépendamment des trois autres : les possibilités se multiplient. ↑énoncé

Ex. 13 – piste Relisez la définition du DNS dans le cours : ce que l'on tape dans le navigateur, est-ce le nom du site ou son numéro ? ↑énoncé

Ex. 14 – piste Sous Windows, le terminal s'appelle « Invite de commandes » ; on tape la commande puis Entrée. Avec `nslookup`, lisez toute la réponse : les premières lignes concernent le serveur qui répond à votre question, l'adresse du site vient après son nom. Avec `ping`, l'adresse apparaît sur la première ligne. ↑énoncé

Ex. 15 – piste Le navigateur a besoin d'une adresse IP pour envoyer sa requête. Qui lui fournit cette adresse quand on tape un nom ? Et si on la lui donne directement ? ↑énoncé

Ex. 16 – piste Relisez le tableau des réseaux physiques du cours (colonnes « Filaire ? » et « Débit »). Pour ceux qui n'y figurent pas, demandez-vous s'il faut brancher un câble. ↑énoncé

Ex. 17 – piste Relisez la section du cours sur le pair-à-pair : dans chaque modèle, quelle machine fournit les fichiers ? ↑énoncé

Ex. 18 – piste Comparez les puissances de 10 : combien de fois 10^9 « tient-il » dans 10^{12} ? Dans 10^{15} ? ↑énoncé

Ex. 19 – piste Convertissez d'abord les octets en bits. Temps (en secondes) = nombre de bits à recevoir divisé par le nombre de bits reçus chaque seconde. ↑énoncé

Ex. 19 – début de solution $4 \times 10^9 \times 8 = 32 \times 10^9$ bits. Avec la fibre : $32 \times 10^9 \div (100 \times 10^6)$; simplifiez les puissances de 10. ↑énoncé

Ex. 20 q. 2 – piste Pour chaque action citée par l'assistant, cherchez dans le tableau « IP et TCP » du cours quel protocole en est chargé. ↑énoncé

Ex. 21 q. 3 – piste Question 2 : `taille // 1500` compte les morceaux complets. Il faut un paquet de plus s'il reste des octets : relisez dans le chapitre *Les bases de Python* l'opérateur qui donne le reste d'une division. ↑énoncé

Ex. 21 q. 3 – début de solution Question 3 : le nombre d'octets envoyés vaut `taille + 40 * nb_paquets(taille)` ; multipliez-le par 8 pour obtenir des bits, puis divisez par le nombre de bits envoyés chaque seconde, `debit * 1000000`. ↑énoncé

TP ET DÉBAT

TP Construire un réseau et faire circuler un paquet

▶ corrigé p. 267

sur le simulateur Filius, gratuit et en français

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

But du TP

Avec le logiciel gratuit **Filius**, on va **construire** un petit réseau, donner une **adresse IP** à chaque machine, puis envoyer un paquet avec la commande `ping` et l'**observer voyager**. On ira du réseau local le plus simple jusqu'à **deux réseaux reliés par un routeur**.

Prise en main. Filius a deux modes (en haut) : le mode **construction** (le marteau) pour poser et relier les composants avec le **Câble**, et le mode **simulation** (la flèche verte) pour lancer les échanges. Les composants (barre de gauche) sont l'**Ordinateur**, le **Switch**, le **Routeur** et le **Câble** :



Un **double-clic** sur une machine ouvre sa configuration (adresse IP, masque, passerelle). En mode simulation, un double-clic permet d'installer la « **Ligne de commande** » : on y tape `ping`, `ipconfig`, `traceroute`. Le clic droit « **Afficher les échanges de données** » montre les paquets qui passent.

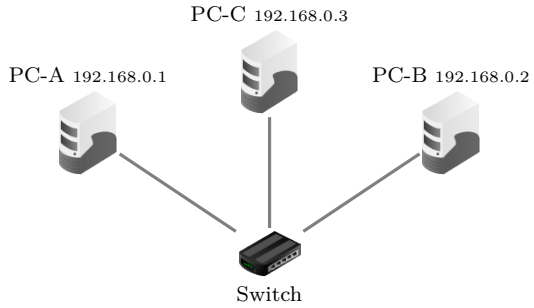
Le vocabulaire du TP

- Un **switch** (ou *commutateur*) est le boîtier qui relie entre elles, par des câbles, les machines d'un **même réseau local** (une salle, une maison) : il fait suivre chaque paquet vers la machine à qui il est destiné.
- Un **routeur** (vu en cours) relie **plusieurs réseaux** entre eux et aiguille les paquets de l'un à l'autre.
- La **passerelle** d'une machine est l'adresse IP du routeur de *son* réseau local : c'est à lui qu'elle confie tout paquet destiné à une machine d'un **autre** réseau, comme on passe par la porte de sortie d'un bâtiment. On l'appelle aussi *routeur par défaut*.
- Filius demande aussi un **masque** : recopiez simplement la valeur donnée (`255.255.255.0`), sans chercher à l'expliquer.

Partie 1 — Un réseau local

Exercice 1 *Construire et adresser*

► En mode **construction**, poser **trois Ordinateurs** reliés à un même **Switch**. Double-cliquer sur chacun et saisir son **adresse IP** et le **masque** `255.255.255.0` (laisser la passerelle vide) :



1. Les trois machines ont des adresses `192.168.0._`. À quoi sert le **switch** ?
2. Double-cliquer sur le switch : Filius propose-t-il de lui donner une adresse IP ? Qu'en déduire ?

Exercice 2 *Pinguer et observer*

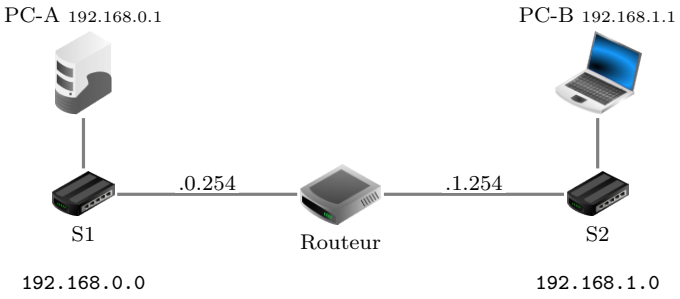
► Passer en **simulation**. Sur PC-A, installer la **Ligne de commande**, l'ouvrir et taper : `ping 192.168.0.2`. Ralentir la vitesse (curseur en haut) pour bien voir.

1. Le `ping` réussit-il ? Décrire le trajet du paquet (quels composants traverse-t-il ?).
2. ► Ouvrir « Afficher les échanges de données » sur PC-B : que voit-on arriver ?
3. ► Taper `ping 192.168.0.9` (machine qui n'existe pas). Que se passe-t-il, et pourquoi ?

Partie 2 — Deux réseaux reliés par un routeur

Exercice 3 *Relier deux réseaux locaux*

► Construire **deux** réseaux locaux reliés par **un Routeur**. À la création du routeur, Filius demande le **nombre d'interfaces** : en mettre **2**.



Interface du routeur côté réseau 0 : `192.168.0.254` ; côté réseau 1 : `192.168.1.254` (masque `255.255.255.0` partout).

Exercice 4 *La passerelle, indispensable pour sortir*

► En simulation, sur PC-A, taper `ping 192.168.1.1`.

1. Sans passerelle, Filius répond « `destination non joignable` ». Pourquoi ? *(Le paquet doit **quitter** le réseau local.)*
2. ► Régler la **passerelle** de PC-A sur `192.168.0.254` (l'interface du routeur de *son* côté), et celle de PC-B sur `192.168.1.254`. Relancer le `ping` : il doit réussir. Décrire le trajet complet du paquet.
3. ► Taper `traceroute 192.168.1.1` sur PC-A. Combien d'étapes ? Que représente l'étape intermédiaire ?

Bilan du TP

Dans un **réseau local**, un **switch** suffit pour relier les machines (mêmes adresses `192.168.0._`). Pour **changer de réseau**, il faut un **routeur**, et chaque machine doit connaître sa **passerelle** (la porte de sortie vers les autres réseaux). Le paquet avance ensuite **de proche en proche**.

DÉBAT Les fournisseurs d'accès à Internet doivent-ils traiter tous les paquets de la même façon ?

▶ corrigé p. 268

la neutralité du Net

Durée : 50 min

Consignes

- En classe entière. Les rôles sont tirés au sort : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement.
- Chaque argument s'appuie sur un **document** ci-dessous ou sur le **cours**. Préparation sans IA ni recherche en ligne.
- Avant de commencer, notez sur votre cahier votre position de départ : oui, non ou je ne sais pas.

Rappel du cours. Les routeurs acheminent les **paquets** de proche en proche. La **neutralité du Net** est le principe selon lequel ils transmettent **tous** les paquets de la même façon, sans favoriser un contenu ou un service. Techniquement, votre fournisseur d'accès à Internet (FAI : Monaco Telecom, Orange, Free…) pourrait ralentir certains paquets ou en accélérer d'autres. En a-t-il le droit ? Et serait-ce une bonne idée ?

Les deux camps

Groupe POUR — oui : tous les paquets doivent être traités de la même façon (neutralité stricte).
Groupe CONTRE — non : un FAI doit pouvoir donner des priorités ou proposer des offres différentes selon les services.

Les documents

Document 1 — La règle en Europe : un Internet ouvert

Depuis 2016, un règlement européen oblige les FAI à traiter tout le trafic **de façon égale**, quels que soient l'expéditeur, le contenu ou l'application. Une gestion « raisonnable » reste permise (congestion passagère, *catégories* de trafic aux besoins différents : un appel vidéo supporte mal le retard, un téléchargement peut attendre), jamais pour des raisons commerciales. En France, l'**Arcep** y veille. Monaco, hors de l'Union européenne, n'est pas directement concerné.

À retenir : en Europe, la neutralité est la loi, avec des exceptions techniques limitées.

Source : règlement (UE) 2015/2120 du 25 novembre 2015, article 3 ; Arcep, rapport annuel « L'état de l'internet en France ».

Document 2 — L'« illimité »… sur certaines applications seulement

Certains forfaits mobiles ont proposé un « tarif zéro » : le trafic de quelques applications partenaires (musique, vidéo, réseau social) n'était pas décompté du forfait, celui des autres si. Bonne affaire pour le client ; handicap pour une petite application concurrente. En 2020 puis en 2021 (affaire Telenor, en Hongrie), la Cour de justice de l'Union européenne a jugé ces offres contraires à l'égalité de traitement du trafic.

À retenir : en Europe, un FAI ne peut pas avantager quelques applications, même au profit du client.

Source : Cour de justice de l'Union européenne, arrêts du 15 septembre 2020 (Telenor Magyarország) et du 2 septembre 2021.

Document 3 — Les opérateurs : « que les plus gros paient leur part »

Quelques grandes plateformes, surtout de vidéo, produisent une grande part du trafic, mais ce sont les opérateurs qui paient antennes, fibre et routeurs. Plusieurs opérateurs européens réclament donc aux plateformes une « juste part » (*fair share*) ; la Commission européenne a ouvert une consultation en février 2023. Les plateformes répondent qu'elles investissent déjà (câbles sous-marins, serveurs proches des abonnés) et que les abonnés paient leur abonnement pour regarder ces vidéos.

À retenir : derrière la neutralité se pose une autre question : qui paie le réseau ?

Source : Commission européenne, consultation « L'avenir du secteur des communications électroniques et de ses infrastructures », février 2023.

Document 4 — Aux États-Unis, la règle fait le yo-yo

Aux États-Unis, l'autorité des télécoms (FCC) a adopté des règles de neutralité en 2015, les a supprimées fin 2017, rétablies en 2024 ; une cour d'appel les a annulées en janvier 2025. Sans règle, un FAI peut vendre des « voies rapides » à ceux qui en ont les moyens. Pour les uns, l'Internet américain s'est développé quand même ; pour les autres, plusieurs États, comme la Californie en 2018, ont voté leur propre loi.

À retenir : la neutralité est un choix politique, qui varie selon les pays.

Source : FCC, décisions de 2015, 2017 et 2024 ; cour d'appel fédérale (6e circuit), 2 janvier 2025.

Rôles, déroulé minuté, règles de prise de parole et trace écrite : voir la page « Méthode : débattre » p. 7.

Tableau d'arguments

Recopier ce tableau sur le cahier et le compléter pendant le débat.

Argument POUR (oui)	Doc.	Argument CONTRE (non)	Doc.
…		…	

Trace écrite finale (individuelle)

Sur le cahier, notez votre position après le débat (oui, non, je ne sais toujours pas) et si vous avez changé d'avis. Puis, en trois à cinq lignes, justifiez-la avec vos **deux meilleurs arguments**, dont **un argument de l'autre camp** que vous trouvez solide (avec le document qui l'appuie).

FICHE DE RÉVISION

L'essentiel

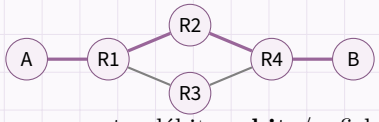
- **Internet ≠ Web** : Internet est le **réseau** ; le Web n'est qu'**un service** qui circule dessus.
- Un message part en **paquets** qui voyagent **séparément** (chemins parfois différents), **réassemblés dans l'ordre** à l'arrivée.
- **Pas de centre** (robuste : en cas de panne, les paquets passent ailleurs) ; TCP/IP est **logiciel**, **indépendant** du support physique.

Notions clés

Paquet	bloc limité (≈ 1 500 octets) : données + adresses **expéditeur**, **destinataire** + **numéro**.
IP / TCP	IP **adresse et achemine** ; TCP **fiabilise** (**redemande**, **remet dans l'ordre**) ; pas de **garantie de temps**.
Routeur, **TTL**	choisit la **prochaine étape** seulement ; TTL = nb max de routeurs, à `0` paquet **détruit**.
DNS	annuaire réparti : **adresse symbolique** (`wikipedia.fr`) → **adresse IP**.
Pair-à-pair	pas de serveur central, chaque machine émet *et* reçoit ; robuste ; dérive : téléchargement **illégal**.
Supports	**filaires** : fibre (débit le plus élevé), Ethernet, ADSL ; **sans fil** : Wi-Fi, 4G/5G, Bluetooth.

Méthode — savoir refaire

Adresse IPv4 : **quatre** nombres de **0 à 255**. `192.168.1.20` valide ; `10.4.256.1` non ($256 > 255$) ; `8.8.8` non (trois nombres).
Chemins : suivre les liaisons de proche en proche : A–R1–R2–R4–B ou A–R1–R3–R4–B. R1 ne décide que de la **prochaine étape**.



`nslookup` : `Name` = nom demandé, `Address` = adresse IP obtenue. **Téléchargement** : débit en **bits**/s, fichier en **octets** ⇒ ×8 d'abord : 50 Mo à 100 Mbit/s → 400 Mbit /100 = 4 s. **Puissances de 10** : ko 10^3, Mo 10^6, Go 10^9, To 10^{12}, Po 10^{15}, Eo 10^{18}, Zo 10^{21} octets.

Repères, enjeux et erreurs à éviter

1969 ARPANET (« `LO` ») ; **1974** TCP/IP (Cerf, Kahn) ; **1983** Internet et DNS ; **1989–1991** le Web. IPv4 : ≈ **4 milliards** d'adresses ⇒ **IPv6**. Trafic 2024 : ≈ 7 Zo (UIT). Enjeux : **DDoS**, **neutralité du Net**. *À éviter* : Internet = Web ; un routeur qui connaîtrait tout le chemin ; IP qui redemanderait un paquet (c'est TCP) ; des paquets tous sur le même chemin ; un nombre > 255 dans une IPv4 ; bits et octets confondus.

Savoir-faire à maîtriser — je sais…

- ☐ raconter en quelques mots l'histoire d'Internet (d'**ARPANET** à Internet) ;
- ☐ expliquer la **commutation par paquets** ;
- ☐ distinguer les rôles des protocoles **IP** et **TCP** ;
- ☐ expliquer le principe du **routage** (un paquet passe de routeur en routeur) ;
- ☐ distinguer une **adresse IP** d'un **nom de domaine** et expliquer le rôle du **DNS** ;
- ☐ expliquer qu'Internet est **indépendant** du support physique ;
- ☐ décrire un réseau **pair-à-pair** (P2P) et un de ses usages.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 269

1. Dire si `172.300.1.4` est une adresse IPv4 valide, en justifiant.
2. Donner le rôle de TCP quand un paquet n'arrive pas.
3. Expliquer ce que devient un paquet dont le TTL atteint 0, et pourquoi.
4. Calculer le temps pour télécharger 25 Mo à 50 Mbit/s.
5. Citer un intérêt du pair-à-pair et un usage illicite.

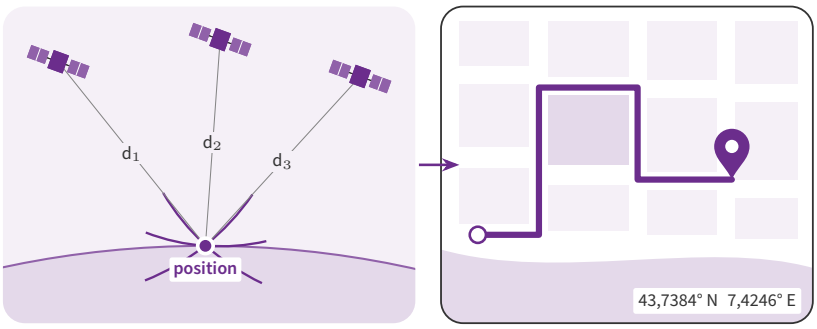
Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 269

1. Dire ce que contient la première ligne d'un fichier CSV (ch. 5).
2. Calculer le poids brut d'une image de 100 × 100 pixels (ch. 3).

7

CHAPITRE 7

Localisation, cartographie et mobilité



DANS CE CHAPITRE

- Cours 174
- Exercices 183
- Débat 189
- Fiche de révision 191
- Corrigés 270

COURS

Programme Principe de la **géolocalisation** (GPS, satellites, trilatération) ; **coordonnées** (latitude, longitude) ; décoder une **trame NMEA** ; **cartes numériques** et calcul d'**itinéraire** ; enjeux de **confidentialité** des traces de localisation.

Idée Votre téléphone sait, à quelques mètres près, *où* vous êtes. Derrière cette magie : des **satellites**, un peu de **géométrie** et beaucoup de **calcul**. Comprendre le GPS, c'est aussi reprendre la main sur ses **traces**.

Objectifs Se repérer par **latitude/longitude** ; expliquer le principe du GPS (temps de trajet du signal, **trilatération**) ; **décoder** une trame NMEA ; utiliser une carte numérique (itinéraire, distance) ; mesurer les enjeux de **vie privée**.

Situation de départ

1. Votre téléphone affiche votre position à quelques mètres près. D'où tient-il cette information ?
2. Parmi des milliers de rues, comment une application de navigation trouve-t-elle le trajet le plus court ?
3. Une application de météo demande à vous localiser « toujours », même quand elle est fermée. Qu'est-ce que cela peut révéler de vous ?

Pour les curieux — *Un peu d'histoire* (facultatif)

Le **GPS** (*Global Positioning System*) est conçu par l'armée américaine dans les années 1970 et devient pleinement opérationnel en **1995**. Jusqu'en **2000**, le signal civil était volontairement **dégradé** (« disponibilité sélective ») : la précision était de l'ordre de 100 m ! Le président Bill Clinton met fin à cette dégradation en mai 2000, et la précision grand public passe d'un coup à quelques mètres (GPS.gov). L'Europe a depuis déployé son propre système **civil**, **Galileo** (premiers services ouverts en décembre 2016 ; Commission européenne, 2016).

Sources : GPS.gov (gouvernement des États-Unis), pages « Selective Availability » (fin de la dégradation le 1er mai 2000) et présentation du système (pleine capacité opérationnelle en 1995), `gps.gov` ; Commission européenne, communiqué « Galileo goes live ! », décembre 2016, `ec.europa.eu`.

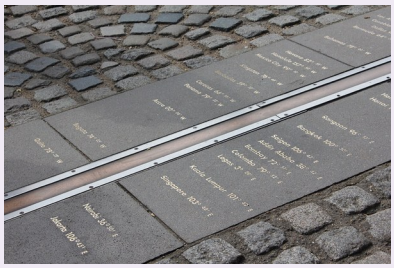
→ frise chronologique p. 10

I Se repérer sur la Terre : latitude et longitude

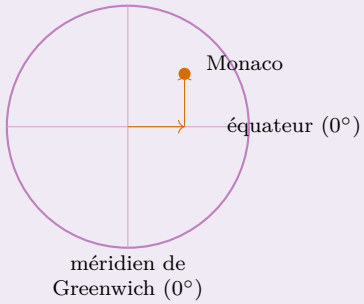
Pour désigner un point sur le globe sans ambiguïté, on utilise deux nombres, comme sur un quadrillage.

Définition 1

La **latitude** indique la position **Nord–Sud** : c'est un angle de $-90°$ (pôle Sud) à $+90°$ (pôle Nord), l'**équateur** valant $0°$.
La **longitude** indique la position **Est–Ouest** : un angle de $-180°$ à $+180°$, le **méridien de Greenwich** (Londres) valant $0°$.



La ligne du méridien de Greenwich : longitude 0°



Exemple

La Principauté de **Monaco** se situe à la latitude **43,7384° Nord** et à la longitude **7,4246° Est**. On note aussi ce couple $(43{,}7384\,;\ 7{,}4246)$.

À retenir — Degrés décimaux ou degrés-minutes

Un même point s'écrit de deux façons : en **degrés décimaux** ($43{,}7384°$) ou en **degrés et minutes** ($43°\ 44{,}304'$), sachant qu'**un degré vaut 60 minutes**. Pour convertir des minutes en degrés, on **divise par 60** : $43°\ 44{,}304' = 43 + \dfrac{44{,}304}{60} \approx 43{,}7384°$. Dans l'autre sens, pour passer des degrés décimaux aux degrés-minutes, on garde la partie entière et on **multiplie la partie décimale par 60** : $7{,}5° = 7°\ (0{,}5 \times 60)' = 7°\ 30'$. Le GPS, on le verra, « parle » en degrés-minutes.

► **Exercices d'application :** exercices 1 à 4 (latitude, longitude, degrés décimaux)

II Le GPS : comment ça marche ?

Autour de la Terre gravitent une trentaine de **satellites** GPS (31 en service en 2023 ; GPS.gov). Chacun connaît sa position et diffuse en permanence un message radio contenant… l'**heure exacte** de l'envoi (il embarque une horloge atomique).



Un satellite GPS en orbite (vue d'artiste)

Règle 1 — Du temps à la distance

Le signal voyage à la **vitesse de la lumière** ($c \approx 300\,000$ km/s). Le récepteur (votre téléphone) compare l'heure d'**envoi** (dans le message) à l'heure de **réception** : la différence est le **temps de trajet**. Il en déduit sa **distance** au satellite :

$$\text{distance} = c \times \text{temps de trajet}.$$

Exemple

Si le signal d'un satellite a mis 0,07 seconde à arriver, le récepteur en est à $300\,000 \times 0{,}07 = 21\,000$ km.

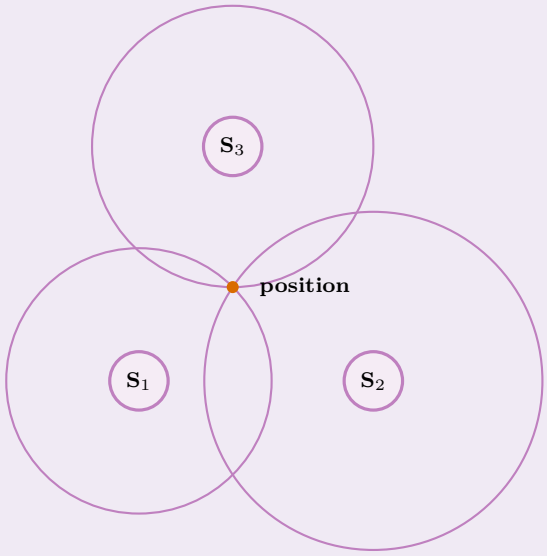
A. La trilatération : se placer grâce aux distances

Connaître sa distance à *un* satellite ne suffit pas : on peut être n'importe où sur une sphère autour de lui. Mais en **croisant plusieurs distances**, la position se resserre. C'est la **trilatération**.

Règle 2 — Le principe de la trilatération

- **1 satellite** : on est quelque part sur une **sphère** (à distance fixée) ;
- **2 satellites** : à l'intersection de deux sphères, un **cercle** ;
- **3 satellites** : l'intersection se réduit à **deux points** (dont un aberrant, loin de la Terre : on l'élimine).

En pratique, on capte un **4e satellite** pour corriger la petite erreur de l'horloge du récepteur.

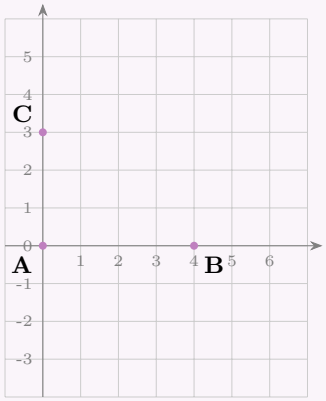


Trois « cercles de distance » se croisent en **un seul** point : votre position.

Remarque En deux dimensions (sur une carte), **trois** cercles suffisent à fixer un point : c'est le schéma ci-dessus. Sur la Terre (en 3D), on raisonne avec des **sphères**, d'où le besoin d'un satellite de plus.

▶ Activité 1 — Trilatération à la règle et au compas

Sur la carte quadrillée ci-dessous (**1 carreau = 1 km**), un randonneur repère trois lieux : un refuge A, un lac B et un sommet C. Son téléphone lui indique qu'il est à 5 km du refuge A, à 3 km du lac B et à 4 km du sommet C.



A(0 ; 0), B(4 ; 0), C(0 ; 3).

Recopier le quadrillage, tracer au compas les trois cercles (centre : le lieu ; rayon : la distance) et marquer sa **position** : le point commun aux trois cercles. Donner ses coordonnées.

▶ **Correction.** Les cercles de centre A (rayon 5) et de centre B (rayon 3) se coupent en **deux** points, $(4\,;3)$ et $(4\,;-3)$. Seul le premier est aussi à 4 km de C : le randonneur est au point $(\mathbf{4}\,;\mathbf{3})$. Le troisième cercle a servi à choisir entre les deux points, exactement comme un satellite supplémentaire pour le GPS.

Source : GPS.gov, page « Space Segment » (31 satellites en service, 2023), `gps.gov`.

▶ **Exercices d'application :** exercices 5 à 8 (le GPS et la trilatération)

III La trame NMEA : ce que « dit » le GPS

Une fois la position calculée, le récepteur GPS la transmet sous forme de courtes phrases de texte : les **trames NMEA** (norme *NMEA 0183*). La plus courante commence par `$GPGGA`.

Document 1 — Décoder une trame `$GPGGA`

```
$GPGGA,101530,4344.304,N,00725.476,E,1,07,1.0,15.0,M,,,,*20
```

On lit les champs, séparés par des virgules :

Champ	Signification
`$GPGGA`	type de trame (position + heure)
`101530`	**heure** UTC : 10 h 15 min 30 s
`4344.304,N`	**latitude** 43° 44,304′ Nord
`00725.476,E`	**longitude** 007° 25,476′ Est
`1`	qualité du positionnement (1 = fixé)
`07`	**nombre de satellites** utilisés
`15.0,M`	altitude : 15,0 mètres
`*20`	somme de contrôle (vérifie l'intégrité)

L'heure est donnée en **UTC** (*temps universel coordonné*) : c'est l'heure de référence mondiale, la même partout sur Terre. Monaco est à UTC+1 en hiver et à UTC+2 en été : 10 h 15 UTC correspond donc à 11 h 15 à Monaco en hiver, 12 h 15 en été.

Règle 3 — Lire la latitude et la longitude

Le GPS donne la latitude au format `ddmm.mmm` (degrés puis minutes) et la longitude au format `dddmm.mmm`. Pour obtenir des **degrés décimaux**, on sépare les degrés des minutes, puis on **divise les minutes par 60**.

Exemple

Dans la trame ci-dessus, la latitude `4344.304` se lit 43° et 44,304′, soit

$$43 + \frac{44{,}304}{60} \approx 43{,}7384^\circ \text{ N},$$

et la longitude `00725.476` donne $7 + \dfrac{25{,}476}{60} \approx 7{,}4246^\circ$ E. Cette trame indique donc… **Monaco** !

▶ **Exercices d'application :** exercice 9 (décoder une trame NMEA)

IV Les cartes numériques et le calcul d'itinéraire

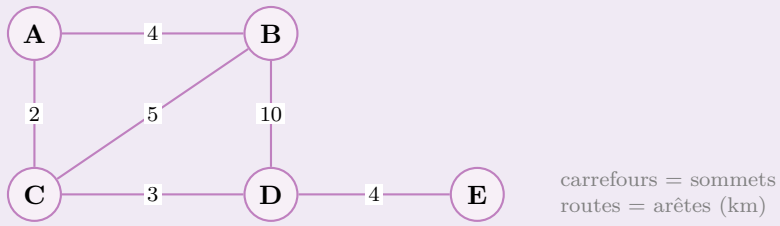
Une position seule ne sert à rien sans une **carte**. Les cartes numériques (Géoportail, OpenStreetMap, applications GPS) affichent votre point et calculent des trajets.

Définition 2

Une **carte numérique** superpose des **couches** d'information (routes, relief, bâtiments, commerces…) sur lesquelles on peut zoomer, mesurer une distance, et surtout **calculer un itinéraire**. **OpenStreetMap** est une carte **libre** (*open data*), construite et corrigée par des millions de contributeurs (plus de 2 millions ont déjà modifié la carte ; OpenStreetMap, 2025).

Règle 4 — Un itinéraire, c'est un plus court chemin dans un graphe

Pour calculer un trajet, l'application modélise le réseau routier par un **graphe** : chaque **carrefour** est un **sommet**, chaque **route** une **arête** *pondérée* par sa longueur (ou son temps de parcours). Trouver « le plus court chemin » revient à additionner les poids et à retenir le trajet de **somme minimale**.



De A à E : le trajet A–C–D–E mesure $2 + 3 + 4 = 9$ km, plus court que A–B–D–E ($4 + 10 + 4 = 18$).

▶ Activité 2 — Sur une carte en ligne

Ouvrir une carte numérique (Géoportail ou `openstreetmap.org`). **1.** Chercher son lycée et **lire ses coordonnées** (clic droit » « Où suis-je ? » / « Afficher l'adresse »). **2.** Calculer un **itinéraire** du lycée à la gare et relever la **distance** et la **durée**. **3.** Mesurer « à vol d'oiseau » la même distance : est-elle plus courte ? Pourquoi ?

Source : OpenStreetMap, page « Stats » du wiki, 2e trimestre 2025 (10 millions d'inscrits, 2,25 millions de contributeurs), wiki.openstreetmap.org.

▶ **Exercices d'application :** exercices 10 à 12 (cartes numériques et itinéraires)

V GPS, GNSS et localisation « assistée »

Le mot « GPS » est passé dans le langage courant, mais c'est en réalité **un** système parmi d'autres.

Règle 5 — Plusieurs systèmes, un nom générique : GNSS

On regroupe tous les systèmes de positionnement par satellites sous le sigle **GNSS**. Les principaux : **GPS** (États-Unis), **GLONASS** (Russie), **Galileo** (Europe), **BeiDou** (Chine). Un smartphone récent en utilise **plusieurs à la fois** pour se localiser mieux et plus vite.

À retenir — Localisation « assistée » : même sans voir le ciel

Le GPS fonctionne mal **en intérieur** ou entre de hauts immeubles (le signal des satellites est bloqué). Le téléphone se repère alors, en complément, grâce aux **antennes-relais** de l'opérateur et aux **réseaux Wi-Fi** environnants, dont les positions sont répertoriées. C'est plus rapide et fonctionne à l'intérieur, mais c'est moins précis.

► **Exercices d'application :** exercice 13 (pas seulement le GPS : GNSS)

VI Localisation et vie privée

Savoir où l'on est, c'est pratique. Mais chaque position est une **donnée personnelle** (chapitre *Les données structurées et leur traitement*), et l'accumulation de positions dessine… toute votre vie.

Règle 6 — Les traces de localisation

Beaucoup d'applications **enregistrent votre position** en continu. Recoupées, ces **traces** révèlent votre domicile, votre lycée, vos trajets, vos habitudes. Une simple **photo** prise au smartphone contient souvent, dans ses **métadonnées** (EXIF), les **coordonnées GPS** du lieu de la prise de vue.

Règle 7 — Reprendre la main

Dans les *réglages* du téléphone, on peut : **couper la localisation** quand elle est inutile, choisir **quelles applications** y ont droit (« toujours » / « seulement pendant l'usage » / « jamais »), et **effacer l'historique** des positions. Avant de publier une photo, on peut aussi **retirer ses métadonnées**.

À retenir — Ce que dit la loi : le RGPD

Les données de localisation sont des **données personnelles** : dans l'Union européenne, elles sont protégées par le **RGPD** (règlement européen 2016/679). Une application doit dire quelles données elle collecte et pourquoi, et chacun dispose de **droits** : accès, rectification, effacement, opposition… Si vous résidez en France (ou dans l'UE), vos droits relèvent du RGPD et de la loi Informatique et libertés (autorité : la **CNIL**).

À retenir — Et à Monaco ?

La loi n° 1.565 (2024) y protège de la même façon les données de localisation, sous le contrôle de l'**APDP** (voir l'encadré « Et à Monaco ? » du chapitre *Les réseaux sociaux*).

► Activité 3 — Enquête sur mes traces

1. Sur un téléphone, ouvrir les réglages de **localisation** et lister trois applications qui y ont accès. **2.** Expliquer, en deux phrases, en quoi l'historique de localisation peut poser un problème de **vie privée**. **3.** Proposer deux gestes concrets pour la protéger.

► **Exercices d'application :** exercice 14 (mes traces : localisation et vie privée)

VII Localisation et environnement

Consulter une carte ou calculer un itinéraire semble « immatériel ». Pourtant, cela mobilise un **téléphone** (qu'il a fallu fabriquer), des **réseaux** et des **centres de données** : les fonds de carte, les itinéraires calculés en ligne et l'état du trafic sont des données servies par des serveurs. Seul le calcul de la position lui-même se fait dans le téléphone, à partir des signaux reçus des satellites.

Règle 8 — Le coût d'une localisation permanente

Laisser la localisation activée en permanence a un coût, même si on ne le voit pas :

- **la batterie** : le récepteur GPS, et les liaisons radio (données mobiles, Wi-Fi) qui l'assistent, consomment de l'énergie ; une application qui suit votre position **en arrière-plan** vide plus vite la batterie. Or une batterie usée trop tôt est une raison fréquente de **remplacer** son téléphone ;
- **les données** : chaque position enregistrée et envoyée à un serveur grossit vos **traces**, stockées dans des centres de données (et, on l'a vu, elles en disent long sur votre vie) ;
- **le réseau** : fonds de carte et trafic en temps réel sont téléchargés à chaque usage, sauf si l'on a enregistré la carte **hors-ligne**.

L'essentiel de l'empreinte d'un téléphone vient pourtant de sa **fabrication** : le calcul complet, chiffres sourcés à l'appui, sera fait au chapitre *Informatique embarquée et objets connectés* (section « L'impact environnemental des objets connectés »).

▶ Activité 4 — Localisation et sobriété

1. Lorsque l'on calcule un itinéraire avec une application en ligne, quelles parties du numérique (terminaux, réseaux, centres de données) sont sollicitées ? **2.** Pourquoi une application qui vous localise « toujours », même fermée, pose-t-elle à la fois un problème de **batterie** et de **vie privée** ? **3.** Proposer deux gestes de **sobriété** liés à la localisation (par exemple : télécharger une carte hors-ligne une fois, en Wi-Fi, avant un voyage ; n'autoriser la localisation que « pendant l'utilisation » de l'application).

▶ **Exercices d'application :** exercice 15 (localisation et environnement), exercice 16 (critiquer une réponse d'IA), exercice 17 (Python au service du thème)

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Déjà vu :** un itinéraire est un plus court chemin dans un **graphe**, la structure même d'un **réseau social** ; la position se glisse dans les **métadonnées EXIF** des photos, et la trame NMEA ressemble à une ligne de **CSV** (chapitres *Photographie numérique* et *Les données structurées et leur traitement*).
- **À venir :** le récepteur GPS est un **capteur**, comme ceux des montres et des objets connectés du dernier chapitre.
- **Plus loin :** en spécialité NSI de Terminale, on apprend à parcourir des graphes et à y chercher des chemins.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Latitude / longitude	Nord–Sud (-90 à $90°$) / Est–Ouest (-180 à $180°$) ; Greenwich $= 0$.
Degrés-minutes	$1° = 60'$; minutes → degrés : diviser par 60 ; degrés → minutes : partie décimale $\times 60$.
Principe du GPS	le récepteur mesure le **temps de trajet** du signal → distance $= c \times t$.
Trilatération	croiser 3 distances (cercles) → 1 point ; 4e satellite pour l'horloge.
Trame NMEA	phrase texte (`$GPGGA`) : heure, latitude, longitude, nb de satellites.
Carte numérique	couches d'infos ; OpenStreetMap = carte libre (open data).
Itinéraire	plus court chemin dans un **graphe** (carrefours + routes pondérées).
GNSS	GPS, GLONASS, Galileo, BeiDou ; Wi-Fi/antennes en complément.
Vie privée	position = **donnée personnelle** ; traces, EXIF ; RGPD (UE), loi n° 1.565 (Monaco).
Environnement	localisation permanente : batterie, traces, réseau ; cartes hors-ligne.

Retour à la situation de départ

7

1. Des **satellites** : il mesure le **temps de trajet** de leurs signaux, en déduit ses **distances** à chacun (distance $= c \times t$), puis sa position par **trilatération** (§ II), exprimée en **latitude** et **longitude** (§ I).
2. Elle modélise les rues par un **graphe** (carrefours = sommets, routes = arêtes pondérées par leur longueur ou leur durée) et y cherche un **plus court chemin** (§ IV).
3. Recoupées, vos **traces** de localisation révèlent domicile, lycée, trajets et habitudes : ce sont des **données personnelles** (§ VI). Dans les réglages, on choisit « seulement pendant l'usage » ou « jamais ».

Débat argumenté

Un **débat**, proposé en fin de chapitre, accompagne ce thème : « Faut-il laisser la géolocalisation de son téléphone activée en permanence ? » (documents, rôles, tableau d'arguments et trace écrite, environ 50 min).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ me repérer sur la Terre avec la **latitude** et la **longitude** ;
- ☐ expliquer le principe du **GPS** (satellites, temps de trajet du signal, trilatération) ;
- ☐ lire une **trame NMEA** et y retrouver l'heure et les coordonnées ;
- ☐ utiliser une **carte numérique** et comprendre le calcul d'un **itinéraire** ;
- ☐ distinguer **GPS**, **GNSS** et localisation **assistée** ;
- ☐ citer les risques du **traçage** de la position pour la vie privée ;
- ☐ situer l'essentiel de l'**empreinte environnementale** d'un smartphone et proposer des gestes de sobriété.

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- [sur papier] *à faire sur papier* ; [sur machine] *à faire sur machine* (carte en ligne).
- Rappels : $1° = 60'$; distance $= c\times$ temps, avec $c \approx 300\,000$ km/s.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Latitude et longitude ↑cours

★ Exercice 1 *Nord ou Sud, Est ou Ouest ?* ▶corrigé p. 271 • essentiel

[sur papier]

1. Que vaut la latitude d'un point situé sur l'**équateur** ? la longitude d'un point sur le **méridien de Greenwich** ?
2. Entre quelles valeurs varie une latitude ? une longitude ?
3. Monaco est à (43,7384 ; 7,4246). Dire, pour chaque nombre, s'il s'agit de la latitude ou de la longitude, et préciser Nord/Sud et Est/Ouest.

▷ coup de pouce

★ Exercice 2 *Trois villes sur le globe* ▶corrigé p. 271

[sur papier] Voici les coordonnées (latitude ; longitude), en degrés décimaux, de trois villes : Monaco (43,74 ; 7,42), Rio de Janeiro (−22,91 ; −43,17) et Tokyo (35,68 ; 139,69).

1. Quelle ville est dans l'hémisphère **Sud** ?
2. Quelle ville est à l'**ouest** du méridien de Greenwich ?
3. Ranger les trois villes de la plus au nord à la plus au sud.

▷ coup de pouce

★★ Exercice 3 *Degrés, minutes et degrés décimaux* ▶corrigé p. 271 • essentiel

[sur papier] On rappelle que $1° = 60'$.

1. Convertir en degrés décimaux : $45°\,30'$; $12°\,15'$; $43°\,44{,}304'$.
2. Convertir en degrés et minutes : $6{,}25°$; $7{,}5°$.

▷ coup de pouce

★★★ Exercice 4 *Des degrés aux kilomètres* ▶corrigé p. 271 ≈15 min

[sur papier] On admet qu'un écart de 1° de latitude correspond, sur le terrain, à environ 111 km (en se déplaçant vers le nord ou vers le sud).

1. Un randonneur part de Monaco (43,74 ; 7,42) et marche plein nord jusqu'au point (44,24 ; 7,42). Quelle distance a-t-il parcourue, à vol d'oiseau ?
2. À combien de kilomètres correspond une minute (1′) de latitude ?
3. Une application affiche la latitude avec quatre décimales (0,0001° près). À quelle distance sur le terrain cela correspond-il ? Est-ce assez précis pour retrouver une maison ?

▷ coup de pouce ▷ début de solution

Le GPS et la trilatération ↑cours

★ **Exercice 5** *Du temps à la distance* ▶corrigé p. 271 • essentiel

[sur papier] Un signal met 0,06 seconde pour parvenir d'un satellite au récepteur.

1. À quelle **distance** (en km) le récepteur se trouve-t-il de ce satellite ?
2. Que devrait mesurer le récepteur pour calculer ce temps de trajet ?

▷coup de pouce

★★ **Exercice 6** *La précision des horloges* ▶corrigé p. 271

[sur papier] Le signal d'un satellite voyage à 300 000 km/s.

1. Quelle distance parcourt-il en un millième de seconde (0,001 s) ? En un millionième de seconde (0,000 001 s) ?
2. L'horloge d'un récepteur avance d'un millième de seconde. Quelle erreur sur la distance au satellite cela provoque-t-il ?
3. En déduire pourquoi les satellites embarquent des horloges **atomiques**, et pourquoi le récepteur capte un quatrième satellite.

▷coup de pouce

★★ **Exercice 7** *Combien de satellites ?* ▶corrigé p. 272 • essentiel

[sur papier]

1. Avec la distance à **un seul** satellite, que peut-on dire de la position (en 3D) ?
2. Pourquoi **trois** distances permettent-elles (sur une carte, en 2D) de fixer un point ?
3. En pratique, pourquoi capte-t-on un **quatrième** satellite ?

▷coup de pouce

★★ **Exercice 8** *Trilatération sur quadrillage* ▶corrigé p. 272

[sur papier] Sur un quadrillage, on place trois balises : A(0 ; 0), B(6 ; 0), C(0 ; 6) (unités en km). Un promeneur est à 5 km de A, $\sqrt{13} \approx 3{,}6$ km de B et 5 km de C.

1. Tracer les trois cercles (compas, à l'échelle) centrés en A, B, C.
2. Lire les coordonnées du point commun aux trois cercles : c'est la position du promeneur.

▷coup de pouce

Décoder une trame NMEA ↑cours

★★★ **Exercice 9** *Que dit le GPS ?* ▶corrigé p. 272 ≈20 min

[sur papier] On capte la trame suivante :

```
$GPGGA,142012,4342.054,N,00716.098,E,1,06,1.2,10.0,M,,,,*20
```

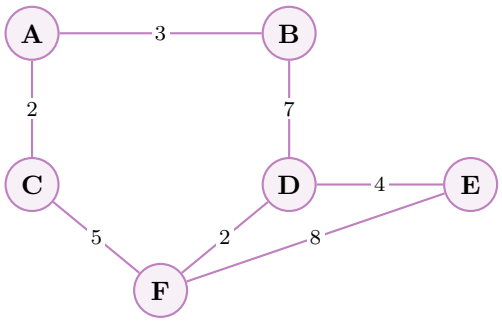
1. Donner l'**heure** (UTC) de la mesure.
2. Donner la **latitude** et la **longitude** en degrés-minutes, puis les convertir en **degrés décimaux** (arrondir à 0,0001).
3. Combien de **satellites** ont été utilisés ?
4. *Recherche :* à quelle ville de la Côte d'Azur ces coordonnées correspondent-elles ?

▷coup de pouce ▷début de solution

Cartes numériques et itinéraires ↑cours

★★ **Exercice 10** *Le plus court chemin* ▶corrigé p. 272 • essentiel

[sur papier] On modélise un réseau routier par le graphe suivant (distances en km) :

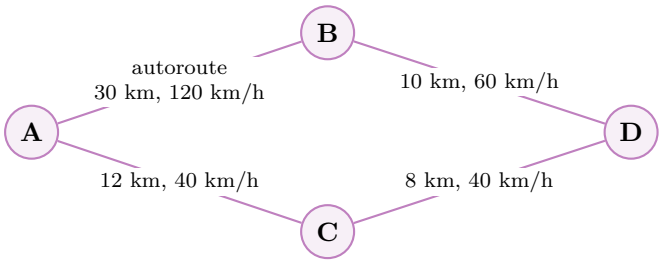


1. Donner deux itinéraires possibles de A à E et leur longueur totale.
2. Quel est le **plus court** chemin de A à E ? Quelle est sa longueur ? Est-ce aussi celui qui passe par le moins de carrefours ?
3. En quoi ce graphe ressemble-t-il à ce que calcule une application GPS ?

▷coup de pouce

★★★ **Exercice 11** *Le chemin le plus rapide* ▶corrigé p. 272 ≈15 min

[sur papier] Pour aller de A à D, deux itinéraires sont possibles (distance et vitesse moyenne sur chaque tronçon) :



1. Quel est l'itinéraire le plus **court** ?
2. Calculer la **durée** de chaque itinéraire, en minutes. Lequel est le plus **rapide** ?
3. Un bouchon fait tomber la vitesse moyenne sur l'autoroute à 40 km/h. Quel itinéraire une application de navigation conseille-t-elle alors ? Justifier.

▷coup de pouce ▷début de solution

★★ **Exercice 12** *Sur une vraie carte* ▶corrigé p. 273

[sur machine] Ouvrir une carte en ligne (Géoportail ou `openstreetmap.org`). Calculer l'**itinéraire** entre le **Musée océanographique** et le **stade Louis-II**, à Monaco, **à pied**, puis **en voiture** : relever chaque fois la **distance** et la **durée** proposées. Pourquoi les deux trajets ne sont-ils pas les mêmes ?

▷coup de pouce

GNSS et vie privée ↑cours

★ **Exercice 13** *Pas seulement le GPS* ▶corrigé p. 273

[sur papier]

1. Citer, en plus du GPS américain, **deux** autres systèmes de positionnement par satellites.
2. Pourquoi le GPS fonctionne-t-il mal à l'intérieur d'un bâtiment ? Comment le téléphone se localise-t-il alors ?

▷coup de pouce

★ **Exercice 14** *Mes traces* ▶ corrigé p. 273 • essentiel

[sur papier] Voici un extrait, simplifié, de l'historique de localisation enregistré par le téléphone de Léa pendant une semaine. Son nom n'y figure nulle part.

Quand ?	Où ?
toutes les nuits, de 21 h à 7 h	toujours le même immeuble d'habitation
du lundi au vendredi, de 8 h à 16 h	un lycée
le mercredi, de 17 h à 19 h	une piscine
le jeudi, de 10 h à 11 h	un cabinet médical

1. Que peut-on **déduire** de ces traces sur Léa ? Pourquoi sont-elles des **données personnelles**, alors que son nom n'y figure pas ?
2. Léa publie une photo de son chat, prise chez elle avec son téléphone. Comment un inconnu pourrait-il retrouver son adresse, alors que rien sur l'image ne la montre ?
3. Trois applications demandent à accéder à la localisation de Léa : une **lampe de poche**, une application de **navigation** et l'application qui permet de **retrouver son téléphone** en cas de perte ou de vol. Pour chacune, quel réglage lui conseiller (« toujours », « seulement pendant l'utilisation » ou « jamais ») ? Justifier.

▷ coup de pouce

Localisation et environnement ↑ cours

★★ **Exercice 15** *Le coût caché d'un itinéraire* ▶ corrigé p. 273

[sur papier] On s'appuie sur l'encadré « Le coût d'une localisation permanente » du cours. Une application de course à pied enregistre la position de Nora **toutes les secondes** pendant ses sorties et envoie le parcours à ses serveurs. On suppose qu'un point (heure, latitude, longitude) occupe environ 30 octets.

1. Nora court 1 heure, trois fois par semaine. Combien de points sont enregistrés pendant une sortie ? Quel volume de données cela représente-t-il pour une sortie, puis pour une année (52 semaines) ? Donner le résultat en mégaoctets (1 Mo $= 10^6$ octets).
2. Ce volume est modeste. Pourquoi ces traces posent-elles pourtant une question de **vie privée** ? Où sont-elles stockées ?
3. Une application de navigation affiche le trafic en temps réel. Pourquoi cela sollicite-t-il des **centres de données** et des **réseaux**, alors que le calcul de la position GPS, lui, se fait dans le téléphone ?
4. Proposer deux gestes de **sobriété** liés à l'usage des cartes et de la localisation (pensez aussi à la batterie), et justifier leur effet.

▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 16** *Le GPS expliqué par un assistant* ▶ corrigé p. 274

[sur papier] Un élève a demandé à un assistant d'IA : « Comment mon téléphone connaît-il ma position grâce au GPS ? » Voici la réponse obtenue :

> *Une trentaine de satellites GPS tournent autour de la Terre ; chacun embarque une horloge atomique et connaît sa position. Votre téléphone envoie un signal radio vers les satellites ; chaque satellite mesure le temps mis par ce signal pour lui parvenir, calcule la distance (vitesse de la lumière × temps de trajet, par exemple* $300\,000 \times 0{,}05 = 15\,000$ *km) et renvoie cette distance au téléphone. Avec trois distances, le téléphone se place par trilatération ; un quatrième satellite sert à corriger l'erreur de son horloge. En intérieur, le signal est bloqué et le téléphone se rabat sur le Wi-Fi et les antennes-relais.*

1. La réponse est-elle correcte ? Vérifier le calcul, puis relire l'encadré « Du temps à la distance » du cours : qui envoie le signal, qui le reçoit ?
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Python au service du thème ↑ cours

★★ **Exercice 17**  *Python au service du thème — convertir des coordonnées* ▶ corrigé p. 274

[sur machine] En Python (Spyder ou Basthon). Un récepteur GPS donne ses positions en degrés-minutes, alors qu'une carte en ligne les attend en degrés décimaux : programmons la conversion du cours.

1. Écrire une fonction `en_decimal(degres, minutes)` qui renvoie la valeur en degrés décimaux. La tester : `en_decimal(45, 30)` doit renvoyer `45.5`, et `en_decimal(43, 44.304)` la latitude de Monaco.
2. Dans une trame, le sens est donné par une lettre : `N` ou `S` pour la latitude, `E` ou `W` (*West*, ouest) pour la longitude. Compléter la fonction suivante, qui renvoie une coordonnée **négative** au sud et à l'ouest :

```
1 def coordonnee(degres, minutes, sens):
2     valeur = en_decimal(degres, minutes)
3     if ...:
4         valeur = -valeur
5     return valeur
```

3. Rio de Janeiro est à 22° 54,6′ S et 43° 10,2′ W. Calculer ses coordonnées en degrés décimaux avec la fonction `coordonnee`, puis les comparer à celles de l'exercice « Trois villes sur le globe ». Utiliser enfin la fonction pour vérifier la question 2 de l'exercice « Que dit le GPS ? ».

▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relisez la définition de la latitude et de la longitude au début du cours : à partir de quelle ligne chacune est-elle mesurée, et entre quelles valeurs ? ↑ énoncé

Ex. 2 – piste D'après la définition du cours, que signifie une latitude négative ? Une longitude négative ? Pour la question 3, comparez les latitudes. ↑ énoncé

Ex. 3 – piste Les minutes sont des soixantièmes de degré : pour convertir des minutes en degrés, divisez par 60. Dans l'autre sens, multipliez la partie décimale par 60. ↑ énoncé

Ex. 4 – piste Commencez par calculer l'écart de latitude entre les deux points, puis utilisez la proportionnalité : 1° correspond à 111 km. ↑ énoncé

Ex. 4 – début de solution Question 1 : l'écart vaut $44{,}24 - 43{,}74 = 0{,}5°$. Question 2 : une minute, c'est 1/60 de degré. Question 3 : 0,0001°, c'est un dix-millième de degré. ↑

Ex. 5 – piste Relisez la règle « Du temps à la distance » du cours et l'exemple qui la suit. ↑ énoncé

Ex. 6 – piste Distance = vitesse × temps, comme dans l'exercice précédent. Une erreur sur le temps de trajet devient une erreur sur la distance, multipliée par 300 000. ↑ énoncé

Ex. 7 – piste Connaître sa distance à un satellite place le récepteur sur une sphère centrée sur ce satellite. Que donne l'intersection de deux sphères ? de trois ? ↑ énoncé

Ex. 8 – piste Le rayon de chaque cercle est la distance à la balise. Choisissez une échelle simple (par exemple 1 cm pour 1 km), puis vérifiez le point trouvé avec les trois distances. ↑ énoncé

Ex. 9 – piste Les informations sont séparées par des virgules, toujours dans le même ordre : heure, latitude, N/S, longitude, E/O, …, nombre de satellites. Relisez le tableau de la trame dans le cours. ↑ énoncé

Ex. 9 – début de solution Dans `4342.054`, les deux premiers chiffres sont les degrés et le reste les minutes : $43°\,42{,}054'$, soit $43 + 42{,}054/60$. Pour la longitude `00716.098`, les degrés occupent les trois premiers chiffres. ↑ énoncé

Ex. 10 – piste Listez tous les chemins de A à E qui ne repassent jamais par un même sommet, puis additionnez les distances de chacun. ↑ énoncé

Ex. 11 – piste Durée (en heures) = distance divisée par vitesse ; multipliez ensuite par 60 pour obtenir des minutes. Calculez la durée de chaque tronçon, puis additionnez. ↑ énoncé

Ex. 11 – début de solution Tronçon A–B : $30 \div 120 = 0{,}25$ h, soit $0{,}25 \times 60 = 15$ min. Faites de même pour les trois autres tronçons. ↑ énoncé

Ex. 12 – piste Sur OpenStreetMap, le bouton d'itinéraire (les flèches) calcule le trajet, et le mode de déplacement se choisit dans sa fenêtre. Demandez-vous quels passages un piéton peut emprunter et pas une voiture. ↑ énoncé

Ex. 13 – piste Relisez la règle sur les GNSS et l'encadré « Localisation assistée » du cours. ↑ énoncé

Ex. 14 – piste Relisez la section « Localisation et vie privée » du cours : ce que révèlent des traces recoupées, les métadonnées d'une photo, puis la règle « Reprendre la main ». Pour la question 3, demandez-vous si l'application a vraiment besoin de savoir où est Léa pour fonctionner, et à quel moment. ↑ énoncé

Ex. 15 – piste Question 1 : une heure compte 3 600 secondes ; multipliez ensuite par la taille d'un point, puis par le nombre de sorties dans l'année. Question 3 : la position est calculée dans le téléphone, mais d'où viennent les informations sur le trafic ? ↑ énoncé

Ex. 16 q. 2 – piste Relisez dans le cours ce que contient le message d'un satellite et ce que fait le récepteur, puis comparez avec le récit de l'assistant, phrase par phrase. ↑ énoncé

Ex. 17 – piste Question 1 : relisez l'encadré « Degrés décimaux ou degrés-minutes » du cours et traduisez son calcul en Python. Question 2 : dans quels cas faut-il changer le signe ? Relisez comment on compare deux chaînes de caractères et comment on relie deux conditions. ↑ énoncé

DÉBAT

DÉBAT Faut-il laisser la géolocalisation de son téléphone activée en permanence ?

▶ corrigé p. 275

service ou surveillance ?

Durée : 50 min

Consignes

- En classe entière. Les rôles sont tirés au sort : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement.
- Chaque argument s'appuie sur un **document** ci-dessous ou sur le **cours**. Préparation sans IA ni recherche en ligne.
- Avant de commencer, notez sur votre cahier votre position de départ : oui, non ou je ne sais pas.

Rappel du cours. Votre téléphone se localise grâce aux satellites (**GPS**, Galileo… : le **GNSS**), mais aussi grâce aux **antennes-relais** et aux **réseaux Wi-Fi**. Chaque position est une **donnée personnelle**, et l'accumulation de positions laisse des **traces**. Les réglages permettent de choisir, pour chaque application, « toujours », « seulement pendant l'utilisation » ou « jamais ». Laisser la localisation allumée en permanence : un service utile, ou une surveillance acceptée ?

Les deux camps

Groupe POUR — oui : les services rendus valent bien de laisser la localisation activée en permanence.
Groupe CONTRE — non : la localisation ne doit être activée que lorsqu'on en a vraiment besoin.

Les documents

Document 1 — Ce que la position rend possible

Calculer un itinéraire, connaître le trafic en temps réel ou la météo du quartier, retrouver un téléphone perdu ou volé, partager sa position lors d'un trajet de nuit : beaucoup de services reposent sur la localisation. Elle peut aussi sauver des vies : depuis 2018, les nouveaux modèles de voitures vendus dans l'Union européenne ont le système **eCall**, qui, en cas d'accident grave, appelle le **112** et transmet la position du véhicule aux secours.

À retenir : la position rend des services, jusqu'à guider les secours.

Source : règlement (UE) 2015/758 du 29 avril 2015 relatif au système eCall.

Document 2 — Quatre points suffisent

En 2013, des chercheurs du MIT (États-Unis) et de l'université de Louvain (Belgique) ont étudié 15 mois de données de téléphonie mobile concernant **1,5 million** de personnes, sans aucun nom. Il suffisait de connaître **4 lieux et heures** approximatifs où une personne s'était trouvée pour l'identifier dans **95 %** des cas. Vos trajets sont comme une empreinte digitale : domicile, lycée, club de sport…

À retenir : quelques positions, même sans nom, suffisent à identifier une personne.

Source : Y.-A. de Montjoye et al., « Unique in the Crowd : The privacy bounds of human mobility *»,* Scientific

Reports, *2013*.

Document 3 — La carte qui en disait trop

En novembre 2017, l'application sportive Strava publie une carte mondiale des parcours enregistrés par le GPS de ses utilisateurs, sans aucun nom. En janvier 2018, des analystes y repèrent, dans des régions presque désertes, des tracés de course réguliers autour de bases militaires, dont certaines n'étaient pas censées être connues. En août 2018, l'armée américaine a restreint la géolocalisation des appareils de ses soldats en zone d'opération.

À retenir : accumulées, des positions anonymes peuvent révéler des lieux sensibles.

Source : presse internationale, janvier 2018 ; département de la Défense des États-Unis, août 2018.

Document 4 — Ce que dit la loi, ce que l'on choisit

La position est une **donnée personnelle** : une application doit dire pourquoi elle la collecte, ne garder que le nécessaire et, en général, demander votre accord (**RGPD** et CNIL en France ; **loi n° 1.565** et APDP à Monaco). Mais si vous avez répondu « toujours », la position peut être enregistrée en continu, même quand vous n'utilisez pas l'application. La plupart des téléphones récents permettent aussi de n'accorder qu'une position **approximative**.

À retenir : la loi encadre, mais ce sont vos réglages qui décident.

Source : RGPD ; loi monégasque n° 1.565 du 3 décembre 2024 ; réglages de localisation d'Android et d'iOS.

Rôles, déroulé minuté, règles de prise de parole et trace écrite : voir la page « Méthode : débattre » p. 7.

Tableau d'arguments

Recopier ce tableau sur le cahier et le compléter pendant le débat.

Argument POUR (oui)	Doc.	Argument CONTRE (non)	Doc.
…		…	

Trace écrite finale (individuelle)

Sur le cahier, notez votre position après le débat (oui, non, je ne sais toujours pas) et si vous avez changé d'avis. Puis, en trois à cinq lignes, justifiez-la avec vos **deux meilleurs arguments**, dont **un argument de l'autre camp** que vous trouvez solide (avec le document qui l'appuie).

FICHE DE RÉVISION

L'essentiel et le vocabulaire

- **Latitude** Nord–Sud de $-90°$ à $90°$ (**équateur** $0°$) ; **longitude** Est–Ouest de $-180°$ à $180°$ (**Greenwich** $0°$).
- Le GPS transforme un **temps de trajet** du signal en **distance**, puis croise plusieurs distances (**trilatération**).
- Le récepteur écrit sa position en texte (**trame NMEA**) ; un itinéraire est un **plus court chemin** (graphe).
- **GNSS** : GPS (É.-U.), GLONASS (Russie), Galileo (Europe), BeiDou (Chine) ; **assistée** : antennes, Wi-Fi.
- **Carte numérique** : des **couches** d'information ; OpenStreetMap est **libre** (open data).
- Une position est une **donnée personnelle** ; se localiser en permanence coûte batterie, données, réseau.

Méthodes à savoir refaire

1. Degrés-minutes → degrés décimaux : $1° = 60'$, on **divise les minutes par 60** : $30°\,45' = 30 + \frac{45}{60} = 30{,}75°$. Dans l'autre sens, on **multiplie la partie décimale par 60** : $7{,}5° = 7°\,(0{,}5 \times 60)' = 7°\,30'$.

2. Du temps à la distance : distance $= c \times t$, $c \approx 300\,000$ km/s. En 0,07 s : $300\,000 \times 0{,}07 = 21\,000$ km.

3. Trilatération : 1 satellite → sphère, 2 → cercle, 3 → 2 points ; le **4e** corrige l'horloge. Sur une carte : **3 cercles**.

4. Décoder une trame `$GPGGA` (champs séparés par des virgules) :

```
$GPGGA,101530,4344.304,N,00725.476,E,1,07,1.0,15.0,M,,,,*20
```

`101530` : heure **UTC** 10 h 15 min 30 s • `4344.304,N` (`ddmm.mmm`) : $43 + \frac{44{,}304}{60} \approx 43{,}7384°$ N • `00725.476,E` (`dddmm.mmm`) : $7 + \frac{25{,}476}{60} \approx 7{,}4246°$ E • `07` : nombre de **satellites** • `15.0,M` : altitude. C'est Monaco !

5. Plus court chemin : carrefours = **sommets**, routes = **arêtes** pondérées (km ou minutes) ; on garde le trajet de **somme minimale** : A–C–D–E $= 2 + 3 + 4 = 9$ km bat A–B–D–E $= 4 + 10 + 4 = 18$ km.

Repères et enjeux

GPS opérationnel en **1995**, signal civil dégradé jusqu'en **2000** ; **31** satellites (2023). **Vie privée** : les traces révèlent domicile et habitudes, une photo garde ses coordonnées (**EXIF**) ; **RGPD** (UE, CNIL), loi n° 1.565 (Monaco, APDP). **Gestes** : localisation « pendant l'utilisation », effacer l'historique, carte hors-ligne.

Erreurs à éviter

Inverser latitude et longitude ; lire $30°\,45'$ comme $30{,}45°$; prendre l'heure UTC pour l'heure locale ; confondre GPS et GNSS.

Savoir-faire à maîtriser — je sais…

- ☐ me repérer sur la Terre avec la **latitude** et la **longitude** ;
- ☐ expliquer le principe du **GPS** (satellites, temps de trajet du signal, trilatération) ;
- ☐ lire une **trame NMEA** et y retrouver l'heure et les coordonnées ;
- ☐ utiliser une **carte numérique** et comprendre le calcul d'un **itinéraire** ;
- ☐ distinguer **GPS**, **GNSS** et localisation **assistée** ;
- ☐ citer les risques du **traçage** de la position pour la vie privée ;
- ☐ situer l'essentiel de l'**empreinte environnementale** d'un smartphone et proposer des gestes de sobriété.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 276

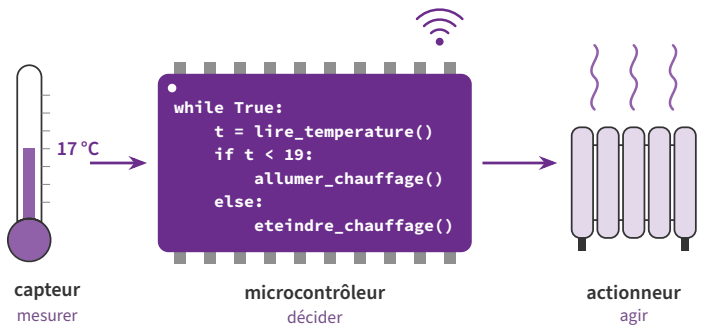
1. Convertir $50°\,15'$ en degrés décimaux.
2. Un signal met 0,08 s pour arriver. Calculer la distance au satellite.
3. Dans `$GPGGA,163045,4545.840,N,...,1,09,...`, donner l'heure UTC et le nombre de satellites.
4. Donner deux gestes pour limiter ses traces de localisation.

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 276

1. Donner le rôle du DNS (ch. 6).
2. Dire ce que garantit le cadenas HTTPS (ch. 4).

CHAPITRE 8

Informatique embarquée et objets connectés



DANS CE CHAPITRE

- Activités préparatoires 194
- Cours 196
- Exercices 204
- Débat 210
- Fiche de révision 212
- Corrigés 277

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 La carte qui réagit

▶ corrigé p. 277

programmer une carte micro:bit sur simulateur

Durée : 20 min · Par deux, sur ordinateur

Consignes

- Ouvrir `makecode.microbit.org` (gratuit, sans compte) et cliquer sur **Nouveau projet**. Si l'éditeur est en anglais : roue dentée (en haut à droite) → *Language* → *Français*.
- On programme avec des **blocs** que l'on fait glisser depuis les catégories du menu de gauche (**Base**, **Entrée**, **Logique**…). La carte dessinée à gauche est un **simulateur** : on peut cliquer sur ses boutons.
- Pas besoin de vraie carte. On répond aux questions sur le cahier.

Exercice 1 *Un bouton, une réponse*

1. Dans **Entrée**, faire glisser le bloc `lorsque le bouton A est pressé`. Dans **Base**, prendre `montrer l'icône` et l'emboîter *à l'intérieur* du premier bloc ; choisir le cœur. Cliquer sur le bouton A du simulateur. Que se passe-t-il ?
Et tant qu'on n'appuie sur rien ?

2. Ajouter un deuxième bloc pour que le bouton B affiche une autre icône. Tester.

3. Sur cette carte, quel élément permet à l'humain de **donner un ordre** ? Lequel permet à la carte de **répondre** ?

Exercice 2 *La carte mesure*

4. Dans le bloc `toujours` (catégorie **Base**), placer `montrer nombre`, puis remplacer le 0 par le bloc `température (°C)` (catégorie **Entrée**). Un thermomètre apparaît à côté du simulateur : faire glisser son curseur. Que montre la carte ?

5. Sur une vraie carte, ce nombre n'est pas tapé par quelqu'un. D'où vient-il ?

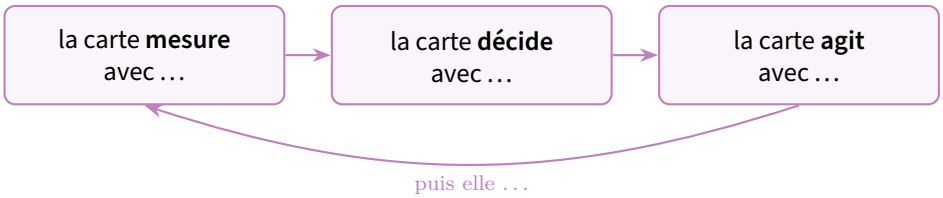
6. Déplacer `montrer nombre` dans `au démarrage`, à la place de `toujours`, puis bouger le curseur. Que change-t-il ? À quoi sert donc `toujours` ?

Exercice 3 *La carte décide*

Votre mission : la carte doit surveiller une salle et afficher l'icône **non** (une croix) si la température **dépasse** 25 °C, et l'icône **oui** (une coche) sinon. Aucun nouveau bloc ne vous est expliqué : cherchez dans les catégories.

7. Dans quelle catégorie avez-vous trouvé le bloc qui permet de choisir entre deux actions ? Comment s'appelle-t-il ?

8. Où avez-vous placé la comparaison avec 25 ? Écrire la condition utilisée.
9. Tester avec le curseur à 20 °C puis à 30 °C : qu'affiche la carte dans chaque cas ?
10. Recopier sur le cahier et compléter le schéma de votre programme :



Exercice 4 *Ce qu'on a découvert*

11. Recopier sur le cahier et compléter, en choisissant parmi ces mots : *actionneur*, *capteur*, *programme*, *sans fin*, *communiquer*.

- La partie de la carte qui **mesure** le monde réel s'appelle un …
- La partie qui **agit** sur le monde réel (ici, qui s'allume) s'appelle un …
- Entre les deux, le … décide, et il recommence …
- Les boutons et l'écran de LED permettent à l'humain et à la machine de …

12. La carte micro:bit est un petit ordinateur. Citer une différence avec l'ordinateur de la salle.

COURS

Programme Systèmes informatiques **embarqués** ; **capteurs** et **actionneurs** ; **interface homme-machine** (IHM) ; **objets connectés** et réseaux d'objets (IoT) ; algorithme de **commande** d'un système physique ; enjeux de **sécurité** et de **vie privée**.

Idée Des ordinateurs minuscules se cachent dans nos objets et pilotent le monde physique : ils *mesurent* (capteurs), *décident* (un programme), *agissent* (actionneurs). Reliés au réseau, ils deviennent **connectés** — pratiques, mais pas sans risques.

Objectifs Reconnaître un système embarqué ; distinguer **capteur** et **actionneur** ; lire et écrire une **boucle de commande** simple ; décrire une **IHM** ; comprendre l'**Internet des objets** ; mesurer les enjeux de sécurité, de vie privée et d'environnement.

Situation de départ

1. Le chauffage de votre maison s'allume tout seul quand il fait froid, et s'éteint quand il fait assez chaud. Qui décide, et comment ?
2. Une caméra de surveillance connectée peut-elle être piratée, et se retourner contre vous ou contre d'autres ?
3. Votre téléphone pollue-t-il surtout quand vous l'utilisez, ou avant même votre premier appel ?

Pour les curieux — *Un peu d'histoire* (facultatif)

L'un des tout premiers ordinateurs *embarqués* est l'**Apollo Guidance Computer** qui, en **1969**, a guidé les astronautes jusqu'à la Lune avec une puissance… ridicule face à un smartphone d'aujourd'hui. Le **premier objet connecté** de l'histoire serait un **grille-pain** branché à Internet, présenté pour s'amuser en **1990**. L'expression « **Internet des objets** » (*Internet of Things*) est lancée en **1999** par **Kevin Ashton** (Ashton, *RFID Journal*, 2009). On comptait fin 2024 **près de 20 milliards** d'objets connectés dans le monde (sans compter smartphones et ordinateurs) — plus de deux fois le nombre d'humains (IoT Analytics, 2024 ; ONU, 2022).



L'ordinateur de guidage d'Apollo



Kevin Ashton (2015)

Sources : K. Ashton, « That "Internet of Things" Thing », *RFID Journal*, 2009 ; IoT Analytics, *State of IoT – Summer 2024*, septembre 2024 (18,8 milliards d'objets connectés estimés fin 2024), `iot-analytics.com` ; ONU (Département des affaires économiques et sociales), *World Population Prospects 2022* (8 milliards d'humains en novembre 2022), `un.org`.

→ frise chronologique p. 10

I L'informatique invisible : les systèmes embarqués

L'ordinateur n'est plus seulement la machine posée sur un bureau. Il a **rétréci** et s'est glissé *dans les objets*.

Définition 1

Un **système embarqué** est un ordinateur **intégré dans un objet**, dédié à **une tâche précise**. Il est le plus souvent minuscule, sobre en énergie, et invisible pour l'utilisateur. Le petit processeur qu'il contient s'appelle un **microcontrôleur**.

Exemple

On en trouve partout : lave-linge, four à micro-ondes, ascenseur, distributeur de billets, montre connectée, régulateur de vitesse d'une voiture, box Internet, feu tricolore, drone, carte bancaire… Une voiture récente en contient à elle seule **plusieurs dizaines**.

À retenir — Différence avec un ordinateur ordinaire

Un système embarqué ne fait **qu'une** chose (réguler une température, afficher l'heure…), n'a en général **pas d'écran ni de clavier** classiques, et doit souvent réagir **en temps réel** : un airbag ne peut pas « ramer ».

► **Exercices d'application :** exercices 1 **et** 2 (systèmes embarqués)

II Sentir et agir : capteurs et actionneurs

Pour interagir avec le monde physique, un système embarqué a besoin de deux sortes d'organes.



Une carte micro:bit : capteurs, boutons et LED

Définition 2

Un **capteur** *mesure* une grandeur physique (température, lumière, mouvement, pression…) et la transforme en un **nombre** exploitable par la machine. C'est une **entrée**.

Un **actionneur** *agit* sur le monde physique (chauffer, éclairer, tourner, ouvrir, émettre un son…). C'est une **sortie**.

Capteurs (entrées)	Actionneurs (sorties)
thermomètre (température)	radiateur, ventilateur
photorésistance (luminosité)	lampe, LED
détecteur de mouvement	moteur (volet, portail)
microphone (son)	haut-parleur
capteur d'image (chapitre *Photographie numérique*)	écran

↻ **Retour sur l'activité.** Dans « La carte qui réagit », le thermomètre de la carte, qui fournissait le nombre affiché, était un **capteur** ; l'écran de LED, qui montrait le cœur ou la coche, était un **actionneur**.

▶ Activité 1 — Capteurs ou actionneurs ?

Pour un **smartphone**, classer en deux colonnes : l'écran tactile, le haut-parleur, l'appareil photo, le vibreur, le micro, la lampe torche, le capteur de luminosité.

▶ **Exercices d'application :** exercices 3 **et** 4 (capteurs et actionneurs)

III La boucle de commande : mesurer, décider, agir

Le cœur d'un objet embarqué est un **programme** qui répète sans fin le même cycle.

Règle 1 — Le cycle acquisition → décision → action



Le système **acquiert** une mesure, **décide** (avec des conditions), **commande** un actionneur, puis **recommence**.

↻ **Retour sur l'activité.** Le schéma complété à la fin de l'activité est exactement ce cycle : la carte *mesurait* la température, *décidait* avec le bloc *si… alors… sinon*, *agissait* en affichant une icône, et le bloc *toujours* la faisait recommencer sans fin.

Exemple — *Le thermostat d'une maison connectée*

On veut maintenir une pièce à 19 °C. On dispose d'un capteur `lire_temperature()` et d'un radiateur, que l'on commande avec `allumer_chauffage()` et `eteindre_chauffage()`.

```
while True:                          # boucle sans fin
    t = lire_temperature()           # ENTREE : le capteur -> un nombre
    if t < 19:
        allumer_chauffage()          # SORTIE : on commande un actionneur
    else:
        eteindre_chauffage()
    attendre(60)                     # on re-teste chaque minute
```

On reconnaît le cycle : **lire** la température, **décider** (`if`), **agir** sur le radiateur, **recommencer**.

À retenir — Deux nouveautés : `while True` et les fonctions de la carte

- La boucle `while True:` se répète « tant que `True` est vrai ». Or `True` est **toujours** vrai : la boucle ne s'arrête jamais. Au chapitre *Les bases de Python*, une boucle sans fin était un piège, car le programme ne se terminait pas. Ici, c'est **voulu** : un objet embarqué doit surveiller et réagir tant qu'il est allumé. Son microcontrôleur n'exécute que ce programme, de la mise sous tension jusqu'à ce qu'on coupe le courant ; il n'a rien d'autre à faire.
- `lire_temperature()`, `allumer_chauffage()`, `attendre(60)`… ne sont pas des fonctions de Python : dans un vrai objet, elles sont fournies avec la carte électronique,

et on les **appelle** comme `print` ou `input`, sans savoir comment elles fonctionnent à l'intérieur. Une fonction `lire_...()` renvoie la mesure du capteur ; `allumer_...()` ou `eteindre_...()` commande un actionneur ; `attendre(n)` met le programme en **pause** pendant `n` secondes. Comme aucune carte n'est branchée à votre ordinateur, ces programmes s'écrivent **sur papier**.

À retenir — C'est un algorithme comme un autre

La « décision » n'est qu'une suite de **conditions** et de **boucles** — ce que vous savez déjà écrire (chapitre *Les bases de Python*), avec en plus la boucle sans fin `while True`. Un volet qui se ferme quand la nuit tombe, un arrosage qui se déclenche quand la terre est sèche : même boucle, autres capteurs et actionneurs.

▶ Activité 2 — Compléter une boucle de commande

Un **éclairage automatique** allume une LED quand la luminosité `lum` devient faible (en dessous de 30). Recopier et compléter :

```
while True:
    lum = lire_luminosite()
    if ... :                    # (a) condition
        allumer_led()           # (b)
    else:
        eteindre_led()
    attendre(1)
```

▶ **Exercices d'application :** exercices 5 à 10 (la boucle de commande : arrosage automatique, alarme, thermostat)

IV L'interface homme-machine (IHM)

Comment un humain *dialogue*-t-il avec un objet ? Par son **interface**.

Définition 3

L'**interface homme-machine** (IHM) est l'ensemble des moyens par lesquels l'utilisateur **commande** la machine et **reçoit** des informations d'elle :

- **entrées** (l'humain vers la machine) : bouton, écran tactile, molette, voix, geste…
- **sorties** (la machine vers l'humain) : écran, LED, son, vibration…

Règle 2 — Une bonne IHM

Une IHM réussie est **simple**, **compréhensible** et donne un **retour** : quand on appuie sur un bouton, un voyant, un bip ou un affichage confirme que l'action a été prise en compte. Une mauvaise IHM laisse l'utilisateur dans le doute (« a-t-il compris ? »).

Exemple

Sur un **lave-linge** : la molette et les boutons sont l'**entrée**, l'écran et le bip de fin sont la **sortie**. Sur un **assistant vocal**, l'entrée est la **voix** et la sortie est… la voix aussi.

▶ **Exercices d'application :** exercices 11 à 13 (l'interface homme-machine)

V Les objets connectés et l'Internet des objets

Un système embarqué qui, en plus, se **relie au réseau** devient un **objet connecté**.

Définition 4

Un **objet connecté** est un objet équipé de capteurs et/ou d'actionneurs qui **échange des données** par un réseau (Wi-Fi, Bluetooth, 4G/5G…). L'ensemble de ces objets forme l'**Internet des objets** (*IoT*, *Internet of Things*).

Exemple

Montre qui envoie votre rythme cardiaque au téléphone ; thermostat réglable à distance ; enceinte connectée ; caméra de surveillance ; capteurs de pollution d'une ville « intelligente » ; balance qui mémorise votre poids dans le nuage.

Pour les curieux — *Le nuage n'est jamais loin* (facultatif)

Beaucoup d'objets connectés envoient leurs mesures vers des **serveurs** (le *cloud*, chapitre *Les données structurées et leur traitement*), où elles sont stockées et analysées. Ces montagnes de données alimentent des services (statistiques, recommandations) mais posent aussi la question : *qui* les détient ?

► **Exercices d'application :** exercice 14 (objets connectés)

VI Sécurité et vie privée

Un objet connecté est un petit ordinateur *relié à Internet* : il peut donc, comme tout ordinateur, être **attaqué**.

Document 1 — Quand des caméras attaquent Internet (Mirai, 2016)

En **2016**, un logiciel malveillant nommé **Mirai** a pris le contrôle de **centaines de milliers** d'objets connectés (jusqu'à environ 600 000 à son pic ; Antonakakis et al., 2017) (caméras, magnétoscopes numériques…) dont les propriétaires n'avaient jamais changé le **mot de passe par défaut** (souvent `admin`/`admin`). Transformés en armée de « robots », ces objets ont **saturé** de grands sites (déni de service, chapitre *Internet*) et rendu inaccessibles, pendant des heures, Twitter, Netflix, Spotify… *La leçon : un objet mal sécurisé met en danger bien plus que son propriétaire.*

Source : M. Antonakakis et al., « Understanding the Mirai Botnet », *USENIX Security Symposium*, 2017.

Règle 3 — Deux enjeux à retenir

- **Sécurité** : un objet connecté est une **porte d'entrée** sur le réseau (et sur votre maison). *Réflexes :* changer le mot de passe par défaut, faire les mises à jour.
- **Vie privée** : un objet connecté est un **capteur** qui observe (caméra), écoute (micro) ou vous suit (montre, GPS). Ses données sont des **données personnelles** (chapitre *Les données structurées et leur traitement*), protégées dans l'Union européenne par le **RGPD**.

À retenir — Et à Monaco ?

Les données d'un objet connecté y sont protégées par la loi n° 1.565 (2024), sous le contrôle de l'**APDP** : ce sont les mêmes grands droits qu'avec le RGPD (voir l'encadré « Et à Monaco ? » du chapitre *Les réseaux sociaux*).

▶ Activité 3 — Enquête sur un objet connecté

Choisir un objet connecté du quotidien (maison, lycée, sport…). **1.** Quels **capteurs** et **actionneurs** possède-t-il ? **2.** Quelles **données** envoie-t-il, et à qui ? **3.** Citer un risque de **sécurité** et un risque de **vie privée**, puis un geste pour s'en protéger.

▶ **Exercices d'application :** exercices 15 **et** 16 (sécurité : mots de passe par défaut ; vie privée)

VII L'impact environnemental des objets connectés

Un objet connecté consomme un peu d'électricité… mais c'est surtout sa **fabrication** qui pèse. Les chiffres ci-dessous sont des **ordres de grandeur**, tirés d'études publiques. C'est ici que l'on fait le calcul complet pour un smartphone ; les thèmes *Localisation* et *Réseaux sociaux* y renvoient.

Document 2 — Le poids du numérique en France

- **Fabriquer avant d'utiliser.** La **fabrication** représente environ **80 %** des émissions de gaz à effet de serre de nos équipements numériques : avant même sa première utilisation, un appareil a déjà produit l'essentiel de son empreinte. *Source : ADEME–Arcep, étude publiée en 2022 (données 2020).*
- **Un parc énorme.** En 2020, la France comptait environ **800 millions d'équipements numériques** (dont 70 millions de smartphones), et les usages numériques produisaient près de **300 kg de déchets par an et par personne**. *Source : ADEME–Arcep, 2022 (données 2020).*
- **Une tendance à la hausse.** Si rien ne change, les émissions du numérique augmenteraient d'environ **45 %** d'ici 2030 et seraient **multipliées par 3** d'ici 2050 (par rapport à 2020), notamment à cause de la **multiplication des objets connectés**. *Source : ADEME–Arcep, 2022 (scénario tendanciel).*
- **Aujourd'hui.** Le numérique représente **4,4 %** de l'empreinte carbone de la France (29,5 millions de tonnes de CO_2e) et **11 %** de sa consommation d'électricité. Répartition : **terminaux** (téléphones, ordinateurs, écrans, objets connectés…) **50 %**, **centres de données 46 %**, **réseaux** 4 %. Ces chiffres ne tiennent pas encore compte de l'essor de l'IA générative. *Source : ADEME–Arcep, mise à jour publiée en janvier 2025 (données 2022).*
- **Le cas du smartphone.** Un smartphone émet environ **80 kg de CO_2e** sur toute sa vie, dont **99 %** (environ 79 kg) lors de sa **fabrication** ; l'usage et la fin de vie ne pèsent qu'environ 1 % en France, où l'électricité est peu carbonée. Ce bilan suppose une durée d'utilisation de **2,5 ans**, soit environ **32 kg de CO_2e par année** d'utilisation. *Source : ADEME, outil Impact CO2 (`impactco2.fr`), fiche « smartphone » (données ADEME–Arcep 2025), consultée en 2026.*

Sources : ADEME–Arcep, *Évaluation de l'impact environnemental du numérique en France et analyse prospective*, 2022 (données 2020 : part de la fabrication, parc d'équipements, déchets, scénario 2030–2050), `arcep.fr` ; ADEME–Arcep, *Évaluation de l'impact environnemental du numérique en France*, mise à jour de janvier 2025 (données 2022 : 4,4 %, 29,5 Mt CO_2e, 11 % de l'électricité, répartition 50/46/4 %), `ademe.fr` ; ADEME, outil *Impact CO2*, fiche « smartphone », consultée en 2026 (80 kg CO_2e, 99 % à la fabrication, 2,5 ans d'utilisation), `impactco2.fr`.

▶ Activité 4 — Un objet connecté, pour quoi faire ?

À partir du document ci-dessus :

1. À l'aide du cas du smartphone, expliquer pourquoi, pour limiter l'impact d'un objet connecté, il vaut mieux le **garder longtemps** plutôt que de chercher seulement à l'éteindre.
2. Choisir un objet connecté (ampoule, prise, montre, enceinte…). Est-il vraiment **utile** ? Quel objet non connecté pourrait rendre le même service ?
3. Citer trois raisons qui obligent à remplacer un objet connecté trop tôt (penser aux **mises à jour** qui s'arrêtent, à la **réparabilité**, à la batterie…).
4. Un objet connecté reste souvent en **veille**, relié au réseau jour et nuit. Proposer deux gestes de **sobriété** pour les objets et le smartphone (durée d'utilisation, qualité vidéo, lecture automatique, extinction…).

▶ **Exercices d'application :** exercice 17 (l'empreinte d'un smartphone), exercice 18 (critiquer une réponse d'IA), exercice 19 (Python au service du thème)

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Déjà vu :** la boucle de commande s'écrit avec les `while` et `if` du chapitre *Les bases de la programmation Python*, et les données de l'objet voyagent en **paquets** sur **Internet**, grâce à une **adresse IP**.
- **Tout se rejoint :** le capteur d'image d'un appareil photo et le récepteur **GPS** sont aussi des **capteurs**, et les mesures envoyées dans le **nuage** sont souvent des **données personnelles** (chapitre *Les données structurées et leur traitement*).
- **Plus loin :** en spécialité NSI de Première, on programme soi-même capteurs, actionneurs et **IHM**.

Bilan — la carte mémoire

Les mots importants du thème sont définis dans le lexique p. 287.

Notion	À retenir
Système embarqué	ordinateur (microcontrôleur) intégré à un objet, pour **une** tâche.
Capteur	mesure une grandeur physique → un **nombre** (entrée).
Actionneur	**agit** sur le monde physique (sortie).
Boucle de commande	**acquisition** (capteur) → **décision** (programme) → **action** (actionneur) → recommencer.
IHM	moyens de **commander** la machine et d'en **recevoir** un retour.
Objet connecté	système embarqué relié au **réseau** ; ensemble = **IoT**.
Nuage	les objets envoient souvent leurs données vers des serveurs distants.
Sécurité	objet = porte d'entrée ; changer le mot de passe, mettre à jour.
Vie privée	objet = capteur ; données personnelles ; RGPD (UE), loi n° 1.565 (Monaco).
Environnement	smartphone ≈ 80 kg CO_2e, dont 99 % à la **fabrication** : garder, réparer, se passer du superflu.

Retour à la situation de départ

1. Un **système embarqué** (§ I) répète sans fin une **boucle de commande** : le **capteur** mesure la température, le programme décide (`if t < 19`), l'**actionneur**, le radiateur, agit, puis tout recommence (§ II et III).
2. Oui : un **objet connecté** est un petit ordinateur relié à Internet, donc une porte d'entrée. En 2016, Mirai a piraté des centaines de milliers d'objets restés avec leur **mot de passe par défaut** pour saturer de grands sites (§ VI). Réflexes : changer ce mot de passe, faire les mises à jour.
3. Avant : environ 99 % des 80 kg de CO_2e d'un smartphone sont émis à sa **fabrication** (§ VII). Le geste le plus efficace est donc de le garder longtemps.

Débat argumenté

Un **débat**, proposé en fin de chapitre, accompagne ce thème : « Faut-il équiper sa maison de caméras et d'enceintes connectées ? » (documents, rôles, tableau d'arguments et trace écrite, environ 50 min).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ reconnaître un **système embarqué** dans un objet du quotidien ;
- ☐ distinguer un **capteur** d'un **actionneur** et donner des exemples de chacun ;
- ☐ dérouler une **boucle de commande** « mesurer → décider → agir » (ex. thermostat) ;
- ☐ décrire le rôle d'une **interface homme-machine** (IHM) ;
- ☐ expliquer ce qu'est un **objet connecté** et l'**Internet des objets** ;
- ☐ citer les risques de **sécurité** et de **vie privée** des objets connectés, et les textes qui protègent les données (RGPD, loi monégasque n° 1.565) ;
- ☐ expliquer pourquoi la **fabrication** pèse le plus dans l'impact environnemental d'un objet connecté.

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône ⧉ en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- [sur papier] *à faire sur papier* ; [sur machine] *à faire sur machine*. Les fonctions des capteurs et actionneurs (`lire_humidite()`, `detecte()`, `attendre()`…) n'existent que sur une vraie carte : les programmes de cette feuille s'écrivent donc sur papier, sauf au dernier exercice, qui simule l'objet sur ordinateur.
- Réflexe « objet embarqué » : **capteur** (mesure) → **programme** (décision) → **actionneur** (action) → on recommence.
- Usage de l'IA, indiqué sur chaque exercice : IA aucune ; IA en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; IA l'exercice s'appuie sur une réponse d'IA. Voir la charte.

Systèmes embarqués ↑cours

★ **Exercice 1** *Embarqué ou pas ?* ▶ corrigé p. 279 • essentiel

[sur papier] Pour chaque objet, dire s'il contient un **système embarqué**, et si oui, la tâche qu'il accomplit : un lave-linge ; un marteau ; un régulateur de vitesse de voiture ; une chaise ; une montre connectée ; un feu tricolore.

▷ coup de pouce

★ **Exercice 2** *Pas tout à fait un ordinateur* ▶ corrigé p. 279

[sur papier] Citer **deux** différences entre un système embarqué (dans un objet) et un ordinateur de bureau.

▷ coup de pouce

Capteurs et actionneurs ↑cours

★ **Exercice 3** *Qui mesure, qui agit ?* ▶ corrigé p. 280 • essentiel

[sur papier] Classer en deux colonnes (**capteur** / **actionneur**) : thermomètre, radiateur, détecteur de mouvement, haut-parleur, microphone, moteur de volet, capteur de luminosité, LED.

▷ coup de pouce

★★ **Exercice 4** *Dans un objet du quotidien* ▶ corrigé p. 280

[sur papier] Pour un **portail automatique** de maison :

1. Citer un **capteur** et un **actionneur** de ce système.
2. Que se passe-t-il, dans l'ordre, entre le moment où une voiture arrive et celui où le portail s'ouvre ? (mesure → décision → action)

▷ coup de pouce

La boucle de commande ↑cours

★★ **Exercice 5** *Que fait ce programme ?* ▶corrigé p. 280 • essentiel

[sur papier] On lit le programme d'un objet embarqué :

```
while True:
    t = lire_temperature()
    if t > 25:
        allumer_ventilateur()
    else:
        eteindre_ventilateur()
    attendre(30)
```

1. Quel est le **capteur** ? Quel est l'**actionneur** ?
2. Décrire en une phrase ce que fait cet objet.
3. Que se passe-t-il si la température mesurée est de 22 °C ? de 28 °C ?

▷coup de pouce

★★ **Exercice 6** *Dérouler une boucle* ▶corrigé p. 280 • essentiel

[sur papier] Une veilleuse exécute le programme suivant :

```
while True:
    lum = lire_luminosite()
    if lum < 40:
        allumer_lampe()
    else:
        eteindre_lampe()
    attendre(60)
```

Le capteur renvoie successivement 70, 30, 10 puis 50. Recopier et compléter le tableau :

Tour	`lum`	`lum < 40` ?	État de la lampe
1	70		
2	30		
3	10		
4	50		

Combien de temps s'est-il écoulé entre la première et la dernière mesure ? ▷coup de pouce

★★ **Exercice 7** *Compléter un arrosage automatique* ▶corrigé p. 280 • essentiel

[sur papier] Un système arrose une plante dès que la terre est trop sèche : l'humidité `h` passe sous `40`. Recopier et compléter :

```
while True:
    h = lire_humidite()
    if ... :                    # (a) condition
        ...                     # (b) ouvrir la pompe
    else:
        fermer_pompe()
    attendre(3600)              # on re-teste chaque heure
```

▷coup de pouce

★★★ **Exercice 8** *Défi — une alarme* ▶corrigé p. 281 ≈10 min

[sur papier] Écrire une boucle de commande pour une **alarme** : tant qu'elle fonctionne, si le détecteur de mouvement renvoie `True` (`detecte()`), elle déclenche la sirène (`sonner()`) et allume une LED (`allumer_led()`) ; sinon elle éteint tout. On testera dix fois par seconde. ▷coup de pouce

▷début de solution

★★★ **Exercice 9** *Le thermostat à deux seuils* ▶ corrigé p. 281 ≈ 15 min

[sur papier] Le thermostat du cours allume le chauffage sous 19 °C et l'éteint sinon : autour de 19 °C, le radiateur s'allume et s'éteint sans arrêt, ce qui l'use. On veut un thermostat plus malin : il **allume** le chauffage si la température passe sous 19 °C, l'**éteint** si elle dépasse 21 °C, et **ne change rien** entre les deux.

1. Écrire cette boucle de commande (une mesure par minute).
2. Le chauffage est éteint et la pièce est à 20 °C. Que fait le programme ? Et si le chauffage était allumé ?

▷ coup de pouce ▷ début de solution

★★★ **Exercice 10** *Le compteur de passages* ▶ corrigé p. 281 ≈ 20 min

[sur papier] À l'entrée d'une petite salle, un détecteur `detecte()` renvoie `True` quand quelqu'un passe. On veut **compter** les entrées, afficher le total sur un écran (`afficher(n)`) et déclencher un signal sonore (`sonner()`) dès que plus de 20 personnes sont entrées.

1. Écrire la boucle de commande (dix tests par seconde).
2. Une personne reste une demi-seconde devant le détecteur. Quel problème cela pose-t-il ? Proposer une solution simple.

▷ coup de pouce ▷ début de solution

L'interface homme-machine (IHM) ↑ cours

★ **Exercice 11** *Entrées et sorties* ▶ corrigé p. 282

[sur papier] Pour un **four à micro-ondes**, citer deux éléments d'**entrée** (l'humain vers la machine) et deux éléments de **sortie** (la machine vers l'humain). ▷ coup de pouce

★★ **Exercice 12** *Bonne ou mauvaise IHM ?* ▶ corrigé p. 282

[sur papier] On appuie sur le bouton d'un objet, mais *rien* ne semble se passer (ni voyant, ni son, ni affichage). Pourquoi est-ce une **mauvaise** IHM ? Que faudrait-il ajouter ? ▷ coup de pouce

★★ **Exercice 13** *Concevoir une IHM* ▶ corrigé p. 282

[sur papier] On conçoit un **pot de fleurs connecté** qui mesure l'humidité de la terre et peut arroser tout seul.

1. Proposer une **entrée** (pour que l'utilisateur commande le pot) et deux **sorties** (pour qu'il soit informé). Justifier chaque choix en une phrase.
2. L'utilisateur lance un arrosage à la main. Quel **retour** le pot doit-il lui donner ?

▷ coup de pouce

Objets connectés, sécurité et vie privée ↑ cours

★ **Exercice 14** *Objet connecté* ▶ corrigé p. 282 • essentiel

[sur papier]

1. Qu'est-ce qui distingue un **objet connecté** d'un simple objet embarqué ?
2. Citer deux objets connectés et, pour chacun, une donnée qu'il transmet.

▷ coup de pouce

★★ **Exercice 15** *Le danger des mots de passe par défaut* ▶ corrigé p. 282

[sur papier] En 2016, le logiciel **Mirai** a détourné des centaines de milliers de caméras connectées pour saturer de grands sites.

1. Quelle négligence des propriétaires a rendu l'attaque possible ?
2. En quoi un objet connecté mal sécurisé met-il en danger **plus** que son seul propriétaire ?
3. Citer deux gestes simples pour sécuriser un objet connecté.

▷ coup de pouce

★ **Exercice 16** *Vie privée* ▶ corrigé p. 282

[sur papier] Une enceinte connectée « écoute » pour réagir à la voix de ses utilisateurs. Expliquer, en deux phrases, en quoi cela pose une question de **vie privée**, et quel texte protège les données personnelles en France (et dans l'UE) ; et à Monaco ? ▷ coup de pouce

Impact environnemental ↑ cours

★★ **Exercice 17** *L'empreinte d'un smartphone* ▶ corrigé p. 283

[sur papier] D'après l'outil *Impact CO2* de l'ADEME (données ADEME–Arcep 2025, consulté en 2026 ; voir le document du cours), un smartphone émet environ `80` kg de CO_2e sur toute sa vie, dont `99` % lors de sa **fabrication** ; le calcul suppose qu'il est gardé `2,5` ans. On néglige ici l'usage et la fin de vie.

1. Calculer les émissions **par année d'utilisation** si le téléphone est gardé 2,5 ans.
2. Même question s'il est gardé **5 ans**. Conclure.
3. Pourquoi éteindre son téléphone la nuit a-t-il un effet bien plus faible que de le garder plus longtemps ?
4. Citer deux raisons qui poussent à changer de téléphone ou d'objet connecté trop tôt, et un moyen d'y remédier.

▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 18** *Le volet automatique d'un assistant* ▶ corrigé p. 283

[sur papier] Un élève a demandé à un assistant d'IA : « Écris la boucle de commande d'un volet roulant qui se ferme quand la nuit tombe et s'ouvre quand il fait jour. Le capteur `lire_luminosite()` renvoie un nombre de `0` (nuit noire) à `100` (plein soleil). » Voici la réponse obtenue :

```
while True:
    lum = lire_luminosite()      # ENTREE : le capteur -> un nombre
    if lum < 20:                 # il fait nuit
        ouvrir_volet()
    else:
        fermer_volet()
    attendre(60)                 # on re-teste chaque minute
```

1. La réponse est-elle correcte ? Dérouler la boucle pour une mesure de nuit (`lum` = 5) puis de jour (`lum` = 80), et dire ce que fait le volet dans chaque cas.
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite *avant* de faire confiance à cette réponse, aurait suffi à détecter l'erreur ?

Python au service du thème ↑ cours

★★ **Exercice 19** *Python au service du thème — simuler un thermostat* ▶ corrigé p. 283

[sur machine] En Python (Spyder ou Basthon). Sans carte électronique, on peut quand même faire tourner un objet embarqué sur ordinateur en le **simulant** : une variable remplace le capteur, un affichage remplace l'actionneur, et une boucle `for` remplace `while True` (on ne simule que quelques minutes).

1. Écrire une fonction `commande(t)` qui renvoie la chaîne `"allumer"` si la température `t` est strictement inférieure à 19 °C, et `"eteindre"` sinon (c'est la décision du thermostat du cours). Tester `commande(17)` et `commande(19)`.

2. On simule la pièce : chaque minute, le radiateur allumé la réchauffe de 2 °C ; éteint, elle se refroidit de 1 °C. Compléter, puis exécuter ce programme, qui part de 15 °C et simule 8 minutes :

```
t = 15
for minute in range(8):
    ordre = commande(t)
    print(f"minute {minute} : {t} degres -> {ordre}")
    if ordre == "allumer":
        t = ...
    else:
        t = ...
```

3. À partir de la minute 2, entre quelles températures la pièce oscille-t-elle ? Que fait le radiateur pendant ce temps ? Pourquoi est-ce gênant, et quel exercice de cette feuille propose une solution ?

▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relisez la définition d'un système embarqué : y a-t-il, à l'intérieur de l'objet, un petit ordinateur qui exécute un programme pour une tâche précise ? ↑énoncé

Ex. 2 – piste Relisez l'encadré « Différence avec un ordinateur ordinaire » : combien de tâches, quels moyens de dialoguer avec l'utilisateur, en combien de temps doit-il réagir ? ↑énoncé

Ex. 3 – piste Pour chaque élément, demandez-vous s'il *mesure* quelque chose (entrée) ou s'il *agit* sur le monde (sortie). ↑énoncé

Ex. 4 – piste Que doit « sentir » le portail pour savoir qu'une voiture arrive ? Qu'est-ce qui le fait bouger ? Placez ensuite le programme entre les deux. ↑énoncé

Ex. 5 – piste Repérez la fonction qui *lit* une valeur (le capteur) et celles qui *agissent* (l'actionneur). Pour 22 °C, la condition `t > 25` est-elle vraie ou fausse ? ↑énoncé

Ex. 6 – piste À chaque tour, on lit une nouvelle valeur, on teste la condition (vraie ou fausse), puis on exécute *une seule* des deux branches. Le programme attend 60 s à la fin de chaque tour. ↑énoncé

Ex. 7 – piste La condition (a) traduit « l'humidité passe sous 40 » avec `<`. L'action (b) fait le contraire de celle écrite sous le `else`. ↑énoncé

Ex. 8 – piste Même structure que l'arrosage : `while True`, une lecture du capteur dans un `if`, un `else`, puis une attente. Pour « éteindre tout », on dispose de `eteindre_sirene()` et `eteindre_led()`. Dix fois par seconde : combien de secondes attendre entre deux tests ? ↑énoncé

Ex. 8 – début de solution

```
while True:
    if detecte():
        sonner()
        allumer_led()
```

(il reste le `else` et l'attente.) ↑énoncé

Ex. 9 – piste Il y a trois cas : sous 19, au-dessus de 21, et entre les deux. Pour « ne rien changer », il suffit de ne commander aucun actionneur : pas besoin de `else`. ↑énoncé

Ex. 9 – début de solution

```
while True:
    t = lire_temperature()
    if t < 19:
        allumer_chauffage()
    elif ...
```

↑énoncé

Ex. 10 – piste Il faut une variable qui garde le nombre d'entrées d'un tour de boucle à l'autre : où la créer, avant ou dans la boucle ? ↑énoncé

Ex. 10 – début de solution
```
nb = 0
while True:
   if detecte():
      nb = nb + 1
```
(il reste l'affichage, le test des 20 personnes et l'attente.) ↑énoncé

Ex. 11 – piste Imaginez-vous devant le four : sur quoi appuyez-vous pour le régler ? Comment vous montre-t-il le temps restant, et comment vous prévient-il que c'est prêt ? ↑énoncé

Ex. 12 – piste Relisez la règle « Une bonne IHM » du cours : que doit-il se passer juste après un appui sur un bouton ? ↑énoncé

Ex. 13 – piste Entrées : par quoi l'humain agit-il sur un objet (bouton, écran tactile, application, voix…) ? Sorties : comment l'objet informe-t-il (voyant, son, écran, notification) ? Pensez à ce que l'utilisateur a besoin de savoir : la terre est-elle sèche ? l'arrosage a-t-il commencé ? ↑énoncé

Ex. 14 – piste Relisez la définition d'un objet connecté : qu'a-t-il de plus qu'un simple système embarqué ? ↑énoncé

Ex. 15 – piste Que font la plupart des gens du mot de passe écrit dans la notice ? Et des centaines de milliers d'objets piratés, que peuvent-ils faire *ensemble* ? ↑énoncé

Ex. 16 – piste Relisez la règle « Deux enjeux à retenir » et l'encadré « Et à Monaco ? » du cours. ↑

Ex. 17 – piste Divisez les 80 kg par le nombre d'années. Où sont émis 99 % des gaz : pendant l'usage ou à la fabrication ? ↑énoncé

Ex. 18 q. 2 – piste Après avoir déroulé la boucle, comparez avec la demande de l'élève : de nuit, que doit faire le volet ? ↑énoncé

Ex. 19 – piste Question 1 : un `if` pour le cas « trop froid », un `else` pour l'autre ; chaque branche renvoie une chaîne avec `return`. Question 2 : « se réchauffer de 2 °C », c'est augmenter la variable `t` de 2, comme dans `a = a + 1`. ↑énoncé

DÉBAT

DÉBAT Faut-il équiper sa maison de caméras et d'enceintes connectées ? ▶ corrigé p. 284

plus de sécurité, ou plus de surveillance ?

Durée : 50 min

Consignes

- En classe entière. Les rôles sont tirés au sort : vous défendrez peut-être une position qui n'est pas la vôtre, c'est un excellent entraînement.
- Chaque argument s'appuie sur un **document** ci-dessous ou sur le **cours**. Préparation sans IA ni recherche en ligne.
- Avant de commencer, notez sur votre cahier votre position de départ : oui, non ou je ne sais pas.

Rappel du cours. Un **objet connecté** possède des **capteurs** (caméra, micro, thermomètre…) et des **actionneurs**, et **échange des données** par un réseau, souvent vers le **nuage**. Il présente deux risques : la **sécurité** (c'est une porte d'entrée sur le réseau, souvenez-vous de Mirai) et la **vie privée** (il observe, écoute, suit). Caméra, sonnette vidéo, enceinte, thermostat : la maison connectée vous protège-t-elle, ou vous surveille-t-elle ?

Les deux camps

Groupe POUR — oui : les services rendus (sécurité, confort, économies) l'emportent sur les risques.
Groupe CONTRE — non : les risques pour la vie privée et la sécurité sont trop grands.

Les documents

Document 1 — Ce que promet la maison connectée

Un **thermostat connecté** règle le chauffage pièce par pièce, à distance ; or, selon l'ADEME, baisser le chauffage de 1 °C économise environ **7 %** d'énergie. Une **caméra** ou une **sonnette vidéo** montre sur le téléphone qui sonne et alerte en cas d'intrusion. Une **enceinte** commandée à la voix et des détecteurs de chute aident les personnes âgées ou en situation de handicap à vivre chez elles en sécurité.

À retenir : la maison connectée promet économies d'énergie, sécurité et autonomie.

Source : ADEME, conseils pour réduire sa consommation de chauffage.

Document 2 — Qui regarde les vidéos ?

En mai 2023, l'autorité américaine de protection des consommateurs (FTC) a obtenu de Ring, fabricant de caméras et de sonnettes connectées appartenant à Amazon, le paiement de **5,8 millions de dollars**. Elle lui reprochait d'avoir laissé des employés et des prestataires accéder aux **vidéos privées** de clients, et d'avoir trop peu protégé les comptes : des pirates avaient pris le contrôle de certaines caméras.

À retenir : les images d'une caméra connectée peuvent être vues par d'autres que soi.

Source : Federal Trade Commission *(États-Unis), accord avec Ring, mai 2023.*

Document 3 — L'enceinte écoute-t-elle ?

Une enceinte connectée écoute en permanence, mais sur l'appareil lui-même, un seul mot : son **mot d'activation**. Elle n'est censée envoyer un enregistrement au nuage qu'après l'avoir entendu. En 2019, la presse a révélé que des salariés ou des sous-traitants d'Amazon, de Google et d'Apple écoutaient une partie de ces enregistrements ; certains, déclenchés par erreur, contenaient des conversations privées. Depuis, des réglages permettent de refuser cette écoute.

À retenir : un micro chez soi suppose de faire confiance au fabricant.

Source : presse internationale, avril à août 2019.

Document 4 — Filmer chez soi : ce que dit la loi

Chez soi, on peut installer une caméra pour surveiller son logement, mais, rappelle la CNIL, sans filmer **ni la voie publique ni la propriété des voisins**, et en informant toute personne employée à domicile. À Monaco, les images de personnes sont protégées par la **loi n° 1.565** (APDP). Un règlement européen imposera fin 2027 aux objets connectés une configuration sûre par défaut et des mises à jour de sécurité.

À retenir : filmer chez soi est permis, filmer les autres est encadré.

*Source : CNIL, fiches pratiques sur la vidéosurveillance chez soi ; loi monégasque n° 1.565 du 3 décembre 2024 ; règlement (UE) 2024/2847 (*Cyber Resilience Act*).*

Rôles, déroulé minuté, règles de prise de parole et trace écrite : voir la page « Méthode : débattre » p. 7.

Tableau d'arguments

Recopier ce tableau sur le cahier et le compléter pendant le débat.

Argument POUR (oui)	Doc.	Argument CONTRE (non)	Doc.
…		…	

Trace écrite finale (individuelle)

Sur le cahier, notez votre position après le débat (oui, non, je ne sais toujours pas) et si vous avez changé d'avis. Puis, en trois à cinq lignes, justifiez-la avec vos **deux meilleurs arguments**, dont **un argument de l'autre camp** que vous trouvez solide (avec le document qui l'appuie).

FICHE DE RÉVISION

L'essentiel

- Les **systèmes embarqués**, ordinateurs cachés dans nos objets, *mesurent* (**capteurs**, entrées), *décident* (**programme**), *agissent* (**actionneurs**, sorties)… et recommencent sans fin.
- Reliés à un réseau, ils deviennent des **objets connectés** (ensemble : l'**Internet des objets**, IoT). Enjeux : **sécurité**, **vie privée**, **environnement**.

Notions clés

Système embarqué	ordinateur (**microcontrôleur**) intégré à un objet pour **une** tâche ; réagit en **temps réel**.
Capteur / actionneur	capteur : **mesure** → un **nombre** (thermomètre, micro) ; actionneur : **agit** (radiateur, LED, moteur).
IHM	**commander** l'objet (bouton, voix) et en **recevoir** un retour (écran, bip) qui confirme l'action.
Objet connecté	**échange des données** par un réseau (Wi-Fi, Bluetooth, 4G/5G), souvent vers des serveurs (*cloud*).

Méthode : lire ou compléter une boucle de commande

```
while True:
    t = lire_temperature()
    if t < 19:
        allumer_chauffage()
    else:
        eteindre_chauffage()
    attendre(60)
```

- **Capteur** : `lire_...()` (le thermomètre) ; **actionneur** : `allumer_...()`, `eteindre_...()` (le radiateur).
- **Dérouler** : si `t` vaut 17, `17 < 19` est vrai, le chauffage s'allume ; on attend 60 s puis on recommence.
- `while True` : boucle sans fin **voulue**.

Repères et enjeux

Fin **2024** : près de **20 milliards** d'objets connectés. **Mirai (2016)** : des centaines de milliers d'objets au **mot de passe par défaut** piratés ⇒ changer ce mot de passe, faire les mises à jour. Voix, position, rythme cardiaque sont des **données personnelles** : **RGPD** (UE), **loi n° 1.565** (Monaco, contrôle de l'APDP). **Environnement** : un smartphone ≈ 80 kg CO_2e, dont 99 % à la **fabrication** (pas à l'usage) : le garder, le réparer.

Erreurs à éviter

Inverser capteur (le micro **écoute**) et actionneur (le haut-parleur **émet**) ; croire qu'un objet embarqué est forcément connecté.

Savoir-faire à maîtriser — je sais…

- ☐ reconnaître un **système embarqué** dans un objet du quotidien ;
- ☐ distinguer un **capteur** d'un **actionneur** et donner des exemples de chacun ;
- ☐ dérouler une **boucle de commande** « mesurer → décider → agir » (ex. thermostat) ;
- ☐ décrire le rôle d'une **interface homme-machine** (IHM) ;
- ☐ expliquer ce qu'est un **objet connecté** et l'**Internet des objets** ;
- ☐ citer les risques de **sécurité** et de **vie privée** des objets connectés, et les textes qui protègent les données (RGPD, loi monégasque n° 1.565) ;
- ☐ expliquer pourquoi la **fabrication** pèse le plus dans l'impact environnemental d'un objet connecté.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 285

1. Classer en capteurs et actionneurs : pompe, thermomètre, LED, microphone.
2. Écrire la condition d'une serre qui ouvre sa fenêtre quand la température `t` dépasse 30.
3. Citer une entrée et une sortie de l'IHM d'un ascenseur.
4. Dire ce qu'un objet connecté a de plus qu'un objet embarqué ; citer deux gestes de sécurité.

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 285

1. Nommer la méthode qui situe un point grâce à ses distances aux satellites (ch. 7).
2. Dire ce que signifie stocker ses données dans le nuage (ch. 5).

II

DEUXIÈME PARTIE

Corrigés

CORRIGÉS DU CHAPITRE 1

1 Les bases de la programmation Python

CORRIGÉS DES EXERCICES

> **Remarque** Plusieurs écritures sont souvent possibles ; on en donne une simple. Les exercices marqués « sur papier » indiquent le résultat attendu. Rappel utile : `input` renvoie toujours du texte, on convertit avec `int(...)` pour calculer.

Premiers pas : afficher

Exercice 1 *Bonjour* ↑énoncé p. 35

▶ Solution.

```
1 print("Bonjour, je m'appelle Marie")
```

Exercice 2 *Le menu de la cantine* ↑énoncé p. 35

▶ Solution. Une instruction `print` par ligne à afficher.

```
1 print("Entree : salade nicoise")
2 print("Plat : lasagnes")
3 print("Dessert : tarte aux pommes")
```

Variables

Exercice 3 *Deux variables* ↑énoncé p. 35

▶ Solution. Le texte va entre guillemets, le nombre non.

```
1 sport = "natation"
2 heures = 4
3 print(sport)
4 print(heures)
```

Exercice 4 *Valeur finale* ↑énoncé p. 35

▶ Solution. *Sur papier.* `a` = **10** et `b` = **6**. La valeur de `b` a été calculée *une fois pour toutes* quand `a` valait 5 : modifier `a` ensuite ne change pas `b`.

▷ **Erreur fréquente.** Répondre que `b` vaut 11. La ligne `b = a + 1` n'est pas une formule qui se met à jour toute seule : c'est un calcul fait une seule fois, avec la valeur que `a` avait à ce moment-là.

Exercice 5 *Des noms parlants* ↑énoncé p. 35

▶ Solution. *Sur papier.*

1. Par exemple `prix_place`, `nb_places` et `total`.
2. `prix place` contient une espace ; `3places` commence par un chiffre ; `total€` contient un symbole (`€`) interdit dans un nom.

Exercice 6 *L'échange* ↑énoncé p. 36

▶ Solution. On garde l'une des valeurs de côté dans une variable `temporaire`, sinon on l'écrase.

```
a = 1
b = 2
temporaire = a
a = b
b = temporaire
```

▷ **Erreur fréquente.** Écrire `a = b` puis `b = a` : après la première ligne, `a` vaut déjà 2, et l'ancienne valeur (1) est perdue. Les deux variables finissent à 2.

Calculs

Exercice 7 *Somme et produit* ↑énoncé p. 36

▶ Solution.

```
a = 6
b = 4
print(a + b)    # 10
print(a * b)    # 24
```

Exercice 8 *Le trajet* ↑énoncé p. 36

▶ Solution. Distance = vitesse × durée. En Python, la virgule décimale s'écrit avec un point.

```
vitesse = 90
duree = 2.5
print(vitesse * duree)    # 225.0
```

Exercice 9 *La tirelire* ↑énoncé p. 36

▶ Solution. On met à jour `tirelire` à partir de sa propre valeur, étape par étape.

```
tirelire = 20
tirelire = tirelire * 2    # 40
tirelire = tirelire - 15   # 25
tirelire = tirelire + 8    # 33
print(tirelire)
```

Exercice 10 *Priorités de calcul* ↑énoncé p. 36

▶ Solution. *Sur papier.*

- `2 + 3 * 4` = 14 (`int`) : la multiplication passe avant ;
- `(2 + 3) * 4` = 20 (`int`) : les parenthèses d'abord ;
- `10 - 6 / 2` = 7.0 (`float`) : `6 / 2` vaut `3.0` ;
- `2 * 3 ** 2` = 18 (`int`) : la puissance passe avant la multiplication ;
- `20 // 3 % 2` = 0 (`int`) : même priorité, on calcule de gauche à droite, `20 // 3` = 6 puis `6 % 2` = 0.

Exercice 11 *La note d'une application* ↑énoncé p. 36

▶ Solution. On compte toutes les étoiles données (chaque nombre d'avis multiplié par sa note), puis on divise par le *nombre total d'avis* (20), et non par 3 (le nombre de notes différentes).

```
avis_5 = 12
avis_4 = 5
avis_1 = 3
etoiles = avis_5 * 5 + avis_4 * 4 + avis_1 * 1
moyenne = etoiles / (avis_5 + avis_4 + avis_1)
print(moyenne)    # 4.15
```

▷ **Erreur fréquente.** Oublier les parenthèses : `etoiles / avis_5 + avis_4 + avis_1` divise seulement par 12 *puis* ajoute 5 et 3 (la division passe avant l'addition), ce qui donne environ 14,9 : une note impossible sur 5 étoiles, qui aurait dû alerter.

Exercice 12 *Le forfait de données* ↑énoncé p. 36

▶ Solution. On calcule d'abord la quantité utilisée (25 % de 80 Go, soit 20 Go), puis on la retire du forfait. (Autre écriture : il reste 75 % du forfait, `forfait * 75 / 100`.)

```
forfait = 80
utilise = forfait * 25 / 100
print(forfait - utilise)   # 60.0
```

Il reste 60 Go.

Exercice 13 *Quel jour serons-nous ?* ↑énoncé p. 36

▶ Solution. `//` compte les semaines complètes ; `%` donne le reste, ce qui « fait le tour » de la semaine.

```
aujourd_hui = 2                    # mercredi
print(100 // 7)                    # 14 semaines completes
print((aujourd_hui + 100) % 7)     # 4, c'est-a-dire vendredi
```

Exercice 14 *Le rendu de monnaie* ↑énoncé p. 37

▶ Solution. On traite les coupures de la plus grande à la plus petite : `//` donne le nombre, `%` garde le reste à traiter ensuite.

```
montant = 27
billets10 = montant // 10     # 2
reste = montant % 10          # 7
pieces2 = reste // 2          # 3
pieces1 = reste % 2           # 1
print(f"{billets10} billet(s) de 10")
print(f"{pieces2} piece(s) de 2")
print(f"{pieces1} piece(s) de 1")
```

Vérification : $2 \times 10 + 3 \times 2 + 1 \times 1 = 27$.

Chaînes de caractères et dialogue

Exercice 15 *Texte ou calcul ?* ↑énoncé p. 37

▶ Solution. Entre guillemets, Python affiche le texte tel quel ; sans guillemets, il calcule. Le `+` entre deux textes les colle (attention à l'espace après *Bonjour*).

```
print("12 + 5")
print(12 + 5)
print("Bonjour " + "Monaco")
```

Exercice 16 *Nom de code* ↑énoncé p. 37

▶ Solution.

```
prenom = input("Ton prenom ? ")
ville = input("Ta ville ? ")
print(f"Agent {prenom}, votre mission vous attend a {ville}.")
```

Exercice 17 *Le type d'un résultat* ↑énoncé p. 37

▶ Solution. *Sur papier.*

- `7 + 2` → `9` (`int`) ; `7 / 7` → `1.0` (`float` : la division `/` donne toujours un flottant) ;
- `"7" + "2"` → `"72"` (`str` : on colle deux textes) ; `3 * 1.0` → `3.0` (`float`) ;
- `int("5") + 1` → `6` (`int`) ; `str(4) + "0"` → `"40"` (`str`).

Exercice 18 *Le temps d'écran* ↑énoncé p. 37

▶ **Solution.** Pour 3 h par jour : 1095 h, soit 45 jours entiers.

```
par_jour = int(input("Heures d'ecran par jour ? "))
par_an = par_jour * 365
print(f"{par_an} heures par an")
print(f"soit {par_an // 24} jours entiers")
```

▷ **Erreur fréquente.** Oublier `int(...)` autour de `input`. La réponse tapée reste du texte : `"3" * 365` ne multiplie pas, il *répète* le texte 365 fois, puis `par_an // 24` provoque une erreur (`TypeError`).

Exercice 19 *Le temps de téléchargement* ↑énoncé p. 37

▶ **Solution.** Chaque seconde, on reçoit `debit` mégaoctets : la durée est donc la taille *divisée* par le débit. `float` convertit le texte saisi en nombre à virgule (`int` planterait sur `2.5`).

```
taille = float(input("Taille du fichier (Mo) : "))
debit = float(input("Debit (Mo/s) : "))
print(f"Un fichier de {taille} Mo se telecharge en {taille / debit} secondes.")
```

Avec 45 et 2.5, le programme affiche : `Un fichier de 45.0 Mo se telecharge en 18.0 secondes.`

Exercice 20 *L'écho* ↑énoncé p. 37

▶ **Solution.** Avec les chaînes, `*` répète.

```
mot = input("Un mot : ")
n = int(input("Combien de fois ? "))
print(mot * n)
```

Exercice 21 *Le ticket de caisse* ↑énoncé p. 37

▶ **Solution.** On convertit chaque saisie dans le bon type, on calcule le total, puis on affiche une ligne de tirets avant et après (`"-" * 20` répète le tiret 20 fois).

```
article = input("Article : ")
prix = float(input("Prix unitaire : "))
quantite = int(input("Quantite : "))
print("-" * 20)
print(f"{quantite} x {article}")
print(f"Prix unitaire : {prix}")
print(f"TOTAL : {prix * quantite} euros")
print("-" * 20)
```

Les fonctions

Exercice 22 *Les codes PIN* ↑énoncé p. 38

▶ **Solution.** Avec `n` chiffres, on multiplie `n` fois par 10 : c'est 10^n, qui s'écrit `10 ** n` en Python.

```
def nb_codes(n):
    return 10 ** n

print(nb_codes(4))   # 10000
```

Seulement 10 000 codes à 4 chiffres : c'est pour cela qu'un téléphone se bloque après quelques essais ratés.

▷ **Erreur fréquente.** Écrire `print(10 ** n)` au lieu de `return 10 ** n` dans la fonction. L'écran montre bien 10000, mais `nb_codes(4)` ne *renvoie* rien (`None`) : impossible de réutiliser le résultat dans un calcul.

Exercice 23 *L'aire du triangle* ↑énoncé p. 38

▶ **Solution.**

```
def aire_triangle(base, hauteur):
    return base * hauteur / 2

print(aire_triangle(6, 4))    # 12.0
```

Exercice 24 *Une note sur 20* ↑énoncé p. 38

▶ **Solution.** C'est une proportionnalité : $\dfrac{\text{points}}{\text{total}} \times 20$.

```
def note_sur_20(points, total):
    return points * 20 / total

print(note_sur_20(12, 15))    # 16.0
```

Exercice 25 *Retrouver le poids d'origine* ↑énoncé p. 38

▶ **Solution.** Le poids envoyé vaut 0,75 fois le poids d'origine, donc on **divise** par 0,75. (Ajouter 25 % à 3 donnerait 3,75, et $3{,}75 \times 0{,}75 = 2{,}8125 \neq 3$.)

```
def poids_origine(poids_envoye):
    return poids_envoye / 0.75

print(poids_origine(3))    # 4.0
```

Exercice 26 *Le poids d'une photo* ↑énoncé p. 38

▶ **Solution.** Nombre de pixels $\times 3$ octets.

```
def poids_photo(largeur, hauteur):
    return largeur * hauteur * 3

print(poids_photo(4000, 3000))    # 36000000
```

Soit 36 Mo : c'est pour cela que les appareils *compressent* les photos (JPEG).

Exercice 27 *Tout en secondes* ↑énoncé p. 38

▶ **Solution.** Une heure vaut 3600 s et une minute 60 s.

```
def en_secondes(h, m, s):
    return h * 3600 + m * 60 + s

print(en_secondes(1, 30, 15))    # 5415
```

Exercice 28 *Deviner l'affichage (fonctions)* ↑énoncé p. 38

▶ **Solution.** On calcule de l'intérieur vers l'extérieur.

- `f(3)` $= 2 \times 3 + 1 = 7$: affiche `7` ;
- `f(1)` $= 3$, puis `g(3)` $= 9$: affiche `9` ;
- `g(2)` $= 4$, puis `f(4)` $= 9$, et $9 + 1 = 10$: affiche `10`.

Exercice 29 *Le coût d'un trajet* ↑énoncé p. 39

▶ **Solution.** `cout_trajet` réutilise `litres` au lieu de réécrire le calcul.

```
def litres(distance, conso):
    return distance * conso / 100

def cout_trajet(distance, conso, prix_litre):
    return litres(distance, conso) * prix_litre

print(cout_trajet(800, 6, 2))    # 96.0
```

Le trajet consomme 48 L et coûte 96 €.

Les conditions

Exercice 30 *Le code secret* ↑énoncé p. 39

▶ Solution. Un `if` pour le bon code, un `else` pour tous les autres.

```
def verifier(code):
    if code == 2468:
        return "Acces autorise"
    else:
        return "Acces refuse"
```

▷ **Erreur fréquente.** Écrire `if code = 2468:` avec un seul signe égal : Python refuse le programme (`SyntaxError`). Le `=` simple *range* une valeur ; pour *comparer*, il faut `==`.

Exercice 31 *Pair ?* ↑énoncé p. 39

▶ Solution. La comparaison `n % 2 == 0` est déjà un booléen : on le renvoie directement.

```
def est_pair(n):
    return n % 2 == 0
```

Exercice 32 *Le signe* ↑énoncé p. 39

▶ Solution. Trois cas, donc `if` / `elif` / `else`.

```
def signe(x):
    if x < 0:
        return "negatif"
    elif x == 0:
        return "nul"
    else:
        return "positif"
```

Exercice 33 *Le découpage en paquets* ↑énoncé p. 39

▶ Solution. `taille // 1500` donne le nombre de paquets *pleins* ; s'il reste des octets (`taille % 1500` différent de 0), il faut un paquet de plus pour eux.

```
taille = int(input("Taille du fichier (octets) : "))
paquets = taille // 1500
if taille % 1500 != 0:
    paquets = paquets + 1
print(f"Il faut {paquets} paquets.")
```

Pour 3000 octets : 2 paquets (reste 0) ; pour 3001 octets : 2 paquets pleins et un dernier paquet d'un seul octet, soit 3 paquets.

Exercice 34 *Valeur absolue maison* ↑énoncé p. 39

▶ Solution. Si le nombre est négatif, on renvoie son opposé.

```
def valeur_absolue(x):
    if x < 0:
        return -x
    else:
        return x
```

Exercice 35 *L'icône de batterie* ↑énoncé p. 39

▶ Solution. On teste les seuils du plus bas au plus haut : le premier qui est vrai l'emporte.

```
def icone(niveau):
    if niveau < 10:
        return "critique"
    elif niveau < 30:
        return "faible"
    elif niveau < 70:
        return "moyenne"
    else:
```

```
        return "pleine"

print(icone(5))      # critique
print(icone(50))     # moyenne
print(icone(100))    # pleine
```

Exercice 36 *La meilleure antenne* ↑énoncé p. 39

▶ **Solution.** On garde le plus grand niveau vu jusqu'ici dans une variable `grand`.

```
def meilleur_signal(a, b, c):
    grand = a
    if b > grand:
        grand = b
    if c > grand:
        grand = c
    return grand

print(meilleur_signal(40, 75, 62))   # 75
```

Exercice 37 *Année bissextile* ↑énoncé p. 40

▶ **Solution.** On traite les cas du plus particulier au plus général : d'abord 400, puis 100, puis 4 (le premier test vrai l'emporte).

```
def bissextile(annee):
    if annee % 400 == 0:
        return True
    elif annee % 100 == 0:
        return False
    elif annee % 4 == 0:
        return True
    else:
        return False

print(bissextile(2024))   # True
print(bissextile(1900))   # False
print(bissextile(2000))   # True
```

Autre écriture, en une seule condition : divisible par 4 **et** pas par 100, **ou** divisible par 400.

```
def bissextile(annee):
    return (annee % 4 == 0 and annee % 100 != 0) or (annee % 400 == 0)
```

Exercice 38 *Une couleur valide ?* ↑énoncé p. 40

▶ **Solution.** Chaque nombre doit être entre 0 et 255 : deux comparaisons reliées par `and` pour chacun, puis les trois résultats reliés par `and`.

```
def couleur_valide(rouge, vert, bleu):
    rouge_ok = rouge >= 0 and rouge <= 255
    vert_ok = vert >= 0 and vert <= 255
    bleu_ok = bleu >= 0 and bleu <= 255
    return rouge_ok and vert_ok and bleu_ok

print(couleur_valide(255, 128, 0))   # True
print(couleur_valide(300, 0, 0))     # False
```

(Avec l'écriture raccourcie vue dans le cours : `0 <= rouge <= 255`.) On retrouvera ces nombres de 0 à 255 au chapitre sur la photographie numérique.

▷ **Erreur fréquente.** Relier les conditions par `or` : `couleur_valide(300, 0, 0)` renverrait alors `True`, puisqu'il suffirait qu'*une* condition soit vraie. Ici, *toutes* doivent l'être.

Les boucles

Exercice 39 *De 1 à 10* ↑énoncé p. 40

▶ **Solution.** `range(1, 11)` va de 1 inclus à 11 exclu.

```
for i in range(1, 11):
    print(i)
```

▷ **Erreur fréquente.** Écrire `range(10)` (qui affiche 0 à 9) ou `range(1, 10)` (qui s'arrête à 9) : la borne de droite n'est jamais atteinte.

Exercice 40 *Trace de boucle* ↑énoncé p. 40

▶ **Solution.** *Sur papier.* `i` prend les valeurs $0, 1, 2, 3$; on affiche $i \times i$:

```
0
1
4
9
```

Exercice 41 *Un message qui se propage* ↑énoncé p. 40

▶ **Solution.** On accumule dans `total` le nombre de personnes touchées à chaque étape, de 0 à `n` inclus.

```
def diffusion(n):
    total = 0
    for i in range(n + 1):
        total = total + 3 ** i
    return total

print(diffusion(3))    # 40
print(diffusion(10))   # 88573
```

$1 + 3 + 9 + 27 = 40$. En 10 étapes seulement, près de 90 000 personnes : c'est ainsi qu'un message devient « viral ».

▷ **Erreur fréquente.** Écrire `total = 0` *à l'intérieur* de la boucle : le total repart de 0 à chaque tour et la fonction ne renvoie que le dernier terme (27 pour `diffusion(3)`). Autre piège : `range(n)` oublie la dernière étape.

Exercice 42 *Les puissances de 2* ↑énoncé p. 40

▶ **Solution.** `range(n + 1)` va de 0 à n inclus. On trouve $2^{10} = 1024$.

```
def puissances_de_2(n):
    for i in range(n + 1):
        print(2 ** i)

puissances_de_2(10)   # 1, 2, 4, ..., 1024
```

Exercice 43 *Les écouteurs* ↑énoncé p. 40

▶ **Solution.** On ne sait pas à l'avance combien de semaines il faudra : c'est un `while`.

```
economies = 0
semaines = 0
while economies < 200:
    economies = economies + 15
    semaines = semaines + 1
print(semaines)   # 14 (210 euros)
```

Exercice 44 *Les multiples de 7* ↑énoncé p. 40

▶ **Solution.** Un compteur qu'on augmente à chaque multiple rencontré (le test `%` dans la boucle).

```
compteur = 0
for n in range(1, 1001):
    if n % 7 == 0:
        compteur = compteur + 1
print(compteur)   # 142
```

Autre méthode, sans boucle : tout simplement `1000 // 7`.

Exercice 45 *Les abonnés* ↑énoncé p. 40

▶ **Solution.** On multiplie le nombre d'abonnés par 1,03 autant de fois qu'il y a de mois.

```
def abonnes(depart, n):
    a = depart
    for mois in range(n):
        a = a * 1.03
    return a

print(abonnes(1000, 10))   # 1343.916...
```

Le calcul donne un nombre à virgule ; on l'arrondit : environ 1344 abonnés. Remarquer que le compte gagne 344 abonnés en 10 mois, et non $10 \times 30 = 300$: chaque mois, les 3 % portent sur un nombre d'abonnés plus grand.

Exercice 46 *Combien de chiffres ?* ↑énoncé p. 41

▶ **Solution.** Chaque division entière par 10 retire le dernier chiffre ; on compte les étapes.

```
def nb_chiffres(n):
    compteur = 0
    while n > 0:
        n = n // 10
        compteur = compteur + 1
    return compteur

print(nb_chiffres(2048))   # 4
```

Exercice 47 *Nombre premier* ↑énoncé p. 41

▶ **Solution.** On cherche un diviseur entre 2 et $n - 1$; s'il en existe un, `n` n'est pas premier.

```
def est_premier(n):
    if n < 2:
        return False
    for d in range(2, n):
        if n % d == 0:
            return False
    return True
```

`return` arrête **immédiatement** la fonction, même au milieu d'une boucle : dès qu'un diviseur est trouvé, on renvoie `False` sans tester les suivants. Si la boucle va jusqu'au bout sans trouver de diviseur, on arrive à la dernière ligne, `return True`.

Petits défis de synthèse

Exercice 48 *Fizz* ↑énoncé p. 41

▶ **Solution.**

```
for i in range(1, 31):
    if i % 3 == 0:
        print("Fizz")
    else:
        print(i)
```

Exercice 49 *L'ordinateur devine* ↑énoncé p. 41

▶ Solution. À chaque réponse, l'intervalle est coupé en deux : c'est la *recherche par dichotomie*.

```
bas = 1
haut = 100
essais = 0
reponse = ""
while reponse != "=":
    proposition = (bas + haut) // 2
    essais = essais + 1
    reponse = input(f"Je propose {proposition} (+, - ou =) : ")
    if reponse == "+":
        bas = proposition + 1
    elif reponse == "-":
        haut = proposition - 1
print(f"Trouve en {essais} essais !")
```

Au maximum 7 essais : chaque essai divise l'intervalle par deux, et $2^7 = 128 \geq 100$.

Exercice 50 *Moyenne d'une série de notes* ↑énoncé p. 41

▶ Solution. On lit une première note, puis on continue **tant que** ce n'est pas -1.

```
somme = 0
nombre = 0
note = float(input("Note (-1 pour finir) : "))
while note != -1:
    somme = somme + note
    nombre = nombre + 1
    note = float(input("Note (-1 pour finir) : "))
if nombre > 0:
    print(f"Moyenne : {somme / nombre}")
else:
    print("Aucune note saisie.")
```

Exercice 51 *Le PGCD* ↑énoncé p. 41

▶ Solution. Tant que les deux nombres diffèrent, on remplace le plus grand par sa différence avec le plus petit.

```
def pgcd(a, b):
    while a != b:
        if a > b:
            a = a - b
        else:
            b = b - a
    return a

print(pgcd(24, 36))   # 12
```

Critiquer une réponse d'IA

Exercice 52 *Une puissance calculée par un assistant* ↑énoncé p. 41

▶ Solution. **1.** Non. L'exécution affiche `16` et non `8` comme l'annonce le commentaire ; de même `puissance(5, 2)` donne `125` au lieu de `25`, et `puissance(3, 0)` donne `3` au lieu de `1`.
2. L'erreur est dans `range(n + 1)` : cette suite contient $n+1$ nombres $(0, 1, \ldots, n)$, donc la boucle multiplie **une fois de trop**. Il faut écrire `range(n)` (rappel du cours : `range(5)` fabrique cinq nombres, de 0 à 4).

```
def puissance(x, n):
    resultat = 1
    for i in range(n):
        resultat = resultat * x
    return resultat
```

```
6
7 print(puissance(2, 3))   # affiche bien 8
```

3. Exécuter la fonction sur **un petit exemple dont on connaît le résultat** ($2 \times 2 \times 2 = 8$) : le commentaire `# affiche 8` n'est qu'une affirmation de l'assistant ; seule la console fait foi.

Réponses des questions de la fiche de révision

↑ questions p. 46

Se tester

1. `3` (`int`) ; `2` (`int`) ; `5.0` (`float`, `/` donne toujours un flottant) (§ V).
2. `6`, `9`, `12`, `15` (un par ligne : i vaut 2, 3, 4, 5) (§ X.A).
3. `def perimetre(longueur, largeur):` puis, indenté, `return 2 * (longueur + largeur)` (§ VII).
4. `input` renvoie du texte : texte + nombre ⇒ `TypeError`. Écrire `n = int(input("Nombre ? "))` (§ VI.C).
5. `False` (une des deux est fausse) ; `True` (il suffit d'une vraie) (§ VIII.B).

Complément — Dessiner avec la tortue

CORRIGÉS DES EXERCICES

Remarque Plusieurs écritures sont possibles ; on en donne une simple. Chaque programme suppose `import turtle as t` au début et `t.done()` à la fin. Rappel de la règle d'or : un polygone régulier à `n` côtés se ferme en tournant de `360 / n` à chaque sommet.

Premiers tracés

Exercice 1 *Le carré* ↑énoncé p. 53

▶ Solution. Quatre fois « avancer de 100, tourner de 90 ».

```
import turtle as t
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
t.done()
```

On verra dans le cours (§ 2) comment la boucle `for` évite de recopier quatre fois les mêmes lignes.

Exercice 2 *Le rectangle* ↑énoncé p. 53

▶ Solution. Deux longueurs et deux largeurs, en alternance, en tournant toujours de 90.

```
import turtle as t
t.forward(200)   # longueur
t.left(90)
t.forward(100)   # largeur
t.left(90)
t.forward(200)   # longueur
t.left(90)
t.forward(100)   # largeur
t.left(90)
t.done()
```

Avec la boucle `for` du § 2, on répétera **deux fois** le couple « long côté / court côté ».

1

Répéter : la boucle `for`

Exercice 3 *L'escalier* ↑énoncé p. 53

▶ Solution. Chaque marche = un pas horizontal, un pas vertical. Après être monté, on se **remet** à l'horizontale avec `right(90)`.

```
import turtle as t
for i in range(5):
    t.forward(40)    # horizontal
    t.left(90)
    t.forward(40)    # vertical
    t.right(90)      # on repart a l'horizontale
t.done()
```

▷ **Erreur fréquente.** Tourner deux fois à gauche dans la marche : à la fin de la première marche, la tortue regarde vers la gauche et repart en arrière, l'escalier ne monte pas. Après avoir monté, il faut tourner *à droite*.

Exercice 4 *Triangle et hexagone* ↑énoncé p. 53

▶ Solution. Triangle : $360/3 = \mathbf{120}$ degrés. Hexagone : $360/6 = \mathbf{60}$ degrés.

```
import turtle as t
for i in range(3):      # triangle equilateral
    t.forward(120)
    t.left(120)
for i in range(6):      # hexagone
    t.forward(80)
    t.left(60)
t.done()
```

▷ **Erreur fréquente.** Tourner de 60 degrés pour le triangle, parce que ses angles mesurent 60 degrés : la tortue tourne de l'angle *extérieur* ($180 - 60 = 120$), et avec 60 la figure ne se ferme pas. La règle $360/n$ évite le piège.

Exercice 5 *L'étoile à cinq branches* ↑énoncé p. 54

▶ Solution. En tournant de `144` degrés à chaque pointe, on « saute » un sommet sur deux, ce qui trace l'étoile.

```
import turtle as t
for i in range(5):
    t.forward(200)
    t.left(144)
t.done()
```

Réutiliser : les fonctions

Exercice 6 *La fonction carré* ↑énoncé p. 54

▶ Solution. On définit `carre` une fois, puis on l'appelle avec des tailles différentes.

```
import turtle as t
def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)

carre(50)
carre(100)
carre(150)
t.done()
```

▷ **Erreur fréquente.** Écrire la définition `def carre(cote):` sans jamais l'appeler : rien ne se dessine. Définir une fonction ne l'exécute pas ; ce sont les lignes `carre(50)`… qui la font travailler.

Exercice 7 *Un polygone au choix* ↑énoncé p. 54

► Solution. On remplace l'angle par `360 / n` : la même fonction dessine tous les polygones réguliers.

```
import turtle as t
def polygone(n, cote):
    for i in range(n):
        t.forward(cote)
        t.left(360 / n)

polygone(3, 100)   # triangle
polygone(4, 100)   # carre
polygone(8, 60)    # octogone
t.done()
```

Colorier une forme

Exercice 8 *Trois carrés colorés* ↑énoncé p. 54

► Solution. On fabrique un « carré plein » (couleur + `begin_fill`/`end_fill`), et on déplace la tortue *sans tracer* entre deux carrés grâce à `penup`/`forward`/`pendown`. Après un carré, la tortue est revenue à son point de départ, tournée vers la droite : avancer de `100` (côté 80 plus un espace de 20) la place au départ du carré suivant.

```
import turtle as t
def carre_plein(cote, couleur):
    t.color(couleur)
    t.begin_fill()
    for i in range(4):
        t.forward(cote)
        t.left(90)
    t.end_fill()

carre_plein(80, "red")
t.penup()
t.forward(100)      # on se deplace sans tracer
t.pendown()
carre_plein(80, "green")
t.penup()
t.forward(100)
t.pendown()
carre_plein(80, "blue")
t.done()
```

Répéter en tournant : les rosaces

Exercice 9 *Rosace de carrés* ↑énoncé p. 54

► Solution. On réutilise `carre`, répété 12 fois en pivotant de `30` degrés ($12 \times 30 = 360$).

```
import turtle as t
def carre(cote):
    for i in range(4):
        t.forward(cote)
        t.left(90)

for k in range(12):
    carre(100)
    t.left(30)
t.done()
```

Exercice 10 *Rosace de cercles* ↑énoncé p. 54

▶ Solution. 18 cercles répartis sur un tour complet : on tourne de $360/18 = \mathbf{20}$ degrés entre chaque.

```
import turtle as t
for k in range(18):
    t.circle(80)
    t.left(20)
t.done()
```

Défis

Exercice 11 *La maison* ↑énoncé p. 55

▶ Solution. On trace d'abord la façade (un carré). La tortue revient en bas à gauche ; on monte au coin haut-gauche, puis on pose un toit triangulaire (triangle équilatéral, angles de `120` degrés). Pourquoi `right(30)` ? Arrivée en haut, la tortue regarde vers le haut (90° par rapport à l'horizontale) ; or le premier côté du toit monte à 60° de l'horizontale (angle d'un triangle équilatéral) : elle doit donc tourner de $90 - 60 = 30$ degrés vers la droite.

```
import turtle as t
# La facade (carre de cote 100)
for i in range(4):
    t.forward(100)
    t.left(90)
# On monte au coin haut-gauche pour poser le toit
t.left(90)
t.forward(100)
t.right(30)          # 90 - 60 = 30 : on vise le sommet du toit
# Le toit (triangle equilateral)
for i in range(3):
    t.forward(100)
    t.right(120)
t.done()
```

Exercice 12 *La spirale carrée* ↑énoncé p. 55

▶ Solution. On garde l'angle à `90` degrés, mais on **augmente** la longueur du côté à chaque tour.

```
import turtle as t
cote = 10
for i in range(20):
    t.forward(cote)
    t.left(90)
    cote = cote + 10    # le cote grandit
t.done()
```

Exercice 13 *Du triangle à l'heptagone* ↑énoncé p. 55

▶ Solution. On réutilise la fonction `polygone(n, cote)` ; une seule boucle fait varier le nombre de côtés `n` de 3 à 7 (`range(3, 8)` s'arrête *avant* 8). Après chaque figure, la tortue est revenue à son point de départ, dans la même direction (elle a tourné de 360 degrés en tout) : on lève le crayon, on avance vers la droite, et on le repose.

```
import turtle as t

def polygone(n, cote):
    for i in range(n):
        t.forward(cote)
        t.left(360 / n)

for n in range(3, 8):
```

```
    polygone(n, 50)
    t.penup()           # on se deplace sans tracer
    t.forward(130)      # assez loin pour ne pas chevaucher
    t.pendown()
t.done()
```

Critiquer une réponse d'IA

Exercice 14 *Le pentagone de l'assistant* ↑énoncé p. 55

▶ Solution. **1.** Non. La tortue tourne de $5 \times 108 = 540$ degrés en tout, et non 360 : la figure ne se referme pas (la tortue termine loin de son point de départ, le cap retourné de 180 degrés).

2. L'assistant a confondu l'**angle intérieur** du pentagone (108 degrés) avec l'angle dont la tortue doit **tourner** à chaque sommet. Règle du cours : pour fermer un polygone régulier à `n` côtés, elle tourne de `360 / n`, soit ici $360/5 = 72$ degrés. Il suffit de remplacer `t.left(108)` par `t.left(72)`.

```
import turtle as t
for i in range(5):
    t.forward(100)
    t.left(72)      # 360 / 5
t.done()
```

3. Vérifier que la **somme des rotations** vaut 360 degrés (5×108 n'y est pas), ou simplement exécuter le programme et regarder si la figure se ferme.

CORRIGÉS DU CHAPITRE 2

Les réseaux sociaux

2

CORRIGÉS DES EXERCICES

Éléments de réponse aux activités du cours

Réponses aux questions fermées ; pour les questions ouvertes ou de débat, des pistes attendues (d'autres formulations sont possibles).

Activité 1 — Dresser le portrait des réseaux

1. Facebook, WhatsApp et Instagram (environ 3 milliards chacun) > YouTube (∼ 2,5 milliards) > TikTok (∼ 1,6 milliard) > Snapchat (près de 1 milliard) > X (centaines de millions). LinkedIn ne se compare pas directement : il annonce ses *inscrits*, pas ses utilisateurs actifs. Les trois premiers ont le même ordre de grandeur et appartiennent à la même entreprise, **Meta**.
2. **YouTube** et **TikTok** (Instagram accepté pour ses vidéos courtes).
3. Le chiffre est publié par l'entreprise elle-même et change tous les jours ; il existe des **faux comptes**, des comptes **inactifs**, des personnes qui ont **plusieurs comptes**, et la définition d'« utilisateur actif » varie. On ne peut donc retenir qu'un **ordre de grandeur**.
4. Réponse personnelle. Exemples : TikTok, des vidéos courtes et divertissantes, surtout pour un public jeune ; LinkedIn, un réseau professionnel pour trouver un stage ou un emploi.

Activité 2 — Régler sa confidentialité

1. Trois réglages exposent Léa (on en attend trois parmi ceux-ci) : le **compte public** → le passer en **privé** ; les **commentaires et messages ouverts à tous** → les limiter à ses **abonnés** ou amis ; la **position ajoutée aux publications** → **désactiver le partage de la position** ; les **abonnés visibles par tous** → masquer la liste de ses abonnés.
2. Réponse personnelle (ne jamais recopier un mot de passe donné en exemple : publié, il n'est plus secret). Méthode attendue : un mot de passe **long** (au moins 12 caractères), par exemple une phrase de passe de plusieurs mots sans lien entre eux, avec majuscules, chiffres et symboles, sans **information personnelle** (prénom, date de naissance) et **différent** pour chaque site.
3. **Identification** : saisir son adresse e-mail ; taper son pseudo. **Authentification** : entrer un code reçu par SMS ; poser son doigt sur le capteur d'empreinte.
4. Débat, pistes attendues : il faut l'**accord** de la personne (droit à l'image, vie privée) ; une fois publiée, la photo **échappe** à son auteur (copies, captures) ; publier sans accord peut **blesser** et, selon les cas, être **puni par la loi**.

Activité 3 — Suivre l'argent

1. La **publicité ciblée**.
2. On vend aux annonceurs notre **attention** et nos **données** (le profil qui permet de choisir la publicité la plus susceptible de nous intéresser).
3. 2 milliards $\times$ 40 e = **80 milliards d'euros par an** (ordre de grandeur).
4. Plus vous restez, plus l'application peut **afficher de publicités** et collecter de données : elle a donc intérêt à vous retenir (notifications, lecture automatique, fil infini).

Activité 4 — Mesurer le réseau des amis

1. Degrés : Alice 2, Bilal 3, Chloé 3, **David** 4, Ella 2, Farid 2. L'influenceur est **David**.
2. Distance Alice–Farid = **3** (Alice–Bilal–David–Farid) ; distance Bilal–Ella = **2** (Bilal–David–Ella).
3. Excentricités : Alice 3, Bilal 2, Chloé 2, David 2, Ella 3, Farid 3.
4. **Diamètre** = 3 (plus grande excentricité) ; **rayon** = 2 (plus petite) ; **centre** = {Bilal, Chloé, David}.
5. Avec les arêtes Alice–Ella et Alice–Farid, toutes les excentricités valent 2 : le **diamètre passe de** 3 **à** 2.

Activité 5 — Combien de degrés nous séparent ?

1. Réponse personnelle. Idée attendue : deux personnes prises au hasard sont reliées par une **courte chaîne de connaissances** (environ six), même si elles ne se connaissent pas.
2. La plus grande distance est le **diamètre**, soit **3** (par exemple Alice et Farid) : c'est **moins** que six, car le réseau est tout petit.
3. Chaque nouvelle amitié crée un **raccourci** entre deux parties du réseau : les chaînes raccourcissent, donc la distance moyenne **diminue**.

Activité 6 — Enquêter sur son fil

1. Pistes attendues : un algorithme **classe** les contenus selon des signaux (temps passé, « j'aime », partages, comptes suivis…) et place en premier ce qui a le plus de chances de vous **retenir**.
2. Réponse personnelle (par exemple : ne plus voir que des vidéos d'un seul type après en avoir regardé quelques-unes).
3. Exemples : **suivre des comptes aux avis différents** ; **vérifier** une information avant de la partager ; couper la lecture automatique ; s'abonner à des médias fiables.

Activité 7 — Ma charte des réseaux

Production personnelle. Une charte réussie contient par exemple une règle sur la **vie privée** (régler sa confidentialité, ne pas publier d'informations personnelles), une sur l'**esprit critique** (vérifier avant de partager), une contre le **harcèlement** (ne pas insulter, garder les preuves, signaler, soutenir une victime), et par exemple le **respect des autres** (demander l'accord avant de publier une photo) et l'**équilibre** (limiter son temps d'écran).

Activité 8 — Sobriété numérique

1. La vidéo est **stockée** en plusieurs définitions et en plusieurs copies dans des centres de données, tant qu'elle n'est pas supprimée ; surtout, elle est **retransmise à chaque visionnage** : vue un million de fois, elle traverse un million de fois les réseaux.
2. Presque toute l'empreinte du téléphone (environ 80 kg CO_2e) est émise à la **fabrication**, donc avant qu'on s'en serve. Le garder plus longtemps **répartit** cette empreinte sur plus d'années et **évite** d'en fabriquer un nouveau (par exemple 80 kg sur 5 ans au lieu de 2,5 ans : deux fois moins par an).

3. Avec la lecture automatique, des vidéos sont téléchargées **sans avoir été choisies** ; en haute qualité, chaque seconde de vidéo contient **plus de données**. Réglages : désactiver la lecture automatique, réduire la qualité vidéo (surtout sur un petit écran).

4. Le réseau gagne de l'argent grâce au **temps passé** (plus de publicités vues) : le fil infini et la lecture automatique servent justement à **retenir l'attention**. Les désactiver irait contre son modèle économique ; c'est donc à l'utilisateur de le faire.

Reconnaître et comparer les réseaux

Exercice 1 *Le bon réseau* ↑énoncé p. 70

▶ **Corrigé.** Instagram ↔ **photos** ; TikTok ↔ **vidéos courtes** ; LinkedIn ↔ **contacts professionnels** ; WhatsApp ↔ **messages privés** ; YouTube ↔ **vidéos**. *(Beaucoup de réseaux mélangent aujourd'hui plusieurs formats ; on retient ici leur usage* principal *d'origine.)*

Exercice 2 *Ordres de grandeur* ↑énoncé p. 70

▶ **Corrigé.** **1.** 3 milliards sur 8 milliards, soit $\frac{3}{8} \approx 0{,}375$: environ **37 %** de l'humanité (un peu plus d'un tiers).

2. Le nombre exact d'utilisateurs *change en permanence* (inscriptions, comptes inactifs ou multiples) et n'est pas vérifiable. Un **ordre de grandeur** (« quelques milliards ») suffit pour comparer et raisonner, sans prétendre à une précision illusoire.

Identité numérique, sécurité

Exercice 3 *Identifier ou authentifier ?* ↑énoncé p. 70

▶ **Corrigé.** **a.** identifiant de l'ENT = **identification** (on annonce qui l'on est) ; **b.** mot de passe = **authentification** (on prouve qu'on connaît un secret) ; **c.** reconnaissance du visage = **authentification** (on prouve qui l'on est par ce que l'on est) ; **d.** numéro de téléphone saisi sur la page de connexion = **identification** (il sert ici d'identifiant) ; **e.** validation de la notification = **authentification** (on prouve qu'on possède le téléphone). *En résumé : l'identification* annonce, *l'authentification* prouve.

▷ **Erreur fréquente.** Ranger la notification « Est-ce bien vous ? » (ou un code reçu par SMS) dans l'identification parce qu'on y « dit que c'est soi ». En la validant, on *prouve* qu'on a le téléphone en main : c'est de l'authentification.

Exercice 4 *E-réputation* ↑énoncé p. 70

▶ **Corrigé.** « Rien ne s'efface vraiment » : une photo, un message ou un commentaire mis en ligne peut être **copié, capturé (*screenshot*), partagé et archivé** par d'autres ; même supprimé de la source, il peut subsister ailleurs. *Exemple :* une publication gênante postée à 15 ans peut être retrouvée des années plus tard par un recruteur, une école ou un employeur, et nuire à une candidature.

Exercice 5 *Un bon mot de passe* ↑énoncé p. 71

▶ **Corrigé.** *Exemple.* Faible : `sofiane2010` — court, contient un **prénom** et une **date de naissance** (informations personnelles faciles à deviner), aucun caractère spécial. Solide : on ne recopie jamais un exemple imprimé (publié, il n'est plus secret), mais on suit une méthode — par exemple une **phrase de passe** de plusieurs mots sans lien entre eux, choisis au hasard, dans laquelle on glisse **majuscules, chiffres et symboles** : elle est **longue**, ne contient **aucune information personnelle** et reste facile à retenir. *Les deux leviers essentiels : la **longueur** et l'**imprévisibilité**.*

Exercice 6 *Mes données, mes droits* ↑énoncé p. 71

▶ **Corrigé.** **1.** Par exemple : son **nom** ou son pseudo, son **adresse e-mail**, ses **photos**, sa **date de naissance**, sa **localisation**, la liste de ses abonnés.
2. Savoir quelles données sont conservées : **droit d'accès**. Faire supprimer une photo : **droit d'effacement**.
3. Résidant à Monaco, elle s'adresse à l'**APDP** (Autorité de protection des données personnelles, loi n° 1.565). Si elle habitait à Nice (France), ce serait la **CNIL** (RGPD et loi Informatique et libertés).
4. Non : en dessous de 15 ans, il faut aussi l'accord des parents (titulaires de l'autorité parentale), à Monaco comme en France.

Le modèle économique

Exercice 7 *« Si c'est gratuit… »* ↑énoncé p. 71

▶ **Corrigé.** **1.** La principale source de revenus est la **publicité ciblée** (et la revente de l'espace publicitaire aux annonceurs).
2. « Si c'est gratuit, c'est que le **produit**, c'est **vous** ». Ce qu'on vend à l'annonceur, ce sont votre **attention** et vos **données personnelles** (centres d'intérêt, comportements), qui permettent de vous montrer des publicités très ciblées.

Exercice 8 *Suivre l'argent* ↑énoncé p. 71

▶ **Corrigé.** **1.** Revenu ≈ 500 millions $\times$ 12 € $= 6\,000$ millions d'euros, soit **6 milliards d'euros par an** (ordre de grandeur).
2. $30 \div 5 = \mathbf{6}$ publicités par jour ; en 1 heure, $60 \div 5 = \mathbf{12}$ publicités. Le nombre de publicités vues, donc ce que paient les annonceurs, est proportionnel au temps passé : l'application a intérêt à **capter votre attention** le plus longtemps possible (notifications, défilement infini, vidéos qui s'enchaînent). C'est l'**économie de l'attention**.

▷ **Erreur fréquente.** Écrire « $500 \times 12 = 6\,000$ milliards » : on comptait en *millions*. 6 000 millions font 6 milliards. Un contrôle de vraisemblance aide : un réseau quatre fois plus petit que celui de l'activité du cours, qui rapporte moins par utilisateur, ne peut pas gagner davantage.

Exercice 9 *Ce que la publicité sait de vous* ↑énoncé p. 71

▶ **Corrigé.** **1.** Hugo **aime le football** (et le pratique sans doute : il cherche des crampons), il a un **petit budget** (« pas chers »), il fréquente le **stade Louis-II**, donc vit probablement **à Monaco ou à proximité**, et il s'intéresse à la **cuisine**.
2. Par exemple : des crampons en promotion dans un magasin de sport proche de Monaco ; des billets pour un match de l'AS Monaco ; une marque de pâtes ou un robot de cuisine.
3. Ces informations permettent de montrer à Hugo des publicités **ciblées**, qui ont plus de chances de le faire cliquer et acheter. Les annonceurs paient donc plus cher pour les afficher : ce qui est vendu, c'est l'**attention** et les **données** d'Hugo.

Un réseau social, c'est un graphe

Exercice 10 *Degrés et distances* ↑énoncé p. 72

▶ **Corrigé.** **1.** Degrés : Gabin 2, Hana 3, Ilyes 3, **Jade** 4, Kenza 2, Lucas 3, Maël 1. La personne la plus « connectée » est **Jade** (degré 4).
2. Distance **Gabin–Kenza = 3** : un plus court chemin est **Gabin–Hana–Jade–Kenza** (ou Gabin–Ilyes–Jade–Kenza). Distance **Kenza–Maël = 2** : chemin **Kenza–Lucas–Maël**.

▷ **Erreur fréquente.** Compter les personnes du chemin au lieu des arêtes : Gabin–Hana–Jade–Kenza passe par 4 personnes, mais n'emprunte que 3 arêtes. La distance vaut 3.

Exercice 11 *Diamètre, rayon, centre* ↑énoncé p. 72

▶ **Corrigé. 1.** L'**excentricité** d'un sommet est la **plus grande distance** qui le sépare d'un autre sommet du réseau (« à quelle distance est la personne la plus éloignée de moi ? »). Ici :

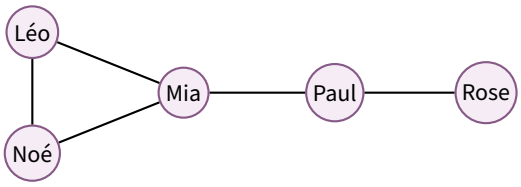
	Gabin	Hana	Ilyes	Jade	Kenza	Lucas	Maël
excentricité	4	3	3	2	3	3	4

2. Le **diamètre** (plus grande excentricité) vaut **4** (Gabin et Maël) ; le **rayon** (plus petite excentricité) vaut **2** ; le **centre** (le sommet d'excentricité minimale) est **Jade**.
3. Non, le diamètre reste égal à **4**. Pour aller de Gabin à Maël, il faut passer par Hana ou Ilyes, puis forcément par Jade (le seul lien entre les deux groupes), puis par Kenza ou Lucas avant d'arriver à Maël : au moins 4 arêtes, avec ou sans la nouvelle amitié. *Contrairement à l'activité du cours, une nouvelle amitié ne réduit pas toujours le diamètre : il faut qu'elle raccourcisse le chemin entre les deux personnes les plus éloignées.*

▷ **Erreur fréquente.** Prendre comme excentricité la *plus petite* distance (qui vaut presque toujours 1) : l'excentricité est la distance à la personne la *plus éloignée*.

Exercice 12 *Dessiner un réseau* ↑énoncé p. 72

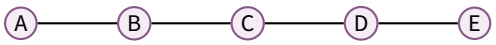
▶ **Corrigé. 1.** Cinq sommets et cinq arêtes : Léo–Mia, Léo–Noé, Mia–Noé, Mia–Paul, Paul–Rose.



2. Degrés : Léo 2, **Mia** 3, Noé 2, Paul 2, Rose 1.
3. Distance **Léo–Rose = 3** (Léo–Mia–Paul–Rose) ; distance **Noé–Paul = 2** (Noé–Mia–Paul).

Exercice 13 *Fabriquer un diamètre* ↑énoncé p. 72

▶ **Corrigé. 1.** On place les cinq personnes en « file indienne » : A–B–C–D–E. Les deux extrémités A et E sont à distance 4, et aucune distance n'est plus grande : le diamètre vaut **4**. Ce réseau compte **4** arêtes. (C'est d'ailleurs le seul dessin possible : avec cinq personnes, une distance de 4 oblige à les aligner toutes.)



2. Diamètre 1 : toutes les distances valent 1, donc **chacun est ami avec tous les autres**. Chaque personne a 4 amis, ce qui fait $5 \times 4 = 20$ « bouts » d'arêtes ; chaque arête ayant deux bouts, il y a $20/2 = \mathbf{10}$ arêtes.
3. Excentricités dans la file : A 4, B 3, **C** 2, D 3, E 4. Le rayon vaut 2 et le **centre** est la personne du milieu, **C**.

Petit monde et recommandation

Exercice 14 *Six degrés de séparation* ↑énoncé p. 73

▶ **Corrigé. 1.** L'idée : deux personnes prises au hasard sur Terre seraient reliées par une **chaîne d'environ six connaissances** (« l'ami d'un ami d'un ami… »). C'est le phénomène du **petit monde**, suggéré par l'expérience de **Milgram** (1967).
2. Une distance moyenne d'environ 3,5 sur Facebook montre que le monde numérique est devenu un « **petit monde** » *encore plus resserré* : les réseaux sociaux raccourcissent les chaînes de connaissances (bien en dessous de six).

Exercice 15 *Qui choisit ce que je vois ?* ↑énoncé p. 73

▶ **Corrigé.** **1.** Un **algorithme de recommandation** choisit et ordonne les contenus qui vous sont montrés (fil d'actualité, vidéos suggérées) à partir de vos actions passées. Son but : **maximiser votre engagement** (temps passé, clics, likes).
2. Une **bulle de filtres** : à force de vous montrer ce qui vous plaît déjà, l'algorithme vous enferme dans des contenus **similaires** et vous cache les autres points de vue. Geste simple pour en sortir : **s'abonner volontairement à des sources variées** (voire opposées), varier ses recherches, ou désactiver la personnalisation.

Exercice 16 *Les amis de mes amis* ↑énoncé p. 73

▶ **Corrigé.** **1.** Alice a pour amis Bilal et Chloé. Leurs autres amis : David seulement. La seule personne à distance 2 d'Alice est donc **David**, avec qui elle a **deux** amis en commun (Bilal et Chloé) : David lui est suggéré en premier.
2. Ella a pour amis David et Farid. Les amis de David qui ne sont pas déjà amis d'Ella sont Bilal et Chloé (Farid, lui, n'a pas d'autre ami). À distance 2 d'Ella : **Bilal** et **Chloé**, avec chacun **un seul** ami en commun (David) : ils sont à égalité. Alice est à distance 3 d'Ella : elle n'est pas suggérée.
3. On vous propose surtout des personnes **déjà proches de vos amis**, qui fréquentent les mêmes groupes et partagent souvent les mêmes goûts et avis. Chaque groupe se resserre, on rencontre rarement des personnes vraiment différentes : comme la bulle de filtres, ce mécanisme rétrécit la variété de ce que l'on voit (ici, des *personnes*).

Exercice 17 *Citoyen des réseaux* ↑énoncé p. 73

▶ **Corrigé.** Face au cyberharcèlement : **(1)** ne pas répondre ni « rendre les coups », **conserver les preuves** (captures d'écran) ; **(2) signaler** et bloquer l'auteur sur la plateforme ; **(3) en parler** à un adulte de confiance (parent, professeur, CPE). À Monaco, la ligne d'écoute du Gouvernement contre le harcèlement, y compris en ligne, est joignable au **98 98 96 96** (24 h/24) ; le service français **3018** (violences numériques, gratuit et anonyme) répond aussi par tchat avec son application.

Impact environnemental

Exercice 18 *Le coût d'un smartphone* ↑énoncé p. 73

▶ **Corrigé.** **1.** 20 Go = 20 000 Mo, et 20 000 ÷ 400 = **50 minutes** de vidéo en 4K.
2. Par exemple : **trier et supprimer** les vidéos ratées ou en double ; filmer en **définition plus basse** (1080p au lieu de 4K, bien suffisant sur un petit écran) ; transférer ses vidéos sur un ordinateur ou un disque dur ; si la mémoire manque vraiment, choisir un téléphone **reconditionné**. Acheter du neuf ajoute environ 80 kg CO_2e, émis presque entièrement à la fabrication.
3. $50 \times 1\,000\,000 = 50\,000\,000$ Mo $= 50 \times 10^6$ Mo, soit **50 To** transportés par les réseaux — un million de fois la taille du fichier.
4. « Le nuage » est fait de **serveurs** rangés dans des **centres de données** : ils fonctionnent jour et nuit, doivent être fabriqués, alimentés en électricité et refroidis, et les fichiers y sont souvent conservés en plusieurs copies. Stocker des vidéos qu'on ne regarde plus a donc un coût, même si on ne le voit pas.

Exercice 19 *Haute ou basse définition ?* ↑énoncé p. 74

▶ **Corrigé.** **1.** En une semaine : $2 \times 7 = 14$ heures de vidéo. Haute définition : $14 \times 3 =$ **42 Go** ; définition standard : $14 \times 1 =$ **14 Go**.
2. $42 \div 14 = 3$: **trois fois moins** de données en définition standard.
3. Conseil : régler la qualité vidéo sur la **définition standard** (ou « économie de données ») et désactiver la lecture automatique. Moins de données échangées, c'est moins de sollicitation des **réseaux** et des **centres de données**, donc moins d'énergie consommée ; cela préserve aussi son forfait.

Critiquer une réponse d'IA

Exercice 20 *Une distance calculée par un assistant* ↑énoncé p. 74

▶ **Corrigé.** **1.** Non. Le chemin proposé (Alice–Bilal–David–Farid) est bien un plus court chemin, David a bien le degré 4 et Ella–Farid sont bien à distance 1. Mais, d'après le cours, la longueur d'un chemin est « le nombre d'**arêtes** empruntées », et la distance est la longueur du plus court chemin : ce chemin compte **3** arêtes, pas 4.
2. L'assistant a compté les **personnes** (sommets) au lieu des **liens** (arêtes) : un chemin qui passe par 4 sommets comporte 3 arêtes. La distance Alice–Farid vaut **3**.
3. Tester la règle sur un cas évident : Ella et Farid, amis directs, sont « à distance 1 » alors que leur chemin passe par *deux* personnes. La façon de compter de l'assistant se contredit donc elle-même en deux phrases.

Python au service du thème

Exercice 21 *Python au service du thème — l'économie de l'attention* ↑énoncé p. 74

▶ **Corrigé.** **1.** Le revenu annuel est le produit du nombre d'utilisateurs par le revenu moyen de chacun :

```
def revenu(utilisateurs, revenu_par_utilisateur):
    return utilisateurs * revenu_par_utilisateur

print(revenu(500000000, 12))    # 6000000000
```

On retrouve 6 000 000 000 €, soit 6 milliards d'euros, comme à l'exercice « Suivre l'argent ».
2. Le nombre de publicités est le nombre de tranches *complètes* de 4 minutes : c'est le quotient de la division entière par 4, `minutes // 4`.

```
def nb_pubs(minutes):
    return minutes // 4

print(nb_pubs(10))   # 2
print(nb_pubs(30))   # 7
print(nb_pubs(60))   # 15
```

3. La variable `i` prend les valeurs 1 à 4 et la durée quotidienne vaut `30 * i` :

```
for i in range(1, 5):
    minutes = 30 * i
    print(f"{minutes} min par jour : {nb_pubs(minutes) * 365} publicites par an")
```

Le programme affiche :

```
30 min par jour : 2555 publicites par an
60 min par jour : 5475 publicites par an
90 min par jour : 8030 publicites par an
120 min par jour : 10950 publicites par an
```

Chaque publicité affichée est payée par un annonceur : plus vous restez, plus le réseau affiche de publicités et plus il gagne d'argent. C'est l'**économie de l'attention** : vidéos qui s'enchaînent, défilement infini et notifications servent à allonger ce temps.

▷ **Erreur fréquente.** Écrire `minutes / 4` : la division `/` donne `7.5` publicités pour 30 minutes, un nombre qui n'a pas de sens ici. La division entière `//` ne compte que les tranches de 4 minutes *terminées*.

CORRIGÉS : DÉBAT

DÉBAT Faut-il interdire les réseaux sociaux aux moins de 15 ans, avec une vérification d'âge obligatoire ?

↑énoncé p. 77

À lire après le débat. Ces éléments de réponse servent à vérifier et à compléter votre trace écrite : les lire avant le débat lui ôterait son intérêt.

Notions du cours mobilisées

- **Données personnelles**, **consentement**, âge de 15 ans (RGPD et loi Informatique et libertés ; loi n° 1.565 et APDP à Monaco).
- **Identification / authentification** : vérifier un âge, c'est prouver une information, comme on prouve son identité.
- **Modèle économique** : publicité ciblée, économie de l'attention, revenu par utilisateur.
- **Algorithme de recommandation**, bulle de filtres, désinformation.
- **Cyberharcèlement** et réflexes (garder les preuves, bloquer, signaler, en parler ; à Monaco, ligne d'écoute 98 98 96 96).
- **Identité numérique** et e-réputation : la permanence des contenus publiés jeune.

Arguments attendus

POUR (oui) : interdire avant 15 ans et vérifier	CONTRE (non) : ni interdiction ni vérification obligatoire
• **Protéger la santé** : sommeil, attention, exposition au harcèlement (doc. 2). • **La règle actuelle n'est pas respectée** : 13 ans affichés, 8 ans et demi en réalité (doc. 1 et 2) ; sans vérification, une règle ne sert à rien. • **Cohérence avec la loi** : on ne peut consentir seul à l'usage de ses données qu'à 15 ans (doc. 1, cours). • **Modèle économique** : l'attention et les données des plus jeunes sont vendues aux annonceurs (cours). • **Effet collectif** : si personne n'y est, on n'est pas exclu du groupe ; la pression sociale diminue. • **C'est faisable** : l'Australie l'applique (doc. 3) ; le double anonymat protège la vie privée (doc. 4).	• **Vie privée de tous** : vérifier l'âge oblige chacun, adultes compris, à fournir des données (doc. 4). • **Contournement** : fausse date, compte d'un adulte (doc. 3) ; l'interdiction peut rester théorique. • **Repli vers des espaces moins surveillés**, sans modération ni signalement (doc. 3). • **Lien social, création, information** : les réseaux ont aussi des effets positifs (doc. 2, cours « Bien vivre avec les réseaux »). • **Éduquer plutôt qu'interdire** : apprendre à régler sa confidentialité et à garder son esprit critique (cours). • **Rôle des familles** : l'accord parental existe déjà (doc. 1) ; c'est aux parents de décider, pas à l'État.

Arguments de l'autre camp souvent jugés solides dans la trace écrite : un élève POUR retient souvent « vérifier l'âge de tout le monde, c'est collecter beaucoup de données personnelles » ; un élève CONTRE, « la règle actuelle des 13 ans n'est presque jamais vérifiée ».

Questions de relance (pour le modérateur)

1. Une interdiction que tout le monde contourne vaut-elle mieux que pas d'interdiction du tout ?
2. Seriez-vous prêts à envoyer votre pièce d'identité à un réseau social ? Et le double anonymat (doc. 4) change-t-il votre réponse ?
3. Messagerie, plateforme de vidéos, jeu en ligne avec discussion : où s'arrête un « réseau social » ?

4. Qui est responsable : la plateforme, les parents, l'élève, l'État ? (doc. 1 et 3)

Synthèse possible

Les réseaux sociaux fixent un âge minimal (souvent 13 ans) rarement vérifié, alors que la loi (RGPD, loi française ; loi n° 1.565 à Monaco) exige l'accord des parents avant 15 ans. Interdire avant 15 ans peut **protéger** (santé, harcèlement, exploitation commerciale de l'attention), mais une interdiction n'a de sens que si l'âge est **vérifié**, ce qui demande de fournir des **données personnelles** : il faut alors des solutions respectueuses de la vie privée, comme le **double anonymat** recommandé par la CNIL. Dans tous les cas, l'éducation (régler sa confidentialité, esprit critique, réflexes contre le harcèlement) reste indispensable.

Réponses des questions de la fiche de révision

↑ questions p. 79

Se tester

1. Pseudo : identification ; code SMS : authentification (§ II).
2. On vend votre attention et vos données aux annonceurs (§ III).
3. Degré de A : 2 ; distance A–E : 1 (§ IV).
4. Bulle de filtres (ou chambre d'écho, désinformation) (§ VI).

Et avant ?

1. `str` (texte) et `int` (entier) (ch. 1, § IV).
2. `6` (ch. 1, § V.A).

CORRIGÉS DU CHAPITRE 3

La photographie numérique

3

CORRIGÉS DES EXERCICES

Éléments de réponse aux activités du cours

Réponses aux questions fermées ; pour les questions ouvertes, des pistes attendues (d'autres formulations sont possibles).

Activité 1 — Expliquer le capteur

Production personnelle. Éléments attendus : le capteur est un quadrillage de millions de **photosites** ; chacun reçoit de la **lumière** à travers un petit filtre rouge, vert ou bleu (le **filtre de Bayer**, avec deux fois plus de filtres verts) et la transforme en un **nombre**, d'autant plus grand que la lumière est intense ; l'appareil combine ensuite les mesures voisines pour obtenir les trois valeurs R, V, B de chaque pixel.

Activité 2 — Jouer avec les couleurs

1. $256 \times 256 \times 256 = 256^3 =$ **16 777 216** couleurs (environ 16,8 millions).
2. Noir (0,0,0) ; blanc (255,255,255) ; jaune (255,255,0) ; violet (255,0,255).
3. (125,125,125) : un **gris moyen** (trois canaux égaux) ; (30,30,30) : un **gris très foncé**, presque noir.

Activité 3 — Calculer sur les images

1. 20 cm $= 20 \div 2{,}54 \approx 7{,}87$ pouces, et $1200 \div 7{,}87 \approx$ **152 ppp** (même résultat en hauteur : 15 cm $\approx 5{,}91$ pouces et $900 \div 5{,}91 \approx 152$).
2. 15 cm $\approx 5{,}91$ pouces, soit $5{,}91 \times 300 \approx 1772$ pixels ; 10 cm $\approx 3{,}94$ pouces, soit $3{,}94 \times 300 \approx 1181$ pixels. Il faut une définition d'au moins **1772** $\times$ **1181** environ (de l'ordre de 2 mégapixels).
3. $4000 \times 3000 \times 3 = 36\,000\,000$ octets, soit **36 Mo**.

Activité 4 — Ses premières retouches

1. On remplace les coordonnées dans la ligne de lecture : `r, v, b = img.getpixel((250, 300))`.
2. Chaque pixel prend pour rouge son ancien vert, pour vert son ancien bleu et pour bleu son ancien rouge : les couleurs sont **permutées** (ce qui était bleu devient vert, le rouge devient bleu, le vert devient rouge).
3. Négatif : dans le programme à deux boucles du cours, on remplace la ligne `putpixel` par `img.putpixel((x, y), (255 - r, 255 - v, 255 - b))` (l'exercice « Le négatif » applique la même ligne à la moitié gauche de l'image seulement).
4. Niveaux de gris : dans les deux boucles, on calcule `gris = (r + v + b) // 3`, puis on écrit le pixel `(gris, gris, gris)` (l'exercice « Niveaux de gris » utilise une autre formule, qui tient compte de la sensibilité de l'œil).

Activité 5 — Enquêter sur une photo

1. Les valeurs dépendent de la photo fournie : la date se lit sur la ligne `DateTime`, le fabricant sur la ligne `Make` (et le modèle sur `Model`), la définition sur la ligne `definition` affichée en premier (largeur, hauteur).
2. Les coordonnées GPS permettent de situer la prise de vue sur une carte : en publiant la photo, on peut révéler sans le vouloir son **domicile**, son école ou ses habitudes (et, avec la date, *quand* on y était).

Le capteur et la formation de l'image

Exercice 1 *Vrai ou faux* ↑énoncé p. 90

▶ **Corrigé.** **a. Faux.** Un photosite ne mesure qu'**une seule** composante (une quantité de lumière derrière *un* filtre rouge, vert *ou* bleu) ; les trois nombres RVB d'un pixel sont **reconstitués** ensuite à partir des photosites voisins (dématriçage).
b. Faux. Dans le filtre de **Bayer**, il y a **deux fois plus de filtres verts** que de rouges ou de bleus (l'œil humain est plus sensible au vert).
c. Vrai. Sans filtre coloré, chaque photosite ne mesurerait qu'une intensité lumineuse : on obtiendrait une image en **niveaux de gris**.
d. Vrai. Plus un photosite reçoit de lumière, plus la valeur numérique produite est **grande** (proche de 255).

Le pixel et le codage RVB

Exercice 2 *Nommer la couleur* ↑énoncé p. 90

▶ **Corrigé.** (255,0,0) : **rouge** ; (0,0,0) : **noir** ; (255,255,255) : **blanc** ; (0,255,0) : **vert** ; (255,255,0) : **jaune** (rouge + vert) ; (40,40,40) : un **gris foncé** (les trois canaux égaux et faibles).

Exercice 3 *Coder une couleur* ↑énoncé p. 90

▶ **Corrigé.** *Exemples plausibles.* Gris clair (200,200,200) (trois canaux égaux et assez grands) ; cyan (0,255,255) (vert + bleu) ; bleu très sombre (0,0,60) ; vert moyen (0,128,0) ; orange (255,140,0) ; rose pâle (255,200,200) : le rouge reste au maximum, et le vert et le bleu, égaux, sont montés pour « blanchir » la couleur (avec 255 partout, on obtiendrait du blanc).

Exercice 4 *Combien de couleurs ?* ↑énoncé p. 90

▶ **Corrigé.** **1.** Chaque canal a 16 valeurs, et il y a trois canaux indépendants :

$$16 \times 16 \times 16 = 16^3 = \mathbf{4\,096} \text{ couleurs,}$$

bien moins que les $256^3 = 16\,777\,216$ couleurs du codage de 0 à 255 (activité 2 du cours) : 4 096 fois moins.
2. Un gris a ses trois canaux **égaux** : il y a donc autant de gris que de valeurs communes possibles, soit **16** nuances (du noir au blanc).

▷ **Erreur fréquente.** Additionner au lieu de multiplier : $16 + 16 + 16 = 48$. Or chaque valeur du rouge peut s'associer à *chacune* des 16 valeurs du vert, et chaque couple à chacune des 16 valeurs du bleu : les possibilités se multiplient.

Définition, résolution et poids

Exercice 5 *Compter les pixels* ↑énoncé p. 91

▶ Corrigé. **1.** $1920 \times 1080 = \mathbf{2\,073\,600}$ pixels, soit environ **2 millions** (2 mégapixels).
2. Poids brut $= 3$ octets $\times\ 2\,073\,600 = 6\,220\,800$ octets $\approx$ **6,2 Mo**.

Exercice 6 *Doubler la définition* ↑énoncé p. 91

▶ Corrigé. **1.** Image A : $800 \times 600 = \mathbf{480\,000}$ pixels. Image B : $1600 \times 1200 = \mathbf{1\,920\,000}$ pixels.
2. $1\,920\,000 \div 480\,000 = \mathbf{4}$: B contient **quatre fois** plus de pixels, et non deux. La largeur *et* la hauteur ont doublé : $2 \times 2 = 4$.
3. Poids brut de B $= 1\,920\,000 \times 3 = 5\,760\,000$ octets $\approx$ **5,8 Mo** (quatre fois le poids brut de A, 1,44 Mo).

▷ **Erreur fréquente.** Répondre « deux fois plus de pixels » parce que la définition « double » : la largeur double *et* la hauteur double, donc le nombre de pixels est multiplié par $2 \times 2 = 4$.

Exercice 7 *Résolution d'une impression* ↑énoncé p. 91

▶ Corrigé. **1.** $12{,}7$ cm $\div\ 2{,}54 = \mathbf{5}$ **pouces**.
2. Résolution $= \dfrac{1500 \text{ pixels}}{5 \text{ pouces}} = \mathbf{300\ ppp}$. C'est exactement la valeur visée pour une « belle » impression : **le résultat convient**. *(En hauteur :* $8{,}47$ *cm* $\approx 3{,}33$ *pouces, et* $1000/3{,}33 \approx 300$ *ppp également.)*

▷ **Erreur fréquente.** Diviser les pixels par la largeur en *centimètres* : $1500 \div 12{,}7 \approx 118$. On obtient des pixels par centimètre, pas des pixels par pouce ; il faut d'abord convertir en pouces.

Exercice 8 *Imprimer en grand* ↑énoncé p. 91

▶ Corrigé. **1.** À 300 ppp, chaque pouce imprimé utilise 300 pixels. Largeur : $4000 \div 300 \approx 13{,}3$ pouces, soit $13{,}3 \times 2{,}54 \approx \mathbf{34\ cm}$. Hauteur : $3000 \div 300 = 10$ pouces, soit **25,4 cm**. La plus grande « belle » impression mesure donc environ $\mathbf{34 \times 25}$ **cm**.
2. 60 cm $= 60 \div 2{,}54 \approx 23{,}6$ pouces, et $4000 \div 23{,}6 \approx \mathbf{170\ ppp}$. C'est nettement moins que 300 ppp : ce n'est pas une « belle » impression au sens strict. Mais une affiche se regarde **de loin** : à quelques mètres, l'œil ne distingue plus les pixels, et 170 ppp suffisent largement.

Exercice 9 *Compression* ↑énoncé p. 91

▶ Corrigé. Poids brut $= 4000 \times 3000 \times 3 = 36\,000\,000$ octets $=$ **36 Mo**. Sur la carte, la photo ne pèse que 4 Mo, soit **9 fois moins** : c'est l'effet de la **compression** (format JPEG). Elle **supprime des détails peu perceptibles** par l'œil (compression *avec pertes*) pour réduire fortement la taille du fichier.

Modifier une image par programme (PIL)

Exercice 10 *Lire un pixel* ↑énoncé p. 91

▶ Corrigé.

```
1 from PIL import Image
2 img = Image.open("terre.jpg").convert("RGB")
3 r, v, b = img.getpixel((120, 200))
4 print("Rouge :", r, " Vert :", v, " Bleu :", b)
```

`getpixel((x, y))` *renvoie le triplet RVB du pixel ; on l'affiche.*

Exercice 11 *Que fait ce programme ?* ↑énoncé p. 92

▶ Corrigé. **1.** Pour chaque pixel, il lit ses valeurs (r, v, b) puis les réécrit en **mettant le rouge à 0** : $(0, v, b)$. Il **supprime la composante rouge** de toute l'image.
2. L'image perd tout son rouge : elle **tire vers le cyan** (bleu-vert). Sur `terre.jpg`, les tons chauds disparaissent et l'image paraît plus froide et bleutée.

Exercice 12 *Le négatif* ↑énoncé p. 92

▶ **Corrigé.** On remplace chaque canal c par $255 - c$, mais seulement pour les colonnes de la moitié gauche : `x` ne prend que les valeurs de 0 à `largeur // 2 - 1`.

```
from PIL import Image
img = Image.open("terre.jpg").convert("RGB")
largeur, hauteur = img.size
for y in range(hauteur):
    for x in range(largeur // 2):      # moitie gauche seulement
        r, v, b = img.getpixel((x, y))
        img.putpixel((x, y), (255 - r, 255 - v, 255 - b))
img.show()
```

Autre solution : garder `range(largeur)` *et ne modifier le pixel que* `if x < largeur // 2:`*. Sur* `terre.jpg` *(*500×500*), l'espace noir devient blanc à gauche et reste noir à droite : par exemple, le pixel* $(10, 250)$*, qui valait* $(3, 1, 4)$*, devient* $(252, 254, 251)$*.*

▷ **Erreur fréquente.** Écrire `range(largeur / 2)` : la division `/` donne un nombre à virgule (`250.0`), que `range` refuse (`TypeError`). Il faut la division entière `//`.

Exercice 13 *Éclaircir l'image* ↑énoncé p. 92

▶ **Corrigé.** On ajoute 60 à chaque canal, sans dépasser 255 grâce à `min`.

```
from PIL import Image
img = Image.open("terre.jpg").convert("RGB")
largeur, hauteur = img.size
for y in range(hauteur):
    for x in range(largeur):
        r, v, b = img.getpixel((x, y))
        img.putpixel((x, y), (min(255, r + 60), min(255, v + 60), min(255, b + 60)))
img.show()
```

Les zones déjà très claires (canaux proches de 255*) sont bloquées à* 255 *: elles deviennent blanches et leurs détails disparaissent (on dit que l'image est « brûlée »). Le fond noir de l'espace, lui, devient gris foncé (par exemple* $(5, 3, 6)$ *devient* $(65, 63, 66)$*).*

Exercice 14 *Niveaux de gris* ↑énoncé p. 92

▶ **Corrigé.** On remplace R, V, B par la valeur m ; les trois canaux deviennent égaux, donc le pixel est gris.

```
from PIL import Image
img = Image.open("terre.jpg").convert("RGB")
largeur, hauteur = img.size
for y in range(hauteur):
    for x in range(largeur):
        r, v, b = img.getpixel((x, y))
        m = (3 * r + 6 * v + b) // 10
        img.putpixel((x, y), (m, m, m))
img.show()
```

Vérification : après ce traitement, tout pixel vaut (m, m, m) *— ses trois canaux sont bien* ***égaux****, ce qui caractérise un gris.* Pour le pixel $(100, 200, 50)$: $m = (300 + 1200 + 50)//10 = 1550//10 = \mathbf{155}$, alors que la simple moyenne donne $350//3 = \mathbf{116}$. Ce pixel, surtout vert, paraît clair à l'œil : la formule pondérée le rend plus fidèlement. Les coefficients $3 + 6 + 1$ font 10 : un pixel déjà gris (g, g, g) garde la valeur g.

▷ **Erreur fréquente.** Oublier les parenthèses : `m = 3 * r + 6 * v + b // 10` ne divise que `b` par 10 (la division passe avant l'addition). Un test sur un pixel connu, par exemple $(90, 90, 90)$, montre l'erreur : on trouve 819 au lieu de 90, une valeur impossible pour un canal, qui ne dépasse jamais 255.

Exercice 15 *Le miroir* ↑énoncé p. 92

▶ Corrigé. Sur chaque ligne, le pixel de la colonne `x` échange sa place avec celui de la colonne symétrique `largeur - 1 - x` (la colonne 0 avec la dernière, la colonne 1 avec l'avant-dernière, etc.). On ne parcourt que la **moitié gauche** : si l'on allait jusqu'au bout, chaque paire serait échangée deux fois et l'image reviendrait à l'identique.

```
from PIL import Image
img = Image.open("terre.jpg").convert("RGB")
largeur, hauteur = img.size
for y in range(hauteur):
    for x in range(largeur // 2):
        gauche = img.getpixel((x, y))
        droite = img.getpixel((largeur - 1 - x, y))
        img.putpixel((x, y), droite)                  # on echange
        img.putpixel((largeur - 1 - x, y), gauche)    # les deux pixels
img.show()
```

*Les deux valeurs sont lues **avant** d'écrire : si l'on écrivait d'abord à gauche, la couleur d'origine de ce pixel serait perdue.*

Métadonnées et vie privée

Exercice 16 *Les données cachées* ↑énoncé p. 92

▶ Corrigé. 1. Les **métadonnées EXIF** sont des informations **enregistrées automatiquement** dans le fichier photo, en plus des pixels. Exemples : **date et heure** de la prise, **modèle d'appareil** (ou de smartphone), **réglages** (ouverture, temps de pose, ISO), et souvent les **coordonnées GPS**.
2. Si une photo publiée contient les **coordonnées GPS**, n'importe qui peut savoir **où** (et quand) elle a été prise : domicile, école, lieux fréquentés. C'est une atteinte à la **vie privée** et à la sécurité.
3. Précaution : **désactiver la géolocalisation** de l'appareil photo, ou **supprimer les métadonnées** (EXIF) de la photo avant de la partager.

Exercice 17 *Une photo est-elle une preuve ?* ↑énoncé p. 93

▶ Corrigé. Non, pas à elle seule. Une image peut être **retouchée**, passée par des **filtres**, sortie de son contexte, ou entièrement **générée par une IA**. Une photo est un indice, pas une preuve absolue : il faut la **recouper** avec d'autres sources.

Critiquer une réponse d'IA

Exercice 18 *Poids et résolution selon un assistant* ↑énoncé p. 93

▶ Corrigé. 1. Le début est juste : $6000 \times 4000 = 24\,000\,000$ pixels (24 Mpx), 3 octets par pixel, donc $72\,000\,000$ octets ≈ 72 Mo brut, réduits par la compression JPEG. En revanche, 30 cm $= 30/2{,}54 \approx 11{,}8$ pouces, et $6000/11{,}8 \approx$ **508** ppp, pas 6000.
2. L'assistant a confondu la **définition** (le nombre de pixels : 6000 en largeur) et la **résolution** (le nombre de pixels *par pouce*) ; le cours précise : « la résolution est le nombre de pixels par unité de longueur ». La valeur juste est environ 508 ppp — la conclusion (impression de bonne qualité) reste vraie, mais le nombre est faux.
3. Un ordre de grandeur suffisait : 6000 ppp sur 11,8 pouces exigerait $6000 \times 11{,}8 \approx 71\,000$ pixels de large, près de **douze fois** plus que la photo n'en possède ($71\,000/6000 \approx 11{,}8$). Dès qu'une réponse annonce une résolution, vérifier l'unité (pixels *par pouce*) et refaire la division.

CORRIGÉS : DÉBAT

DÉBAT Faut-il obliger à signaler toute photo retouchée, dans la publicité comme sur les réseaux sociaux ?

↑ énoncé p. 103

À lire après le débat. Ces éléments de réponse servent à vérifier et à compléter votre trace écrite : les lire avant le débat lui ôterait son intérêt.

Notions du cours mobilisées

- **Pixel**, codage **RVB** (trois nombres de 0 à 255), **définition** et **résolution**.
- **Modifier une image par programme** (Python + PIL) : éclaircir, changer une couleur, c'est changer les valeurs des pixels.
- **Algorithmes des appareils** : HDR, débruitage, accentuation, embellissement automatique.
- **Une photo n'est plus une preuve** en soi : esprit critique (d'où vient l'image ? qui l'a publiée ?).
- **Métadonnées EXIF** : le fichier peut garder la trace du logiciel, mais elles s'effacent.

Arguments attendus

POUR (oui) : toujours signaler	CONTRE (non) : pas d'obligation générale
• **Santé publique** : des corps impossibles à atteindre abîment l'image de soi (doc. 1). • **Droit d'être informé** : une retouche est souvent invisible, celui qui regarde doit savoir (cours). • **La loi le fait déjà** pour la publicité et l'influence commerciale ; il suffit d'aller plus loin (doc. 1 et 2). • **Les filtres sont partout** sur les réseaux, or la loi ne vise que l'influence commerciale : une photo personnelle retouchée échappe à toute mention (doc. 2). Les jeunes se comparent pourtant à ces images. • **Confiance** : la presse montre qu'on peut fixer une règle claire (doc. 4). • **C'est techniquement simple** : une mention, ou une trace dans les métadonnées (doc. 4).	• **Toute photo est déjà calculée** par le smartphone (doc. 3, cours) : où placer la limite ? • **Mention inutile si elle est partout** : on ne la lit plus (doc. 3). • **Liberté artistique** : la retouche fait partie du travail du photographe depuis toujours (doc. 3). • **Impossible à contrôler** pour des milliards de photos personnelles ; les métadonnées s'effacent (doc. 4). • **Distinguer les cas** : la loi vise déjà ce qui trompe dans un but commercial (silhouette, visage) (doc. 1 et 2). • **Retouche légère ≠ tromperie** : corriger la lumière ne change pas ce que montre l'image (doc. 1 et 4).

Arguments de l'autre camp souvent jugés solides dans la trace écrite : un élève POUR retient souvent « une mention mise sur toutes les photos finirait par ne plus être lue » ; un élève CONTRE, « une silhouette affinée dans une publicité trompe vraiment celui qui regarde ».

Questions de relance (pour le modérateur)

1. Éclaircir une photo et affiner une silhouette : est-ce la même chose ? (pensez aux pixels et à ce que montre l'image)
2. La retouche automatique du smartphone (doc. 3) doit-elle être signalée ?
3. Pourquoi la loi vise-t-elle la publicité et les influenceurs, et pas vos photos de vacances (doc. 1 et 2) ?
4. La règle de la presse (doc. 4) pourrait-elle s'appliquer aux réseaux sociaux ?

Synthèse possible

Retoucher une photo numérique, c'est **modifier les valeurs de ses pixels** : c'est facile, et le résultat peut être invisible. Toute photo de smartphone est déjà transformée par des algorithmes (HDR, débruitage). La loi française n'impose pas de signaler toute retouche, mais celles qui **trompent dans un but commercial** : la silhouette dans la publicité (« Photographie retouchée », 2017), la silhouette ou le visage dans l'influence commerciale (« Image retouchée », 2023). La presse s'impose des règles plus strictes encore : on ne touche pas au contenu. Face à une image, l'esprit critique reste indispensable : d'où vient-elle, qui l'a publiée, dans quel but ?

Réponses des questions de la fiche de révision

↑ questions p. 105

Se tester

1. (255,255,0) (rouge + vert) ; un gris clair (trois canaux égaux) (§ II).
2. $2000 \times 1500 \times 3 = 9\,000\,000$ octets ≈ 9 Mo (§ III).
3. 25,4 cm = 10 pouces ; $1000 \div 10 = 100$ ppp (§ III).
4. (55,205,165) (§ IV).
5. Date, modèle d'appareil… ; le GPS révèle le lieu (domicile, école) (§ V).

Et avant ?

1. Le nombre d'arêtes qui en partent (ch. 2, § IV).
2. `for i in range(256):` puis, indenté, `print(i)` (ch. 1, § X.A).

CORRIGÉS DU CHAPITRE 4

Le Web

4

CORRIGÉS DES EXERCICES

Éléments de réponse aux activités du cours

Réponses aux questions fermées ; pour les questions ouvertes, des pistes attendues (d'autres réponses sont possibles).

Activité 1 — Ma première page

Le résultat dépend de chacun ; voici une réalisation possible des trois questions (zone HTML, puis zone CSS de l'éditeur en ligne).

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Ma premiere page</title>
  </head>
  <body>
    <h1>Bienvenue</h1>
    <p>Ceci est ma page web.</p>
    <p id="important">Ce paragraphe est important.</p>
    <img src="renard.jpg" alt="Un renard dans la neige" />
  </body>
</html>
```

```css
h1 {
  text-align: center;
  color: crimson;
}
p {
  font-family: Verdana;
}
#important {
  color: red;
}
```

Seul le paragraphe qui porte `id="important"` passe en rouge : en CSS, l'identifiant se cible avec un dièse `#`.

Activité 2 — Aiguiser son esprit critique

1. Réponse personnelle. On constate en général que les premiers résultats ne sont pas tout à fait les mêmes, ni dans le même ordre : chaque moteur a son propre **index** et son propre **algorithme de classement**.
2. Les publicités sont signalées par une mention du type « **Sponsorisé** » (ou « Annonce ») et sont souvent placées tout en haut ; les autres résultats sont classés par l'algorithme, sans paiement.
3. La plupart des internautes cliquent sur les tout premiers résultats et vont rarement en deuxième page : être en tête apporte beaucoup de visiteurs, donc de ventes ou de revenus publicitaires. C'est pourquoi des entreprises paient des annonces ou cherchent à améliorer leur classement.

Le Web et Internet

Exercice 1 *Vrai ou faux* ↑énoncé p. 119

▶ Corrigé.

a. **Faux.** Internet est le *réseau* ; le Web est un *service* (les pages) qui circule dessus.
b. **Faux.** L'e-mail utilise le protocole SMTP, pas le Web.
c. **Vrai.** HTTP, URL et HTML.
d. **Vrai.** Tim Berners-Lee, au CERN, en 1989–1991.

Exercice 2 *Web ou Internet ?* ↑énoncé p. 119

▶ Corrigé.

- **Web** : une page consultée dans un navigateur ; un lien hypertexte ; le protocole HTTP.
- **Internet (hors Web)** : un e-mail (SMTP) ; un transfert de fichier par FTP.

▷ **Erreur fréquente.** Ranger l'e-mail dans le Web parce qu'on le lit parfois dans un navigateur (messagerie en ligne). Le courrier lui-même voyage avec son propre protocole (SMTP) : c'est un autre service d'Internet que le Web.

La requête HTTP, client et serveur

Exercice 3 *Décomposer une requête* ↑énoncé p. 119

▶ Corrigé.

1. Ressource demandée : `/prevoir`.
2. **Deux** paramètres : `ville` vaut `Nice` ; `jour` vaut `mardi`.
3. `GET` signifie « *donne-moi* cette ressource ».

Exercice 4 *Réussite ou échec ?* ↑énoncé p. 119

▶ Corrigé. **200** = tout va bien ; **403** = accès interdit ; **404** = page non trouvée ; **500** = erreur du serveur.

Exercice 5 *Client ou serveur ?* ↑énoncé p. 120

▶ Corrigé.

1. **JavaScript** s'exécute dans le navigateur (côté client) ; PHP, lui, tourne côté serveur.
2. Par un **clic droit** → « **Afficher le code source** » (ou les outils de développement).
3. Une page **dynamique** (construite sur mesure par le serveur).

Exercice 6 *Suivre une requête* ↑énoncé p. 120

▶ Corrigé. Ordre : **C** (le navigateur envoie la requête `GET /cours.html`), **B** (le serveur répond, code `200`), **A** (le navigateur affiche la page). Si le fichier n'existait pas, le serveur aurait renvoyé le code **404** (page non trouvée).

Exercice 7 *Le formulaire de recherche* ↑énoncé p. 120

▶ Corrigé.

1. Ressource demandée : `/chercher`. **Deux** paramètres : `titre` vaut `Harry Potter` (écrit `Harry+Potter`) ; `format` vaut `livre`.
2. `https://www.mediatheque.mc/chercher?titre=Astérix&format=BD` (le navigateur encode lui-même la lettre accentuée).
3. Une page **dynamique** : le serveur ne peut pas avoir préparé à l'avance une page pour chaque titre possible ; il **construit** la liste des résultats d'après les paramètres reçus.
4. Les paramètres apparaissent **en clair dans la barre d'adresse** (quelqu'un peut les lire par-dessus l'épaule) et restent enregistrés dans l'**historique** du navigateur et dans les marque-pages. Un mot de passe ne doit jamais être visible ainsi.

L'URL et les chemins

Exercice 8 *Décomposer une URL* ↑énoncé p. 120

▶ Corrigé.

1. Protocole : `https`. Il **chiffre** les données échangées (connexion sécurisée, cadenas).
2. Nom du serveur (domaine) : `www.lycee.mc`.
3. Chemin du fichier : `/cours/snt/web.html`.

Exercice 9 *Chemins absolu et relatif* ↑énoncé p. 120

▶ Corrigé.

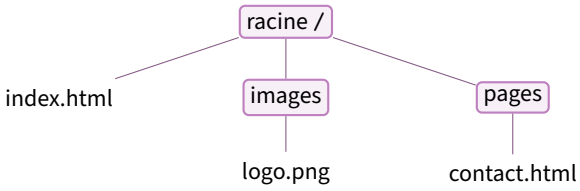
1. Chemin absolu : `/photos/ete/sable.jpg`.
2. Chemin relatif depuis `photos` : `ete/sable.jpg` (pas de `/` au début).
3. Depuis `musique` : `../photos/ete/mer.png` (on remonte à la racine avec `..`, puis on redescend dans `photos`, puis dans `ete`).

▷ **Erreur fréquente.** Écrire `/ete/sable.jpg` pour le chemin relatif : avec un `/` au début, le chemin part de la *racine* du site, où il n'y a pas de dossier `ete`. Le lien est cassé.

Exercice 10 *Le site déménage* ↑énoncé p. 121

▶ Corrigé.

1. Arborescence :



2. Un chemin relatif part du dossier de la page qui l'utilise. Depuis `pages`, `images/logo.png` désigne `pages/images/logo.png`, qui n'existe pas. Il faut d'abord **remonter** à la racine : `<img src="../images/logo.png" alt="Logo du club" />`.
3. `<a href="../index.html">Retour à l'accueil</a>`.

4. Il faut modifier **les deux pages** : dans `index.html`, `images/logo.png` devient `img/logo.png` ; dans `contact.html`, `../images/logo.png` devient `../img/logo.png`. *(Tout chemin qui passe par ce dossier doit être corrigé.)*

Lire du HTML

Exercice 11 *Que voit-on ?* ↑ énoncé p. 121

▶ **Corrigé.** Un grand titre « Recette », puis le paragraphe « Il faut **trois** oeufs. » (le mot **trois** en gras), puis une liste à puces :

- farine
- sucre

Exercice 12 *Corriger les erreurs* ↑ énoncé p. 121

▶ **Corrigé.** La balise `<strong>` est refermée trop tard (mauvaise imbrication), et le `<h2>` n'est pas fermé. Code corrigé :

```
<p>Bonjour <strong>tout le monde</strong></p>
<h2>Chapitre 1</h2>
```

▷ **Erreur fréquente.** Refermer les balises dans le désordre, comme `<p>...<strong>...</p></strong>` : la dernière balise ouverte doit être la première refermée (les balises s'emboîtent comme des boîtes).

Exercice 13 *Du rendu au code* ↑ énoncé p. 121

▶ **Corrigé.** Une solution :

```
<h1>Mon club de foot</h1>
<p>Entrainement le <strong>mercredi</strong>.</p>
<ul>
  <li>gardiens</li>
  <li>joueurs</li>
</ul>
<a href="https://www.monclub.mc">Le site du club</a>
```

Écrire du HTML et du CSS

Exercice 14 *Ma page de présentation* ↑ énoncé p. 122

▶ **Corrigé.** Une solution possible :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Ma page</title>
  </head>
  <body>
    <h1>Alex</h1>
    <p>Bonjour, je suis eleve de seconde au lycee.</p>
    <ul>
      <li>football</li>
      <li>guitare</li>
      <li>jeux video</li>
    </ul>
    <a href="https://www.gouv.mc">Un site que j'aime</a>
  </body>
</html>
```

Exercice 15 *Un peu de style* ↑énoncé p. 122

▶ **Corrigé.** On ajoute en bas du `<body>` une ligne de signature munie d'un `id`, par exemple `<p id="signature">Page réalisée par Léa</p>`, puis les règles CSS :

```
h1 {
  text-align: center;
  background-color: teal;
  color: white;
}
p {
  font-family: Verdana;
}
#signature {
  color: gray;
}
```

Exercice 16 *Le bon sélecteur* ↑énoncé p. 122

▶ **Corrigé.**

1. **Verdana** : les **trois paragraphes** `<p>` (pas le titre `<h1>`). **En vert** : les deux éléments de classe `info`, c'est-à-dire « Ouvert du lundi au vendredi. » et « Contact : 93 00 00 00 ». **Aligné à droite** : seulement l'élément d'identifiant `fin`, « Contact : 93 00 00 00 » (qui est donc à la fois en Verdana, en vert et à droite).
2. Par la balise (il n'y a qu'un `<h1>`) ou par son identifiant :

```
h1 { color: blue; }
#haut { color: blue; }
```

▷ **Erreur fréquente.** Confondre le point et le dièse : `.info` cible tous les éléments de *classe* `info`, `#fin` le seul élément d'*identifiant* `fin`. Écrire `#info` ne cible rien ici.

Exercice 17 *Prévoir le rendu* ↑énoncé p. 122

▶ **Corrigé.** La classe `.titre` cible *à la fois* le `<h1>` et le `<p>` (ils ont tous deux `class="titre"`). Les deux seront donc **centrés** et écrits en **bleu foncé**. Le `<h1>` reste un grand titre (gras), le `<p>` un paragraphe de taille normale.

Droit d'auteur

Exercice 18 *Ai-je le droit ?* ↑énoncé p. 122

▶ **Corrigé.**

1. **Non** : toute création est protégée par le **droit d'auteur**. La réutiliser exige l'autorisation de l'auteur (ou une licence adaptée).
2. Une **licence** fixe le **droit d'usage** d'une œuvre. Une licence **Creative Commons** autorise d'avance certains usages (copier, parfois modifier/diffuser), souvent à condition de **citer l'auteur**.
3. Faux : un fichier se copie pour presque rien, mais il garde une **valeur** (travail fourni, usage, valeur commerciale). *Copiable* ≠ *sans valeur*.

Trouver une page

Exercice 19 *Le moteur de recherche* ↑énoncé p. 123

► Corrigé.

1. L'**index** est l'immense répertoire des pages déjà explorées par les robots. Le moteur cherche **dans son index** (déjà prêt), pas « dans le Web » en direct.
2. Le **PageRank** : une page est jugée importante si beaucoup d'autres pages importantes pointent vers elle. (On accepte aussi : présence des mots-clés, langue, localisation.)
3. Les publicités sont signalées par une mention du type « **Sponsorisé** » (autrefois *Annonce*) et apparaissent souvent tout en haut.

Exercice 20 *Le graphe des liens* ↑énoncé p. 123

► Corrigé.

1. Liens reçus : A en reçoit **1** (de C) ; B en reçoit **1** (de A) ; C en reçoit **3** (de A, B et D) ; D n'en reçoit **aucun**.
2. **C**, qui reçoit le plus de liens.
3. **A** : son unique lien vient de C, la page la plus importante du graphe, alors que celui de B vient de A, moins importante. Selon l'idée du PageRank, un lien « compte » plus quand il vient d'une page importante.
4. Il cherche à **tricher** pour remonter artificiellement dans le classement (et attirer plus de visiteurs, donc plus d'argent). Les moteurs luttent contre cela pour que les premiers résultats restent **pertinents** pour l'utilisateur : ils repèrent les fausses pages et pénalisent les sites qui les utilisent.

Sécurité et confidentialité

Exercice 21 *Régler et sécuriser son navigateur* ↑énoncé p. 123

► Corrigé.

1. Par exemple : refuser les cookies (tiers), effacer l'historique / navigation privée, gérer les autorisations (caméra, micro, position), bloquer les traceurs.
2. L'**hameçonnage** : une fausse page imitant un vrai site pour voler vos identifiants.
3. **Survoler** le lien (sans cliquer) pour lire la *vraie* URL de destination et vérifier le nom de domaine.

Critiquer une réponse d'IA

Exercice 22 *Le cadenas rend-il anonyme ?* ↑énoncé p. 123

► Corrigé.

1. Non. Le début est juste (HTTPS chiffre les échanges, indispensable avant de saisir un mot de passe) et la fin aussi (un site d'hameçonnage peut être en HTTPS). Mais le cours dit exactement le contraire du milieu : « Naviguer n'est jamais tout à fait anonyme » et « chaque visite laisse des traces : historique, adresse IP, cookies ».
2. L'erreur : « avec HTTPS, les cookies sont désactivés et votre navigation est totalement anonyme ». HTTPS protège le **contenu** des échanges contre une lecture en chemin, rien de plus : le site (et ses cookies) sait très bien qui vous êtes et ce que vous y faites, et votre adresse IP reste visible. Correction : « le cadenas ne rend pas anonyme ; pour limiter le suivi, il faut régler les cookies ou utiliser la navigation privée ».
3. Ouvrir un site en HTTPS où l'on est resté connecté (le portail du lycée, une boutique) : il vous reconnaît malgré le cadenas, donc il n'y a pas d'anonymat. Ou, plus simplement, relire l'encadré « Cookies et traces » du cours avant de croire l'assistant.

Python au service du thème

Exercice 23 *Python au service du thème — fabriquer du HTML* ↑énoncé p. 124

▶ **Corrigé. 1.** Un élément HTML, c'est la balise ouvrante, le contenu, puis la balise fermante (avec sa barre oblique). Seuls le nom et le contenu changent : ce sont les deux paramètres.

```
def balise(nom, contenu):
    return "<" + nom + ">" + contenu + "</" + nom + ">"

print(balise("h1", "Mon club"))        # <h1>Mon club</h1>
print(balise("strong", "mercredi"))    # <strong>mercredi</strong>
```

2. `i` prend les valeurs 1, 2 et 3 ; le programme affiche :

```
<ul>
<li>Photo 1</li>
<li>Photo 2</li>
<li>Photo 3</li>
</ul>
```

C'est le code d'une **liste à puces** : le navigateur afficherait trois puces, « Photo 1 », « Photo 2 » et « Photo 3 ». Avec `range(1, 101)`, la même boucle écrirait une liste de cent photos : c'est ainsi qu'un serveur fabrique une page dynamique (une galerie, des résultats de recherche) sans que personne n'écrive le HTML à la main.

3. On recolle les trois morceaux avec les séparateurs `"://"` et `"/"` :

```
def url(protocole, serveur, chemin):
    return protocole + "://" + serveur + "/" + chemin

print(url("https", "www.lycee.mc", "cours/snt/web.html"))
# https://www.lycee.mc/cours/snt/web.html
```

▷ **Erreur fréquente.** Oublier la barre oblique de la balise fermante : `"<" + nom + ">" + contenu + "<" + nom + ">"` renvoie `<h1>Mon club<h1>`, où la balise `<h1>` est ouverte deux fois et jamais fermée. Afficher le résultat d'un appel avec `print` et le comparer à un code HTML correct montre tout de suite l'erreur.

CORRIGÉS : DÉBAT

DÉBAT Un moteur de recherche doit-il rester libre de classer ses résultats comme il le veut ?

↑énoncé p. 126

À lire après le débat. Ces éléments de réponse servent à vérifier et à compléter votre trace écrite : les lire avant le débat lui ôterait son intérêt.

Notions du cours mobilisées

- **Moteur de recherche**, robots (*crawlers*), **index**, requête.
- **Classement par pertinence**, *PageRank* (une page est importante si des pages importantes pointent vers elle), **hyperliens** et graphe du Web.
- **Publicités** signalées par une mention (« Sponsorisé ») ; enjeu commercial de la première page.
- **Cookies et traces** : la personnalisation des résultats repose sur des données personnelles.

- **Notions juridiques** du cours (droit d'auteur, licences) : le Web n'est pas une zone sans droit.

Arguments attendus

POUR (oui) : liberté du moteur	CONTRE (non) : règles fixées par la loi
• **Entreprise privée, service gratuit** : elle a investi dans son algorithme, c'est son savoir-faire. • **Le secret protège** contre la triche des référenceurs (doc. 3). • **Liberté de choix** : l'utilisateur peut changer de moteur (doc. 3). • **Qualité** : le moteur a intérêt à donner les meilleurs résultats, sinon il perd ses utilisateurs. • **Risque de censure** : si l'État décide du classement, il décide de ce que l'on voit. • **Les règles existent déjà** sur la façon de classer (doc. 2) : inutile d'aller plus loin, ni de décider à la place du moteur.	• **Position dominante** : quand presque tout le monde utilise le même moteur, son classement décide de qui existe (doc. 1). • **Autofavoritisme** : le moteur peut mettre ses propres services devant (doc. 1), d'où le DMA (doc. 2). • **Publicité et résultats mélangés** : l'utilisateur doit savoir ce qui est payé (doc. 3, cours). • **Transparence** : un classement secret ne peut pas être contrôlé (doc. 2). • **Vie privée** : le droit au déréférencement montre que la loi doit pouvoir intervenir (doc. 4). • **Profilage** : sans règle, chacun voit des résultats taillés pour lui (lien avec la bulle de filtres des réseaux sociaux).

Arguments de l'autre camp souvent jugés solides dans la trace écrite : un élève POUR retient souvent « un moteur dominant ne doit pas avantager ses propres produits » ; un élève CONTRE, « si l'État choisissait l'ordre des résultats, ce serait une forme de censure ».

Questions de relance (pour le modérateur)

1. Qui décide aujourd'hui qu'un article disparaît des résultats sur un nom (doc. 4) ? Est-ce normal que ce soit une entreprise ?
2. Changer de moteur, est-ce vraiment simple ? Lequel utilisez-vous, et pourquoi ?
3. Faut-il connaître la « recette » du classement pour lui faire confiance (doc. 3) ?
4. Encadrer la façon de classer, est-ce la même chose que décider de l'ordre des résultats (doc. 2) ?

Synthèse possible

Un moteur de recherche **classe** les pages de son index par un algorithme : l'ordre des résultats n'est jamais neutre, et la première page a une valeur commerciale énorme. Parce que quelques moteurs concentrent la plupart des recherches, l'Union européenne **encadre** ce classement sans le dicter : interdiction de favoriser ses propres services (DMA), option sans profilage (DSA), transparence des principaux critères (règlement de 2019), droit au déréférencement (RGPD ; loi n° 1.565 à Monaco). Le débat oppose la **liberté** de l'entreprise et l'**intérêt général** d'un accès loyal à l'information.

Réponses des questions de la fiche de révision

↑ questions p. 128

Se tester

1. L'e-mail (SMTP) (§ I).
2. `www.bus.mc` ; `/horaires` ; `ligne=2` et `arret=Casino` (§ II.A et III).
3. `<p>Bonne <strong>lecture</strong></p>` (imbrication propre) (§ IV).
4. `.alerte { color: green; }` (§ V)
5. Page introuvable sur le serveur (§ II).

Et avant ?

1. (255,255,255) (ch. 3, § II).
2. `www.lycee.mc` (ch. 1, § VI.A).

CORRIGÉS DU CHAPITRE 5

5 Les données structurées et leur traitement

CORRIGÉS DES EXERCICES

Éléments de réponse aux activités du cours

Réponses aux questions fermées ; pour les questions ouvertes, des pistes attendues (d'autres réponses sont possibles).

Activité 1 — Chasse aux métadonnées

Les valeurs dépendent du fichier choisi (par exemple : 2,4 Mo, modifié le 12/09, format `.jpg`). En général, la **taille** et le **format** ne sont pas des données personnelles ; la **date** peut en devenir une quand elle est reliée à vous (elle dit quand vous avez travaillé ou pris la photo). D'autres métadonnées sont nettement personnelles : le **nom de l'auteur** d'un document, ou les **coordonnées GPS** d'une photo.

Activité 2 — Lire une table

1. **7** descripteurs et **6** enregistrements (la ligne d'en-tête ne compte pas).
2. *Mario Kart 8* : `ventes` vaut `60` (millions d'exemplaires).
3. Des **nombres** : `annee`, `pegi`, `ventes` (ou `id`, `studio`) ; du **texte** : `titre`, `genre`.

Activité 3 — Data-détective : premières fouilles

1. *Red Dead Redemption 2* est sorti en **2018**.
2. `pegi` = 18 : *GTA V* et *Red Dead Redemption 2*.
3. Par `annee` croissante : Wii Sports (2006), Minecraft (2011), GTA V (2013), Mario Kart 8 (2014), Red Dead Redemption 2 (2018), Animal Crossing (2020).
4. *Mario Kart 8* a `studio` = `1` ; dans la table *studio*, l'`id` `1` est Nintendo : le jeu est édité au **Japon**.

Activité 4 — Explorer un vrai jeu de données ouvert

Réponse personnelle. On attend : le titre exact du jeu de données et son format (le plus souvent CSV) ; le nombre de colonnes, lu sur la ligne d'en-tête, et trois noms de descripteurs ; une question précise du type « Quel est le plus grand… ? » (tri décroissant, puis lecture de la première ligne) ou « Combien de…vérifient… ? » (filtre, puis comptage des lignes).

Données, données personnelles, métadonnées

Exercice 1 *Donnée personnelle ou pas ?* ↑énoncé p. 139

▶ Corrigé.

a. Oui : une adresse e-mail identifie une personne.
b. Non : la température ne concerne personne en particulier.
c. Oui : un numéro de téléphone est rattaché à une personne.
d. Non : un chiffre global, aucune personne identifiée.
e. Oui : la position GPS révèle *indirectement* où était la personne (souvent son domicile).

Exercice 2 *Donnée ou métadonnée ?* ↑énoncé p. 139

▶ Corrigé.

- **Donnée** (le contenu) : les pixels de l'image ; le chat visible sur la photo.
- **Métadonnée** (ce qui décrit) : la date de prise de vue ; le modèle d'appareil ; les coordonnées GPS ; le poids du fichier.

Lire une table

Exercice 3 *Vocabulaire de la table* ↑énoncé p. 140

▶ Corrigé.

1. La table *jeu* a 7 descripteurs (colonnes) et 6 enregistrements (lignes, sans compter l'en-tête). Une colonne de plus et trois lignes de plus : $7+1 = \mathbf{8}$ descripteurs et $6+3 = \mathbf{9}$ enregistrements.
2. `id`, `titre`, `studio`, `annee`, `genre`, `pegi`, `ventes` et `plateforme`.
3. L'`id 5` est *Red Dead Redemption 2*, de genre `action`.
4. Deux jeux pourraient porter le **même titre** (rééditions, versions) : le titre n'est pas forcément *unique*, contrairement à l'`id`.

▷ **Erreur fréquente.** Compter la ligne d'en-tête comme un enregistrement et trouver 10 enregistrements : la première ligne donne les noms des descripteurs, elle ne décrit aucun jeu. Autre piège : ajouter le nouveau descripteur aux enregistrements (ou les nouveaux jeux aux descripteurs) ; une colonne de plus ne crée aucune ligne.

Exercice 4 *Concevoir une table* ↑énoncé p. 140

▶ Corrigé.

1. Par exemple : `id_pret`, `titre_livre`, `nom_eleve`, `classe`, `date_pret`, `date_retour`.
2. L'identifiant est `id_pret`, un numéro **unique** pour chaque prêt. Le nom de l'élève ne convient pas : deux élèves peuvent porter le même nom, et un même élève emprunte plusieurs livres (son nom revient sur plusieurs lignes).
3. Par exemple :

id_pret	titre_livre	nom_eleve	classe	date_pret	date_retour
1	Le Petit Prince	Lina Rossi	2nde 3	02/10	16/10
2	Le Petit Prince	Hugo Martin	2nde 1	17/10	31/10

Le format CSV

Exercice 5 *Lire du CSV* ↑énoncé p. 140

▶ Corrigé.

1. Le séparateur est le **point-virgule** `;`.
2. **4** descripteurs (`titre`, `editeur`, `duree`, `age_min`) et **3** enregistrements.
3. Pour *Dixit*, `duree` vaut `30`.
4. Format **ouvert** (c'est du texte). Avantage : lisible par n'importe quel logiciel, durable, et **sans limite** de nombre de lignes.

Exercice 6 *Écrire du CSV* ↑énoncé p. 140

▶ Corrigé.

```
id,nom,pays
1,Nintendo,Japon
2,Mojang,Suede
```

(La 3e ligne est le 2e enregistrement. On écrit sans accent si l'encodage pose problème, ou en UTF-8.)

Exercice 7 *Un CSV mal formé* ↑énoncé p. 140

▶ Corrigé. Les trois erreurs :

- ligne *Hugo* : le séparateur est la **virgule** au lieu du point-virgule (le tableur verra une seule valeur) ;
- ligne *Zoe* : il manque une valeur (**deux valeurs** pour trois descripteurs) ;
- ligne *Tom* : l'âge est écrit **en lettres** ; ce n'est plus un nombre, le tableur ne pourra ni trier ni calculer correctement sur la colonne `age`.

Fichier corrigé (l'âge de Zoé est inventé) :

```
nom;ville;age
Lina;Monaco;15
Hugo;Nice;16
Zoe;Menton;15
Tom;Beausoleil;15
```

Rechercher, filtrer, trier

Exercice 8 *Rechercher* ↑énoncé p. 141

▶ Corrigé.

1. *Mario Kart 8* est de genre `course`.
2. Le jeu vendu à **300** millions est *Minecraft*.

Exercice 9 *Filtrer* ↑énoncé p. 141

▶ Corrigé.

1. genre `action` : *GTA V*, *Red Dead Redemption 2*.
2. `pegi` $\leqslant 7$: *Minecraft* (7), *Wii Sports* (7), *Mario Kart 8* (3), *Animal Crossing* (3).
3. après 2013 **et** plus de 50 M : *Mario Kart 8* (2014, 60) et *Red Dead Redemption 2* (2018, 55). *(Animal Crossing est écarté : 45 M seulement.)*

▷ **Erreur fréquente.** Retenir *GTA V* : il est sorti *en* 2013, pas *après* 2013. Autre piège : lire « ET » comme « OU », ce qui ferait aussi retenir *Minecraft*, *GTA V*, *Wii Sports* et *Animal Crossing*.

Exercice 10 *Trier* ↑énoncé p. 141

▶ Corrigé.

1. ventes décroissantes : Minecraft (300), GTA V (190), Wii Sports (83), Mario Kart 8 (60), Red Dead Redemption 2 (55), Animal Crossing (45).
2. ordre alphabétique des titres : Animal Crossing, GTA V, Mario Kart 8, Minecraft, Red Dead Redemption 2, Wii Sports. *(Mario Kart 8 passe avant Minecraft : à première lettre égale, on compare la deuxième, et « a » vient avant « i ».)*
3. `pegi` = 18, du plus récent au plus ancien : *Red Dead Redemption 2* (2018), *GTA V* (2013).

Exercice 11 *Calculer sur une colonne* ↑énoncé p. 141

▶ Corrigé.

1. **Deux** jeux ont un `pegi` de 18 : *GTA V* et *Red Dead Redemption 2*.
2. Jeux d'action : 190 et 55 ; moyenne = $(190 + 55) \div 2 = \mathbf{122{,}5}$ millions.

   ▷ **Erreur fréquente.** Diviser par 6 (le nombre total de jeux) au lieu de 2 : on doit d'abord *filtrer* les jeux d'action, puis faire la moyenne sur eux seuls.
3. Le jeu le plus ancien est sorti en **2006** : c'est *Wii Sports*.

Exercice 12 *Croiser deux tables* ↑énoncé p. 141

▶ Corrigé.

1. *Minecraft* a `studio` = `2` : édité par **Mojang**, un studio de **Suède**.
2. Studio japonais = **Nintendo** (`id 1`). Ses jeux : *Wii Sports*, *Mario Kart 8*, *Animal Crossing*.
3. Ventes cumulées par pays :
   - Suède (Mojang) : 300 ;
   - États-Unis (Rockstar) : $190 + 55 = 245$;
   - Japon (Nintendo) : $83 + 60 + 45 = 188$.

   Le pays au plus grand total est la **Suède** (300 M), à elle seule grâce à *Minecraft*.

Exercice 13 *Pourquoi deux tables ?* ↑énoncé p. 141

▶ Corrigé.

1. Nintendo a édité trois jeux : « Nintendo » et « Japon » seraient écrits **trois fois** chacun (une fois par jeu), au lieu d'une seule fois dans la table *studio*.
2. Rockstar Games a édité deux jeux. Avec une seule table : **deux** cases à modifier ; avec les deux tables : **une seule** case (dans la table *studio*).
3. Si l'on oublie une case, les données deviennent **incohérentes** : le même studio apparaît sous deux noms différents, et une recherche ou un total par studio sera faux.
4. Il suffit d'écrire son **numéro de studio**, 2 ; le nom et le pays se retrouvent en croisant avec la table *studio*. *Ranger chaque information à un seul endroit évite les répétitions et les erreurs.*

Au tableur

Exercice 14 *Fouiller au tableur* ↑énoncé p. 142

▶ Corrigé.

1. Tri par `ventes` décroissantes : *Minecraft* apparaît en tête.
2. Le filtre automatique sur `genre` = `action` n'affiche que *GTA V* et *Red Dead Redemption 2*.
3. Les ventes sont en colonne `G`, lignes 2 à 7 : `=SOMME(G2:G7)` donne **733** ; `=MOYENNE(G2:G7)` donne environ **122,2**.
4. Graphique en barres : une barre par jeu, dont la hauteur est le nombre de ventes (Minecraft largement en tête).

Exercice 15 *Lire des formules* ↑énoncé p. 142

► Corrigé.

1. `=MAX(G2:G7)` affiche **300** (les ventes de Minecraft) ; `=MIN(D2:D7)` affiche **2006** (l'année la plus ancienne) ; `=SOMME(G3:G4)` additionne les ventes des lignes 3 et 4 (GTA V et Wii Sports) : $190 + 83 = \mathbf{273}$.
2. Les `pegi` sont en colonne F : `=MOYENNE(F2:F7)`, qui donne $(7 + 18 + 7 + 3 + 18 + 3) \div 6 = 56 \div 6 \approx \mathbf{9{,}3}$.

Open data, vie privée et nuage

Exercice 16 *Les données ouvertes* ↑énoncé p. 142

► Corrigé.

1. L'*open data* désigne des données **publiques librement réutilisables**, par exemple sur `data.gouv.fr` (ou le portail open data de Monaco).
2. Le CSV est un **format ouvert** (texte) : lisible par tous les logiciels, durable et sans limite de lignes, alors que le `.xlsx` est un format de tableur bien plus complexe, conçu pour un type de logiciel (et l'ancien `.xls` était même **propriétaire**, avec une limite de 65 536 lignes).

Exercice 17 *Mes données, mes droits* ↑énoncé p. 142

► Corrigé.

1. Le **RGPD** protège les **données personnelles**. Droits : accéder à ses données, les corriger, les faire **effacer** (droit à l'oubli), donner (ou refuser) son consentement.
2. Nuage : **avantage** = accès partout et partage facile ; **inconvénient** = données chez un tiers (souvent à l'étranger), besoin d'une connexion, question de confidentialité.
3. Les « 3 V » : **Volume**, **Vélocité**, **Variété**.
4. À Monaco, c'est la **loi n° 1.565** du 3 décembre 2024 relative à la protection des données personnelles (mêmes grands droits que le RGPD) ; l'autorité de contrôle est l'**APDP** (Autorité de protection des données personnelles), qui a succédé à la CCIN. *(En France : RGPD et loi Informatique et libertés, autorité CNIL.)*

Exercice 18 *Défi — data-détective sur l'open data* ↑énoncé p. 142

► Corrigé. Réponse ouverte : on attend une **question** claire, l'**opération** réalisée (tri ou filtre) et la **réponse** lue dans les données. *(Exemple : « Quelle commune a la plus forte population ? » → tri décroissant sur la colonne population → on lit la première ligne.)*

Critiquer une réponse d'IA

Exercice 19 *Le jeu le plus récent, selon un assistant* ↑énoncé p. 143

► Corrigé.

1. Non. Le croisement est bien fait (studio 2 = Mojang, Suède) et Minecraft est bien le premier jeu *par ventes*, mais sa valeur pour `annee` est 2011 : quatre jeux de la table sont plus récents.
2. L'erreur : le tri porte sur la **mauvaise colonne**. « Le plus récent » se lit sur le descripteur `annee` (tri décroissant), pas sur `ventes`. Bonne réponse : *Animal Crossing* (2020), studio 1 = **Nintendo**, au Japon.
3. Relire la **question** et vérifier que le descripteur trié est celui qu'elle nomme (« récent » → `annee`) ; puis contrôler la réponse sur la table : l'année de Minecraft (2011) n'est visiblement pas la plus grande.

Python au service du thème

Exercice 20 *Python au service du thème — filtrer avec une fonction* ↑énoncé p. 143

▶ **Corrigé. 1.** Le critère relie deux comparaisons par `and` ; la fonction renvoie directement le booléen obtenu :

```
def garder(genre, annee):
    return genre == "action" and annee > 2015
```

2. GTA V est un jeu d'action, mais sorti en 2013 : une des deux conditions est fausse. Red Dead Redemption 2 vérifie les deux. Animal Crossing est récent, mais ce n'est pas un jeu d'action. Le programme affiche :

```
False
True
False
```

On retrouve le résultat du cours : seul *Red Dead Redemption 2* est gardé. Avec `or`, il suffit qu'*une* des deux conditions soit vraie : les trois lignes affichent `True` (GTA V est un jeu d'action ; Animal Crossing est sorti après 2015). Sur toute la table, le filtre « `action` **OU** après 2015 » garde *GTA V*, *Red Dead Redemption 2* et *Animal Crossing*.

3. Même méthode, avec les descripteurs `annee` et `ventes` :

```
def succes(annee, ventes):
    return annee > 2013 and ventes > 50

print(succes(2014, 60))    # Mario Kart 8 : True
print(succes(2013, 190))   # GTA V : False
print(succes(2020, 45))    # Animal Crossing : False
```

Comme à l'exercice « Filtrer » : GTA V est sorti *en* 2013, pas après, et Animal Crossing s'est vendu à moins de 50 millions.

▷ **Erreur fréquente.** Écrire `genre = "action"` avec un seul signe égal : `=` range une valeur dans une variable, il ne compare pas, et Python signale une erreur de syntaxe (`SyntaxError`). Pour comparer, il faut le double égal `==`.

CORRIGÉS : DÉBAT

DÉBAT Faut-il ouvrir à tous le plus possible de données publiques, même quand elles concernent des personnes ?

↑énoncé p. 150

À lire après le débat. Ces éléments de réponse servent à vérifier et à compléter votre trace écrite : les lire avant le débat lui ôterait son intérêt.

Notions du cours mobilisées

- **Donnée**, **donnée personnelle** (identification directe ou indirecte), **métadonnée**.
- **Table** : descripteurs (colonnes), enregistrements (lignes) ; **format CSV**, **format ouvert**.
- **Rechercher, filtrer, trier**, et surtout **croiser deux tables** : c'est le mécanisme de la réidentification (doc. 4).
- **Open data** (`data.gouv.fr`, portail de Monaco) et pouvoir citoyen, data-journalisme.
- **RGPD** (consentement, but précis, droits) ; **loi n° 1.565** et **APDP** à Monaco ; **big data**.

Arguments attendus

POUR (oui) : ouvrir le plus possible	CONTRE (non) : limiter fortement
• **Transparence démocratique** : contrôler l'usage de l'argent public (doc. 2). • **Utilité concrète** : transports, santé, environnement, recherche (doc. 2). • **Données payées par les contribuables** : elles appartiennent à tous (doc. 2). • **C'est la loi** en France depuis 2016, avec retrait des données personnelles (doc. 1). • **On peut protéger** : regrouper les cas rares (INSEE), interdire la réidentification (doc. 3 et 4). • **Format ouvert** (CSV) : tout le monde peut vérifier et réutiliser, pas seulement les grandes entreprises (cours).	• **Réidentification** : trois informations suffisent à retrouver une personne (doc. 4) ; croiser des tables est facile (cours). • **Curiosité mal placée** : le prix payé par son voisin (doc. 3). • **Une fois publiée, une table est copiée** : on ne peut plus la retirer (permanence, comme sur les réseaux sociaux). • **Une interdiction d'usage ne garantit rien** : qui contrôle ceux qui téléchargent (doc. 3) ? • **Pas de consentement** des personnes concernées : elles n'ont pas choisi d'être dans la table. • **Big data** : plus il y a de tables publiques, plus les croisements possibles sont nombreux (cours).

Arguments de l'autre camp souvent jugés solides dans la trace écrite : un élève POUR retient souvent « retirer le nom ne suffit pas à rendre une table anonyme » ; un élève CONTRE, « les données ouvertes ont rendu de vrais services pendant l'épidémie ».

Questions de relance (pour le modérateur)

1. Dans la base DVF (doc. 3), quelle colonne faudrait-il retirer ou rendre moins précise pour mieux protéger les personnes ? Que perd-on alors ?
2. Avec quelles colonnes d'une autre table pourrait-on « croiser » une table sans nom pour retrouver quelqu'un (doc. 4) ?
3. Qui doit décider de ce qui est publié : l'administration, la loi, les personnes concernées ?
4. Les résultats d'une élection par bureau de vote sont publics : pourquoi est-ce important ? Est-ce une donnée personnelle ?

Synthèse possible

L'**open data** met à la disposition de tous des tables publiques (souvent en CSV) : transparence, recherche, nouveaux services. En France, l'ouverture est la règle depuis 2016, à condition de retirer les données personnelles. Mais **retirer le nom ne suffit pas** : en **croisant** plusieurs tables, quelques descripteurs (code postal, date de naissance, sexe, adresse) suffisent souvent à retrouver une personne. Une donnée pseudonymisée reste une donnée personnelle (RGPD ; loi n° 1.565 à Monaco). Ouvrir les données suppose donc de choisir **quelles colonnes publier, avec quelle précision**, et d'encadrer leur réutilisation.

Réponses des questions de la fiche de révision

↑ questions p. 152

Se tester

1. ; et 3 descripteurs (§ III).
2. GTA V ; Wii Sports, Minecraft, GTA V, Mario Kart 8 (§ IV).
3. Elle identifie indirectement une personne (§ I).
4. Par exemple la date, le modèle d'appareil, la position GPS, la taille (§ I.A).

Et avant ?

1. Le CSS (ch. 4, § V).
2. Accès, rectification, effacement (ch. 2, § II).

CORRIGÉS DU CHAPITRE 6

Internet

6

CORRIGÉS DES EXERCICES

Éléments de réponse aux activités du cours

Activité 1 — Le routage débranché

Activité d'observation. Ce qu'il faut en retenir : aucun « routeur » ne connaît tout le réseau, chacun ne connaît que ses **voisins directs** et choisit seulement la **prochaine étape** ; de proche en proche, le paquet arrive quand même. Quand un routeur disparaît, ses voisins font passer le paquet par un **autre chemin** : c'est ce qui rend Internet résistant aux pannes.

Activité 2 — Retrouver une adresse IP

L'adresse obtenue dépend du moment et du lieu : on note celle qui s'affiche (par exemple `141.94.212.184` pour `wikipedia.fr` le 28/09/2026) ; de même pour le site du lycée. **Oui**, deux noms différents peuvent mener à la même adresse IP : un même serveur peut héberger plusieurs sites, et un site a souvent plusieurs noms (avec ou sans `www`, par exemple). Le DNS associe un nom à une adresse, mais rien n'interdit que plusieurs noms pointent vers la même.

Un peu d'histoire

Exercice 1 *Remettre dans l'ordre* ↑énoncé p. 162

▶ Corrigé. premier message ARPANET (**1969**) → protocole TCP/IP (**1974**) → naissance d'Internet, bascule TCP/IP (**1983**) → invention du Web (**1989–1991**).

Exercice 2 *« LO… »* ↑énoncé p. 162

▶ Corrigé. On voulait transmettre `LOGIN`. La machine distante a **planté** après la 2e lettre ; les deux premières lettres reçues furent donc **« LO »**.

Les paquets

Exercice 3 *Pourquoi découper ?* ↑énoncé p. 162

▶ Corrigé.

1. Un **paquet** est un petit bloc de taille limitée contenant une partie du message ; son étiquette porte l'**adresse de l'expéditeur**, l'**adresse du destinataire** et un **numéro**.
2. Le numéro permet de **remettre les paquets dans l'ordre** à l'arrivée.

Exercice 4 *Arrivés dans le désordre* ↑énoncé p. 162

► Corrigé.

1. Ce n'est pas un problème : grâce aux **numéros**, la machine réordonne les paquets (1, 2, 3, 4) avant de reconstituer le message.
2. C'est **TCP** qui remet dans l'ordre.

Exercice 5 *Découper une photo* ↑énoncé p. 162

► Corrigé.

1. $3\,000\,000 \div 1500 = \mathbf{2000}$ paquets.
2. $2000 \times 40 = \mathbf{80\,000}$ octets d'étiquettes, soit $80\,000 \div 3\,000\,000 \approx 2{,}7\,\%$ de la photo : c'est **peu**, le prix à payer pour que chaque paquet puisse voyager seul.
3. 1 % de 2000 : **20** paquets à renvoyer. C'est **TCP** qui constate qu'ils manquent et les redemande.

IP et TCP

Exercice 6 *Qui fait quoi ?* ↑énoncé p. 163

► Corrigé. a) **IP** ; b) **TCP** ; c) **IP** ; d) **TCP**.

▷ **Erreur fréquente.** Confier à IP la remise dans l'ordre (d) : IP achemine chaque paquet séparément, sans s'occuper des autres. C'est TCP, à l'arrivée, qui vérifie que tout est là et remet les paquets dans l'ordre.

Exercice 7 *Fiable mais pas à l'heure* ↑énoncé p. 163

► Corrigé. TCP **garantit** que tout paquet finira par arriver (il redemande les manquants), **mais** sans délai garanti. En visioconférence, un paquet en retard arrive « trop tard » (le direct a continué) : d'où saccades et coupures.

Le routage

Exercice 8 *Suivre un paquet* ↑énoncé p. 163

► Corrigé.

1. Deux chemins : A – R1 – R2 – R4 – B et A – R1 – R3 – R4 – B.
2. Non : R1 ne connaît que ses **voisins**. Il décide seulement de la **prochaine étape** (vers R2 ou R3).
3. Oui : le paquet passe par l'**autre chemin** (R1 – R3 – R4). C'est la **robustesse** du routage.

Exercice 9 *Le compteur TTL* ↑énoncé p. 163

► Corrigé.

1. Il évite qu'un paquet **tourne en rond** indéfiniment et encombre le réseau. À 0, le paquet est **détruit**.
2. Non : après 3 routeurs le compteur atteint 0 et le paquet est détruit avant d'arriver. **TCP** constate l'absence et **redemande** le paquet.

Exercice 10 *Le chemin le plus court* ↑énoncé p. 164

▶ Corrigé.

1. PC-A – R1 – R2 – R6 – PC-B : il traverse **3** routeurs (aucun chemin n'en traverse moins : il faut au moins un routeur entre R1 et R6, qui ne sont pas reliés).
2. Après la panne, il faut traverser au moins **4** routeurs (R1 et R6 n'ont plus aucun voisin commun). Trois chemins réalisent ce minimum : R1 – R2 – R4 – R6 ; R1 – R2 – R5 – R6 ; R1 – R3 – R5 – R6.
3. Même quand un câble tombe en panne, il existe encore plusieurs chemins : les routeurs envoient les paquets par une autre route, un peu plus longue, et la communication continue. C'est le principe d'Internet : **pas de point de passage unique**.

Adresses IP et DNS

Exercice 11 *Est-ce une adresse IPv4 valide ?* ↑énoncé p. 164

▶ Corrigé.

- `192.168.0.1` : **valide** (4 nombres entre 0 et 255).
- `10.0.300.5` : **invalide** ($300 > 255$).
- `172.16.4` : **invalide** (seulement 3 nombres).
- `8.8.8.8` : **valide**.

▷ **Erreur fréquente.** Déclarer `172.16.4` valide parce que chacun de ses nombres est entre 0 et 255 : une adresse IPv4 compte *exactement quatre* nombres. Les deux conditions doivent être vérifiées.

Exercice 12 *Combien d'adresses IPv4 ?* ↑énoncé p. 164

▶ Corrigé.

1. Chaque nombre prend **256** valeurs (de 0 à 255). Nombre d'adresses : $256 \times 256 \times 256 \times 256 = 256^4 =$ **4 294 967 296**, soit environ **4 milliards**.

   ▷ **Erreur fréquente.** Calculer $256 \times 4 = 1\,024$: les quatre nombres se choisissent indépendamment, il faut donc *multiplier* 256 par lui-même quatre fois (256^4).
2. **Non** : il y a moins d'adresses que d'humains (8 milliards), alors que chacun peut avoir plusieurs appareils connectés (sans compter les objets connectés et les serveurs). Solution : **IPv6**, aux adresses bien plus longues (des milliards de milliards d'adresses).

Exercice 13 *Le rôle du DNS* ↑énoncé p. 164

▶ Corrigé.

1. Le DNS **traduit** une adresse **symbolique** en adresse **IP** : de `wikipedia.fr` **vers** son adresse IP, par exemple `141.94.212.184` (le 28/09/2026) ; toute adresse obtenue est acceptée.
2. Parce qu'un nom en toutes lettres est **facile à retenir**, contrairement à une suite de chiffres.

Exercice 14 *Sur machine* ↑énoncé p. 164

▶ Corrigé. *Les adresses dépendent du poste, du moment et du lieu : on note celles qui s'affichent.* Chaque **nom** de site est associé à une **adresse IP** numérique : c'est le DNS qui fait la traduction. En général, `ping` et `nslookup` donnent la **même adresse**, car `ping` commence lui aussi par demander l'adresse au DNS avant de tester la liaison. Pour un très gros site, réparti sur plusieurs serveurs, on peut obtenir des adresses différentes d'une fois sur l'autre. Si un serveur ne répond pas au `ping` (certains l'ignorent), son adresse s'affiche quand même : la traduction par le DNS a bien eu lieu.

▷ **Erreur fréquente.** Relever, dans la réponse de `nslookup`, la première adresse affichée : c'est celle du *serveur DNS* interrogé (souvent la box, en `192.168...`), pas celle du site. L'adresse du site vient après la ligne qui rappelle son nom.

Exercice 15 *Quand le DNS ne répond plus* ↑énoncé p. 165

► Corrigé.

1. La page **ne s'affiche pas** (message d'erreur) : le navigateur ne peut pas obtenir l'adresse IP du serveur, donc il ne sait pas à qui envoyer sa requête.
2. **Oui, en principe** : avec l'adresse IP, le navigateur n'a plus besoin du DNS ; la requête part directement vers le serveur (le reste du réseau fonctionne).
3. Comme un annuaire, le DNS fait correspondre un **nom** facile à retenir à un **numéro** (l'adresse IP) ; sans lui, il faudrait connaître les numéros par cœur.

Réseaux physiques, P2P et trafic

Exercice 16 *Filaire ou sans fil ?* ↑énoncé p. 165

► Corrigé.

- **Filaire** : fibre optique, Ethernet, ADSL.
- **Sans fil** : Wi-Fi, 4G, Bluetooth.
- Plus gros débit : la **fibre optique**.

Exercice 17 *Client/serveur ou pair-à-pair ?* ↑énoncé p. 165

► Corrigé.

1. Client/serveur : un **serveur central** distribue à des clients. Pair-à-pair : **pas de serveur central**, chaque machine est à la fois émetteur et récepteur.
2. Intérêt : robustesse / partage efficace de gros fichiers. Usage illicite : **téléchargement illégal** d'œuvres protégées.

Exercice 18 *Ordres de grandeur* ↑énoncé p. 165

► Corrigé.

1. 1 To $= 10^{12}$ o $= 10^3$ Go $=$ **1000** Go. 1 Po $= 10^{15}$ o $= 10^6$ Go $=$ **1 000 000** Go.
2. 7 Zo $= 7 \times 10^{21}$ octets ; le préfixe **Zo** (zetta) vaut 10^{21}.

Exercice 19 *Le temps d'un téléchargement* ↑énoncé p. 165

► Corrigé.

1. $4 \times 10^9 \times 8 =$ **32×10^9** bits (32 milliards de bits).
2. Fibre : $32 \times 10^9 \div (100 \times 10^6) =$ **320 s**, soit environ **5 min 20 s**. ADSL : $32 \times 10^9 \div (8 \times 10^6) =$ **4000 s**, soit $4000 \div 60 \approx$ **66,7 min**, environ **1 h 07 min** (plus de douze fois plus long).
3. En plus du film, il faut transmettre les **étiquettes** des paquets, renvoyer les paquets **perdus**, et le débit réel est souvent inférieur au débit annoncé (réseau partagé avec d'autres utilisateurs).

Critiquer une réponse d'IA

Exercice 20 *Un paquet perdu, selon un assistant* ↑énoncé p. 165

▶ **Corrigé.**

1. Non. Paquets, routeurs, TTL et « aucune garantie de temps » sont conformes au cours. Mais le cours attribue à **TCP** le rôle de « vérifier que *tous* les paquets arrivent, **redemander** ceux qui manquent, et les **remettre dans l'ordre** », et à IP celui d'**adresser** et d'**acheminer** les paquets.
2. L'erreur : « c'est le protocole IP qui s'en aperçoit et redemande le paquet ». IP ne fait qu'acheminer ; c'est **TCP** qui s'aperçoit de la perte, redemande le paquet et remet les paquets dans l'ordre. Correction : remplacer « IP » par « TCP » dans cette phrase.
3. Reprendre les quatre tâches de l'exercice 6 (« Qui fait quoi ? ») et vérifier que chacune est attribuée au bon protocole : « redemander un paquet perdu » est une tâche de TCP.

Python au service du thème

Exercice 21 *Python au service du thème — paquets et durée d'envoi* ↑énoncé p. 166

▶ **Corrigé.** **1.** $4500 = 3 \times 1500$: **3** paquets pleins. 4501 octets : 3 paquets pleins et il reste 1 octet, qui occupe un paquet de plus, soit **4** paquets. 700 octets tiennent dans **1** seul paquet, incomplet.

2. `taille // 1500` compte les morceaux complets ; s'il reste des octets (`taille % 1500` n'est pas nul), on ajoute un paquet pour le dernier morceau.

```
def nb_paquets(taille):
    n = taille // 1500
    if taille % 1500 != 0:
        n = n + 1
    return n

print(nb_paquets(4500))   # 3
print(nb_paquets(4501))   # 4
print(nb_paquets(700))    # 1
```

3. On ajoute les 40 octets d'étiquette de chaque paquet, on convertit en bits ($\times 8$), puis on divise par le nombre de bits envoyés chaque seconde :

```
def duree(taille, debit):
    total = taille + 40 * nb_paquets(taille)   # octets envoyes, etiquettes comprises
    return total * 8 / (debit * 1000000)

print(duree(4500000, 20))   # 1.848
```

La photo est découpée en 3000 paquets, soit $4\,500\,000 + 40 \times 3000 = 4\,620\,000$ octets envoyés, ou 36 960 000 bits : à 20 Mbit/s, l'envoi dure **1,848 s**, un peu moins de deux secondes (sans compter les paquets perdus à renvoyer).

▷ **Erreur fréquente.** Écrire seulement `return taille // 1500` : `nb_paquets(4501)` renvoie alors 3 au lieu de 4, et le dernier octet n'est jamais envoyé. La division entière « oublie » le reste ; c'est le test sur `taille % 1500` qui le rattrape.

CORRIGÉS : TP ET DÉBAT

TP Construire un réseau et faire circuler un paquet

↑ énoncé p. 168

Partie 1 — Un réseau local

Exercice 1 *Construire et adresser*

► Corrigé.

1. Le **switch** relie entre elles les machines d'un *même* réseau local et aiguille les données vers la bonne machine. *(Toutes les IP commencent par* `192.168.0` *: même réseau local.)*
2. **Non**, Filius ne propose aucune adresse IP pour le switch : il n'en a pas besoin (il ne fait que distribuer dans le réseau local, il ne « raisonne » pas sur les adresses IP comme un routeur).

Exercice 2 *Pinguer et observer*

► Corrigé.

1. Le `ping` **réussit**. Trajet : PC-A → Switch → PC-B, et la réponse revient par le même chemin.
2. On voit **arriver les paquets** (requête « echo » puis réponse). C'est l'échange « requête echo » / « réponse echo » du `ping`.
3. `ping 192.168.0.9` : **échec** (« délai dépassé » / pas de réponse) car aucune machine ne porte cette adresse dans le réseau.

Partie 2 — Deux réseaux reliés par un routeur

Exercice 3 *Relier deux réseaux locaux*

► Corrigé. Construction conforme au schéma de l'énoncé : PC-A et S1 dans le réseau `192.168.0.0`, PC-B et S2 dans le réseau `192.168.1.0`, et le routeur avec ses deux interfaces `192.168.0.254` et `192.168.1.254`.

Exercice 4 *La passerelle, indispensable pour sortir*

► Corrigé.

1. PC-A (`192.168.0.1`) et PC-B (`192.168.1.1`) ne sont **pas dans le même réseau** (`.0.` vs `.1.`). Sans passerelle, PC-A ne sait pas *par où sortir* de son réseau local → « destination non joignable ».
2. Avec les passerelles réglées, le `ping` réussit. Trajet : **PC-A** → **S1** → **Routeur** → **S2** → **PC-B** (et retour). Le routeur fait le lien entre les deux réseaux.
3. `traceroute` montre **deux étapes** : d'abord le **routeur** (`192.168.0.254`), puis **PC-B**. L'étape intermédiaire est le routeur (la passerelle).

Pour aller plus loin : couper le câble PC-B–S2 en pleine simulation et relancer le ping → échec ; le rebrancher → succès. Illustre une ***panne*** *: ce câble est le seul chemin vers PC-B, donc plus rien ne passe. La* ***robustesse*** *d'Internet viendrait d'un* ***second chemin*** *entre routeurs, par lequel les paquets pourraient contourner la panne.*

DÉBAT Les fournisseurs d'accès à Internet doivent-ils traiter tous les paquets de la même façon ?

↑ énoncé p. 170

À lire après le débat. Ces éléments de réponse servent à vérifier et à compléter votre trace écrite : les lire avant le débat lui ôterait son intérêt.

Notions du cours mobilisées

- **Paquet**, **routeur**, **routage** de proche en proche, sans chef d'orchestre central.
- **IP** achemine, **TCP** fiabilise mais *sans garantie de temps* : d'où les saccades du streaming (argument pour la priorité au temps réel).
- **Ordres de grandeur du trafic** (zetta-octets par an) : la congestion est un vrai problème.
- **Neutralité du Net** et **déni de service** (cadre « Deux enjeux de société »).
- Indépendance d'Internet vis-à-vis du réseau physique : qui paie la fibre, les antennes 4G/5G ?
- **Internet ≠ Web** : la neutralité concerne le réseau, pas le contenu des sites.

Arguments attendus

POUR (oui) : neutralité stricte	CONTRE (non) : priorités ou offres différenciées
• **Égalité des chances** : une petite application ou un site naissant doit arriver aussi vite qu'un géant (doc. 2). • **Concurrence et innovation** : Internet s'est développé sans « péage » ; les « voies rapides » payantes favoriseraient ceux qui ont déjà de l'argent (doc. 4). • **Liberté d'accès à l'information** : le FAI ne doit pas choisir ce que l'abonné voit ou utilise. • **C'est la loi en Europe**, confirmée par la justice (doc. 1 et 2) ; la Californie a recréé une règle quand la règle fédérale a disparu (doc. 4). • **Le client paie déjà** son abonnement pour accéder à tout Internet (doc. 3). • La gestion technique « raisonnable » est *déjà* permise sans casser la neutralité (doc. 1).	• **Tous les paquets n'ont pas les mêmes besoins** : appel vidéo, téléconsultation, jeu en ligne supportent mal le retard ; TCP ne garantit pas le temps (cours, doc. 1). • **Congestion** : le trafic explose (cours) ; donner la priorité permet de mieux utiliser le réseau. • **Qui paie les réseaux ?** Les grandes plateformes produisent beaucoup de trafic sans financer les antennes et la fibre (doc. 3). • **Offres moins chères** pour le client : le « tarif zéro » était une bonne affaire pour certains abonnés (doc. 2). • **L'exemple américain** : l'Internet a continué à se développer sans règle fédérale (doc. 4). • La neutralité est un **principe européen** : Monaco n'est pas tenu par ce règlement (doc. 1), chaque pays peut faire un autre choix.

Arguments de l'autre camp souvent jugés solides dans la trace écrite : un élève POUR retient souvent « certains usages (appel vidéo, secours) ont vraiment besoin de passer vite » ; un élève CONTRE, « une petite application n'a aucune chance si les géants achètent la priorité ».

Questions de relance (pour le modérateur)

1. Si un hôpital fait une consultation à distance, doit-elle passer avant une vidéo de divertissement ? Est-ce contraire à la neutralité (doc. 1) ?
2. Qui devrait payer les nouvelles antennes et la fibre : l'abonné, les plateformes, l'État ?
3. Une option « réseau social illimité » à 2 € : bonne affaire pour vous, mais pour l'application concurrente ?
4. Que se passerait-il si votre FAI ralentissait le site d'un concurrent de son propre service de vidéo ?

Synthèse possible

La **neutralité du Net** est le principe selon lequel les FAI traitent tous les paquets de la même façon, sans favoriser un contenu, un service ou une entreprise. Dans l'Union européenne, c'est une **obligation légale** depuis 2016 (règlement 2015/2120), contrôlée en France par l'Arcep, avec des exceptions techniques limitées (congestion, catégories de trafic aux besoins différents). Le débat porte sur deux questions : **qui paie** les réseaux dont le trafic explose, et **qui décide** de ce qui passe en premier. Les choix diffèrent selon les pays (États-Unis, Monaco hors UE).

Réponses des questions de la fiche de révision

↑ questions p. 172

Se tester

1. Non : $300 > 255$ (§ V.A).
2. Il le redemande (§ III).
3. Détruit, pour ne pas tourner en rond (§ IV).
4. $25 \times 8 = 200$ Mbit ; $200/50 = 4$ s (§ VI).
5. Robuste ; téléchargement illégal (§ VII).

Et avant ?

1. L'en-tête : les descripteurs (ch. 5, § III).
2. $100 \times 100 \times 3 = 30\,000$ octets (ch. 3, § III).

CORRIGÉS DU CHAPITRE 7

7 Localisation, cartographie et mobilité

CORRIGÉS DES EXERCICES

Éléments de réponse aux activités du cours

Réponses aux questions fermées ; pour les questions ouvertes ou les relevés sur machine, des pistes attendues.

Activité 1 — Trilatération à la règle et au compas

Corrigée dans le cours, juste après l'énoncé : le randonneur est au point (**4** ; **3**) (à 5 km de A, 3 km de B et 4 km de C).

Activité 2 — Sur une carte en ligne

1. et **2.** Relevés personnels (pour un lycée de Monaco, la latitude est proche de 43,7° N et la longitude de 7,4° E). **3.** La distance « à vol d'oiseau » est **plus courte** : c'est la ligne droite, alors que l'itinéraire suit les routes, avec leurs virages, les sens interdits et le relief (à Monaco, beaucoup de rues en lacets).

Activité 3 — Enquête sur mes traces

1. Réponse personnelle (par exemple : cartes, météo, réseau social, appareil photo). **2.** Recoupées, les positions enregistrées révèlent le **domicile**, le lycée, les trajets et les **habitudes** ; stockées chez une entreprise, elles peuvent être revendues, piratées ou servir à suivre quelqu'un. **3.** Par exemple : couper la localisation quand elle est inutile ; n'autoriser une application que « pendant l'utilisation » ; effacer l'historique des positions ; retirer les métadonnées d'une photo avant de la publier.

Activité 4 — Localisation et sobriété

1. Les trois : le **terminal** (le téléphone calcule sa position à partir des signaux des satellites et affiche la carte), les **réseaux** (données mobiles ou Wi-Fi) et les **centres de données** (fonds de carte, calcul de l'itinéraire, état du trafic). **2.** En arrière-plan, le récepteur GPS et les liaisons radio fonctionnent sans arrêt : la **batterie** se vide plus vite (et une batterie usée trop tôt pousse à changer de téléphone) ; dans le même temps, les positions sont enregistrées et envoyées en continu, ce qui grossit les **traces** sur la vie privée. **3.** Les deux gestes cités dans l'énoncé conviennent ; autres exemples : couper la localisation quand on ne s'en sert pas, désactiver l'historique des positions.

Latitude et longitude

Exercice 1 *Nord ou Sud, Est ou Ouest ?* ↑énoncé p. 183

► Corrigé.

1. Sur l'équateur, la latitude vaut 0° ; sur le méridien de Greenwich, la longitude vaut 0°.
2. Latitude : de −90° (pôle Sud) à +90° (pôle Nord). Longitude : de −180° à +180°.
3. 43,7384 est la **latitude** (Nord) ; 7,4246 est la **longitude** (Est).

Exercice 2 *Trois villes sur le globe* ↑énoncé p. 183

► Corrigé.

1. **Rio de Janeiro** : sa latitude est **négative** (−22,91), donc au sud de l'équateur.
2. **Rio de Janeiro** : sa longitude est négative (−43,17), donc à l'ouest de Greenwich.
3. Du nord au sud (latitude décroissante) : **Monaco** (43,74), **Tokyo** (35,68), **Rio de Janeiro** (−22,91).

Exercice 3 *Degrés, minutes et degrés décimaux* ↑énoncé p. 183

► Corrigé.

1. $45°30' = 45 + \frac{30}{60} = 45{,}5°$; $12°15' = 12 + \frac{15}{60} = 12{,}25°$; $43°44{,}304' = 43 + \frac{44{,}304}{60} \approx 43{,}7384°$.
2. $6{,}25° = 6° + 0{,}25 \times 60' = 6°15'$; $7{,}5° = 7°30'$.

▷ **Erreur fréquente.** Écrire $45°\,30' = 45{,}30°$: les minutes sont des *soixantièmes* de degré, pas des centièmes. 30′, c'est la moitié de 60′, donc 0,5°.

Exercice 4 *Des degrés aux kilomètres* ↑énoncé p. 183

► Corrigé.

1. Écart de latitude : $44{,}24 - 43{,}74 = 0{,}5°$, soit $0{,}5 \times 111 \approx$ **55,5 km** (la longitude ne change pas : il marche plein nord).
2. $1' = \frac{1}{60}$ de degré : $111 \div 60 \approx$ **1,85 km** (c'est le « mille nautique » des marins).
3. $0{,}0001 \times 111 = 0{,}0111$ km $\approx$ **11 m**. C'est assez précis pour repérer une maison ou une entrée d'immeuble (et c'est l'ordre de grandeur de la précision réelle d'un GPS de téléphone, quelques mètres).

Le GPS et la trilatération

Exercice 5 *Du temps à la distance* ↑énoncé p. 184

► Corrigé.

1. $d = c \times t = 300\,000 \times 0{,}06 =$ **18 000** km.
2. Il doit mesurer le **temps de trajet** : comparer l'heure d'*envoi* (contenue dans le message) à l'heure de *réception* — d'où le besoin d'une horloge précise.

Exercice 6 *La précision des horloges* ↑énoncé p. 184

► Corrigé.

1. En 0,001 s : $300\,000 \times 0{,}001 =$ **300 km**. En 0,000 001 s : $300\,000 \times 0{,}000\,001 = 0{,}3$ km, soit **300 m**.
2. Une avance d'un millième de seconde fausse le temps de trajet d'autant : l'erreur sur la distance est de **300 km** — la position serait complètement fausse.
3. Une toute petite erreur de temps donne une énorme erreur de distance : il faut des horloges **extrêmement précises** (atomiques) dans les satellites. L'horloge d'un téléphone, elle, n'est pas assez précise : le **quatrième satellite** sert à calculer et corriger son erreur.

Exercice 7 *Combien de satellites ?* ↑énoncé p. 184

▶ Corrigé.

1. Avec un seul satellite, on sait seulement qu'on est sur une **sphère** (à distance fixée) autour de lui.
2. Deux cercles se coupent en deux points ; le **troisième** lève l'ambiguïté et laisse **un** point.
3. Le 4e satellite sert à **corriger l'erreur de l'horloge** du récepteur.

Exercice 8 *Trilatération sur quadrillage* ↑énoncé p. 184

▶ Corrigé. Le point commun aux trois cercles est $(\mathbf{4}\,;\,\mathbf{3})$. *Vérification :* distance à $\mathrm{A}(0;0) = \sqrt{4^2+3^2} = 5$; à $\mathrm{C}(0;6) = \sqrt{4^2+3^2} = 5$; à $\mathrm{B}(6;0) = \sqrt{(-2)^2+3^2} = \sqrt{13} \approx 3{,}6$. ✓

Décoder une trame NMEA

Exercice 9 *Que dit le GPS ?* ↑énoncé p. 184

▶ Corrigé.

1. `142012` → **14 h 20 min 12 s** (UTC).
2. Latitude `4342.054,N` $= 43°42{,}054'$ N $= 43 + \frac{42{,}054}{60} \approx$ **43,7009°** N. Longitude `00716.098,E` $= 7°16{,}098'$ E $= 7 + \frac{16{,}098}{60} \approx$ **7,2683°** E.
3. `06` : **6** satellites utilisés.
4. Ces coordonnées correspondent à **Nice**.

▷ **Erreur fréquente.** Lire `4342.054` comme 43,42054° : après les deux chiffres des degrés viennent des *minutes* (42,054′), qu'il faut diviser par 60.

Cartes numériques et itinéraires

Exercice 10 *Le plus court chemin* ↑énoncé p. 185

▶ Corrigé.

1. Par exemple A–B–D–E $= 3+7+4 = 14$ km ; A–C–F–E $= 2+5+8 = 15$ km.
2. Il n'y a que quatre chemins de A à E sans repasser par un même sommet : A–B–D–E (14 km), A–C–F–E (15 km), A–C–F–D–E ($2+5+2+4 = 13$ km) et A–B–D–F–E ($3+7+2+8 = 20$ km). Le plus court est **A–C–F–D–E**, de longueur **13 km**. Ce n'est **pas** celui qui passe par le moins de carrefours (A–B–D–E n'a que deux étapes intermédiaires, mais mesure 14 km) : seule la **somme des poids** compte.

   ▷ **Erreur fréquente.** S'arrêter aux deux chemins de la question 1 sans chercher les autres : le plus court (A–C–F–D–E) fait un détour par F et D. Il faut lister *tous* les chemins avant de conclure.
3. Une application GPS modélise de même le réseau routier par un **graphe** (carrefours = sommets, routes pondérées) et cherche le trajet de **somme minimale**.

Exercice 11 *Le chemin le plus rapide* ↑énoncé p. 185

▶ Corrigé.

1. A–B–D $= 30+10 = 40$ km ; A–C–D $= 12+8 = 20$ km : le plus **court** est **A–C–D**.
2. A–B–D : $30 \div 120 = 0{,}25$ h $= 15$ min, puis $10 \div 60 \approx 0{,}167$ h $= 10$ min, soit **25 min**. A–C–D : $12 \div 40 = 0{,}3$ h $= 18$ min, puis $8 \div 40 = 0{,}2$ h $= 12$ min, soit **30 min**. Le plus **rapide** est **A–B–D**, bien qu'il soit deux fois plus long.
3. Avec le bouchon : A–B dure $30 \div 40 = 0{,}75$ h $= 45$ min, donc A–B–D dure 55 min. L'application conseille alors **A–C–D** (30 min). Elle cherche le chemin de **durée** minimale, recalculée avec le trafic en temps réel : le meilleur chemin dépend du « poids » choisi pour les arêtes (distance ou durée).

Exercice 12 *Sur une vraie carte* ↑énoncé p. 185

► Corrigé. Relevés sur machine : les valeurs varient selon l'application. L'application calcule un plus court chemin dans un **graphe** qui dépend du mode de déplacement : à pied, elle peut emprunter escaliers, ascenseurs publics et rues piétonnes, interdits aux voitures ; en voiture, elle doit suivre les rues ouvertes à la circulation et respecter les sens interdits. Les deux trajets, leurs longueurs et surtout leurs durées (la vitesse n'est pas la même) diffèrent donc.

GNSS et vie privée

Exercice 13 *Pas seulement le GPS* ↑énoncé p. 185

► Corrigé.

1. **GLONASS** (Russie), **Galileo** (Europe), **BeiDou** (Chine).
2. En intérieur, les signaux des satellites sont **bloqués** par les murs. Le téléphone se localise alors grâce aux **antennes-relais** et aux réseaux **Wi-Fi** environnants (plus rapide, moins précis).

Exercice 14 *Mes traces* ↑énoncé p. 186

► Corrigé.

1. On devine son **domicile** (le lieu de toutes ses nuits), qu'elle est **lycéenne** et dans quel lycée, qu'elle fait de la **natation** le mercredi, et même une information de **santé** (un rendez-vous médical), particulièrement sensible. Recoupées, ces traces désignent une seule personne : très peu de gens dorment dans cet immeuble *et* vont dans ce lycée *et* dans cette piscine. Elles permettent donc d'**identifier** Léa et de la **suivre** : ce sont des données personnelles, même sans son nom.
2. Par les **métadonnées EXIF** de la photo : prise au smartphone, elle contient souvent les **coordonnées GPS** du lieu de la prise de vue, c'est-à-dire son domicile. Il suffit de télécharger le fichier pour les lire, sauf si Léa les a retirées avant de publier.
3. Lampe de poche : **« jamais »**, elle n'a aucun besoin de la position. Navigation : **« seulement pendant l'utilisation »**, elle en a besoin pour guider, mais pas une fois fermée. Retrouver son téléphone : **« toujours »** se justifie, car l'application doit pouvoir localiser le téléphone perdu, quand personne ne l'utilise ; c'est un choix à réserver à un service de confiance. En plus : effacer régulièrement l'historique des positions.

Localisation et environnement

Exercice 15 *Le coût caché d'un itinéraire* ↑énoncé p. 186

► Corrigé.

1. Une sortie d'une heure : 3 600 secondes, donc **3 600 points**, soit $3\,600 \times 30 = 108\,000$ octets $\approx$ **0,1 Mo**. Sur une année : $3 \times 52 = 156$ sorties, soit $156 \times 108\,000 = 16\,848\,000$ octets $\approx$ **17 Mo**.
2. Ces traces sont des **données personnelles** : recoupées, elles révèlent le domicile (point de départ), les horaires et les habitudes de Nora. Elles sont stockées sur les **serveurs** de l'application (dans des centres de données), souvent sans limite de durée, et peuvent être partagées, revendues ou piratées.
3. Le trafic en temps réel (et les fonds de carte, les itinéraires calculés en ligne) sont des **données servies par des serveurs** : elles sont stockées et calculées dans des **centres de données** puis acheminées par les **réseaux**. Le GPS, lui, ne fait que *recevoir* des signaux et le téléphone calcule seul sa position.

4. Par exemple : télécharger une **carte hors-ligne** une fois (en Wi-Fi) avant un trajet, ce qui évite de recharger les données à chaque usage ; n'autoriser la localisation que **« pendant l'utilisation »** de l'application et la couper quand elle est inutile, ce qui ménage la **batterie** (une batterie usée trop tôt pousse à changer de téléphone, alors que l'essentiel de son empreinte vient de la **fabrication**, comme on le verra au thème *Objets connectés*) et limite les traces envoyées aux serveurs.

Critiquer une réponse d'IA

Exercice 16 *Le GPS expliqué par un assistant* ↑énoncé p. 186

▶ Corrigé.

1. Le calcul est juste ($300\,000 \times 0{,}05 = 15\,000$ km), ainsi que la trilatération, le quatrième satellite et le repli sur le Wi-Fi. Mais le cours dit : chaque satellite « **diffuse** en permanence un message radio contenant l'heure exacte de l'envoi » et « le **récepteur** (votre téléphone) compare l'heure d'envoi à l'heure de réception ».
2. L'erreur : le sens de la communication est inversé. Le téléphone n'**envoie** rien aux satellites : il ne fait que **recevoir** leurs messages, calcule lui-même sa distance à chacun, puis sa position. Correction : « chaque satellite diffuse l'heure d'envoi ; le téléphone la compare à l'heure de réception et en déduit la distance ».
3. Un argument de bon sens suffisait : le GPS fonctionne sans réseau ni carte SIM (en mode avion, en pleine montagne), donc le téléphone ne peut qu'*écouter*. Ou relire, avant de croire l'assistant, la phrase du cours qui décrit ce que fait le récepteur.

Python au service du thème

Exercice 17 *Python au service du thème — convertir des coordonnées* ↑énoncé p. 187

▶ Corrigé. 1. Comme dans le cours, on divise les minutes par 60 avant de les ajouter aux degrés ; la priorité de la division sur l'addition évite les parenthèses.

```
def en_decimal(degres, minutes):
    return degres + minutes / 60

print(en_decimal(45, 30))       # 45.5
print(en_decimal(43, 44.304))   # 43.7384
```

2. On change le signe si le sens est `S` **ou** `W` :

```
def coordonnee(degres, minutes, sens):
    valeur = en_decimal(degres, minutes)
    if sens == "S" or sens == "W":
        valeur = -valeur
    return valeur
```

3. On appelle la fonction avec les données de Rio, puis avec celles de la trame :

```
print(coordonnee(22, 54.6, "S"))     # -22.91
print(coordonnee(43, 10.2, "W"))     # -43.17
print(coordonnee(43, 42.054, "N"))   # 43.7009
print(coordonnee(7, 16.098, "E"))    # 7.2683
```

Rio de Janeiro est en $(-22{,}91\,;\ -43{,}17)$: ce sont exactement les coordonnées de l'exercice « Trois villes sur le globe », négatives car Rio est au sud de l'équateur et à l'ouest de Greenwich. La trame de l'exercice « Que dit le GPS ? » donne bien 43,7009° N et 7,2683° E (Nice).

▷ **Erreur fréquente.** Ne tester que le sud (`if sens == "S":`) : la latitude de Rio devient bien négative, mais sa longitude reste positive, 43,17 : on placerait Rio à l'*est* de Greenwich, dans l'océan Indien, au large de Madagascar. Il faut changer le signe dans les deux cas, reliés par `or`.

CORRIGÉS : DÉBAT

DÉBAT Faut-il laisser la géolocalisation de son téléphone activée en permanence ?

↑énoncé p. 189

À lire après le débat. Ces éléments de réponse servent à vérifier et à compléter votre trace écrite : les lire avant le débat lui ôterait son intérêt.

Notions du cours mobilisées

- **Latitude, longitude** ; **GPS / GNSS** (Galileo…) et **localisation assistée** (antennes-relais, Wi-Fi).
- **Cartes numériques** et **calcul d'itinéraire** : services qui ont besoin de la position.
- **Traces de localisation**, **métadonnées EXIF** des photos (coordonnées GPS).
- **Données personnelles**, **RGPD** et CNIL ; **loi n° 1.565** et **APDP** à Monaco ; réglages « toujours / pendant l'utilisation / jamais ».
- **Sobriété** (partie « Localisation et environnement ») : la localisation permanente consomme batterie et données.

Arguments attendus

POUR (oui) : toujours activée	CONTRE (non) : seulement quand c'est utile
• **Sécurité** : appel aux secours avec la position, partage de position lors d'un trajet de nuit (doc. 1) ; l'eCall montre l'intérêt de la position pour les secours, même s'il est intégré à la voiture et non au téléphone. • **Retrouver un téléphone** perdu ou volé : impossible si la localisation est coupée (doc. 1). • **Confort** : itinéraires, trafic en temps réel, météo locale, sans rien régler (doc. 1, cours). • **La loi protège** : but précis, droits d'accès et d'effacement, autorités de contrôle (doc. 4). • **On garde la main** : position approximative, choix application par application (doc. 4, cours). • **Intérêt collectif** : les positions agrégées servent à calculer l'état du trafic pour tous.	• **Quatre points suffisent** à identifier quelqu'un, même sans nom (doc. 2). • **L'anonymat ne protège pas** : la carte Strava a révélé des lieux secrets (doc. 3). • **Les traces révèlent une vie** : domicile, lycée, habitudes (cours). • **« Toujours » = enregistrement en continu**, même sans utiliser l'application (doc. 4). • **Les services fonctionnent quand même** avec « pendant l'utilisation » : itinéraire, météo (cours). • **Sobriété** : batterie et données consommées en permanence (cours).

Arguments de l'autre camp souvent jugés solides dans la trace écrite : un élève POUR retient souvent « des positions sans nom suffisent à retrouver quelqu'un » ; un élève CONTRE, « en cas d'accident ou de perte, la localisation peut vraiment aider ».

Questions de relance (pour le modérateur)

1. Quelles applications ont vraiment besoin de votre position *en permanence* ? Lesquelles seulement pendant l'utilisation ?
2. Si un service utile ne fonctionne qu'avec « toujours », que faites-vous ?
3. Le doc. 2 parle de données sans nom : pourquoi cela ne suffit-il pas à protéger ?
4. Les données de position agrégées (trafic, affluence) : utiles à tous, ou risquées pour chacun ?

Synthèse possible

La localisation rend de vrais services (itinéraires, secours, objet perdu). Mais chaque position est une **donnée personnelle**, et leur **accumulation** trace notre vie : quatre points suffisent souvent à identifier une personne, même dans des données sans nom. La loi (RGPD ; loi n° 1.565 et APDP à Monaco) impose un but précis et donne des droits, et les téléphones permettent de choisir, application par application, « toujours », « pendant l'utilisation » ou « jamais », et une position approximative. Garder la main, c'est **n'accorder la position qu'au service qui en a besoin, quand il en a besoin**.

Réponses des questions de la fiche de révision

↑ questions p. 191

Se tester

1. $50 + \frac{15}{60} = 50{,}25°$ (§ I).
2. $300\,000 \times 0{,}08 = 24\,000$ km (§ II).
3. 16 h 30 min 45 s UTC ; 9 satellites (§ III).
4. Couper la localisation (ou « pendant l'utilisation ») ; effacer l'historique ; retirer les métadonnées des photos (§ VI).

Et avant ?

1. Traduire un nom de domaine en adresse IP (ch. 6, § V.B).
2. Un échange chiffré, pas un site honnête (ch. 4, § II).

CORRIGÉS DU CHAPITRE 8

8 Informatique embarquée et objets connectés

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 La carte qui réagit

↑énoncé p. 194

Exercice 1 *Un bouton, une réponse*

▶ **Solution.** **1.** Un clic sur A affiche le **cœur** sur l'écran de LED. Tant qu'on n'appuie sur rien, la carte **attend** (rien ne s'affiche, ou l'icône reste affichée).

2. Programme attendu (deux blocs indépendants) :

```
lorsque le bouton A est presse
    montrer l'icone  (coeur)
lorsque le bouton B est presse
    montrer l'icone  (triste)
```

3. L'humain donne un ordre avec les **boutons** (une *entrée*) ; la carte répond avec l'**écran de LED** (une *sortie*).

Exercice 2 *La carte mesure*

▶ **Solution.** **4.** La carte fait défiler la **température** indiquée par le curseur ; le nombre change dès qu'on bouge le curseur.

5. D'un **capteur de température** intégré à la carte : il mesure une grandeur physique et la transforme en nombre.

6. Dans
au démarrage, la température n'est lue et affichée **qu'une seule fois** : bouger le curseur ne change plus rien.
toujours **répète sans fin** la mesure et l'affichage : c'est ce qui permet à la carte de suivre le monde réel.

Exercice 3 *La carte décide*

▶ **Solution.** **7.** Dans la catégorie Logique : le bloc
si … alors … sinon (on peut aussi prendre
si … alors et ajouter le
sinon avec le petit « + »).

8. La comparaison (bloc
0 < 0 de Logique, réglé sur « > ») se place dans la case du

si , avec
température (°C) à gauche et 25 à droite : *température* > 25. Programme attendu :

```
toujours
    si  temperature (degres C) > 25  alors
        montrer l'icone  (non)
    sinon
        montrer l'icone  (oui)
```

(Les accents sont omis dans ces listings. Équivalent affiché par l'onglet Python de MakeCode :)

```
1 def on_forever():
2     if input.temperature() > 25:
3         basic.show_icon(IconNames.NO)
4     else:
5         basic.show_icon(IconNames.YES)
6 basic.forever(on_forever)
```

9. À 20 °C : la condition est fausse → icône **oui** (coche). À 30 °C : la condition est vraie → icône **non** (croix).

10. La carte **mesure** avec le **capteur de température** ; elle **décide** avec le bloc **si … alors … sinon** (le programme) ; elle **agit** avec l'**écran de LED** ; puis elle **recommence** (bloc toujours).

Erreurs fréquentes : bloc si placé dans au démarrage (une seule mesure) ; comparaison inversée (< au lieu de >) ; icônes inversées. Faire tester les deux valeurs du curseur, de part et d'autre de 25, suffit à les repérer.

Exercice 4 *Ce qu'on a découvert*

▶ Solution. 11.

- La partie qui mesure le monde réel s'appelle un **capteur**.
- La partie qui agit sur le monde réel s'appelle un **actionneur**.
- Entre les deux, le **programme** décide, et il recommence **sans fin** (en boucle).
- Les boutons et l'écran permettent à l'humain et à la machine de **communiquer** (de dialoguer) : le cours appellera cela l'**interface homme-machine**.

12. Par exemple : la carte ne fait qu'**une tâche** à la fois (celle qu'on lui a programmée) ; elle n'a ni clavier ni vrai écran ; elle est minuscule et consomme très peu ; elle réagit tout de suite à ce qu'elle mesure.

CORRIGÉS DES EXERCICES

Éléments de réponse aux activités du cours

Réponses aux questions fermées ; pour les questions ouvertes, des pistes attendues (d'autres réponses sont possibles).

Activité 1 — Capteurs ou actionneurs ?

Capteurs : l'appareil photo, le micro, le capteur de luminosité, et la surface tactile de l'écran (elle détecte le doigt). **Actionneurs** : le haut-parleur, le vibreur, la lampe torche, et l'affichage de l'écran. L'écran tactile est donc **les deux à la fois**.

Activité 2 — Compléter une boucle de commande

(a) La condition est `lum < 30` (la luminosité devient faible). (b) La ligne `allumer_led()` commande l'**actionneur** : la LED s'allume. Programme complété :

```
while True:
    lum = lire_luminosite()
    if lum < 30:                # (a) il fait sombre
        allumer_led()           # (b)
    else:
        eteindre_led()
    attendre(1)
```

Activité 3 — Enquête sur un objet connecté

Réponse personnelle. Exemple attendu pour une **montre connectée** : capteurs (cardiofréquencemètre, GPS, accéléromètre, écran tactile) et actionneurs (écran, vibreur) ; elle envoie le rythme cardiaque, les pas et les positions au téléphone, puis aux serveurs du fabricant. Risque de **sécurité** : un compte ou un objet piraté (mot de passe par défaut, mises à jour non faites) ; risque de **vie privée** : des données de santé et de déplacement collectées et parfois partagées. Geste : changer le mot de passe par défaut, faire les mises à jour, limiter les partages dans les réglages.

Activité 4 — Un objet connecté, pour quoi faire ?

1. D'après le document, 99 % des 80 kg CO_2e d'un smartphone sont émis à la **fabrication** : l'éteindre n'agit que sur le 1 % restant. Le garder longtemps **répartit** l'empreinte de fabrication : environ 32 kg par an sur 2,5 ans, mais $80 \div 5 = 16$ kg par an sur 5 ans, et cela évite d'en fabriquer un nouveau.
2. Réponse personnelle (par exemple : une ampoule connectée rend le même service qu'un interrupteur ou une minuterie).
3. Par exemple : les **mises à jour** qui s'arrêtent (l'objet n'est plus sûr, des applications ne fonctionnent plus) ; une **réparation** impossible ou trop chère (pièces collées, pièces introuvables) ; une **batterie** usée qu'on ne peut pas remplacer ; un service en ligne que le fabricant arrête.
4. Par exemple : éteindre vraiment les objets la nuit (multiprise à interrupteur) ; garder son téléphone plus longtemps ; réduire la qualité vidéo et couper la lecture automatique.

Systèmes embarqués

Exercice 1 *Embarqué ou pas ?* ↑énoncé p. 204

▶ Corrigé.

- **Oui** : lave-linge (piloter le cycle de lavage) ; régulateur de vitesse (maintenir la vitesse) ; montre connectée (afficher l'heure, mesurer l'activité) ; feu tricolore (gérer l'alternance des feux).
- **Non** : le marteau et la chaise (aucun ordinateur).

Exercice 2 *Pas tout à fait un ordinateur* ↑énoncé p. 204

▶ Corrigé. Un système embarqué ne réalise **qu'une tâche** (au lieu d'être polyvalent), n'a en général **ni écran ni clavier** classiques, est **minuscule et sobre**, et doit souvent réagir **en temps réel**.

Capteurs et actionneurs

Exercice 3 *Qui mesure, qui agit ?* ↑énoncé p. 204

▶ Corrigé.

- **Capteurs** : thermomètre, détecteur de mouvement, microphone, capteur de luminosité.
- **Actionneurs** : radiateur, haut-parleur, moteur de volet, LED.

▷ **Erreur fréquente.** Ranger le capteur de luminosité avec la LED parce que les deux « parlent de lumière » : le capteur *mesure* la lumière (entrée), la LED en *produit* (sortie). On classe un élément selon ce qu'il fait, pas selon le sujet.

Exercice 4 *Dans un objet du quotidien* ↑énoncé p. 204

▶ Corrigé. *(Portail automatique.)*

1. Capteur : détecteur de présence (ou récepteur de la télécommande/du badge). Actionneur : le **moteur** du portail.
2. Le capteur détecte la voiture (ou reçoit le signal) → le programme **décide** d'ouvrir → il commande le **moteur** (actionneur) → le portail s'ouvre.

La boucle de commande

Exercice 5 *Que fait ce programme ?* ↑énoncé p. 205

▶ Corrigé.

1. Capteur : le thermomètre (`lire_temperature`). Actionneur : le ventilateur.
2. L'objet **rafraîchit** la pièce : il allume le ventilateur dès que la température dépasse 25 °C, et l'éteint sinon.
3. À 22 °C : $22 > 25$ est faux → ventilateur **éteint**. À 28 °C : $28 > 25$ est vrai → ventilateur **allumé**.

Exercice 6 *Dérouler une boucle* ↑énoncé p. 205

▶ Corrigé.

Tour	`lum`	`lum < 40 ?`	État de la lampe
1	70	faux	éteinte
2	30	vrai	allumée
3	10	vrai	allumée (elle le reste)
4	50	faux	éteinte

Entre la première et la dernière mesure, il y a trois attentes de 60 s : **3 minutes**.

▷ **Erreur fréquente.** Répondre 4 minutes parce qu'il y a 4 mesures : entre 4 mesures, il n'y a que 3 attentes (comme il n'y a que 3 intervalles entre 4 poteaux).

Exercice 7 *Compléter un arrosage automatique* ↑énoncé p. 205

▶ Corrigé.

```
while True:
    h = lire_humidite()
    if h < 40:                  # (a) la terre est trop seche
        ouvrir_pompe()          # (b)
    else:
        fermer_pompe()
    attendre(3600)
```

▷ **Erreur fréquente.** Écrire `if h > 40:` : la pompe s'ouvrirait quand la terre est déjà humide et se fermerait quand elle est sèche. Pour vérifier une condition, il suffit de la tester sur deux valeurs, par exemple $h = 20$ (terre sèche) et $h = 70$.

Exercice 8 *Défi — une alarme* ↑énoncé p. 205

▶ Corrigé.

```
while True:
    if detecte():                # mouvement detecte
        sonner()
        allumer_led()
    else:
        eteindre_sirene()
        eteindre_led()
    attendre(0.1)                # 10 fois par seconde
```

Exercice 9 *Le thermostat à deux seuils* ↑énoncé p. 206

▶ Corrigé.

1. Trois cas ; entre 19 et 21 °C, on ne commande rien (pas de `else`) :

```
while True:
    t = lire_temperature()
    if t < 19:
        allumer_chauffage()      # trop froid
    elif t > 21:
        eteindre_chauffage()     # trop chaud
    attendre(60)                 # entre 19 et 21 : on ne change rien
```

2. À 20 °C, aucune des deux conditions n'est vraie : le programme **ne change rien**. Le chauffage éteint **reste éteint** ; s'il était allumé, il **reste allumé** (jusqu'à dépasser 21 °C). Le radiateur ne bascule donc plus sans arrêt autour d'un seul seuil.

Exercice 10 *Le compteur de passages* ↑énoncé p. 206

▶ Corrigé.

1. Le compteur `nb` est créé **avant** la boucle (sinon il repartirait de 0 à chaque tour) :

```
nb = 0
while True:
    if detecte():
        nb = nb + 1
        afficher(nb)
        if nb > 20:
            sonner()
    attendre(0.1)                # 10 tests par seconde
```

2. En une demi-seconde, la boucle teste 5 fois le détecteur : la même personne serait **comptée 5 fois**. Solution simple : après une détection, attendre une seconde avant de reprendre les tests (ajouter `attendre(1)` juste après `afficher(nb)`).

Pour aller plus loin : on peut aussi ne compter que lorsque le détecteur **passe** de `False` à `True`. Il faut alors se souvenir, d'un tour à l'autre, de la mesure précédente, rangée dans une variable `avant` :

```
nb = 0
avant = False
while True:
    maintenant = detecte()
    if maintenant and not avant:     # le detecteur vient de passer a True
        nb = nb + 1
        afficher(nb)
        if nb > 20:
            sonner()
    avant = maintenant               # on retient la mesure pour le tour suivant
    attendre(0.1)
```

L'interface homme-machine (IHM)

Exercice 11 *Entrées et sorties* ↑énoncé p. 206

▶ **Corrigé.** *(Four à micro-ondes.)* **Entrées** : les boutons / la molette (durée, puissance), le bouton « départ ». **Sorties** : l'écran (temps restant), le **bip** de fin, la lumière intérieure.

Exercice 12 *Bonne ou mauvaise IHM ?* ↑énoncé p. 206

▶ **Corrigé.** Sans aucun **retour**, l'utilisateur ne sait pas si son appui a été pris en compte : il doute, ré-appuie, se trompe. Il faudrait ajouter un **retour** : un voyant/une LED, un bip, ou un message à l'écran confirmant l'action.

Exercice 13 *Concevoir une IHM* ↑énoncé p. 206

▶ **Corrigé.**

1. Par exemple. **Entrée** : un bouton « arroser maintenant » (ou une application sur le téléphone), pour déclencher l'arrosage à la main. **Sorties** : un voyant qui passe au rouge quand la terre est sèche (on voit d'un coup d'œil l'état de la plante) ; une notification sur le téléphone quand le réservoir d'eau est vide (on est prévenu même absent).
2. Un **retour** immédiat : un voyant qui clignote ou un bip au début de l'arrosage, puis un signal à la fin. Sans retour, l'utilisateur ne sait pas si son appui a été pris en compte.

Objets connectés, sécurité et vie privée

Exercice 14 *Objet connecté* ↑énoncé p. 206

▶ **Corrigé.**

1. Un objet connecté est **relié au réseau** et **échange des données** ; un objet seulement embarqué ne communique pas.
2. Exemples : montre connectée → rythme cardiaque / nombre de pas ; caméra → images ; thermostat → température (au choix).

Exercice 15 *Le danger des mots de passe par défaut* ↑énoncé p. 206

▶ **Corrigé.** *(Mirai.)*

1. Les propriétaires n'avaient jamais **changé le mot de passe par défaut** (souvent `admin`/`admin`).
2. L'objet piraté devient un « robot » qui participe à une **attaque** (déni de service) contre d'autres sites : il nuit à *tout* Internet, pas seulement à son propriétaire.
3. Changer le mot de passe par défaut ; installer les **mises à jour** (et couper l'objet quand il ne sert pas).

Exercice 16 *Vie privée* ↑énoncé p. 207

▶ **Corrigé.** Une enceinte qui « écoute » en permanence peut enregistrer des conversations privées et les envoyer vers un serveur : ce sont des **données personnelles**, dont l'usage est encadré en France (et dans l'UE) par le **RGPD** (autorité : CNIL). Monaco n'étant pas dans l'UE, c'est la **loi n° 1.565 du 3 décembre 2024** qui s'y applique (autorité : APDP).

Impact environnemental

Exercice 17 *L'empreinte d'un smartphone* ↑énoncé p. 207

▶ Corrigé.

1. $80 \div 2{,}5 = 32$: environ **32 kg de CO_2e par an**.
2. $80 \div 5 = 16$: environ **16 kg de CO_2e par an**. Garder le téléphone deux fois plus longtemps divise environ **par deux** son empreinte annuelle, car la fabrication (99 %) est « amortie » sur plus d'années.
3. Éteindre le téléphone ne réduit que la part « usage », qui ne pèse qu'environ 1 % ici ; garder le téléphone plus longtemps agit sur la fabrication, qui pèse environ 99 %.
4. Raisons : batterie usée, écran cassé difficile à réparer, **mises à jour** de sécurité qui s'arrêtent, mode, stockage plein… Remèdes : faire réparer (changer la batterie ou l'écran), choisir un modèle réparable et suivi longtemps, acheter reconditionné.

Critiquer une réponse d'IA

Exercice 18 *Le volet automatique d'un assistant* ↑énoncé p. 207

▶ Corrigé.

1. Non. La structure est la bonne (acquisition → décision → action → attente, comme le thermostat du cours), mais le déroulé donne : `lum` = 5 → condition vraie → `ouvrir_volet()` ; `lum` = 80 → condition fausse → `fermer_volet()`. Le volet s'**ouvre** la nuit et se **ferme** le jour : l'inverse de la demande.
2. L'erreur : les deux **actions** sont inversées dans le `if` / `else` (le commentaire « il fait nuit » est juste, l'action qui le suit ne l'est pas). Correction : `fermer_volet()` sous `if lum < 20:` et `ouvrir_volet()` sous `else:`.
3. Dérouler la boucle « à la main » avec **une valeur de chaque côté du seuil** (une mesure de nuit, une de jour) avant d'accepter le programme : c'est exactement la vérification de l'exercice 5.

Python au service du thème

Exercice 19 *Python au service du thème — simuler un thermostat* ↑énoncé p. 207

▶ Corrigé. 1. La fonction reprend la décision du thermostat du cours, mais **renvoie** l'ordre au lieu de commander le radiateur :

```
def commande(t):
    if t < 19:
        return "allumer"
    else:
        return "eteindre"

print(commande(17))   # allumer
print(commande(19))   # eteindre
```

2. Allumé, le radiateur ajoute 2 à la température ; éteint, la pièce perd 1 :

```
t = 15
for minute in range(8):
    ordre = commande(t)
    print(f"minute {minute} : {t} degres -> {ordre}")
    if ordre == "allumer":
        t = t + 2
    else:
        t = t - 1
```

Le programme affiche :

```
minute 0 : 15 degres -> allumer
minute 1 : 17 degres -> allumer
minute 2 : 19 degres -> eteindre
minute 3 : 18 degres -> allumer
minute 4 : 20 degres -> eteindre
minute 5 : 19 degres -> eteindre
minute 6 : 18 degres -> allumer
minute 7 : 20 degres -> eteindre
```

3. À partir de la minute 2, la pièce oscille entre **18** et **20 °C**. Le radiateur est rallumé (minutes 3 et 6) puis éteint (minutes 4 et 7) presque à chaque minute : ces marche-arrêt incessants l'usent et gaspillent de l'énergie. C'est le défaut que corrige l'exercice 9, « Le thermostat à deux seuils » : on n'allume que sous 19 °C et on n'éteint qu'au-dessus de 21 °C.

▷ **Erreur fréquente.** Écrire `t = 15` *dans* la boucle : la température repart de 15 à chaque tour, le programme affiche huit fois `15 degres -> allumer` et la simulation n'avance jamais. Comme pour le compteur de passages (exercice 10), une variable qui doit garder sa valeur d'un tour à l'autre se crée **avant** la boucle.

CORRIGÉS : DÉBAT

DÉBAT Faut-il équiper sa maison de caméras et d'enceintes connectées ?

↑ énoncé p. 210

À lire après le débat. Ces éléments de réponse servent à vérifier et à compléter votre trace écrite : les lire avant le débat lui ôterait son intérêt.

Notions du cours mobilisées

- **Système embarqué**, **objet connecté**, **Internet des objets**.
- **Capteurs** (caméra, micro, thermomètre) et **actionneurs** (chauffage, haut-parleur) ; **boucle mesurer – décider – agir** (thermostat).
- **IHM** : commande vocale, application sur téléphone.
- **Nuage** : où vont les vidéos et les enregistrements ? qui les détient ?
- **Sécurité** (Mirai, mot de passe par défaut, mises à jour) et **vie privée** (RGPD ; loi n° 1.565, APDP).
- **Impact environnemental** : fabrication, veille permanente, durée de vie.

Arguments attendus

POUR (oui) : équiper sa maison	CONTRE (non) : ne pas équiper
• **Économies d'énergie** : thermostat connecté, 1 °C de moins ≈ 7 % d'économie (doc. 1). • **Sécurité du logement** : alerte d'intrusion, voir qui sonne (doc. 1). • **Autonomie** des personnes âgées ou en situation de handicap (doc. 1). • **La loi encadre** ce que l'on peut filmer (doc. 4) et va imposer la sécurité des objets (doc. 4). • **On peut se protéger** : changer le mot de passe par défaut, mises à jour, refuser l'écoute (doc. 3, cours). • **Les abus sont sanctionnés** (doc. 2).	• **Des inconnus peuvent voir vos images** : employés, pirates (doc. 2). • **Le micro écoute** : enregistrements déclenchés par erreur, écoutés par des humains (doc. 3). • **Porte d'entrée sur le réseau** : Mirai a transformé des caméras en armée de robots (cours). • **Surveiller les autres** : invités, voisins, personnes employées à domicile (doc. 4). • **Les données partent dans le nuage**, chez une entreprise souvent étrangère (cours). • **Impact environnemental** : fabrication et veille permanente ; un objet non connecté rend parfois le même service (cours).

Arguments de l'autre camp souvent jugés solides dans la trace écrite : un élève POUR retient souvent « une caméra mal sécurisée peut être regardée par d'autres que soi » ; un élève CONTRE, « un thermostat connecté permet de vraies économies d'énergie ».

Questions de relance (pour le modérateur)

1. Une caméra dans le salon : rassurante pour qui ? gênante pour qui (invités, enfants, aide à domicile) ?
2. Un thermostat connecté et une caméra présentent-ils les mêmes risques ? Classer les objets du moins au plus intrusif.
3. Qu'est-ce qui changerait si les vidéos restaient dans la maison au lieu de partir dans le nuage ?
4. Un objet non connecté pourrait-il rendre le même service ? (cours, impact environnemental)

Synthèse possible

Un objet connecté **mesure** (capteurs), **décide**, **agit**, et envoie souvent ses données dans le **nuage**. La maison connectée rend des services réels (économies d'énergie, alerte, autonomie), mais une caméra ou une enceinte est un capteur qui **observe et écoute** : images et sons peuvent être vus par d'autres (employés, pirates). Deux risques sont à distinguer : la **sécurité** (mot de passe par défaut, mises à jour) et la **vie privée** (ce qui est collecté, où, par qui). La loi encadre ce que l'on peut filmer (CNIL ; loi n° 1.565 et APDP à Monaco). Avant d'équiper sa maison : se demander quel **service**, avec quel **capteur**, et si un objet non connecté suffirait.

Réponses des questions de la fiche de révision

↑ questions p. 212

Se tester

1. Capteurs : thermomètre, micro ; actionneurs : pompe, LED (§ II).
2. `if t > 30:` puis `ouvrir_fenetre()` (§ III).
3. Entrée : boutons ; sortie : affichage de l'étage, bip (§ IV).
4. Un réseau ; changer le mot de passe par défaut, mettre à jour (§ V et VI).

Et avant ?

1. La trilatération (ch. 7, § II.A).
2. Les garder sur des serveurs distants, accessibles par Internet (ch. 5, § VI.B).

Lexique

Les définitions sont résumées ; le renvoi (chapitre et page, cliquable) mène à la définition complète dans le cours.

A

Actionneur Organe qui agit sur le monde physique (sortie) : moteur, lampe, haut-parleur… ch. 8, p. 197

Adresse IP Numéro d'une machine sur le réseau ; en IPv4, quatre nombres de 0 à 255 (IPv6 : adresses bien plus longues). ch. 6, p. 157

Affectation Instruction qui donne une valeur à une variable ; en Python, le signe `=`. ch. 1, p. 18

Algorithme de recommandation Programme qui choisit et ordonne les contenus affichés dans votre fil, d'après ce que vous avez déjà regardé ou aimé. ch. 2, p. 65

B

Balise Marque HTML, ouvrante et fermante (`<p>…</p>`), qui délimite un élément de la page. ch. 4, p. 112

Big data Données trop volumineuses pour les outils classiques : les « 3 V » (Volume, Vélocité, Variété). ch. 5, p. 136

Booléen Valeur qui ne peut être que `True` (vrai) ou `False` (faux). ch. 1, p. 26

Boucle Structure qui répète des instructions : `for` (nombre de répétitions connu) ou `while` (tant qu'une condition est vraie). ch. 1, p. 29

Boucle de commande Programme qui répète sans fin : mesurer (capteur), décider, agir (actionneur). ch. 8, p. 198

Bulle de filtres Effet d'un algorithme de recommandation qui finit par ne plus montrer que ce qui vous plaît. ch. 2, p. 65

C

Capteur (objet) Organe qui mesure une grandeur physique et la transforme en nombre (entrée). ch. 8, p. 197

Capteur (photographique) Surface couverte de millions de photosites qui transforment la lumière reçue en nombres. ch. 3, p. 83

Carte numérique Carte en couches d'informations sur laquelle on peut zoomer, mesurer et calculer un itinéraire. ch. 7, p. 179

Chaîne de caractères Du texte, écrit entre guillemets (type `str`). ch. 1, p. 20

Client, serveur Le client (le navigateur) demande une page ; le serveur, allumé en permanence, la stocke et la renvoie. ch. 4, p. 110

Compilateur, interpréteur Programmes qui traduisent en langage machine des instructions écrites avec des mots. Python est un langage interprété. ch. 1, p. 14

Concaténation Mise bout à bout de deux chaînes de caractères avec le signe `+`. ch. 1, p. 23

Cookie Petit fichier déposé par un site dans le navigateur pour vous reconnaître, ou vous suivre de site en site. ch. 4, p. 117

CSS Langage qui décrit l'apparence d'une page web (couleurs, polices, mise en page). ch. 4, p. 114

CSV Format texte ouvert pour les tables : une ligne par enregistrement, valeurs séparées par des virgules ou des points-virgules. ch. 5, p. 133

Cyberharcèlement Insultes, menaces ou moqueries répétées en ligne ; c'est interdit par la loi. Réflexes : garder les preuves, bloquer, signaler, en parler. ch. 2, p. 66

D

Définition (d'une image) Nombre de pixels d'une image : largeur × hauteur. ch. 3, p. 85

Degré Nombre d'arêtes qui partent d'un sommet d'un graphe (le nombre d'amis). ch. 2, p. 63

Descripteur, enregistrement Un descripteur est une colonne d'une table ; un enregistrement en est une ligne. ch. 5, p. 132

Diamètre, rayon, centre Plus grande et plus petite excentricité d'un graphe ; le centre est le sommet dont l'excentricité est la plus petite. ch. 2, p. 63

Distance (graphe) Longueur (en nombre d'arêtes) de la plus courte chaîne qui relie deux sommets. ch. 2, p. 63

DNS Annuaire d'Internet : il traduit une adresse symbolique (`google.fr`) en adresse IP. ch. 6, p. 158

Donnée Information élémentaire enregistrée : un nombre, un mot, une date, une image… ch. 5, p. 131

Donnée personnelle Donnée qui permet d'identifier une personne, directement ou indirectement. ch. 2, p. 60 ; ch. 5, p. 131

Droit d'auteur, licence Le droit d'auteur protège toute création ; une licence fixe ce qu'on a le droit d'en faire. ch. 4, p. 116

E

E-réputation Image que votre identité numérique donne de vous aux autres. ch. 2, p. 60

Empreinte environnementale, sobriété Effets du numérique sur l'environnement (gaz à effet de serre, ressources), surtout dus à la fabrication des appareils ; la sobriété consiste à en limiter l'usage. ch. 8, p. 201

Excentricité Distance d'un sommet au sommet le plus éloigné de lui. ch. 2, p. 63

F

Filtre de Bayer Mosaïque de petits filtres rouges, verts et bleus placée devant les photosites pour capter la couleur. ch. 3, p. 83

Fonction Mini-programme qui porte un nom, reçoit des valeurs d'entrée (les paramètres) et renvoie souvent un résultat (`return`). ch. 1, p. 24

Format ouvert Format dont la structure est publique, lisible par n'importe quel logiciel ; s'oppose au format fermé (propriétaire). ch. 5, p. 133

G

GPS, GNSS Système de positionnement par satellites (américain) ; GNSS désigne l'ensemble de ces systèmes (GPS, Galileo, GLONASS, BeiDou). ch. 7, p. 176

Graphe Ensemble de sommets (les personnes) reliés par des arêtes (les relations) ; il modélise un réseau social. ch. 2, p. 63

H

Hameçonnage Faux message (courriel, SMS) qui imite un site connu pour vous faire saisir vos identifiants. ch. 4, p. 117

HTML Langage de balisage qui décrit le contenu et la structure d'une page web. ch. 4, p. 112

HTTP, HTTPS Protocole du dialogue entre client et serveur ; HTTPS en est la version sécurisée (données chiffrées). ch. 4, p. 110

Hypertexte, hyperlien Texte augmenté de renvois cliquables (les hyperliens) vers d'autres documents. ch. 4, p. 108

I

Identité numérique Ensemble des informations qui vous représentent en ligne, y compris les traces laissées sans y penser. ch. 2, p. 60

Indentation Décalage des lignes qui indique, en Python, quelles instructions forment un bloc. ch. 1, p. 25

Instruction conditionnelle Bloc d'instructions exécuté seulement si une condition est vraie (`if`, `elif`, `else`). ch. 1, p. 28

Interface homme-machine (IHM) Moyens de commander une machine et d'en recevoir un retour (écran, boutons, voix…). ch. 8, p. 199

Internet Réseau physique mondial de machines qui échangent des données grâce au protocole IP. ch. 4, p. 109

IPv6 Nouvelle version des adresses IP, bien plus longues, pour numéroter des milliards de milliards de machines. ch. 6, p. 157

L

Langage de programmation Langage qui permet d'écrire des programmes compréhensibles par la machine après traduction ; de bas niveau (proche de la machine) ou de haut niveau (proche du langage humain), comme Python. ch. 1, p. 14

Langage machine Seul langage que le processeur comprend vraiment : une suite de 0 et de 1. ch. 1, p. 14

Latitude, longitude Angles qui repèrent un point : Nord–Sud par rapport à l'équateur, Est–Ouest par rapport au méridien de Greenwich. ch. 7, p. 175

M

Métadonnée Donnée qui décrit une autre donnée (date, taille, appareil, position d'une photo…). ch. 5, p. 131

Métadonnées EXIF Données rangées dans un fichier photo : date, appareil, réglages et souvent la position GPS. ch. 3, p. 87

Microcontrôleur Petit processeur d'un système embarqué. ch. 8, p. 197

Module Boîte à outils de fonctions supplémentaires que l'on importe (`math`, `turtle`…). ch. 1, p. 22

Moteur de recherche Service qui explore le Web avec des robots, range les pages dans un index et classe les réponses à une requête. ch. 4, p. 116

N

Navigateur Logiciel client qui demande les pages web et les affiche (Firefox, Chrome…). ch. 4, p. 110

Neutralité du Net Principe selon lequel les routeurs transmettent tous les paquets de la même façon, sans favoriser un service. ch. 6, p. 160

Nuage (*cloud*) Données stockées sur des serveurs distants, accessibles par Internet. ch. 5, p. 136

O

Objet connecté, IoT Objet à capteurs ou actionneurs qui échange des données par un réseau ; leur ensemble forme l'Internet des objets (IoT). ch. 8, p. 200

Open data Données publiques librement téléchargeables et réutilisables (data.gouv.fr, portail de Monaco…). ch. 5, p. 134

P

Pair-à-pair (P2P) Réseau sans serveur central, où chaque machine est à la fois émetteur et récepteur. ch. 6, p. 159

Paquet Petit morceau de données qui voyage sur Internet avec l'adresse de l'expéditeur, celle du destinataire et un numéro. ch. 6, p. 156

Paramètre Valeur d'entrée d'une fonction, placée entre parenthèses après son nom. ch. 1, p. 24

Petit monde Idée qu'une courte chaîne de connaissances relie deux inconnus (« six degrés de séparation », expérience de Milgram, 1967). ch. 2, p. 64

Photosite Petite cellule d'un capteur qui mesure la quantité de lumière reçue. ch. 3, p. 83

Pixel Point coloré de l'image numérique (*picture element*). ch. 3, p. 84

Poids (d'une image) Taille du fichier : environ largeur × hauteur × 3 octets avant compression. ch. 3, p. 85

Polygone régulier (tortue) Pour tracer un polygone régulier à n côtés, la tortue tourne de $360/n$ degrés à chaque sommet. compl., p. 50

Programme Suite d'instructions données à un ordinateur pour qu'il effectue une tâche. ch. 1, p. 14

R

Réseau social Service en ligne qui permet de se créer un profil, de se relier à d'autres personnes et de partager des contenus. ch. 2, p. 59

Résolution Nombre de pixels par unité de longueur, souvent en ppp (pixels par pouce). ch. 3, p. 85

RGPD Règlement européen qui protège les données personnelles (consentement, droit d'accès, droit à l'oubli) ; à Monaco, loi n° 1.565 (2024). ch. 2, p. 60

Rosace Figure obtenue en répétant un même motif et en tournant d'un petit angle à chaque fois. compl., p. 51

Routeur, routage Le routeur aiguille chaque paquet vers la prochaine étape ; le routage est ce cheminement de proche en proche. ch. 6, p. 157

RVB (synthèse additive) Codage d'une couleur par trois nombres de 0 à 255 (Rouge, Vert, Bleu) ; mélanger ces trois lumières donne toutes les couleurs. ch. 3, p. 84

S

Système embarqué Ordinateur intégré dans un objet, dédié à une tâche précise. ch. 8, p. 197

T

Table de données Données rangées en lignes (les enregistrements) et en colonnes (les descripteurs). ch. 5, p. 132

Tableur Logiciel pour trier, filtrer, calculer et représenter les données d'une table. ch. 5, p. 135

TCP/IP Protocoles d'Internet : IP adresse et achemine les paquets, TCP vérifie qu'ils arrivent tous et les remet dans l'ordre. ch. 6, p. 156

Trame NMEA Ligne de texte envoyée par un récepteur GPS : heure, latitude, longitude, nombre de satellites… ch. 7, p. 178

Trilatération Calcul d'une position en croisant les distances à plusieurs satellites. ch. 7, p. 177

TTL Compteur porté par chaque paquet : le nombre maximal de routeurs qu'il peut encore traverser ; à 0, le paquet est détruit. ch. 6, p. 157

Type Nature d'une valeur : entier (`int`), nombre à virgule (`float`), chaîne de caractères (`str`), booléen (`bool`). ch. 1, p. 20

U

URL Adresse d'une page web. ch. 4, p. 111

UTC Temps universel coordonné : l'heure de référence mondiale, la même partout sur Terre (Monaco est à UTC+1 en hiver, UTC+2 en été). ch. 7, p. 178

V

Variable Donnée rangée dans la mémoire, repérée par un nom et qui contient une valeur, laquelle peut changer. ch. 1, p. 18

W

Web Service d'Internet : des pages reliées par des hyperliens, consultées avec un navigateur. ch. 4, p. 109

Crédits iconographiques

Illustrations d'ouverture de chapitre — dessins originaux de l'auteur.

Chapitre 1 — Les bases de la programmation Python

Captures d'écran du logiciel Spyder (logiciel libre, licence MIT) — captures réalisées par l'auteur — licence MIT pour le logiciel — p. 16
`https://www.spyder-ide.org`

Grace Hopper en 1984 — James S. Davis (U.S. Navy) — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Commodore_Grace_M._Hopper,_USN_(covered).jpg`

Guido van Rossum en 2006 — Doc Searls — CC BY-SA 2.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Guido_van_Rossum_OSCON_2006.jpg`

Complément : dessiner avec la tortue

Tortue robot Logo (Valiant Turtle, 1983) — Valiant Technology Ltd. — CC BY-SA 3.0 — p. 48
`https://commons.wikimedia.org/wiki/File:Turtle_draw.jpg`

Seymour Papert présentant Logo à Moscou, 1987 — Shen-montpellier — CC BY-SA 4.0 — p. 48
`https://commons.wikimedia.org/wiki/File:Papert-moscow-1987.jpg`

Chapitre 2 — Les réseaux sociaux

Stanley Milgram en 1974 — auteur inconnu (*Dayton Daily News*, 1974) — domaine public — p. 58
`https://commons.wikimedia.org/wiki/File:Stanley_Milgram_1974.jpg`

Mark Zuckerberg en 2019 — Anthony Quintano — CC BY 2.0 — p. 58
`https://commons.wikimedia.org/wiki/File:Mark_Zuckerberg_2019_(cropped).jpg`

Chapitre 3 — La photographie numérique

« Point de vue du Gras », Nicéphore Niépce, vers 1826 — Joseph Nicéphore Niépce — domaine public — p. 82
`https://commons.wikimedia.org/w/index.php?curid=107219`

Premier appareil photo numérique (Steven Sasson, 1975) — Bertrand Labévue — CC BY-SA 4.0 — p. 82
`https://commons.wikimedia.org/wiki/File:Kodak_Sasson_BL.jpg`

Capteur CMOS et module de stabilisation d'un Sony A7R III (image réduite) — Harry Munday — CC BY-SA 4.0 — p. 83
`https://commons.wikimedia.org/wiki/File:Sony_A7RIII_CMOS_Sensor_and_IBIS_Module_.jpg`

« The Blue Marble », la Terre vue d'Apollo 17 le 7 décembre 1972 (image `terre.jpg` *des programmes)* — NASA, équipage d'Apollo 17 — domaine public
`https://commons.wikimedia.org/wiki/File:The_Earth_seen_from_Apollo_17.jpg`

Capteur photo vu au macro — Prosthetic Head — CC BY-SA 4.0 — p. 83
`https://commons.wikimedia.org/wiki/File:Colour_Sensor_Macro.jpg`

La Nuit étoilée, *Vincent van Gogh, 1889 (Museum of Modern Art, New York ; numérisation Google Art Project, image réduite)* — Vincent van Gogh — domaine public — p. 101
`https://commons.wikimedia.org/wiki/File:Van_Gogh_-_Starry_Night_-_Google_Art_Project.jpg`

The Blue Boy, *Thomas Gainsborough, vers 1770 (The Huntington Library, San Marino ; image réduite)* — Thomas Gainsborough — domaine public — p. 102
`https://commons.wikimedia.org/wiki/File:Thomas_Gainsborough_-_The_Blue_Boy_(The_Huntington_Library,_San_Marino_L._A.).jpg`

Chapitre 4 — Le Web

La première page web (1991), capture d'écran de la page restaurée — CERN — page publiée par le CERN — p. 109
`http://info.cern.ch/hypertext/WWW/TheProject.html`

Le NeXT de Tim Berners-Lee, premier serveur web (CERN) — Coolcaesar (en.wikipedia) — CC BY-SA 3.0 — p. 108
`https://commons.wikimedia.org/wiki/File:First_Web_Server.jpg`

Tim Berners-Lee en 2009 — Enrique Dans — CC BY 2.0 — p. 108
`https://commons.wikimedia.org/wiki/File:Tim_Berners-Lee_April_2009.jpg`

Chapitre 5 — Les données structurées et leur traitement

Machines de Hollerith au recensement de 1890 — *Scientific American*, 30 août 1890 — domaine public — p. 130
`https://commons.wikimedia.org/wiki/File:`
`1890_Census_Hollerith_Electrical_Counting_Machines_Sci_Amer.jpg`

Carte perforée de Hollerith — auteur inconnu (Library of Congress) — domaine public — p. 130
`https://commons.wikimedia.org/wiki/File:Hollerith_Punched_Card.jpg`

Chapitre 6 — Internet

Icônes des schémas de réseau (ordinateur, switch, routeur, serveur, nuage) — logiciel libre Filius — GNU GPL v2 ou v3 — p. 157
`https://www.lernsoftware-filius.de`

Cahier de bord de l'IMP, 29 octobre 1969 — Andrew « FastLizard4 » Adams — CC BY-SA 2.0 — p. 154
`https://commons.wikimedia.org/wiki/File:`
`The_IMP_Log_The_Very_First_Message_Sent_on_the_Internet_(6293913865).jpg`

Schéma d'ARPANET (4 nœuds), 1969 — DARPA — domaine public — p. 154
`https://commons.wikimedia.org/wiki/File:Map_of_the_4_node_arpanet,_of_1969,_drawn_on_a_napkin.jpg`

Vinton Cerf en 2010 — Veni Markovski — CC BY 3.0 — p. 154
`https://commons.wikimedia.org/wiki/File:Vint_Cerf_-_2010.jpg`

Chapitre 7 — Localisation, cartographie et mobilité

Le méridien de Greenwich (Londres) — Perituss — CC0 — p. 175
`https://commons.wikimedia.org/wiki/File:Prime_meridian_Greenwich_2011_1.jpg`

Satellite GPS Navstar (bloc II-F) — U.S. Air Force — domaine public — p. 176
`https://commons.wikimedia.org/wiki/File:Navstar-2F.jpg`

Chapitre 8 — Informatique embarquée et objets connectés

Apollo Guidance Computer (1969) — Steve Jurvetson — CC BY 2.0 — p. 196
`https://commons.wikimedia.org/wiki/File:Apollo_Guidance_Computer_(AGC).jpg`

Kevin Ashton en 2015 — Larry D. Moore — CC BY 4.0 — p. 196
`https://commons.wikimedia.org/wiki/File:Kevin_Ashton_2015.jpg`

Carte micro:bit (v2) — SimonWaldherr — CC BY 4.0 — p. 197
`https://commons.wikimedia.org/wiki/File:BBC_micro_bit_v2.jpg`